Presentation of the course and setting up R

MATH 2740 – Mathematics of Data Science – Unit U01

Julien Arino
julien.arino@umanitoba.ca

Department of Mathematics @ University of Manitoba

Fall 2026

The University of Manitoba campuses are located on original lands of Anishinaabeg, Ininew, Anisininew, Dakota and Dene peoples, and on the National Homeland of the Red River Métis. We respect the Treaties that were made on these territories, we acknowledge the harms and mistakes of the past, and we dedicate ourselves to move forward in partnership with Indigenous communities in a spirit of Reconciliation and collaboration.

Outline

General information about the course

Mathematics of data science?

Setting things up for the course

Introduction to R Markdown

(Super brief) Introduction to programming in R

General information about the course

Mathematics of data science?

Setting things up for the course

Introduction to R Markdown

(Super brief) Introduction to programming in R

Foreword

- "Numerical dates" are in the form YYYY-MM-DD (e.g, these slides were compiled on 2026-10-27)
- Times are 24h (HHMM)
- Units are SI

In case you want to know, the slides in the course are `Rnw` compiled using `R`. All (including source code) are available on GitHub here

Getting in touch

- julien.arino@umanitoba.ca
- Please use your `myumanitoba` email address. Use a tag such as `[MATH 2740]` in your subject line, if you want to be read..
- Word of warning: I am bad with email! I do answer questions after class or during office hours

Office hours

- Because of the ongoing renovation of Machray Hall, I am sharing an office with 8 colleagues, connected to another office shared by 8 colleagues
- $\Longrightarrow$ it is **impossible** to see you in my office
- I have booked 316 St Paul's from 1400 to 1800 on Tuesday for office hours
- Seeing you outside of office hours is impossible without ample warning, since I actually need to book a room

Course website - UMLearn

- All information about the course is posted on UMLearn
- It is **your responsibility** to check the UMLearn site regularly: *Announcements* is how I normally communicate with you about the course
- (Remember to hit the link at the top of the page that says *MATH-2740-A01 - Mathematics of Data Science*, sometimes UMLearn takes you directly to Content, which is not where Announcements are)

Important – I **DO NOT** answer emails that ask about things easy to find on UMLearn

Lectures

- TR 1130–1245 in 100 Fletcher Argue

- Videos for the course as I taught it in 2021 are available as a YouTube playlist. There is no guarantee that that the content will be the same this year, but there will be commonalities for sure

Tutorials

Section	Day and time	Location
B01	W 0830–0920	P230 Duff Roblin
B03	W 0930–1020	303 Tier
B04	W 1130–1220	P230 Duff Roblin

- In tutorials, you will review some of the mathematical content and work on computations
- Three (3) Lab Tests (45 minutes) will be held during tutorials (dates later in the slides)

Work in labs

- The lecture notes contain problems that will be worked on in labs (when not in a Lab Test)
- During the tutorial, the TA will go over some of the problems
- A dedicated solutions manual will be posted on UMLearn
- Solutions will only be posted after the corresponding tutorial period has ended

R worksheets

- Working on R coding is important in this course, although because of LLMs, it is not realistic anymore to use assignments to test this capacity
- Worksheets will be posted on UMLearn at the start of each week
- Solutions will be posted on UMLearn the next week
- They are meant as practice and will not be graded
- Lab Tests, the Midterm and Final Examinations will all contain *one* question asking you to explain what a piece of R code does

Evaluation

- 3 Lab Tests during tutorials (**10%** each for a total **30%**)
- 1 midterm on Thursday 5 November 2026 1800–2000 (**30%**)
- 1 final examination during the December 2026 Examination period (**40%**)
- In all written work, explain what you are doing. No explanation $\implies$ lost marks (even if correct)

Lab Tests

- Held during tutorials on 23 Sept, 21 Oct, 25 Nov
- 45 minutes long
- You **must** write Lab Tests in the tutorial you are registered in
- Tests your capacity to remember definitions or results, do simple proofs, carry out computations and explain R code

Absences from scheduled evaluations

- If you miss a lab test or the midterm examination, you can either use a self-declaration form or provide proper documentation (MD's note)
- Self-declaration **must be made within 48 hours** of the event you are using it for
- They are intended for occasional and unforeseen circumstances, I **will not accept more than one during the term**
- You **must** use the form on this page to self-declare your absence

What if you missed an evaluation?

If you have proper documentation (self-declaration or MD's note), then

- Lab Tests: the weight of the missed test is shifted to the final examination
- Midterm Examination: you will write a Deferred Midterm Examination (date set by Department of Mathematics)

Miss more than two (2) evaluations (Lab Tests or Midterm) $\Longrightarrow$ **F in the course**

Unhappy about a mark

If you realise that some points were forgotten (this happens) or feel that you have been marked unfairly, feel free to reach **me** about it, preferably in person after class or during office hours

Do not bother the TAs with this, they do not have the authority to change your marks

- I will only consider requests made within seven (7) calendar days from the date you receive your mark
- When processing requests, I **remark the entire examination** $\Longrightarrow$ your overall mark could increase, stay the same **or even decrease**

Colour coding in slides & lecture notes

Memorising the definitions is part of the course. To help, definitions are colour coded

Definition 1 (Important definitions)

These definitions are important, you need to know them

Definition 2 (Definitions)

These definitions are a little less important, you will not be asked to state them (although it is a good idea to know them anyway)

In lecture notes, colour coding is only for important stuff

Results are colour coded too

Results are also colour coded, the same way are definitions. For example, with theorems (the same is true for propositions and others)...

Theorem 3 (Important theorems)

Theorems in orange boxes are important, you should know them and be able to reproduce them

Theorem 4 (Theorems)

Theorems in blue boxes are worth knowing but you will not be asked to reproduce them

You must know how to do some proofs

There are a few proofs (not many!) that I want you to know how to do

Such proofs appear on slides like the present one, with a background as on this slide (i.e., like important definitions and results)

BECOME A VOLUNTEER NOTETAKER

Why?

- Help reduce barriers for your peers
- Contribute to creating an inclusive University of Manitoba
- Receive recognition on your Experience Record

How?

- Login and upload your notes directly at https://sasclockwork.cc.umanitoba.ca/ClockWork/
- Submit notes in your preferred style (typed or handwritten)
- It only takes an extra 15 minutes weekly
- Full instructions on to the Student Accessibility Services website

https://umanitoba.ca/student-supports/accessibility#volunteer-note-sharing

Student Accessibility Services | UM

General information about the course

Mathematics of data science?

Setting things up for the course

Introduction to R Markdown

(Super brief) Introduction to programming in R

In days of yore (circa 2010)

> *Big data is like teenage sex: everyone talks about it, nobody really knows how to do it, everyone thinks everyone else is doing it, so everyone claims they are doing it...*

Attributed to Dan Ariely (Duke University)

The vocabulary has evolved, big data $\rightarrow$ complex data $\rightarrow$ data science, but Data Science remains a loosely defined concept, although things are becoming better

Data Science (according to Wikipedia)

Data science is an **interdisciplinary field** that uses scientific methods, processes, algorithms and systems to extract knowledge and insights from **structured** and **unstructured data**, and apply knowledge and actionable insights from data across a broad range of application domains.

[..] It uses techniques and theories drawn from many fields within the context of **mathematics**, **statistics**, **computer science**, **information science**, and **domain knowledge**.

The data deluge

► Data science is nothing new (some statisticians argue it is just another name for statistics), but it has become prominent in recent years as a consequence of the unprecedented mass of information generated and collected by our modern societies

► One speaks of *information explosion* or *data deluge*. See some considerations, e.g., here

A wide variety of jobs

We have absolutely insane amounts of data and we try to make sense of it

$\Longrightarrow$ data science

However, except for the name, the situation has not improved significantly since the days of yore of Ariely's quote: data science is a hodge-podge that contains everything but the kitchen sink

To caricature

- two main types of data: structured and unstructured
- two main branches: statistics and computer science
- two main types of jobs: users and developpers

Math of Data Science?

- DS has two main branches: statistics and computer science
- DS has two main types of jobs: users and developpers

So why a course on Math of Data Science?

If you plan to be a user and are not curious about *the how* and *the why* and can tolerate errors due to misuse of methods, then you probably don't care about this course

In other cases, many of the concepts used have their roots in math and to understand where the methods are coming from and, even more importantly, to develop new methods, math is often required

Warning! We barely brush the surface

- Some techniques from linear algebra
- Some graph theory ideas
- A little bit of multivariable calculus

There is a lot more to see!!!

Prerequisites / What you will learn

► (MATH 1210 or MATH 1220 or MATH 1300) and (MATH 1232 or MATH 1700 or MATH 1710)

$\Rightarrow$ You **must know and be comfortable** with 1st year linear algebra (3 CH) and 1st year calculus (6 CH)

► We need more: some stuff you would learn in 2090 (Linear Algebra 2), some stuff from 2130, 2150 or 2720 (Multivariable Calculus) and some stuff from 2070 (Graph Theory)

Focus here is not on mathematical “precision”

- We won’t do complicated proofs. I will show some when they are useful in understanding *why* something works

- We will cover just enough of the mentioned math topics that you can understand *how* to do things

- In classic math courses, we work on “small” examples so we can work them by hand and are able to do them in tests

Here, we will do small examples by hand, indeed. But you will do regularly sized examples in computer worksheets

General information about the course

Mathematics of data science?

Setting things up for the course

Introduction to R Markdown

(Super brief) Introduction to programming in R

We will be programming quite a bit

As already indicated, Data Science is a very hands-on discipline. We will be programming quite a bit in this course

Indeed, we can work out some of the examples "by hand", but to make things interesting, we typically need to consider larger examples where hand calculations are not pleasant or not even feasible

R versus Python

Slightly different take on life :)

In short: `Python` is more CS, `R` is more Stats/Math

Both are good languages for data science

In this course, we use `R`

R was originally for stats but is now more

- Open source version of S
- Appeared in 1993
- Now version 4.6
- Uses a lot of C and Fortran code. E.g., `deSolve`:

  The functions provide an interface to the FORTRAN functions 'lsoda', 'lsodar', 'lsode', 'lsodes' of the 'ODEPACK' collection, to the FORTRAN functions 'dvode', 'zvode' and 'daspk' and a C-implementation of solvers of the 'Runge-Kutta' family with fixed or variable time steps

- Very active community on the web, easy to find solutions

Getting your computer ready for the course

All computer coding is in R

$\Longrightarrow$ you need to find a way to run R. Next slide: some methods, from the easiest to the most challenging.

Note that all options described below are Open Source (completely free)

In short...

- Terminal version, not very friendly
- Nicer terminal: radian
- Execute R scripts by using `Rscript name_of_script.R`. Useful to run code in `cron`, for instance
- Use IDEs:
  - RStudio has become the reference
  - RKWard is useful if you are for instance using an ARM processor (Raspberry Pi, some Chromebooks..)
- Integrate into jupyter notebooks

Install R and RStudio

This is probably the best option if you intend to go a little further than what we will do in the course. `R` is available on most platforms, while `RStudio` is available on most platforms except for Linux ARM devices (but can be compiled there)

Visit https://www.r-project.org/

Choose your version: Windows or Mac. Under Linux, you can install directly from your package manager (e.g., `sudo apt install R-base` for Debian-based distros)

To install RStudio, see here

Going further

- RStudio server: run RStudio on a Linux server and connect via a web interface
- Shiny: easily create an interactive web site running R code
- Shiny server: run Shiny apps on a Linux server
- Rmarkdown: markdown that incorporates R commands. Useful for generating reports in html or pdf, can make slides as well..
- Sweave/knitr: LaTeX incorporating R commands. Useful for generating reports. Not used as much as Rmarkdown these days (but used for these slides)

General information about the course

Mathematics of data science?

Setting things up for the course

Introduction to R Markdown

(Super brief) Introduction to programming in R

R Markdown?

- File format for making dynamic documents with R
- Combines code, its results and narrative text in a single document
- Uses simple `Markdown` syntax for text and `R` code chunks for analysis
- From one '.Rmd' file, you can create HTML, PDF, Word documents, presentations, ...

The YAML header

Every R Markdown document starts with a YAML header, enclosed by `---`. This controls the overall properties of the document

Example YAML for a PDF document

```
---
title: "My Report"
author: "My Name"
date: "2025-08-17"
output:
  pdf_document:
    toc: true
    number_sections: true
---
```

Basic Text Formatting

Markdown provides a simple syntax for formatting text

- `# Header` creates a section title, `## Sub-header` creates a subsection title, etc.
- `*italic text*` produces *italic text*
- `**bold text**` produces **bold text**
- `` `code font` `` produces `code font`

Lists

Unordered lists start with * or -

```
* Item 1
* Item 2
```

Ordered lists use numbers

```
1. First Item
2. Second Item
```

(Once the numbered list is “initiated”, the number doesn’t matter, you can write 1., 1., 1., etc., provided you have a new line each time)

Chunks: the core of R Markdown

R code is placed in “chunks”, which start with ```` ```{r chunk-name} ```` and end with ```` ``` ````

````
```{r cars-summary}
# Your R code goes here
summary(cars)
```
````

Chunk names (`cars-summary` here) are not mandatory (you could write ‘‘‘`{r}` but are very useful, in particular when debugging

Chunk output

When the document is "knit," the R code is executed and its output is embedded in the final document

````
```{r cars-summary}
summary(cars)
```
````

```
##      speed           dist
##  Min.   : 4.0   Min.   :  2.00
##  1st Qu.:12.0   1st Qu.: 26.00
##  Median :15.0   Median : 36.00
##  Mean   :15.4   Mean   : 42.98
##  3rd Qu.:19.0   3rd Qu.: 56.00
##  Max.   :25.0   Max.   :120.00
```

Controlling chunks with options

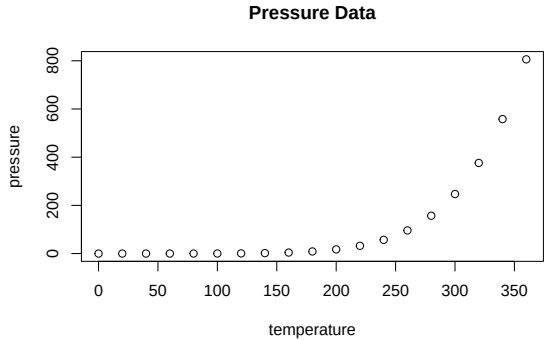
Chunk behavior is controlled by options inside the curly braces

- `echo=FALSE` hides the `R` code, but shows the output
- `eval=FALSE` shows the code, but does not execute it
- `include=FALSE` executes the code, but hides both code and output
- `warning=FALSE`, `message=FALSE` hides warnings or messages
- `fig.width=5`, `fig.height=4` sets figure dimensions

Figure chunks

Plots are automatically embedded

```
```{r pressure-plot, echo=FALSE, fig.cap="Pressure Data"}
plot(pressure)
```
```



Inline R Code

You can embed R code directly into text with backticks: `` `r ...` ``

```
The `cars` dataset has `r nrow(cars)` rows.
```

The cars dataset has 50 rows.

General information about the course

Mathematics of data science?

Setting things up for the course

Introduction to R Markdown

(Super brief) Introduction to programming in R

R is a scripted language

- Interactive
- Allows you to work in real time
  - Be careful: what is in memory might involve steps not written down in a script
  - If you want to reproduce your steps, it is good to write all the steps down in a script and to test from time to time running using `Rscript`: this will ensure that all that is required to run is indeed loaded to memory when it needs to, i.e., that it is not already there..

Assignment

Two ways:

```
X <- 10
```

or

```
X = 10
```

First version is preferred by R purists.. I don't really care

Lists

A very useful data structure, quite flexible and versatile. Empty list: `L <- list()`. Convenient for things like parameters. For instance

```
L <- list()
L$a <- 10
L$b <- 3
L[["another_name"]] <- "Plouf plouf"
```

```r
L[1]
```

```
## $a
## [1] 10
```

```r
L[[2]]
```

```
## [1] 3
```

```r
L$a
```

```
## [1] 10
```

```r
L[["b"]]
```

```
## [1] 3
```

```r
L$another_name
```

```
## [1] "Plouf plouf"
```

Vectors

```
x = 1:10
y <- c(x, 12)
y
z = c("red", "blue")
z
z = c(z, 1)
z
```

Note that in z, since the first two entries are characters, the added entry is also a character. Contrary to lists, vectors have all entries of the same type

Matrices

Matrix (or vector) of zeros

```
A <- mat.or.vec(nr = 2, nc = 3)
A
```

```
##      [,1] [,2] [,3]
## [1,]    0    0    0
## [2,]    0    0    0
```

Matrix are column-first

```
B <- matrix(c(1,2,3,4), nr = 2, nc = 2)
B
```

```
##      [,1] [,2]
## [1,]    1    3
## [2,]    2    4
```

```
C <- matrix(c(1,2,3,4), nr = 2, nc = 2, byrow = TRUE)
C
```

```
##      [,1] [,2]
## [1,]    1    2
## [2,]    3    4
```

Remark that here and elsewhere, naming the arguments (e.g., `nr = 2`) allows to use arguments in any order

Matrix operations

Probably the biggest annoyance in R compared to other languages

- The notation `A*B` is the *Hadamard product* $A \circ B$ (what would be denoted `A.*B` in matlab), not the standard matrix multiplication
- Matrix multiplication is written `A %*% B`

Vector operations

Vector addition is also frustrating. Say you create the vector

```
x = 1:10
x

##  [1]  1  2  3  4  5  6  7  8  9 10
```

Then `x+1` gives

```
x+1

##  [1]  2  3  4  5  6  7  8  9 10 11
```

i.e., adds 1 to all entries in the vector. Beware of this in particular when addressing sets of indices in lists, vectors or matrices

Flow control

```
if (condition_is_true) {
  list of stuff to do
}
```

Even if `list of stuff to do` is a single instruction, best to use curly braces

```
if (condition_is_true) {
  list of stuff to do
} else if (another_condition) {
  ...
} else {
  ...
}
```

For loops

`for` applies to lists or vectors

```
for (i in 1:10) {
  something using integer i
}
for (j in c(1,3,4)) {
  something using integer j
}
for (n in c("truc", "muche", "chose")) {
  something using string n
}
for (m in list("truc", "muche", "chose", 1, 2)) {
  something using string n or integer n, depending
}
```

lapply

Very useful function (a few others in the same spirit: `sapply`, `vapply`, `mapply`) Applies a function to each entry in a list/vector/matrix. Because there is a parallel version (`parLapply`) that we will see later, worth learning

```
l = list()
for (i in 1:10) {
        l[[i]] = runif(i)
}
lapply(X = l, FUN = mean)
```

or, to make a vector

```
unlist(lapply(X = l, FUN = mean))
```

or

```
sapply(X = l, FUN = mean)
```

“Advanced” lapply

Can “pick up” nontrivial list entries

```
l = list()
for (i in 1:10) {
        l[[i]] = list()
        l[[i]]$a = runif(i)
        l[[i]]$b = runif(2*i)
}
sapply(X = l, FUN = function(x) length(x$b))
```

gives

```
# [1]  2  4  6  8 10 12 14 16 18 20
```

Just recall: the argument to the function you define is a list entry (`l[[1]]`, `l[[2]]`, etc., here)

Avoid parameter variation loops with expand.grid

```
# Suppose we want to vary 3 parameters
variations = list(
    p1 = seq(1, 10, length.out = 10),
    p2 = seq(0, 1, length.out = 10),
    p3 = seq(-1, 1, length.out = 10)
)

# Create the list
tmp = expand.grid(variations)
PARAMS = list()
for (i in 1:dim(tmp)[1]) {
    PARAMS[[i]] = list()
    for (k in 1:length(variations)) {
        PARAMS[[i]][[names(variations)[k]]] = tmp[i, k]
    }
}
```

There is still a loop, but you can split this list, use it on different machines, etc. And can use `parLapply`

Can I have this wrapped up to go?

For each slide set including `R` code, we generate an `R` file in the `CODE` directory with all the code chunks in this `Rnw` file and no LaTeX

Source the file (if you are reading `SLIDES/filename.pdf`, then you want `CODE/filename.R`) to reproduce all results in the slide set without generating a pdf

Some small changes might be required (for instance, in the file for this lecture, quite a lot is commented out)

Math prelims – Linear algebra

MATH 2740 – Mathematics of Data Science – Unit U02

Julien Arino
julien.arino@umanitoba.ca

Department of Mathematics @ University of Manitoba

Fall 2026

The University of Manitoba campuses are located on original lands of Anishinaabeg, Ininew, Anisininew, Dakota and Dene peoples, and on the National Homeland of the Red River Métis. We respect the Treaties that were made on these territories, we acknowledge the harms and mistakes of the past, and we dedicate ourselves to move forward in partnership with Indigenous communities in a spirit of Reconciliation and collaboration.

In MATH 2740, we rely on notions you acquired in MATH 1210/1220/1300. We also use some material from first-year calculus

So let us (briefly) go over material in these courses

I also add (for some of you) a few things that will be handy and establish some terminology that we use throughout the course

Outline

Sets and logic

Complex numbers

Vectors and vector spaces

Linear systems and matrices

A crash course in multivariable calculus

Sets and logic

Complex numbers

Vectors and vector spaces

Linear systems and matrices

A crash course in multivariable calculus

Sets and elements

Definition 1 (Set)

A **set** X is a collection of **elements**

We write $x \in X$ or $x \notin X$ to indicate that the element x belongs to the set X or does not belong to the set X, respectively

Definition 2 (Subset)

Let X be a set. The set S is a **subset** of X, which is denoted $S \subset X$, if all its elements belong to X

Not used here but worth noting: we say S is a **proper subset** of X and write $S \subsetneq X$, if it is a subset of X and not equal to X

Quantifiers

A shorthand notation for “for all elements x belonging to X” is $\forall x \in X$

For example, if $X = \mathbb{R}$, the *field* of real numbers, then $\forall x \in \mathbb{R}$ means “for all real numbers x”

A shorthand notation for “there exists an element x in the set X” is $\exists x \in X$

$\forall$ and $\exists$ are **quantifiers**

Intersection and union of sets

Let X and Y be two sets

Definition 3 (Intersection)

The intersection of X and Y, $X \cap Y$, is the set of elements that belong to X ***and*** *to Y,*

$$X \cap Y = \{x : x \in X \textbf{ and } x \in Y\}$$

Definition 4 (Union)

The union of X and Y, $X \cup Y$, is the set of elements that belong to X ***or*** *to Y,*

$$X \cup Y = \{x : x \in X \textbf{ or } x \in Y\}$$

In mathematics, or=and/or in common parlance. We also have an **exclusive or** (xor)

A teeny bit of logic

In a logical sense, a **proposition** is an assertion (or statement) whose truth value (true or false) can be asserted. For example, a theorem is a proposition that has been shown to be true. “The sky is blue” is also a proposition

Let A be a proposition. We generally write

$$A$$

to mean that A is true, and

$$\textbf{not } A$$

to mean that A is false. **not** A is the **negation** of A (or **not** A is the negative of A)

A teeny bit of logic (cont.)

Let A, B be propositions. Then

- $A \Rightarrow B$ (read A implies B) means that whenever A is true, then so is B
- $A \Leftrightarrow B$, also denoted A if and only if B (A iff B for short), means that $A \Rightarrow B$ **and** $B \Rightarrow A$
  We also say that A and B are **equivalent**

Let A and B be propositions. Then

$$(A \Rightarrow B) \Leftrightarrow (\textbf{not } B \Rightarrow \textbf{not } A)$$

Necessary or sufficient conditions

Suppose we want to establish whether a given statement P is true, depending on the truth value of a statement H. Then we say that

- H is a **necessary condition** if $P \Rightarrow H$
  (It is necessary that H be true for P to be true; so whenever P is true, so is H)

- H is a **sufficient condition** if $H \Rightarrow P$
  (It suffices for H to be true for P to also be true)

- H is a **necessary and sufficient condition** if $H \Leftrightarrow P$, i.e., H and P are equivalent

Playing with quantifiers

For the quantifiers $\forall$ (for all) and $\exists$ (there exists),

$\exists$ is the negation of $\forall$

Therefore, for example, the negation of

$$\forall x \in X, \exists y \in Y$$

is

$$\exists x \in X, \forall y \in Y$$

Sets and logic

Complex numbers

Vectors and vector spaces

Linear systems and matrices

A crash course in multivariable calculus

Complex numbers

Definition 5 (Complex numbers)

*A **complex number** is an ordered pair (a, b), where $a, b \in \mathbb{R}$. Usually written $a + ib$ or $a + bi$, where $i^2 = -1$ (i.e., $i = \sqrt{-1}$)*
The set of all complex numbers is denoted $\mathbb{C}$,

$$\mathbb{C} = \{a + ib : a, b \in \mathbb{R}\}$$

Definition 6 (Addition and multiplication on $\mathbb{C}$)

Letting $a + ib$ and $c + id \in \mathbb{C}$, addition on $\mathbb{C}$ is defined by

$$(a + ib) + (c + id) = (a + c) + i(b + d)$$

and multiplication on $\mathbb{C}$ is defined by

$$(a + ib)(c + id) = (ac - bd) + i(ad + bc)$$

Latter is easy to obtain using regular multiplication and $i^2 = -1$

Properties

$\forall \alpha, \beta, \gamma \in \mathbb{C}$,

$\alpha + \beta = \beta + \alpha$ and $\alpha\beta = \beta\alpha$ **[commutativity]**

$(\alpha + \beta) + \gamma = \alpha + (\beta + \gamma)$ and $(\alpha\beta)\gamma = \alpha(\beta\gamma)$ **[associativity]**

$\gamma + 0 = \gamma$ and $\gamma 1 = \gamma$ **[identities]**

$\forall \alpha \in \mathbb{C}$, $\exists \beta \in \mathbb{C}$ unique s.t. $\alpha + \beta = 0$ **[additive inverse]**

$\forall \alpha \neq 0 \in \mathbb{C}$, $\exists \beta \in \mathbb{C}$ unique s.t. $\alpha\beta = 1$ **[multiplicative inverse]**

$\gamma(\alpha + \beta) = \gamma\alpha + \gamma\beta$ **[distributivity]**

Additive & multiplicative inverse, subtraction, division

Definition 7

Let $\alpha, \beta \in \mathbb{C}$

- $-\alpha$ is the **additive inverse** of α, i.e., the unique number in $\mathbb{C}$ s.t. $\alpha + (-\alpha) = 0$
- **Subtraction** on $\mathbb{C}$:
$$\beta - \alpha = \beta + (-\alpha)$$
- For $\alpha \neq 0$, $1/\alpha$ is the **multiplicative inverse** of α, i.e., the unique number in $\mathbb{C}$ s.t.
$$\alpha(1/\alpha) = 1$$
- **Division** on $\mathbb{C}$:
$$\beta/\alpha = \beta(1/\alpha)$$

Definition 8 (Real and imaginary parts)

*Let $z = a + ib$. Then $Re\, z = a$ is **real part** and $Im\, z = b$ is **imaginary part** of z*

If ambiguous, write $\mathrm{Re}\,(z)$ and $\mathrm{Im}\,(z)$

Definition 9 (Conjugate and Modulus)

Let $z = a + ib \in \mathbb{C}$. Then

- ***Complex conjugate** of z is*

$$\bar{z} = a - ib$$

- ***Modulus** (or **absolute value**) of z is*

$$|z| = \sqrt{a^2 + b^2} \geq 0$$

Properties of complex numbers

Let $w, z \in \mathbb{C}$, then

- $z + \bar{z} = 2\mathrm{Re}\, z$
- $z - \bar{z} = 2i\mathrm{Im}\, z$
- $z\bar{z} = |z|^2$
- $\overline{w+z} = \bar{w} + \bar{z}$ and $\overline{wz} = \bar{w}\bar{z}$
- $\bar{\bar{z}} = z$
- $|\mathrm{Re}\, z| \leq |z|$ and $|\mathrm{Im}\, z| \leq |z|$
- $|\bar{z}| = |z|$
- $|wz| = |w|\,|z|$
- $|w+z| \leq |w| + |z|$ **[triangle inequality]**

Solving quadratic equations

Consider the polynomial

$$P(x) = a_0 + a_1 x + a_2 x^2$$

where $x, a_0, a_1, a_2 \in \mathbb{R}$. Letting

$$\Delta = a_1^2 - 4 a_0 a_2$$

you know that if $\Delta > 0$, then

$$P(x) = 0$$

has two distinct *real* solutions,

$$x_1 = \frac{-a_1 - \sqrt{\Delta}}{2a_2} \quad \text{and} \quad x_2 = \frac{-a_1 + \sqrt{\Delta}}{2a_2}$$

if $\Delta = 0$, then there is a (multiplicity 2) unique *real* solution

$$x_1 = \frac{-a_1}{2a_2}$$

while if $\Delta < 0$, there is no solution

Solving quadratic equations with complex numbers

Consider the polynomial

$$P(x) = a_0 + a_1 x + a_2 x^2$$

where $x, a_0, a_1, a_2 \in \mathbb{R}$. If instead of seeking $x \in \mathbb{R}$, we seek $x \in \mathbb{C}$, then the situation is the same, except when $\Delta < 0$

In the latter case, note that

$$\sqrt{\Delta} = \sqrt{(-1)(-\Delta)} = \sqrt{-1}\sqrt{-\Delta} = i\sqrt{-\Delta}$$

Since $\Delta < 0$, $-\Delta > 0$ and the square root is the usual one

Solving quadratic equations with complex numbers

To summarize, consider the polynomial

$$P(x) = a_0 + a_1 x + a_2 x^2$$

where $x, a_0, a_1, a_2 \in \mathbb{R}$. Letting

$$\Delta = a_1^2 - 4a_0 a_2$$

Then

$$P(x) = 0$$

has two solutions,

$$x_{1,2} = \frac{-a_1 \pm \sqrt{\Delta}}{2a_2}$$

where, if $\Delta < 0$, $x_1, x_2 \in \mathbb{C}$ and take the form

$$x_{1,2} = \frac{-a_1 \pm i\sqrt{-\Delta}}{2a_2}$$

Why this matters

Recall (we will come back to this later) that to find the *eigenvalues* of the matrix

$$A = \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix}$$

we seek λ solutions to $\det(A - \lambda\mathbb{I}) = 0$, i.e., λ solutions to

$$|A - \lambda\mathbb{I}| = \begin{vmatrix} a_{11} - \lambda & a_{12} \\ a_{21} & a_{22} - \lambda \end{vmatrix} = (a_{11} - \lambda)(a_{22} - \lambda) - a_{12}a_{21} = 0$$

i.e., λ solutions to

$$\lambda^2 - (a_{11} + a_{22})\lambda + a_{11}a_{22} - a_{12}a_{21} = 0$$

Why this matters (cont.)

Let

$$P(\lambda) = \lambda^2 - (a_{11} + a_{22})\lambda + a_{11}a_{22} - a_{12}a_{21}$$

From previous discussion, letting

$$\begin{aligned} \Delta &= (a_{11} + a_{22})^2 - 4(a_{11}a_{22} - a_{12}a_{21}) \\ &= a_{11}^2 + a_{22}^2 + 2a_{11}a_{22} - 4a_{11}a_{22} + 4a_{12}a_{21} \\ &= a_{11}^2 + a_{22}^2 - 2a_{11}a_{22} + 4a_{12}a_{21} \\ &= (a_{11} - a_{22})^2 + 4a_{12}a_{21} \end{aligned}$$

we have two (potentially equal) solutions to $P(\lambda) = 0$

$$x_{1,2} = \frac{a_{11} + a_{22} \pm \sqrt{\Delta}}{2}$$

that are complex if $\Delta < 0$

Example: $\begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}$

Sets and logic

Complex numbers

Vectors and vector spaces

Linear systems and matrices

A crash course in multivariable calculus

Vectors

A **vector** $\boldsymbol{v}$ is an ordered n-tuple of real or complex numbers

Denote $\mathbb{F} = \mathbb{R}$ or $\mathbb{C}$ (real or complex numbers). For $v_1, \ldots, v_n \in \mathbb{F}$,

$$\boldsymbol{v} = (v_1, \ldots, v_n) \in \mathbb{F}^n$$

is a vector. $v_1, \ldots, v_n$ are the **components** of $\boldsymbol{v}$

If unambiguous, we write v. Otherwise, $\boldsymbol{v}$ or $\vec{v}$

Vector space

Definition 10 (Vector space)

A **vector space** over $\mathbb{F}$ is a set V together with two binary operations, **vector addition**, denoted $+$, and **scalar multiplication**, that satisfy the relations:

1. $\forall \boldsymbol{u}, \boldsymbol{v}, \boldsymbol{w} \in V$, $\boldsymbol{u} + (\boldsymbol{v} + \boldsymbol{w}) = (\boldsymbol{u} + \boldsymbol{v}) + \boldsymbol{w}$
2. $\forall \boldsymbol{v}, \boldsymbol{w} \in V$, $\boldsymbol{v} + \boldsymbol{w} = \boldsymbol{w} + \boldsymbol{v}$
3. $\exists \mathbf{0} \in V$, the zero vector, such that $\boldsymbol{v} + \mathbf{0} = \boldsymbol{v}$ for all $\boldsymbol{v} \in V$
4. $\forall \boldsymbol{v} \in V$, there exists an element $\boldsymbol{w} \in V$, the additive inverse of $\boldsymbol{v}$, such that $\boldsymbol{v} + \boldsymbol{w} = \mathbf{0}$
5. $\forall \alpha \in \mathbb{R}$ and $\forall \boldsymbol{v}, \boldsymbol{w} \in V$, $\alpha(\boldsymbol{v} + \boldsymbol{w}) = \alpha\boldsymbol{v} + \alpha\boldsymbol{w}$
6. $\forall \alpha, \beta \in \mathbb{R}$ and $\forall \boldsymbol{v} \in V$, $(\alpha + \beta)\boldsymbol{v} = \alpha\boldsymbol{v} + \beta\boldsymbol{v}$
7. $\forall \alpha, \beta \in \mathbb{R}$ and $\forall \boldsymbol{v} \in V$, $\alpha(\beta\boldsymbol{v}) = (\alpha\beta)\boldsymbol{v}$
8. $\forall \boldsymbol{v} \in V$, $1\boldsymbol{v} = \boldsymbol{v}$

Subspaces

Definition 11 (Subspace)

Let V be a vector space over $\mathbb{F}$. A non-empty subset W of V is a **subspace** of V if W is itself a vector space under the same operations of addition and scalar multiplication

Definition 12 (Subspace – Practical definition)

Let V be a vector space over $\mathbb{F}$. A non-empty subset W of V is a subspace if

1. *for all $\boldsymbol{u}, \boldsymbol{v} \in W$, $\boldsymbol{u} + \boldsymbol{v} \in W$ (W is closed under vector addition)*
2. *for all $\alpha \in \mathbb{F}$ and $\boldsymbol{v} \in W$, $\alpha \boldsymbol{v} \in W$ (W is closed under scalar multiplication)*

Norms

Definition 13 (Norm)

Let V be a vector space over $\mathbb{F}$, and $\boldsymbol{v} \in V$ be a vector. The ***norm*** *of $\boldsymbol{v}$, denoted $\|\boldsymbol{v}\|$, is a function from V to $\mathbb{R}_+$ that has the following properties:*

1. *For all $\boldsymbol{v} \in V$, $\|\boldsymbol{v}\| \geq 0$ with $\|\boldsymbol{v}\| = 0$ iff $\boldsymbol{v} = \boldsymbol{0}$*
2. *For all $\alpha \in \mathbb{F}$ and all $\boldsymbol{v} \in V$, $\|\alpha\boldsymbol{v}\| = |\alpha| \, \|\boldsymbol{v}\|$*
3. *For all $\boldsymbol{u}, \boldsymbol{v} \in V$, $\|\boldsymbol{u} + \boldsymbol{v}\| \leq \|\boldsymbol{u}\| + \|\boldsymbol{v}\|$*

Let V be a vector space (for example, $\mathbb{R}^2$ or $\mathbb{R}^3$)

The **zero element** (or **zero vector**) is the vector $\mathbf{0} = (0, \ldots, 0)$

The **additive inverse** of $\boldsymbol{v} = (v_1, \ldots, v_n)$ is $-\boldsymbol{v} = (-v_1, \ldots, -v_n)$

For $\boldsymbol{v} = (v_1, \ldots, v_n) \in V$, the length (or Euclidean norm) of $\boldsymbol{v}$ is the **scalar**

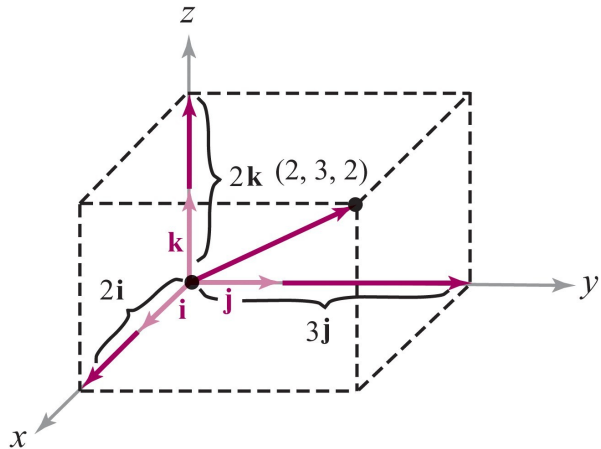
$$\|\boldsymbol{v}\| = \sqrt{v_1^2 + \cdots + v_n^2}$$

To **normalize** the vector $\boldsymbol{v}$ consists in considering $\tilde{\boldsymbol{v}} = \boldsymbol{v}/\|\boldsymbol{v}\|$, i.e., the vector in the same direction as $\boldsymbol{v}$ that has unit length

Standard basis vectors

Vectors $\boldsymbol{i} = (1,0,0)$, $\boldsymbol{j} = (0,1,0)$ and $\boldsymbol{k} = (0,0,1)$ are the **standard basis vectors** of $\mathbb{R}^3$. A vector $\boldsymbol{v} = (v_1, v_2, v_3)$ can then be written

$$\boldsymbol{v} = v_1\boldsymbol{i} + v_2\boldsymbol{j} + v_3\boldsymbol{k}$$



For V ($\mathbb{R}^n$), the standard basis vectors are usually denoted $\boldsymbol{e}_1, \ldots, \boldsymbol{e}_n$, with

$$\boldsymbol{e}_k = (\underbrace{0, \ldots, 0}_{k-1}, 1, \underbrace{0, \ldots, 0}_{n-k+1})$$

Dot product

Definition 14 (Dot product)

Let $\boldsymbol{a} = (a_1, \ldots, a_n) \in \mathbb{R}^n$, $\boldsymbol{b} = (b_1, \ldots, b_n) \in \mathbb{R}^n$. *The* ***dot product*** *of* $\boldsymbol{a}$ *and* $\boldsymbol{b}$ *is the* ***scalar***

$$\boldsymbol{a} \bullet \boldsymbol{b} = \sum_{i=1}^{n} a_i b_i = a_1 b_1 + \cdots + a_n b_n$$

The dot product is a special case of **inner product**

Properties of the dot product

Theorem 15

For $\boldsymbol{a}, \boldsymbol{b}, \boldsymbol{c} \in \mathbb{R}^n$ *and* $\alpha \in \mathbb{R}$,

- $\boldsymbol{a} \bullet \boldsymbol{a} = \|\boldsymbol{a}\|^2$ *(so* $\boldsymbol{a} \bullet \boldsymbol{a} \geq 0$*, with* $\boldsymbol{a} \bullet \boldsymbol{a} = 0$ *iff* $\boldsymbol{a} = \boldsymbol{0}$*)*
- $\boldsymbol{a} \bullet \boldsymbol{b} = \boldsymbol{b} \bullet \boldsymbol{a}$ *(*$\bullet$ *is commutative)*
- $\boldsymbol{a} \bullet (\boldsymbol{b} + \boldsymbol{c}) = \boldsymbol{a} \bullet \boldsymbol{b} + \boldsymbol{a} \bullet \boldsymbol{c}$ *(*$\bullet$ *distributive over* $+$*)*
- $(\alpha \boldsymbol{a}) \bullet \boldsymbol{b} = \alpha(\boldsymbol{a} \bullet \boldsymbol{b}) = \boldsymbol{a} \bullet (\alpha \boldsymbol{b})$
- $\boldsymbol{0} \bullet \boldsymbol{a} = 0$

Some results stemming from the dot product

Theorem 16

*If θ is the angle between the vectors **a** and **b**, then*

$$\boldsymbol{a} \bullet \boldsymbol{b} = \|\boldsymbol{a}\| \, \|\boldsymbol{b}\| \, \cos\theta$$

Corollary 17 (Cauchy-Schwarz inequality)

*For any two vectors **a** and **b**, we have*

$$|\boldsymbol{a} \bullet \boldsymbol{b}| \leq \|\boldsymbol{a}\| \, \|\boldsymbol{b}\|$$

*with equality if and only if **a** is a scalar multiple of **b**, or one of them is **0**.*

Theorem 18

***a** and **b** are orthogonal if and only if $\boldsymbol{a} \bullet \boldsymbol{b} = 0$.*

Scalar and vector projections

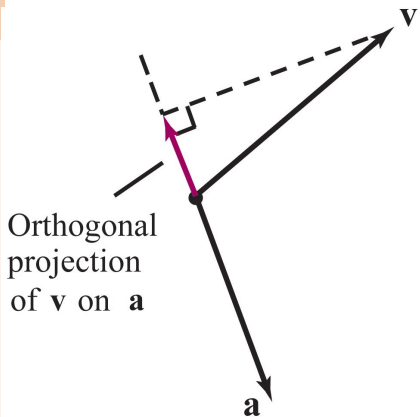
Definition 19

Let $\boldsymbol{a}$, $\boldsymbol{v}$ *be vectors. The* **scalar projection** *of* $\boldsymbol{v}$ *onto* $\boldsymbol{a}$ *(or* **component** *of* $\boldsymbol{v}$ *along* $\boldsymbol{a}$*) is*

$$\text{comp}_{\boldsymbol{a}}\boldsymbol{v} = \frac{\boldsymbol{a} \bullet \boldsymbol{v}}{\|\boldsymbol{a}\|}$$

The **vector projection** *(or* **orthogonal projection***) of* $\boldsymbol{v}$ *onto* $\boldsymbol{a}$ *is*

$$\text{proj}_{\boldsymbol{a}}\boldsymbol{v} = \left(\frac{\boldsymbol{a} \bullet \boldsymbol{v}}{\|\boldsymbol{a}\|}\right)\frac{\boldsymbol{a}}{\|\boldsymbol{a}\|} = \frac{\boldsymbol{a} \bullet \boldsymbol{v}}{\|\boldsymbol{a}\|^2}\boldsymbol{a}$$



Sets and logic

Complex numbers

Vectors and vector spaces

Linear systems and matrices

A crash course in multivariable calculus

Linear systems and matrices

Linear systems and matrices

Matrix arithmetic

Eigenpairs

Similarity and diagonalisation

Linear independence/Bases/Dimension

Linear systems

Definition 20 (Linear system)

A **linear system** of n equations in p unknowns takes the form

$$\begin{array}{ccccccccc} a_{11}x_1 & + & a_{12}x_2 & + & \cdots & + & a_{1p}x_p & = & b_1 \\ a_{21}x_1 & + & a_{22}x_2 & + & \cdots & + & a_{2p}x_p & = & b_2 \\ \vdots & & \vdots & & \vdots & & \vdots & & \vdots \\ a_{n1}x_1 & + & a_{n2}x_2 & + & \cdots & + & a_{np}x_p & = & b_n \end{array} \tag{1}$$

The a_{ij}, x_j and b_i could be in $\mathbb{R}$ or $\mathbb{C}$, although here we typically assume they are in $\mathbb{R}$

The aim is to find $x_1, x_2, \ldots, x_p$ that satisfy all equations simultaneously

Theorem 21 (Nature of solutions to a linear system)

A linear system can have

- *no solution*
- *a unique solution*
- *infinitely many solutions*

Operations on linear systems

You learned to manipulate linear systems using

- Gaussian elimination
- Gauss-Jordan elimination

with the aim to put the system in **row echelon form** (REF) or **reduced row echelon form** (RREF)

Matrices and linear systems

Writing

$$A = \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1p} \\ a_{21} & a_{22} & \cdots & a_{2p} \\ \vdots & \vdots & & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{np} \end{pmatrix}, \quad \boldsymbol{x} = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_p \end{pmatrix} \quad \text{and} \quad \boldsymbol{b} = \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{pmatrix}$$

where A is an $n \times p$ **matrix**, $\boldsymbol{x}$ is a p (column) **vector** (or $p \times 1$ matrix) and $\boldsymbol{b}$ is an n (column) **vector** (or $n \times 1$ matrix), then the linear system in the previous slide takes the form

$$A\boldsymbol{x} = \boldsymbol{b}$$

Notation for vectors

We usually assume vectors are column vectors and thus write, e.g.,

$$\boldsymbol{x} = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} = (x_1, x_2, \ldots, x_n)^T$$

Here, ^{T} is the **transpose operator** (more on this soon)

Consider the system

$$A\boldsymbol{x} = \boldsymbol{b}$$

If $\boldsymbol{b} = \mathbf{0}$, the system is **homogeneous** and always has the solution $\boldsymbol{x} = 0$ and so the “no solution” option in Theorem 21 goes away

Linear systems and matrices

Linear systems and matrices
Matrix arithmetic
Eigenpairs
Similarity and diagonalisation
Linear independence/Bases/Dimension

Definition 22 (Matrix)

An n-by-p or $n \times p$ matrix is a rectangular array of elements of $\mathbb{R}$ or $\mathbb{C}$ with n rows and p columns,

$$A = [a_{ij}] = \begin{pmatrix} a_{11} & \cdots & a_{1p} \\ \vdots & & \vdots \\ a_{n1} & \cdots & a_{np} \end{pmatrix}$$

We always list indices as “row,column”

We denote $\mathcal{M}_{np}(\mathbb{F})$ or $\mathbb{F}^{np}$ the set of $n \times p$ matrices with entries in $\mathbb{F} = \{\mathbb{R}, \mathbb{C}\}$. Often, we omit $\mathbb{F}$ in $\mathcal{M}_{np}$ if the nature of $\mathbb{F}$ is not important

When $n = p$, we usually write $\mathcal{M}_n$

Basic matrix arithmetic

Let $A \in \mathcal{M}_{np}, B \in \mathcal{M}_{np}$ be matrices (of the same size) and $c \in \mathbb{F} = \{\mathbb{R}, \mathbb{C}\}$ be a scalar

- **Scalar multiplication**
$$cA = [ca_{ij}]$$
- **Addition**
$$A + B = [a_{ij} + b_{ij}]$$
- **Subtraction** (addition of $-B = (-1)B$ to A)
$$A - B = A + (-1)B = [a_{ij} + (-1)b_{ij}] = [a_{ij} - b_{ij}]$$
- **Transposition** of A gives a matrix $A^T \in \mathcal{M}_{pn}$ with
$$A^T = [a_{ji}], \quad j = 1, \ldots, p, \quad i = 1, \ldots, n$$

Matrix multiplication

The (matrix) **product** of A and B, AB, requires the "inner dimensions" to match, i.e., the number of columns in A must equal the number of rows in B

Suppose that is the case, i.e., let $A \in \mathcal{M}_{np}$, $B \in \mathcal{M}_{pq}$. Then the i, j entry in $C := AB \in \mathcal{M}_{nq}$ takes the form

$$c_{ij} = \sum_{k=1}^{p} a_{ik} b_{kj}$$

Recall that the matrix product is not commutative, i.e., in general, $AB \neq BA$ (when both those products are defined, i.e., when $A, B \in \mathcal{M}_n$)

Special matrices

Definition 23 (Zero and identity matrices)

*The **zero** matrix is the matrix 0_{np} whose entries are all zero. The **identity** matrix is a square $n \times n$ matrix $\mathbb{I}_n$ with all entries on the main diagonal equal to one and all off diagonal entries equal to zero*

Definition 24 (Symmetric matrix)

*A square matrix $A \in \mathcal{M}_n$ is **symmetric** if $\forall i, j = 1, \ldots, n$, $a_{ij} = a_{ji}$. In other words, $A \in \mathcal{M}_n$ is symmetric if $A = A^T$*

Making symmetric matrices

Theorem 25

1. *If $A \in \mathcal{M}_n$, then $A + A^T$ is symmetric*
2. *If $A \in \mathcal{M}_{np}$, then $AA^T \in \mathcal{M}_n$ and $A^TA \in \mathcal{M}_p$ are symmetric*

Proof of Theorem 25

X symmetric $\iff$ $X = X^T$, so use $X =$ the matrix whose symmetric property you want to check

1. True if $A + A^T = (A + A^T)^T$. We have

$$(A + A^T)^T = A^T + (A^T)^T = A^T + A = A + A^T$$

2. AA^T symmetric if $AA^T = (AA^T)^T$. We have

$$(AA^T)^T = (A^T)^T A^T = AA^T$$

$A^T A$ works similarly

Determinants

Definition 26 (Determinant)

Let $A \in \mathcal{M}_n$ with $n \geq 2$. The **determinant** of A is the *scalar*

$$\det(A) = |A| = \sum_{j=1}^{n} a_{ij} C_{ij}$$

where $C_{ij} = (-1)^{i+j} \det(A_{ij})$ is the (i, j)-**cofactor** of A and A_{ij} is the submatrix of A from which the ith row and jth column have been removed

This is a cofactor expansion along the ith row
This is a recursive formula: it gives result in terms of n $\mathcal{M}_{n-1}$ matrices, to which it must in turn be applied, all the way down to

$$\det \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix} = a_{11} a_{22} - a_{12} a_{21}$$

Two special matrices and their determinants

Definition 27

*$A \in \mathcal{M}_n$ is **upper triangular** if $a_{ij} = 0$ when $i > j$, **lower triangular** if $a_{ij} = 0$ when $j > i$, **triangular** if it is either upper or lower triangular and **diagonal** if it is both upper and lower triangular*

When A diagonal, we often write $A = \text{diag}(a_{11}, a_{22}, \ldots, a_{nn})$

Theorem 28

Let $A \in \mathcal{M}_n$ be triangular or diagonal. Then

$$\det(A) = \prod_{i=1}^{n} a_{ii} = a_{11}a_{22}\cdots a_{nn}$$

Inversion/Singularity

Definition 29 (Matrix inverse)

*$A \in \mathcal{M}_n$ is **invertible** (or **nonsingular**) if $\exists A^{-1} \in \mathcal{M}_n$ s.t.*

$$AA^{-1} = A^{-1}A = \mathbb{I}$$

*A^{-1} is the **inverse** of A. If A^{-1} does not exist, A is **singular***

If A is invertible, its inverse A^{-1} is unique

Linear algebra in a nutshell

Theorem 30

Let $A \in \mathcal{M}_n$. The following statements are equivalent (TFAE):

1. *The matrix A is invertible (or nonsingular)*
2. *$\forall \boldsymbol{b} \in \mathbb{F}^n$, $A\boldsymbol{x} = \boldsymbol{b}$ has a unique solution ($\boldsymbol{x} = A^{-1}\boldsymbol{b}$)*
3. *The only solution to $A\boldsymbol{x} = \mathbf{0}$ is the trivial solution $\boldsymbol{x} = \mathbf{0}$*
4. *$RREF(A) = \mathbb{I}_n$*
5. *The matrix A is equal to a product of elementary matrices*
6. *$\forall \boldsymbol{b} \in \mathbb{F}^n$, $A\boldsymbol{x} = \boldsymbol{b}$ has a solution*
7. *There is a matrix $B \in \mathcal{M}_n$ such that $AB = \mathbb{I}_n$*
8. *There is an invertible matrix $B \in \mathcal{M}_n$ such that $AB = \mathbb{I}_n$*
9. $\det(A) \neq 0$
10. *0 is not an eigenvalue of A*

Revisiting matrix arithmetic

With addition, subtraction, scalar multiplication, multiplication, transposition and inversion, you can perform arithmetic on matrices essentially as on scalar, if you bear in mind a few rules

- The sizes have to be compatible
- The order is important since matrix multiplication is not commutative
- Transposition and inversion change the order of products:

$$(AB)^T = B^T A^T \text{ and } (AB)^{-1} = B^{-1} A^{-1}$$

Linear systems and matrices

- Linear systems and matrices
- Matrix arithmetic
- **Eigenpairs**
- Similarity and diagonalisation
- Linear independence/Bases/Dimension

Eigenvalues / Eigenvectors / Eigenpairs

Definition 31

*Let $A \in \mathcal{M}_n$. A vector $\boldsymbol{x} \in \mathbb{F}^n$ such that $\boldsymbol{x} \neq \mathbf{0}$ is an **eigenvector** of A if $\exists \lambda \in \mathbb{F}$ called an **eigenvalue**, s.t.*

$$A\boldsymbol{x} = \lambda \boldsymbol{x}$$

*A couple $(\lambda, \boldsymbol{x})$ with $\boldsymbol{x} \neq \mathbf{0}$ s.t. $A\boldsymbol{x} = \lambda \boldsymbol{x}$ is an **eigenpair***

Parallel eigenvectors

Theorem 32

If $(\lambda, \boldsymbol{x})$ *eigenpair, then for* $c \neq 0$, $(\lambda, c\boldsymbol{x})$ *also eigenpair*

Proof.

Assume $(\lambda, \boldsymbol{x})$ is an eigenpair of $A \in \mathcal{M}_n$, i.e., $\boldsymbol{x} \neq \boldsymbol{0}$ and $A\boldsymbol{x} = \lambda\boldsymbol{x}$

Take the nonzero scalar $c \in \mathbb{F}$. Then $A(c\boldsymbol{x}) = cA\boldsymbol{x}$ because scalars move freely in vector/matrix products. Therefore $A(c\boldsymbol{x}) = cA\boldsymbol{x} = c\lambda\boldsymbol{x}$ since $A\boldsymbol{x} = \lambda\boldsymbol{x}$

So finally, moving c again, $A(c\boldsymbol{x}) = \lambda(c\boldsymbol{x})$, i.e., $c\boldsymbol{x}$ is an eigenvector of A associated to λ □

Linear systems and matrices

Linear systems and matrices
Matrix arithmetic
Eigenpairs
Similarity and diagonalisation
Linear independence/Bases/Dimension

Similarity

Definition 33 (Similarity)

*$A, B \in \mathcal{M}_n$ are **similar** ($A \sim B$) if $\exists P \in \mathcal{M}_n$ invertible s.t.*

$$P^{-1}AP = B$$

Theorem 34 ($\sim$ is an equivalence relation)

$A, B, C \in \mathcal{M}_n$, then

- $A \sim A$ *($\sim$ **reflexive**)*
- $A \sim B \implies B \sim A$ *($\sim$ **symmetric**)*
- $A \sim B$ *and* $B \sim C \implies A \sim C$ *($\sim$ **transitive**)*

Similarity (cont.)

Theorem 35

$A, B \in \mathcal{M}_n$ with $A \sim B$. Then

- $\det A = \det B$
- *A invertible* $\iff$ *B invertible*
- *A and B have the same eigenvalues*

Diagonalisation

Definition 36 (Diagonalisability)

*$A \in \mathcal{M}_n$ is **diagonalisable** if $\exists D \in \mathcal{M}_n$ diagonal s.t. $A \sim D$*

In other words, $A \in \mathcal{M}_n$ is diagonalisable if there exists a diagonal matrix $D \in \mathcal{M}_n$ and a nonsingular matrix $P \in \mathcal{M}_n$ s.t. $P^{-1}AP = D$

Could of course write $PAP^{-1} = D$ since P invertible, but $P^{-1}AP$ makes more sense for computations

Theorem 37

$A \in \mathcal{M}_n$ diagonalisable $\iff$ A has n linearly independent eigenvectors

Corollary 38 (Sufficient condition for diagonalisability)

$A \in \mathcal{M}_n$ has all its eigenvalues distinct $\implies$ A diagonalisable

For $P^{-1}AP = D$: in P, put the linearly independent eigenvectors as columns and in D, the corresponding eigenvalues

Linear systems and matrices

- Linear systems and matrices
- Matrix arithmetic
- Eigenpairs
- Similarity and diagonalisation
- **Linear independence/Bases/Dimension**

Linear combination and span

Definition 39 (Linear combination)

Let V be a vector space. A ***linear combination*** *of a set $\{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_k\}$ of vectors in V is a vector*

$$c_1 \boldsymbol{v}_1 + \cdots + c_k \boldsymbol{v}_k$$

where $c_1, \ldots, c_k \in \mathbb{F}$

Definition 40 (Span)

The set of all linear combinations of a set of vectors $\boldsymbol{v}_1, \ldots, \boldsymbol{v}_k$ is the ***span*** *of $\{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_k\}$,*

$$\operatorname{span}(\boldsymbol{v}_1, \ldots, \boldsymbol{v}_k) = \{c_1 \boldsymbol{v}_1 + \cdots + c_k \boldsymbol{v}_k : c_1, \ldots, c_k \in \mathbb{F}\}$$

Finite/infinite-dimensional vector spaces

Theorem 41

The span of a set of vectors in V is the smallest subspace of V containing all the vectors in the set

Definition 42 (Set of vectors spanning a space)

If $\text{span}(\boldsymbol{v}_1, \ldots, \boldsymbol{v}_k) = V$, we say $\boldsymbol{v}_1, \ldots, \boldsymbol{v}_k$ **spans** V

Definition 43 (Dimension of a vector space)

A vector space V is **finite-dimensional** if some set of vectors in it spans V. A vector space V is **infinite-dimensional** if it is not finite-dimensional

Linear (in)dependence

Definition 44 (Linear independence/Linear dependence)

A set $\{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_k\}$ of vectors in a vector space V is ***linearly independent*** *if*

$$(c_1 \boldsymbol{v}_1 + \cdots + c_k \boldsymbol{v}_k = 0) \Leftrightarrow (c_1 = \cdots = c_k = 0),$$

where $c_1, \ldots, c_k \in \mathbb{F}$. A set of vectors is ***linearly dependent*** *if it is not linearly independent.*

If linearly dependent, assume w.l.o.g. that $c_1 \neq 0$, then

$$\boldsymbol{v}_1 = -\frac{c_2}{c_1}\boldsymbol{v}_2 - \cdots - \frac{c_k}{c_1}\boldsymbol{v}_k$$

i.e., $\boldsymbol{v}_1$ is a linear combination of the other vectors in the set

Theorem 45

Let V be a finite-dimensional vector space. Then the ***cardinal*** *(number of elements) of every linearly independent set of vectors is less than or equal to the number of elements in every spanning set of vectors*

E.g., in $\mathbb{R}^3$, a set with 4 or more vectors is automatically linearly dependent

Basis

Definition 46 (Basis)

Let V be a vector space. A ***basis*** *of V is a set of vectors in V that is both linearly independent and spanning*

Theorem 47 (Criterion for a basis)

A set $\{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_k\}$ of vectors in a vector space V is a basis of $V \iff \forall \boldsymbol{v} \in V$, $\boldsymbol{v}$ can be written uniquely in the form

$$\boldsymbol{v} = c_1 \boldsymbol{v}_1 + \cdots + c_k \boldsymbol{v}_k,$$

where $c_1, \ldots, c_k \in \mathbb{F}$

More on bases

Theorem 48 (Basis of finite-dimensional vector space)

Every finite-dimensional vector space has a basis

Theorem 49

Any two bases of a finite-dimensional vector space have the same number of vectors

Definition 50 (Dimension)

The ***dimension*** $\dim V$ *of a finite-dimensional vector space* V *is the number of vectors in any basis of the vector space*

Theorem 51 (Dimension of a subspace)

Let V *be a finite-dimensional vector space and* $U \subset V$ *be a subspace of* V*. Then* $\dim U \leq \dim V$

Constructing bases

Theorem 52

Let V be a finite-dimensional vector space. Then every linearly independent set of vectors in V with $\dim V$ *elements is a basis of V*

Theorem 53

Let V be a finite-dimensional vector space. Then every spanning set of vectors in V with $\dim V$ *elements is a basis of V*

Sets and logic

Complex numbers

Vectors and vector spaces

Linear systems and matrices

A crash course in multivariable calculus

A crash course in multivariable calculus

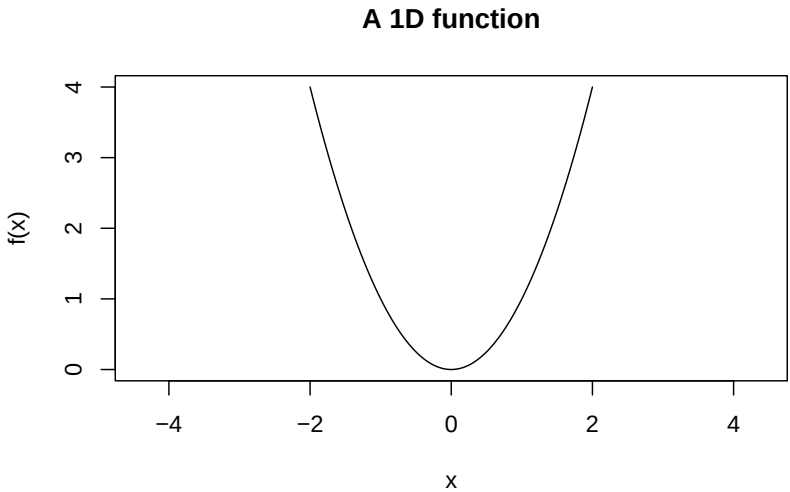
What is Multivariable Calculus?

Optimization with Constraints

The Lagrangian Method

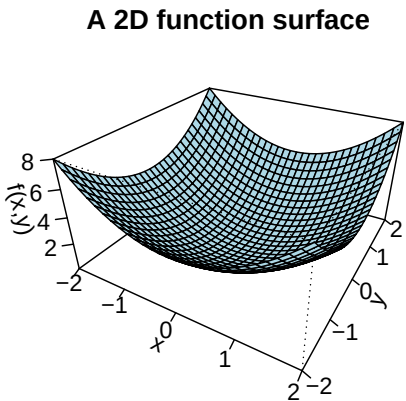
One dimension

MATH 1500 & 1700 deal with functions of one variable, like $f(x) = x^2$



Multivariable calculus

Multivariable calculus extends this to functions of two or more variables, like $f(x, y) = x^2 + y^2$



Partial derivatives

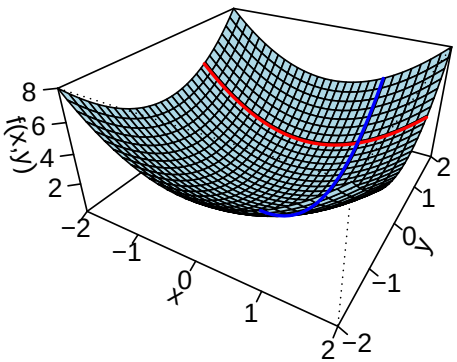
How do we measure the “slope” on a 3D surface?

Definition 54 (Partial derivative)

Let $f : \mathbb{R}^n \to \mathbb{R}$ be a function of variables $x_1, \ldots, x_n$. The ***partial derivative*** *of f with respect to x_i, denoted $\dfrac{\partial f}{\partial x_i}$, is the derivative of f with respect to x_i while treating all other variables as constants*

- $\dfrac{\partial f}{\partial x}$ measures height change as we move only in the x direction. Treat y as a constant
- $\dfrac{\partial f}{\partial y}$ measures height change as we move only in the y direction. Treat x as a constant

Partial derivatives



The Steepest path: the gradient

Definition 55 (Gradient)

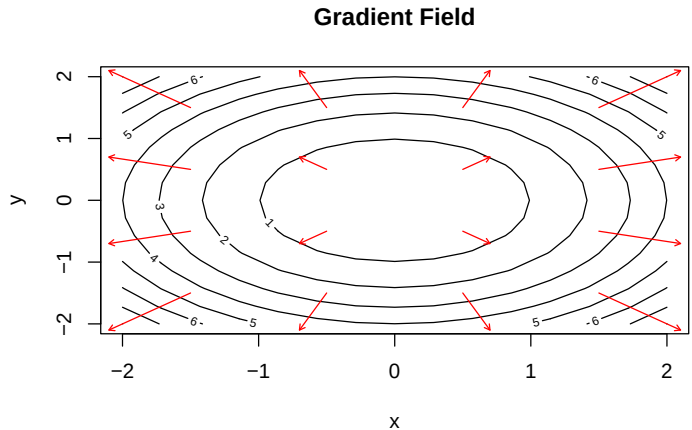
*The **gradient** of a differentiable function $f : \mathbb{R}^n \to \mathbb{R}$, denoted ∇f, is the vector-valued function composed of all its first-order partial derivatives:*

$$\nabla f(x_1, \ldots, x_n) = \left(\frac{\partial f}{\partial x_1}, \ldots, \frac{\partial f}{\partial x_n} \right)$$

What does it tell us?

- **Direction:** it points in the direction of the *steepest ascent*
- **Magnitude:** its length represents the steepness of that ascent

Follow the gradient



At a peak or a valley (a local max/min), the ground is flat. So, $\nabla f = (0, 0)$

A crash course in multivariable calculus

What is Multivariable Calculus?

Optimization with Constraints

The Lagrangian Method

The real-world problem

Often, we want to maximize or minimize a function, but we don't have unlimited freedom. We have **constraints**

- Maximize the profit of your company... *subject to a limited budget*
- Minimize the material used for a can... *that must hold a specific volume*
- Find the highest point on a mountain... *while staying on a specific trail*

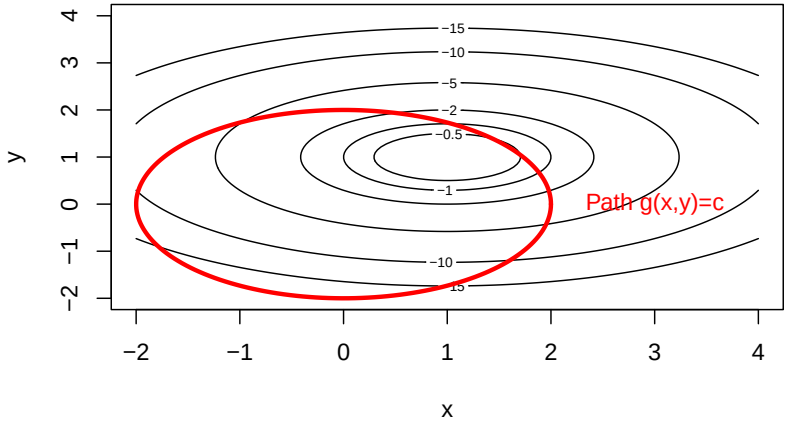
Setting the gradient to zero ($\nabla f = 0$) finds the highest point on the whole mountain, which might not be on our trail!

Visualizing the problem

Imagine our function $f(x, y)$ is the altitude on a map (contour lines)

Our constraint, $g(x, y) = c$, is a specific path we must walk on

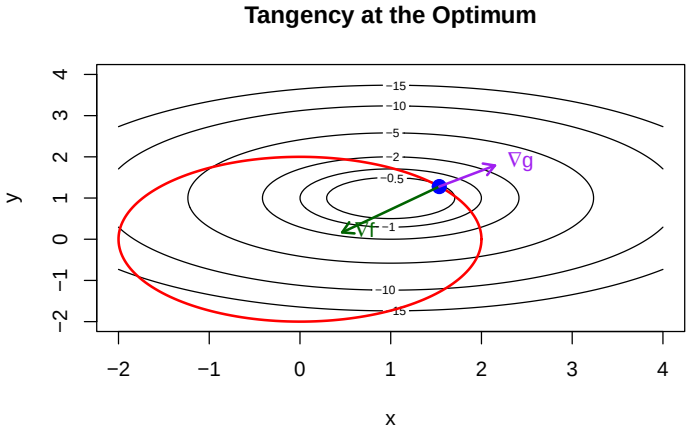
Optimization with a Constraint



We are looking for the highest (or lowest) point *along the red path*

The key insight

At the optimal point on the path, the path will be perfectly **tangent** to the contour line of the surface



Why? If the path crossed the contour line, you could move along the path to get to a higher (or lower) contour

Mathematically, this tangency means the gradient vectors of the function and the constraint are **parallel**

$$\nabla f = \lambda \nabla g$$

The scalar λ (lambda) is called the **Lagrange multiplier**

A crash course in multivariable calculus

What is Multivariable Calculus?

Optimization with Constraints

The Lagrangian Method

The Lagrangian function

The condition $\nabla f = \lambda \nabla g$ is clever, but solving it can be messy

Instead, we combine our function and constraint into a single, new function called the **Lagrangian**

$$\mathcal{L}(x, y, \lambda) = f(x, y) - \lambda[g(x, y) - c]$$

- $f(x, y)$ the function we want to optimize
- $g(x, y) = c$ the constraint we must follow
- λ the Lagrange multiplier

Finding the unconstrained optimum of $\mathcal{L}$ solves the original constrained problem!

The method – step-by-step

To find the optimum of the Lagrangian $\mathcal{L}(x, y, \lambda)$, we find where its gradient is zero

We take the partial derivative with respect to *all* its variables (x, y, and λ) and set them to zero

1. $\frac{\partial \mathcal{L}}{\partial x} = \frac{\partial f}{\partial x} - \lambda \frac{\partial g}{\partial x} = 0$
2. $\frac{\partial \mathcal{L}}{\partial y} = \frac{\partial f}{\partial y} - \lambda \frac{\partial g}{\partial y} = 0$
3. $\frac{\partial \mathcal{L}}{\partial \lambda} = -(g(x, y) - c) = 0 \implies g(x, y) = c$

The first two equations rearrange to $\nabla f = \lambda \nabla g$ and the third equation is the original constraint

Example: Fencing a Field

Problem: You have 40 meters of fence. What is the largest rectangular area you can enclose?

- **Maximize Area:** $A(x, y) = xy$
- **Constraint (Perimeter):** $2x + 2y = 40$

1. Form the Lagrangian:

$$\mathcal{L}(x, y, \lambda) = xy - \lambda(2x + 2y - 40)$$

2. Take Partial Derivatives:

- $\frac{\partial \mathcal{L}}{\partial x} = y - 2\lambda = 0 \implies y = 2\lambda$
- $\frac{\partial \mathcal{L}}{\partial y} = x - 2\lambda = 0 \implies x = 2\lambda$
- $\frac{\partial \mathcal{L}}{\partial \lambda} = -(2x + 2y - 40) = 0$

Example: Solution

From the first two equations, we see that $x = y$

Now, substitute this into the third equation (the constraint):

$$2x + 2(x) = 40$$

$$4x = 40$$

$$x = 10$$

Since $x = y$, we have $y = 10$, i.e., optimal dimensions are 10m by 10m (a square), giving a maximum area of 100 m^2

Least squares problems

MATH 2740 – Mathematics of Data Science – Unit U03

Julien Arino
julien.arino@umanitoba.ca

Department of Mathematics @ University of Manitoba

Fall 2026

The University of Manitoba campuses are located on original lands of Anishinaabeg, Ininew, Anisininew, Dakota and Dene peoples, and on the National Homeland of the Red River Métis. We respect the Treaties that were made on these territories, we acknowledge the harms and mistakes of the past, and we dedicate ourselves to move forward in partnership with Indigenous communities in a spirit of Reconciliation and collaboration.

Outline

Getting the Canadian census data

Least squares problems

Example – Canadian population data

Getting the Canadian census data

Least squares problems

Example – Canadian population data

Remark

This is the slide version of `Canada-census-get-plot-data.Rmd` in the folder `CODE` of the course repository

The `html` file generated there (link) is much easier to read than these slides

Canadian census data

We want to consider the evolution of the population of Canada through time

For this, we grab the Canadian census data

Search for (Google) “Canada historical census data csv", since csv (comma separated values) is a very easy format to use with R

Here, we find a csv for 1851 to 1976

We follow the link to Table A2-14, where we find another link, this time to a csv file. This is what we use in R

Grabing the Canadian census data

The function `read.csv` reads in a file (potentially directly from the web)

```
prefix = "https://www150.statcan.gc.ca/n1/en/pub/11-516-x/sectiona/"
page = "A2_14-eng.csv?st=L7vSnqio"
url = paste0(prefix, page)
data_old = read.csv(url)
```

This directly assigns the result to the variable `data`

We then use the function `head` to show the first few lines in the result

```
head(data_old)
```

```
##    X Series.A2.14.
## 1 NA
## 2 NA          Year
## 3 NA
## 4 NA
## 5 NA
## 6 NA
##   Population.of.Canada..by.province..census.dates..1851.to.1976 X.1
## 1                                                                NA
## 2                                                        Canada  NA Newfo
## 3                                                                NA
## 4                                                                NA
## 5                                                             2  NA
## 6                                                                NA
##      X.3 X.4    X.5       X.6    X.7     X.8       X.9 X.10    X.11 X.12
## 1             NA                                          NA           NA
```

Obviously, this does not make a lot of sense. This is normal: take a look at the first few lines in the file. They take the form

```
head(data_old, n = 1)

##    X Series.A2.14.
## 1 NA
##   Population.of.Canada..by.province..census.dates..1851.to.1976 X.1 X.2 X
## 1                                                                 NA
##   X.5 X.6 X.7 X.8 X.9 X.10 X.11 X.12 X.13 X.14 X.15 X.16 X.17 X.18 X.19 X
## 1                      NA        NA        NA        NA             NA
```

This happens often: the first few lines are here to describe the information contained in the data set, i.e., the so-called **metadata**

It is easy to deal with this: the function `read.csv` takes the optional argument `skip`, which indicates how many lines to skip at the beginning of the file

The second line is also empty, so let us skip it too

```
data_old = read.csv(url, skip = 2)
head(data_old, n = 2)
```

```
##    X Year Canada X.1 Newfound. Prince X.2   Nova       New Quebec Ontario
## 1 NA              NA      land Edward  NA Scotia Brunswick
## 2 NA              NA           Island  NA
##   Manitoba X.3 Saskat. X.4 Alberta X.5  British X.6     Yukon   Northwest
## 1           NA  chewan  NA          NA Columbia  NA Territory Territories
## 2           NA          NA          NA           NA
##   X.8
## 1  NA
## 2  NA
```

To make things legible, the table authors used 3 rows to write long names (e.g., Prince Edward Island is written over 3 rows)

Note, however, that `read.csv` has rightly picked up on the first row being the column names

Because we are only interested in the total population of the country and the year, let us get rid of the first 4 rows and of all columns except the second (Year) and third (Canada)

```
data_old = data_old[5:dim(data_old)[1], 2:3]
head(data_old, n=4)
```

```
##   Year     Canada
## 5 1976 22,992,604
## 6
## 7 1971 21,568,311
## 8 1966 20,014,880
```

Still not perfect...

```
head(data_old, n=3)

##   Year     Canada
## 5 1976 22,992,604
## 6
## 7 1971 21,568,311

tail(data_old, n=2)

##                            Year Canada
## 28 see notes to series A15-53.
## 29                1848 figure.
```

1. there are some empty rows
2. the last few rows need to be removed too, they contain remarks about the data
3. the population counts contain commas
4. it would be better if years were increasing

Fixing these issues

1 and 2 are easy: remark that the Canada column is empty for both issues. Entries in the column are strings. Looking for empty content therefore means looking for empty strings

So to fix 1 and 2, we keep the rows where Canada does not equal the empty string

To get rid of commas, we just need to substitute an empty string for ","

To sort, we find the order for the years and apply it to the entire table

Finally, as remarked above, for now, both the year and the population are considered as strings. To plot, we will have to indicate that these are numbers, not characters

```r
data_old = data_old[which(data_old$Canada != ""),]
data_old$Canada = gsub(",", "", data_old$Canada)
order_data = order(data_old$Year)
data_old = data_old[order_data,]
data_old$Year = as.numeric(data_old$Year)
data_old$Canada = as.numeric(data_old$Canada)
head(data_old)
```

```
##    Year  Canada
## 23 1851 2436297
## 22 1861 3229633
## 21 1871 3689257
## 20 1881 4324810
## 19 1891 4833239
## 17 1901 5371315
```

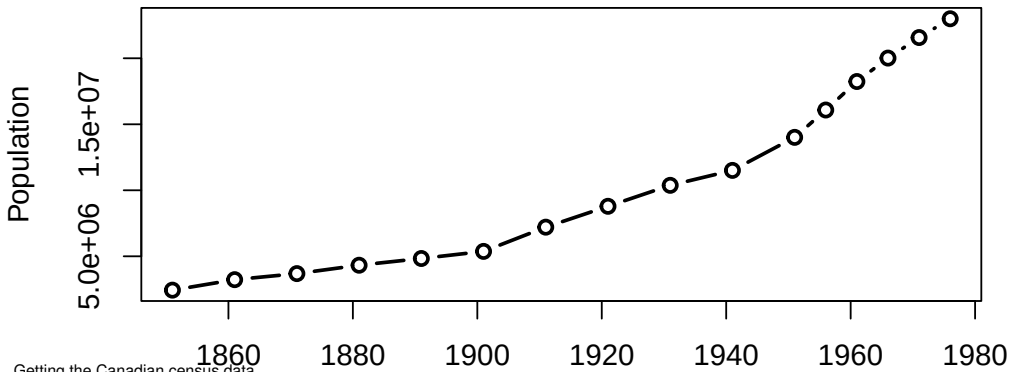
Row numbers are a little weird

```
row.names(data_old) = 1:dim(data_old)[1]
head(data_old)
```

```
##   Year  Canada
## 1 1851 2436297
## 2 1861 3229633
## 3 1871 3689257
## 4 1881 4324810
## 5 1891 4833239
## 6 1901 5371315
```

Well, that looks about right! Let's see what this looks like in a graph

```r
plot(data_old$Year, data_old$Canada,
    type = "b", lwd = 2,
    xlab = "Year", ylab = "Population")
```



But wait, this is only to 1976..!

Looking around, we find another table here with a download csv link (this one)

StatCan has added Cloudflare bot protection, so automated downloads from R fail. We download it via browser and save to `DATA/98-315-XWE2011001-001.csv`

```
data_new = read.csv("../DATA/98-315-XWE2011001-001.csv",
                    fileEncoding = "latin1")
```

The table is 720KB, so surely there must be more to this than just the population

To get a sense of that, we dump a few random rows

```
data_new[sort(unique(c(1,sample(nrow(data_new), 4)))), ]
```

```
##                 GEOGRAPHY.NAME
## 1                       Canada
## 453  Newfoundland and Labrador
## 3126            Trois-Rivières
## 5586                    London
## 5636                   Windsor
##                                                    CHARACTERISTIC YEAR.S
## 1                                        Population (in thousands)    1956
## 453                    Population by single years of age: 76 years    2011
## 3126                                % of three-person households    2006
## 5586 % of private households containing a couple without children    2011
## 5636                     % of the population aged 0 to 14 years    2011
##        TOTAL FLAG_TOTAL
## 1    16081.0
## 453   3145.0
## 3126    14.3
```

Flat tables

This is a **flat table**: each row contains a piece of information, the columns describe what this piece of information is about

This has *way more* information than we need: we just want the population of Canada and here we get 9960 rows over 213 characteristics

Also, population is expressed in thousands, so once we selected what we want, we need to multiply by 1,000

Selecting rows

We want the rows where the geography is “Canada” and the characteristic is “Population (in thousands)”

Find indices of rows that satisfy the first criterion and those that satisfy the second; intersecting these two sets of indices, we have selected the rows we want

```
idx_CAN = which(data_new$GEOGRAPHY.NAME == "Canada")
idx_char = which(data_new$CHARACTERISTIC == "Population (in thousands)")
idx_keep = intersect(idx_CAN, idx_char)
```

Let us keep only these rows

```
data_new = data_new[idx_keep,]
head(data_new, n = 8)
```

```
##   GEOGRAPHY.NAME            CHARACTERISTIC YEAR.S. TOTAL FLAG_TOTAL
## 1         Canada Population (in thousands)    1956 16081
## 2         Canada Population (in thousands)    1961 18238
## 3         Canada Population (in thousands)    1966 20015
## 4         Canada Population (in thousands)    1971 21568
## 5         Canada Population (in thousands)    1976 22993
## 6         Canada Population (in thousands)    1981 24343
## 7         Canada Population (in thousands)    1986 25309
## 8         Canada Population (in thousands)    1991 27297
```

We want to concatenate this data frame with the one from earlier

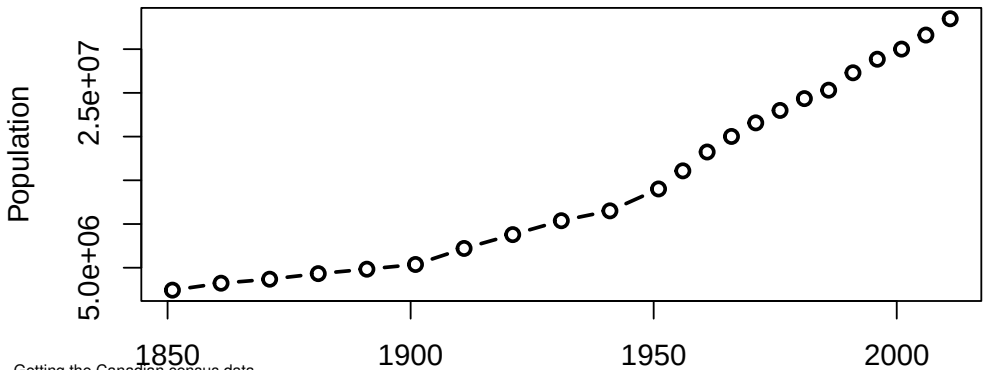
To do this, we need the two data frames to have the same number of columns, same column names and same entry types (notice that `YEAR.S.` in `data_new` is a column of characters)

What remains to be done

- Rename the columns in the pruned old data (data_pruned) to `year` and `population`. Personally, I prefer lowercase column names.. and `population` is more informative than `Canada`
- Keep only the relevant columns in `data_new`, rename them accordingly and multiply population by 1,000 there
- Transform year in `data_new` to numbers
- We already have data up to and including 1976 in `data_old`, so get rid of that in `data_new`
- Append the rows of `data_new` to those of `data_pruned`

```r
colnames(data_old) = c("year", "population")
data_new = data_new[,c("YEAR.S.","TOTAL")]
colnames(data_new) = c("year", "population")
data_new$year = as.numeric(data_new$year)
data_new = data_new[which(data_new$year>1976),]
data_new$population = data_new$population*1000

data = rbind(data_old,data_new)
```

```r
plot(data$year, data$population,
    type = "b", lwd = 2,
    xlab = "Year", ylab = "Population")
```



Save the processed data

In case we need the data elsewhere, we save the data to a `csv` file

```
write.csv(data, file = "../CODE/Canada_census.csv")
```

Using `readr` saves the data without row numbers (by default), so we can do this instead

```
readr::write_csv(data, file = "../CODE/Canada_census.csv")
```

Getting the Canadian census data

Least squares problems

Example – Canadian population data

Least squares problems

Initial considerations

The least squares problem

Fitting something more complicated

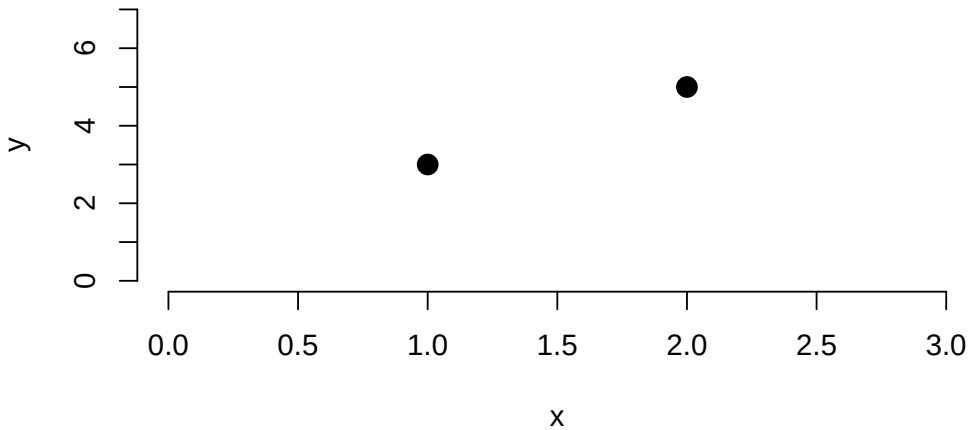
We just collected the census data for Canada

Suppose we want to predict the population of Canada in 20 years given the historical population growth seen in the previous plot. What can we do?

If there were just two points, we could easily "drive" a line through these two points. However, we have much more than two points, so we will use *fitting*, *i.e.*, try to make a curve come as close to possible to the points

We start with a line, giving rise to **linear least squares**

Least squares approximation – A trivial case



We want to find the equation of a line $y = a + bx$ that goes through these two points, i.e., we seek a and b such that

$$3 = a + b$$
$$5 = a + 2b$$

i.e., they satisfy $y = a + bx$ for $(x, y) = (1, 3)$ and $(x, y) = (2, 5)$

This is a linear system with 2 equations and 2 unknowns a and b

$$\begin{pmatrix} 1 & 1 \\ 1 & 2 \end{pmatrix} \begin{pmatrix} a \\ b \end{pmatrix} = \begin{pmatrix} 3 \\ 5 \end{pmatrix}$$

We know from Theorem **??** (Linear algebra in a nutshell) that this system has a unique solution if the matrix

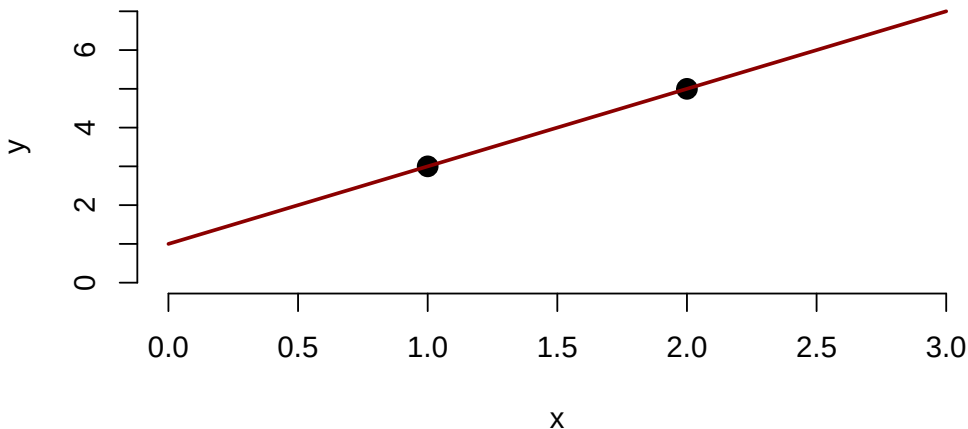
$$M = \begin{pmatrix} 1 & 1 \\ 1 & 2 \end{pmatrix}$$

is invertible

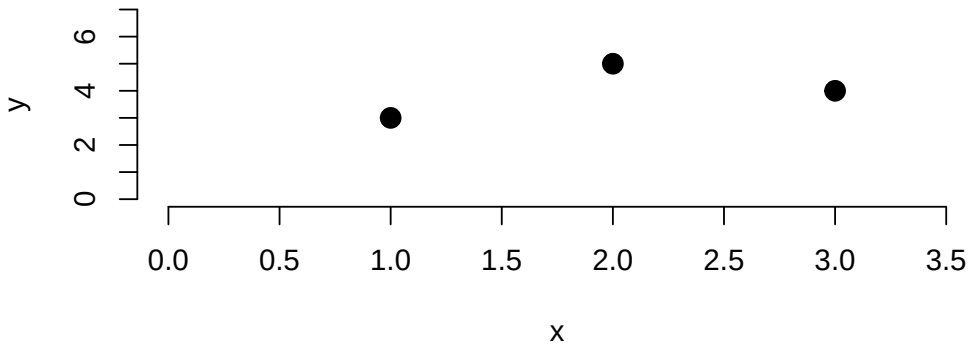
$\det(M) = 1$, so we are good, we'll find a and b easily..

```
A = matrix(c(1,1,1,2), nr = 2, nc = 2, byrow = TRUE)
rhs = matrix(c(3,5), nr = 2, nc =1)
coefs = solve(A,rhs)
coefs
```

```
##      [,1]
## [1,]    1
## [2,]    2
```


y
0
2
4
6
0.0
0.5
1.0
1.5
2.0
2.5
3.0
x

Now let's add another point



These points are clearly not colinear, so there is not one line going through the 3

We end up with an *overdetermined* system

$$
\begin{aligned}
3 &= a + b \\
5 &= a + 2b \\
4 &= a + 3b
\end{aligned}
$$

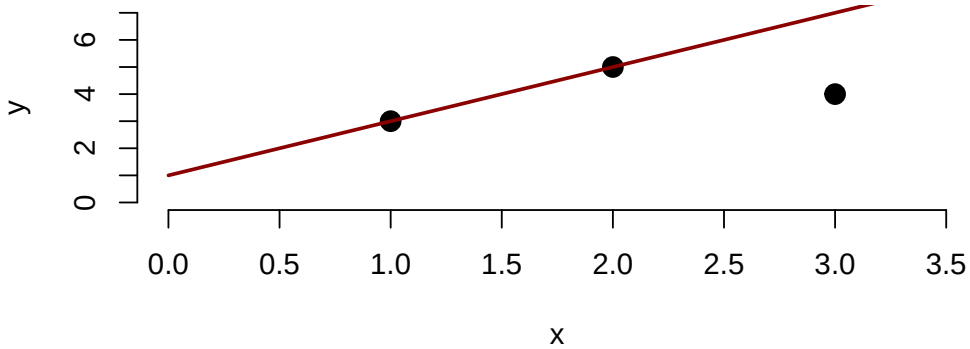
i.e.,

$$
\begin{pmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{pmatrix} \begin{pmatrix} a \\ b \end{pmatrix} = \begin{pmatrix} 3 \\ 5 \\ 4 \end{pmatrix}
$$

We have verified visually that the points are not colinear, so this system has no solution

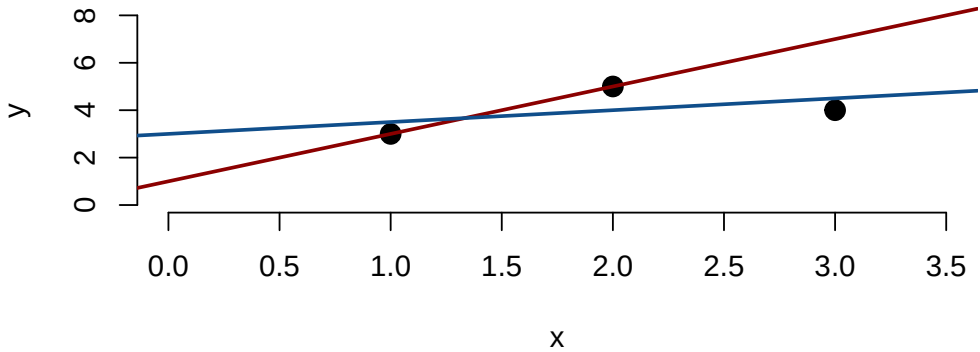
(If you had to do it for good, you consider two vectors stemming from these 3 points and compute the angle between them or check that one is a multiple of the other)

So let us instead try to find the line that comes "closest" to the 3 points



Obviously, not sensational..

Let's eyeball it – Less steep and higher y-intercept



The blue line looks better, but how do we quantify this?

How do we find "how far away" we are?

- We could use projections onto the line (which we know minimises the distance)

- However, this will be a problem if we later decide that rather than a straight line, we want to use something more "funky" like a quadratic or an exponential

Compare, for a given value x, the distance between the true value y and the value of y obtained using the curve (line, here) that we use to fit the data

Let (x_i, y_i) be the data points, i.e., here, $(x_1, y_1) = (1, 3)$, $(x_2, y_2) = (2, 5)$ and $(x_3, y_3) = (3, 4)$

Suppose we use a line with equation $y = a + bx$ and that we pick a value for a and b. Then at x_1,

$$\tilde{y}_1 = a + bx_1 = a + b$$

at x_2

$$\tilde{y}_2 = a + bx_2 = a + 2b$$

and at x_3,

$$\tilde{y}_3 = a + bx_3 = a + 3b$$

Consider x_1, for instance. The error we made by using the line with coefficients (a, b) is $\overrightarrow{(x_1, y_1)(x_1, \tilde{y}_1)}$

For future use, let us create a function for $y = a_0 + a_1 x$

```
my_line = function(x, a_0, a_1){
    return(a_0 + a_1*x)
}
```

Functions are super useful when programming

```
my_line(1,2,3)
```

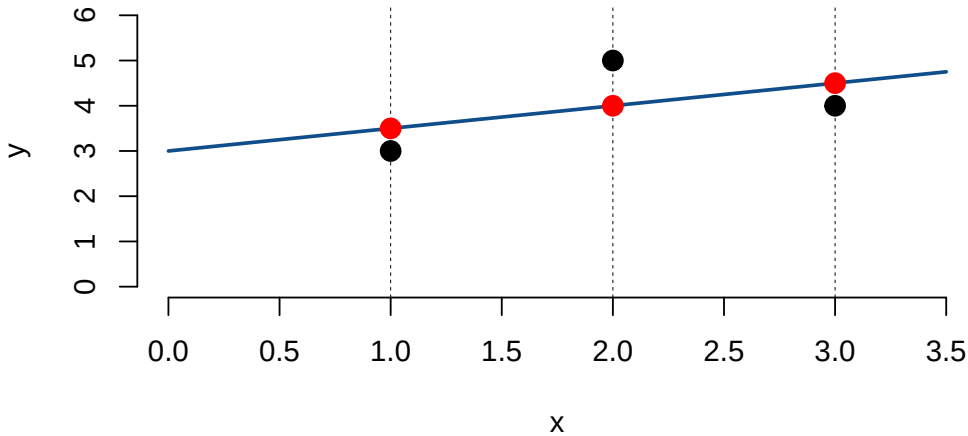
```
## [1] 5
```

```
my_line(a_0 = 2, a_1 = 3, x = 1)
```

```
## [1] 5
```

```
my_line(x = c(1,2,3), a_0 = 2, a_1 = 3)
```

```
## [1]  5  8 11
```


y
0 1 2 3 4 5 6
0.0 0.5 1.0 1.5 2.0 2.5 3.0 3.5
x

Let us return to the error

$$\overrightarrow{(x_1, y_1)(x_1, \tilde{y}_1)}$$

We have

$$\overrightarrow{(x_1, y_1)(x_1, \tilde{y}_1)} = (x_1 - x_1, y_1 - \tilde{y}_1) = (0, y_1 - \tilde{y}_1)$$

Let us call

$$\varepsilon_1 = y_1 - \tilde{y}_1$$

We can compute ε_2 and ε_3 too. And we can then form the **error vector**

$$\mathbf{e} = (\varepsilon_1, \varepsilon_2, \varepsilon_3)^T$$

The norm of $\mathbf{e}$, $\|\mathbf{e}\|$, then tells us how much error we are making for the choice of (a, b) we are using

The norm of $\|\mathbf{e}\|$ tells us how much error we are making for the choice of (a, b) we are using

So our objective is to find (a, b) such that $\|\mathbf{e}\|$ is minimal

We could use various norms, but the Euclidean norm has some very interesting properties, so we use

$$\|\mathbf{e}\| = \sqrt{\varepsilon_1^2 + \varepsilon_2^2 + \varepsilon_3^2}$$

The linear least squares problem

Given a collection of data points $(x_1, y_1), \ldots, (x_n, y_n)$, find the coefficients a, b of the line $y = a + bx$ such that

$$\|\mathbf{e}\| = \sqrt{\varepsilon_1^2 + \cdots + \varepsilon_n^2} = \sqrt{(y_1 - \tilde{y}_1)^2 + \cdots + (y_n - \tilde{y}_n)^2}$$

is minimal, where $\tilde{y}_i = a + bx_i$, for $i = 1, \ldots, n$

Let us first hack a brute force solution! (For the example we have been using this far)

We have our three points in the list `points`, the function `my_line` that computes the value $\tilde{y}$ given x and a, b, so let us make a new function that, given a, b, computes **e**

We'll also pass the points `points`

A function to compute the error

```
error = function(a_0, a_1, points) {
    y_tilde = my_line(points$x, a_0 = a_0, a_1 = a_1)
    e = points$y - y_tilde
    return(sqrt(sum(e^2)))
}
error(a_0 = 2, a_1 = 3, points)
```

```
## [1] 7.874008
```

```
error(a_0 = 3, a_1 = 0.5, points)
```

```
## [1] 1.224745
```

```
error(a_0 = 3.1, a_1 = 0.48, points)
```

```
## [1] 1.229471
```

We can't be doing this by hand..!

Let's minimise using something cool – *genetic algorithms*

- Genetic algorithms are a stochastic *optimisation* method. There are other types, e.g., gradient descent (deterministic)
- The idea is to use a mechanism mimicking evolution's drive towards higher fitness
- The function value is its fitness
- We try different genes (here, different values of a, b) and evaluate their fitness.. keep the good ones
- We mutate or crossover genes, throw in new ones, etc.
- We keep doing this until we reach a stopping criterion
- We then return the best gene we found

```r
if (!require("GA", quietly = TRUE)) {
  install.packages("GA")
  library(GA)
}
GA = ga(type = "real-valued",
        fitness = function(x) -error(a_0 = x[1], a_1 = x[2], points),
        suggestions = c(a_0 = 2, a_1 = 3),
        lower = c(-10, -10), upper = c(10, 10),
        popSize = 200, maxiter = 150)
```

GA returns an S4 object of class ga

```
GA

## An object of class "ga"
##
## Call:
## ga(type = "real-valued", fitness = function(x) -error(a_0 = x[1],     a_1
##
## Available slots:
##  [1] "call"         "type"         "lower"        "upper"        "nBits"
##  [6] "names"        "popSize"      "iter"         "run"          "maxiter
## [11] "suggestions"  "population"   "elitism"      "pcrossover"   "pmutati
## [16] "optim"        "fitness"      "summary"      "bestSol"      "fitness
## [21] "solution"
```

Access slots using @. For instance, the GA finds the coefficients

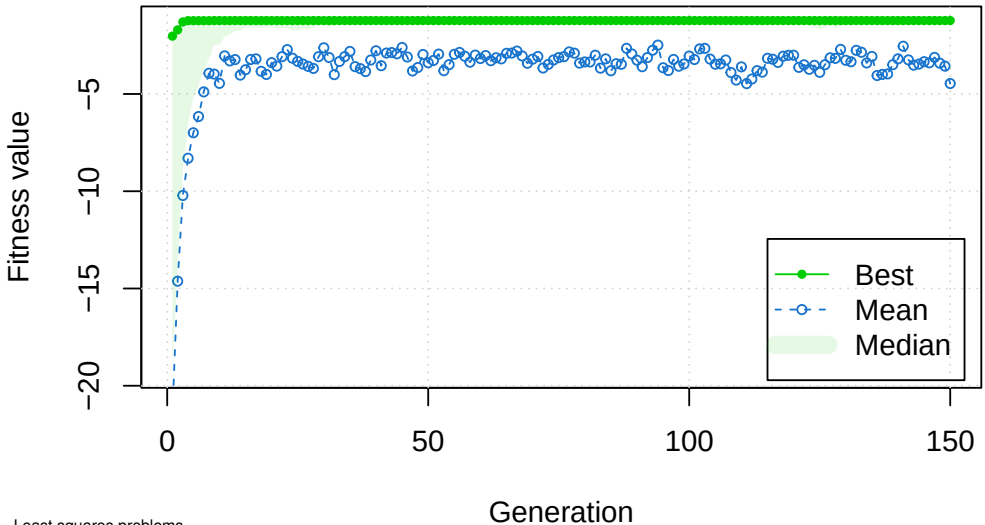
```
GA@solution
```

```
##            x1        x2
## [1,] 2.999243 0.5004865
```

with fitness value (i.e., $\|\boldsymbol{e}\|$)

```
-GA@fitnessValue
```

```
## [1] 1.224745
```


Fitness value
−5
−10
−15
−20
0
50
100
150
Generation
Best
Mean
Median

- Here, however, we do not have to go brute force: we can reason using mathematics

- We now take a little detour on the math side of things, we will come back to code in a while..

Least squares problems

- Initial considerations
- **The least squares problem**
- Fitting something more complicated

The least squares problem (simplest version)

Definition 1

Given a collection of points $(x_1, y_1), \ldots, (x_n, y_n)$*, find the coefficients* a, b *of the line* $y = a + bx$ *such that*

$$\|\mathbf{e}\| = \sqrt{\varepsilon_1^2 + \cdots + \varepsilon_n^2} = \sqrt{(y_1 - \tilde{y}_1)^2 + \cdots + (y_n - \tilde{y}_n)^2}$$

is minimal, where $\tilde{y}_i = a + bx_i$ *for* $i = 1, \ldots, n$

We just saw how to solve this by brute force using a genetic algorith to minimise $\|e\|$, let us now see how to solve this problem "properly"

For a data point $i = 1, \ldots, n$

$$\varepsilon_i = y_i - \tilde{y}_i = y_i - (a + bx_i)$$

So if we write this for all data points,

$$\begin{aligned} \varepsilon_1 &= y_1 - (a + bx_1) \\ &\vdots \\ \varepsilon_n &= y_n - (a + bx_n) \end{aligned}$$

In matrix form

$$\boldsymbol{e} = \boldsymbol{b} - A\boldsymbol{x}$$

with

$$\boldsymbol{e} = \begin{pmatrix} \varepsilon_1 \\ \vdots \\ \varepsilon_n \end{pmatrix}, A = \begin{pmatrix} 1 & x_1 \\ \vdots & \vdots \\ 1 & x_n \end{pmatrix}, \boldsymbol{x} = \begin{pmatrix} a \\ b \end{pmatrix} \text{ and } \boldsymbol{b} = \begin{pmatrix} y_1 \\ \vdots \\ y_n \end{pmatrix}$$

The least squares problem (reformulated)

Definition 2 (Least squares solutions)

Consider a collection of points $(x_1, y_1), \ldots, (x_n, y_n)$*, a matrix* $A \in \mathcal{M}_{np}$*,* $\boldsymbol{b} \in \mathbb{R}^n$*. A* ***least squares solution*** *of* $A\boldsymbol{x} = \boldsymbol{b}$ *is a vector* $\tilde{\boldsymbol{x}} \in \mathbb{R}^p$ *s.t.*

$$\forall \boldsymbol{x} \in \mathbb{R}^p, \quad \|\boldsymbol{b} - A\tilde{\boldsymbol{x}}\| \leq \|\boldsymbol{b} - A\boldsymbol{x}\|$$

Needed to solve the problem

Definition 3 (Best approximation)

Let V be a vector space, $W \subset V$ and $\mathbf{v} \in V$. The **best approximation** to $\mathbf{v}$ in W is $\tilde{\mathbf{v}} \in W$ s.t.

$$\forall \mathbf{w} \in W, \mathbf{w} \neq \tilde{\mathbf{v}}, \quad \|\mathbf{v} - \tilde{\mathbf{v}}\| < \|\mathbf{v} - \mathbf{w}\|$$

Theorem 4 (Best approximation theorem)

Let V be a vector space with an inner product, $W \subset V$ and $\mathbf{v} \in V$. Then $\text{proj}_W(\mathbf{v})$ *is the best approximation to* $\mathbf{v}$ *in* W

Let us find the least squares solution

$\forall \boldsymbol{x} \in \mathbb{R}^p$, $A\boldsymbol{x}$ is a vector in the **column space** of A (the space spanned by the vectors making up the columns of A)

Since $\boldsymbol{x} \in \mathbb{R}^p$, $A\boldsymbol{x} \in \text{col}(A)$

$\implies$ least squares solution of $A\boldsymbol{x} = \boldsymbol{b}$ is a vector $\tilde{\boldsymbol{y}} \in \text{col}(A)$ s.t.

$$\forall \boldsymbol{y} \in \text{col}(A), \quad \|\boldsymbol{b} - \tilde{\boldsymbol{y}}\| \leq \|\boldsymbol{b} - \boldsymbol{y}\|$$

This looks very much like Best approximation and Best approximation theorem

Putting things together

We just stated: The least squares solution of $A\boldsymbol{x} = \boldsymbol{b}$ is a vector $\tilde{\boldsymbol{y}} \in \text{col}(A)$ s.t.

$$\forall \boldsymbol{y} \in \text{col}(A), \quad \|\boldsymbol{b} - \tilde{\boldsymbol{y}}\| \leq \|\boldsymbol{b} - \boldsymbol{y}\|$$

We know (reformulating a tad):

Theorem 5 (Best approximation theorem)

Let V be a vector space with an inner product, $W \subset V$ and $\mathbf{v} \in V$. Then $\text{proj}_W(\mathbf{v}) \in W$ is the best approximation to $\mathbf{v}$ in W, i.e.,

$$\forall \mathbf{w} \in W, \mathbf{w} \neq \text{proj}_W(\mathbf{v}), \quad \|\mathbf{v} - \text{proj}_W(\mathbf{v})\| < \|\mathbf{v} - \mathbf{w}\|$$

$\implies$ $W = \text{col}(A)$, $\boldsymbol{v} = \boldsymbol{b}$ and $\tilde{\boldsymbol{y}} = \text{proj}_{\text{col}(A)}(\mathbf{b})$

So if $\tilde{\boldsymbol{x}}$ is a least squares solution of $A\boldsymbol{x} = \boldsymbol{b}$, then

$$\tilde{\boldsymbol{y}} = A\tilde{\boldsymbol{x}} = \text{proj}_{\text{col}(A)}(\mathbf{b})$$

We have

$$\boldsymbol{b} - A\tilde{\boldsymbol{x}} = \boldsymbol{b} - \text{proj}_{\text{col}(A)}(\mathbf{b}) = \text{perp}_{\text{col}(A)}(\mathbf{b})$$

and it is easy to show that

$$\text{perp}_{\text{col}(A)}(\mathbf{b}) \perp \text{col}(A)$$

So for all columns $\boldsymbol{a}_i$ of A

$$\boldsymbol{a}_i \cdot (\boldsymbol{b} - A\tilde{\boldsymbol{x}}) = 0$$

which we can also write as $\boldsymbol{a}_i^T(\boldsymbol{b} - A\tilde{\boldsymbol{x}}) = 0$

For all columns $\boldsymbol{a}_i$ of A,

$$\boldsymbol{a}_i^T(\boldsymbol{b} - A\tilde{\boldsymbol{x}}) = 0$$

This is equivalent to saying that

$$A^T(\boldsymbol{b} - A\tilde{\boldsymbol{x}}) = \mathbf{0}$$

We have

$$\begin{aligned} A^T(\boldsymbol{b} - A\tilde{\boldsymbol{x}}) = \mathbf{0} &\iff A^T\boldsymbol{b} - A^TA\tilde{\boldsymbol{x}} = \mathbf{0} \\ &\iff A^T\boldsymbol{b} = A^TA\tilde{\boldsymbol{x}} \\ &\iff A^TA\tilde{\boldsymbol{x}} = A^T\boldsymbol{b} \end{aligned}$$

The latter system constitutes the **normal equations** for $\tilde{\boldsymbol{x}}$

Invertibility of A^TA

Theorem 6 (Invertibility of A^TA)

Let $A \in \mathcal{M}_{np}$. Then A has linearly independent columns $\iff$ A^TA invertible

Proof. Since $A^TA \in \mathcal{M}_p$ is square, it is invertible $\iff A^TA\boldsymbol{x} = \mathbf{0} \implies \boldsymbol{x} = \mathbf{0}$. Also, A has linearly independent columns $\iff A\boldsymbol{x} = \mathbf{0} \implies \boldsymbol{x} = \mathbf{0}$. Thus, it suffices to show that $A^TA\boldsymbol{x} = \mathbf{0} \iff A\boldsymbol{x} = \mathbf{0}$.

- ($\Longleftarrow$) If $A\boldsymbol{x} = \mathbf{0}$, then multiplying on the left by A^T gives $A^TA\boldsymbol{x} = A^T\mathbf{0} = \mathbf{0}$.
- ($\Longrightarrow$) Suppose $A^TA\boldsymbol{x} = \mathbf{0}$. Multiplying on the left by $\boldsymbol{x}^T$ gives

$$\boldsymbol{x}^TA^TA\boldsymbol{x} = \boldsymbol{x}^T\mathbf{0} = 0$$

But $\boldsymbol{x}^TA^TA\boldsymbol{x} = (A\boldsymbol{x})^T(A\boldsymbol{x}) = \|A\boldsymbol{x}\|^2$. Since the norm of a vector is zero if and only if the vector is the zero vector, we have $\|A\boldsymbol{x}\|^2 = 0 \implies A\boldsymbol{x} = \mathbf{0}$. □

Least squares theorem

Theorem 7 (Least squares theorem)

$A \in \mathcal{M}_{np}$, $\boldsymbol{b} \in \mathbb{R}^n$. Then

1. *$A\boldsymbol{x} = \boldsymbol{b}$ always has at least one least squares solution $\tilde{\boldsymbol{x}}$*
2. *$\tilde{\boldsymbol{x}}$ least squares solution to $A\boldsymbol{x} = \boldsymbol{b} \iff \tilde{\boldsymbol{x}}$ is a solution to the normal equations $A^T A\tilde{\boldsymbol{x}} = A^T\boldsymbol{b}$*
3. *If $A^T A$ invertible, then the least squares solution is unique and*

$$\tilde{\boldsymbol{x}} = \left(A^T A\right)^{-1} A^T \boldsymbol{b}$$

We have seen 1 and 2, and we just proved 3!

Least squares problems

Initial considerations

The least squares problem

Fitting something more complicated

Suppose we want to fit something a bit more complicated..

For instance, instead of the affine function

$$y = a + bx$$

suppose we want to do the quadratic

$$y = a_0 + a_1 x + a_2 x^2$$

or even

$$y = k_0 e^{k_1 x}$$

How do we proceed?

Fitting the quadratic

We have the data points $(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)$ and want to fit

$$y = a_0 + a_1 x + a_2 x^2$$

At (x_1, y_1),

$$\tilde{y}_1 = a_0 + a_1 x_1 + a_2 x_1^2$$

$\vdots$

At (x_n, y_n),

$$\tilde{y}_n = a_0 + a_1 x_n + a_2 x_n^2$$

In terms of the error

$$\begin{aligned}\varepsilon_1 &= y_1 - \tilde{y}_1 = y_1 - (a_0 + a_1 x_1 + a_2 x_1^2)\\ &\vdots\\ \varepsilon_n &= y_n - \tilde{y}_n = y_n - (a_0 + a_1 x_n + a_2 x_n^2)\end{aligned}$$

i.e.,

$$\boldsymbol{e} = \boldsymbol{b} - A\boldsymbol{x}$$

where

$$\boldsymbol{e} = \begin{pmatrix}\varepsilon_1\\ \vdots\\ \varepsilon_n\end{pmatrix}, A = \begin{pmatrix}1 & x_1 & x_1^2\\ \vdots & \vdots & \vdots\\ 1 & x_n & x_n^2\end{pmatrix}, \boldsymbol{x} = \begin{pmatrix}a_0\\ a_1\\ a_2\end{pmatrix} \text{ and } \boldsymbol{b} = \begin{pmatrix}y_1\\ \vdots\\ y_n\end{pmatrix}$$

Theorem 7 applies, with here $A \in \mathcal{M}_{n3}$ and $\boldsymbol{b} \in \mathbb{R}^n$

Fitting the exponential

Things are a bit more complicated here

If we proceed as before, we get the system

$$\begin{aligned} y_1 &= k_0 e^{k_1 x_1} \\ &\vdots \\ y_n &= k_0 e^{k_1 x_n} \end{aligned}$$

$e^{k_1 x_i}$ is a nonlinear term, it cannot be put in a matrix

However: take the ln of both sides of the equation

$$\ln(y_i) = \ln(k_0 e^{k_1 x_i}) = \ln(k_0) + \ln(e^{k_1 x_i}) = \ln(k_0) + k_1 x_i$$

If $y_i, k_0 > 0$, then their ln are defined and we're in business..

$$\ln(y_i) = \ln(k_0) + k_1 x_i$$

So the system is

$$\boldsymbol{y} = A\boldsymbol{x} + \boldsymbol{b}$$

with

$$A = \begin{pmatrix} x_1 \\ \vdots \\ x_n \end{pmatrix}, \boldsymbol{x} = \begin{pmatrix} k_1 \end{pmatrix}, \boldsymbol{b} = \begin{pmatrix} \ln(k_0) \end{pmatrix} \text{ and } \boldsymbol{y} = \begin{pmatrix} \ln(y_1) \\ \vdots \\ \ln(y_n) \end{pmatrix}$$

Getting the Canadian census data

Least squares problems

Example – Canadian population data

Example: Canadian census data

Let's consider data from the Canadian census. We first load and visualize it.

```
canada = read.csv("../DATA/Canada_census.csv")
canada = as.data.frame(canada)
plot(canada, main = "Canadian Population over Time",
     xlab = "Year", ylab = "Population")
```

Fitting a linear model

Let's first try to fit a linear model $y = a_0 + a_1 \times \text{year}$.

We construct the matrix A and the vector $\boldsymbol{b}$ (the population data):

```
b = canada$population
A = matrix(ncol = 2, nrow = dim(canada)[1],
           data = c(rep(1, dim(canada)[1]),
                    canada$year))
```

Recall that the normal equations are $A^T A\tilde{\boldsymbol{x}} = A^T \boldsymbol{b}$, where $\tilde{\boldsymbol{x}} = (a_0, a_1)^T$.

Solving the normal equations

We can compute $A^T A$ and solve the normal equations in R:

```
ATA = t(A) %*% A
invATA = solve(ATA)
x_tilde = invATA %*% t(A) %*% b
```

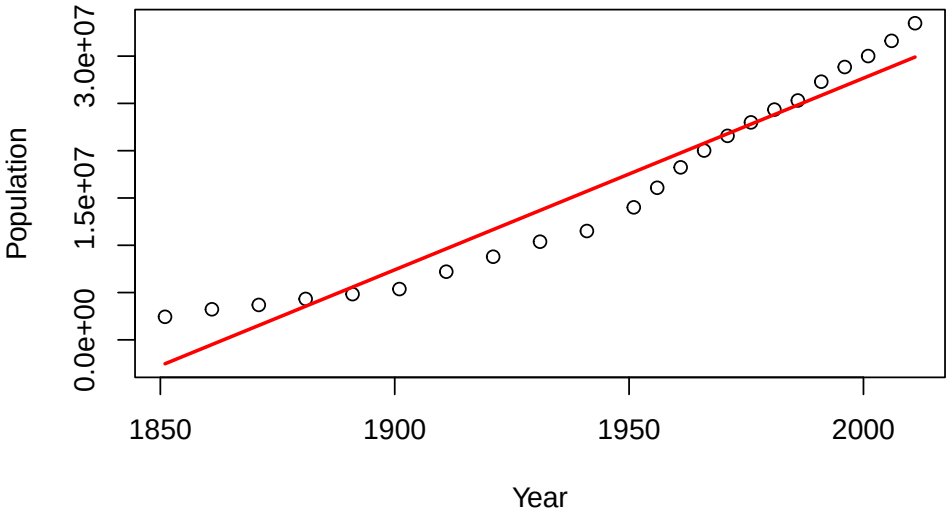
The resulting coefficients $\tilde{\boldsymbol{x}}$ are:

```
##               [,1]
## [1,] -377660589.2
## [2,]     202664.8
```

Visualizing the linear fit

We compute the fitted values and plot them alongside the data:

```
x_axis = seq(canada$year[1], max(canada$year), 1)
y_fitted = x_tilde[1] + x_tilde[2] * x_axis

plot(canada, ylim = c(min(y_fitted), max(canada$population)),
xlab = "Year", ylab = "Population")
lines(x_axis, y_fitted, col = "red", lwd = 2)
```


Population
0.0e+00
1.5e+07
3.0e+07
1850
1900
1950
2000
Year

Fitting a quadratic model

The linear fit misses the curvature in the population growth. Let us try a quadratic model:

$$y = a_0 + a_1 \times \text{year} + a_2 \times \text{year}^2$$

We can set up the matrix A_2:

```
A2 = matrix(ncol = 3, nrow = dim(canada)[1],
            data = c(rep(1, dim(canada)[1]),
                     canada$year,
                     canada$year^2))
```

Numerical issues

Let's try to compute $A_2^T A_2$ and its inverse:

```
ATA2 = t(A2) %*% A2
invATA2 = solve(ATA2)

## Error in `solve.default()`:
## !  system is computationally singular:  reciprocal condition number
= 2.05592e-20
```

The years are around 1900-2000, so year$^2 \approx 4 \times 10^6$. The columns of A_2 have vastly different scales, which makes $A_2^T A_2$ very poorly **conditioned**!

A better conditioned formulation

To avoid numerical issues, we can shift years so they start at 0:

$$\text{shifted_year} = \text{year} - \min(\text{year})$$

We are now fitting $y = a_0 + a_1 \times \text{shifted_year} + a_2 \times \text{shifted_year}^2$.

```
A3 = matrix(ncol = 3, nrow = dim(canada)[1],
            data = c(rep(1, dim(canada)[1]),
                     canada$year - min(canada$year),
                     (canada$year - min(canada$year))^2))
```

This makes the values in the columns of A_3 much closer in magnitude.

Solving the shifted quadratic model

We can now safely compute the normal equations for the shifted model:

```
ATA3 = t(A3) %*% A3
invATA3 = solve(ATA3)
x_tilde = invATA3 %*% t(A3) %*% b
```

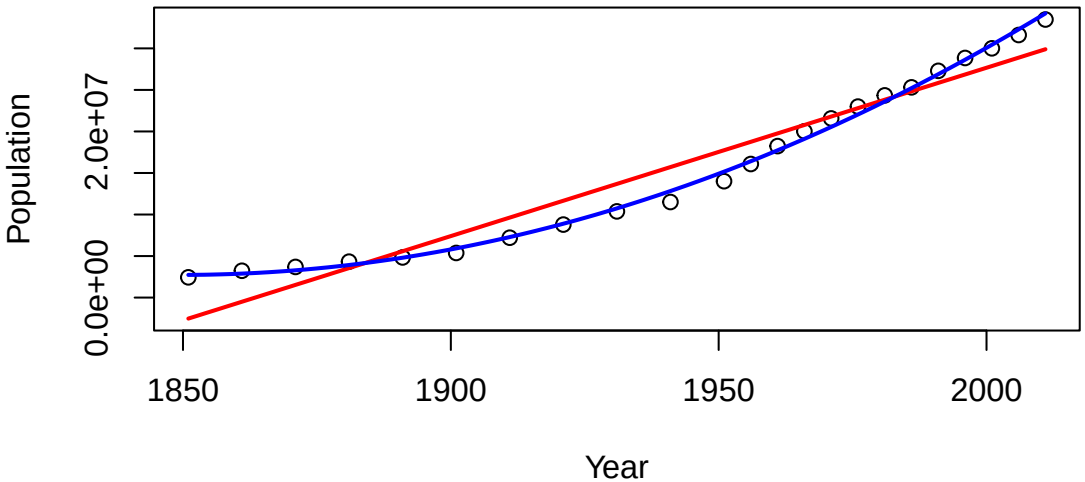
The resulting coefficients $\tilde{\boldsymbol{x}}$ are:

```
##             [,1]
## [1,] 2730500.544
## [2,]    3442.989
## [3,]    1208.270
```

Visualizing the quadratic fit

We compute the fitted quadratic curve and plot it:

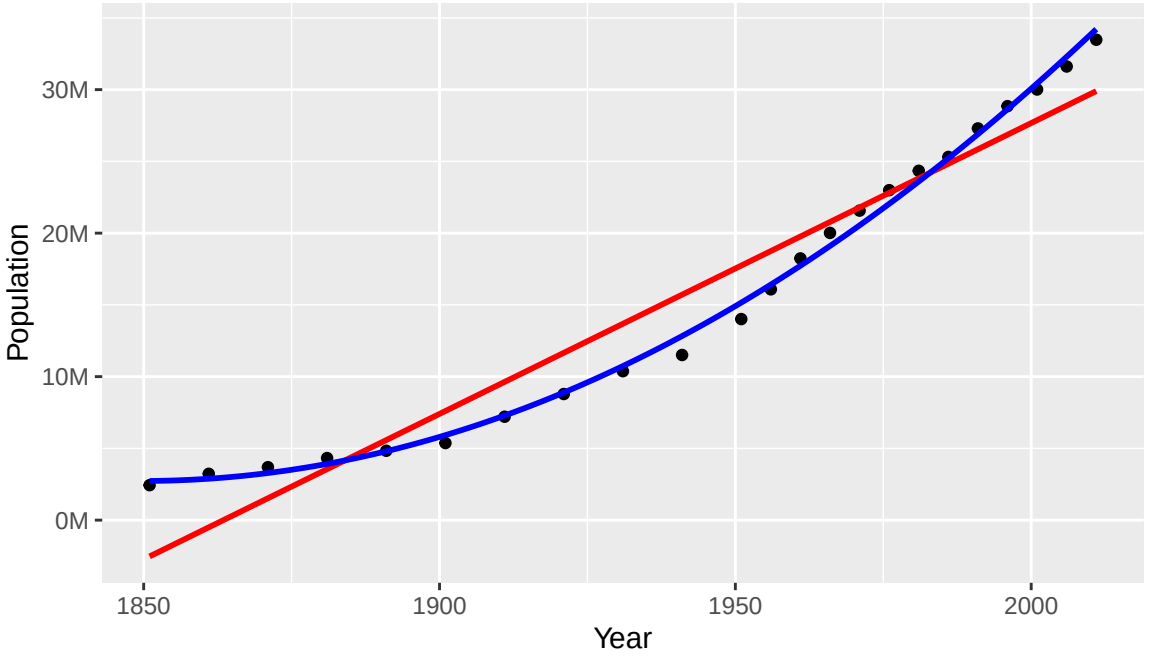
```
y_fitted_quad = x_tilde[1] +
  x_tilde[2] * (x_axis - min(canada$year)) +
  x_tilde[3] * (x_axis - min(canada$year))^2

plot(canada, ylim = c(min(y_fitted), max(canada$population)),
xlab = "Year", ylab = "Population")
lines(x_axis, y_fitted, col = "red", lwd = 2) # Linear
lines(x_axis, y_fitted_quad, col = "blue", lwd = 2) # Quadratic
```


Population
0.0e+00
2.0e+07
1850
1900
1950
2000
Year

Visualizing the quadratic fit (ggplot2)

```
df_fit <- data.frame(year = x_axis,
                     linear = y_fitted,
                     quad = y_fitted_quad)

ggplot() +
  geom_point(data = canada, aes(x = year, y = population)) +
  geom_line(data = df_fit, aes(x = year, y = linear), color = "red", linewic
  geom_line(data = df_fit, aes(x = year, y = quad), color = "blue", linewidt
  scale_y_continuous(labels = function(x) paste0(x / 1e6, "M")) +
  labs(x = "Year", y = "Population")
```

`ggplot2` is a graphic package providing a “grammar of graphics”. Nicer output than default `R` plots, but also harder (LLMs help a lot here)


30M
20M
10M
0M
Population
1850
1900
1950
2000
Year

Takeaways

- Matrix formulations of least squares give us a straightforward way to solve polynomial curve fitting problems
- By adding columns corresponding to higher powers (or other transformations), we can easily expand the model
- **Numerical stability is crucial**: always be careful when dealing with polynomials of large numbers. Shifting and scaling your data can prevent matrices from becoming ill-conditioned
- You could shift to the mean or even $[-1, 1]$. Just remember to "unshift" back the years

QR factorisation & SVD

MATH 2740 – Mathematics of Data Science – Unit U04

Julien Arino
julien.arino@umanitoba.ca

Department of Mathematics @ University of Manitoba

Fall 2026

The University of Manitoba campuses are located on original lands of Anishinaabeg, Ininew, Anisininew, Dakota and Dene peoples, and on the National Homeland of the Red River Métis. We respect the Treaties that were made on these territories, we acknowledge the harms and mistakes of the past, and we dedicate ourselves to move forward in partnership with Indigenous communities in a spirit of Reconciliation and collaboration.

Outline

The QR decomposition

The SVD (singular values decomposition)

Matrix factorisations

Matrix factorisations are popular because they allow to perform some computations more easily

There are several different types of factorisations. Here, we study just the QR factorisation (useful for many least squares problems) and SVD (useful in many contexts)

The QR decomposition

The SVD (singular values decomposition)

The QR decomposition

Orthogonality and projections

The Gram-Schmidt orthogonalisation process

Orthogonal matrices

The QR factorisation & Least squares

Definition 57 (Orthogonal set of vectors)

The set of vectors $\{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_k\} \in \mathbb{R}^n$ *is an* ***orthogonal set*** *if*

$$\forall i, j = 1, \ldots, k, \quad i \neq j \implies \boldsymbol{v}_i \bullet \boldsymbol{v}_j = 0$$

Theorem 58

$\{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_k\} \in \mathbb{R}^n$ *with* $\forall i$, $\boldsymbol{v}_i \neq \boldsymbol{0}$, *orthogonal set* $\implies$ $\{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_k\} \in \mathbb{R}^n$ *linearly independent*

Definition 59 (Orthogonal basis)

Let S be a basis of the subspace $W \subset \mathbb{R}^n$ composed of an orthogonal set of vectors. We say S is an **orthogonal basis** of W

Proof of Theorem 58

Assume $\{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_k\}$ orthogonal set with $\boldsymbol{v}_i \neq \boldsymbol{0}$ for all $i = 1, \ldots, k$. Recall $\{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_k\}$ is LI if

$$c_1\boldsymbol{v}_1 + \cdots + c_k\boldsymbol{v}_k = \boldsymbol{0} \iff c_1 = \cdots = c_k = 0$$

So assume $c_1, \ldots, c_k \in \mathbb{R}$ are s.t. $c_1\boldsymbol{v}_1 + \cdots + c_k\boldsymbol{v}_k = \boldsymbol{0}$. Recall that $\forall \boldsymbol{x} \in \mathbb{R}^k$, $\boldsymbol{0}_k \bullet \boldsymbol{x} = 0$. So for some $\boldsymbol{v}_i \in \{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_k\}$

$$\begin{aligned} 0 &= \boldsymbol{0} \bullet \boldsymbol{v}_i \\ &= (c_1\boldsymbol{v}_1 + \cdots + c_k\boldsymbol{v}_k) \bullet \boldsymbol{v}_i \\ &= c_1\boldsymbol{v}_1 \bullet \boldsymbol{v}_i + \cdots + c_k\boldsymbol{v}_k \bullet \boldsymbol{v}_i \end{aligned} \tag{1}$$

As $\{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_k\}$ orthogonal, $\boldsymbol{v}_j \bullet \boldsymbol{v}_i = 0$ when $i \neq j$, (1) reduces to

$$c_i\boldsymbol{v}_i \bullet \boldsymbol{v}_i = 0 \iff c_i\|\boldsymbol{v}_i\|^2 = 0$$

As $\boldsymbol{v}_i \neq 0$ for all i, $\|\boldsymbol{v}_i\| \neq 0$ and so $c_i = 0$. This is true for all i, hence the result □

Example – Vectors of the standard basis of $\mathbb{R}^3$

For $\mathbb{R}^3$, we denote

$$\boldsymbol{i} = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}, \quad \boldsymbol{j} = \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix} \text{ and } \boldsymbol{k} = \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}$$

($\mathbb{R}^k$ for $k > 3$, we denote them $\boldsymbol{e}_i$)
Clearly, $\{\boldsymbol{i}, \boldsymbol{j}\}$, $\{\boldsymbol{i}, \boldsymbol{k}\}$, $\{\boldsymbol{j}, \boldsymbol{k}\}$ and $\{\boldsymbol{i}, \boldsymbol{j}, \boldsymbol{k}\}$ orthogonal sets. The standard basis vectors are also $\neq \mathbf{0}$, so the sets are LI. And

$$\{\boldsymbol{i}, \boldsymbol{j}, \boldsymbol{k}\}$$

is an orthogonal basis of $\mathbb{R}^3$ since it spans $\mathbb{R}^3$ and is LI

$$c_1\boldsymbol{i} + c_2\boldsymbol{j} + c_3\boldsymbol{k} = c_1 \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix} + c_2 \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix} + c_3 \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix} = \begin{pmatrix} c_1 \\ c_2 \\ c_3 \end{pmatrix}$$

Orthonormal version of things

Definition 60 (Orthonormal set)

The set of vectors $\{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_k\} \in \mathbb{R}^n$ *is an* ***orthonormal set*** *if it is an orthogonal set and furthermore*

$$\forall i = 1, \ldots, k, \quad \|\boldsymbol{v}_i\| = 1$$

Definition 61 (Orthonormal basis)

A basis of the subspace $W \subset \mathbb{R}^n$ *is an* ***orthonormal basis*** *if the vectors composing it are an orthonormal set*

$\{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_k\} \in \mathbb{R}^n$ is orthonormal if

$$\boldsymbol{v}_i \bullet \boldsymbol{v}_j = \begin{cases} 1 & \text{if } i = j \\ 0 & \text{otherwise} \end{cases}$$

The QR decomposition

Orthogonality and projections

The Gram-Schmidt orthogonalisation process

Orthogonal matrices

The QR factorisation & Least squares

What this aims to do is to construct an orthogonal basis for a subspace $W \subset \mathbb{R}^n$

To do this, we use the *Gram-Schmidt orthogonalisation process*, which turn s a basis of W into an orthogonal basis of W

Projections

Definition 62 (Orthogonal projection onto a subspace)

*$W \subset \mathbb{R}^n$ a subspace and $\{\boldsymbol{u}_1, \ldots, \boldsymbol{u}_k\}$ an orthogonal basis of W. $\forall \boldsymbol{v} \in \mathbb{R}^n$, the **orthogonal projection** of **v onto** W is*

$$\text{proj}_W(\boldsymbol{v}) = \frac{\boldsymbol{u}_1 \bullet \boldsymbol{v}}{\|\boldsymbol{u}_1\|^2}\boldsymbol{u}_1 + \cdots + \frac{\boldsymbol{u}_k \bullet \boldsymbol{v}}{\|\boldsymbol{u}_k\|^2}\boldsymbol{u}_k$$

Definition 63 (Component orthogonal to a subspace)

*$W \subset \mathbb{R}^n$ a subspace and $\{\boldsymbol{u}_1, \ldots, \boldsymbol{u}_k\}$ an orthogonal basis of W. $\forall \boldsymbol{v} \in \mathbb{R}^n$, the **component** of **v orthogonal to** W is*

$$\text{perp}_W(\boldsymbol{v}) = \boldsymbol{v} - \text{proj}_W(\boldsymbol{v})$$

Theorem 64 (Gram-Schmidt orthogonalisation process)

Let W be a subspace of $\mathbb{R}^n$ and $\{\boldsymbol{x}_1, \ldots, \boldsymbol{x}_k\}$ be a basis of W. Let

$$
\begin{aligned}
\boldsymbol{v}_1 &= \boldsymbol{x}_1 \\
\boldsymbol{v}_2 &= \boldsymbol{x}_2 - \frac{\boldsymbol{v}_1 \bullet \boldsymbol{x}_2}{\|\boldsymbol{v}_1\|^2}\boldsymbol{v}_1 \\
\boldsymbol{v}_3 &= \boldsymbol{x}_3 - \frac{\boldsymbol{v}_1 \bullet \boldsymbol{x}_3}{\|\boldsymbol{v}_1\|^2}\boldsymbol{v}_1 - \frac{\boldsymbol{v}_2 \bullet \boldsymbol{x}_3}{\|\boldsymbol{v}_2\|^2}\boldsymbol{v}_2 \\
&\vdots \\
\boldsymbol{v}_k &= \boldsymbol{x}_k - \frac{\boldsymbol{v}_1 \bullet \boldsymbol{x}_k}{\|\boldsymbol{v}_1\|^2}\boldsymbol{v}_1 - \cdots - \frac{\boldsymbol{v}_{k-1} \bullet \boldsymbol{x}_k}{\|\boldsymbol{v}_{k-1}\|^2}\boldsymbol{v}_{k-1}
\end{aligned}
$$

and

$$W_1 = \operatorname{span}(\boldsymbol{x}_1), W_2 = \operatorname{span}(\boldsymbol{x}_1, \boldsymbol{x}_2), \ldots, W_k = \operatorname{span}(\boldsymbol{x}_1, \ldots, \boldsymbol{x}_k)$$

Then $\forall i = 1, \ldots, k$, $\{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_i\}$ is an orthogonal basis of W_i

Theorem 65 (Gram-Schmidt orthonormalisation process)

Let W be a subspace of $\mathbb{R}^n$ and $\{\boldsymbol{x}_1, \ldots, \boldsymbol{x}_k\}$ be a basis of W. Let

$$\tilde{\boldsymbol{v}}_1 = \boldsymbol{x}_1 / \|\boldsymbol{x}_1\|$$

$$\boldsymbol{v}_2 = \boldsymbol{x}_2 - (\tilde{\boldsymbol{v}}_1 \bullet \boldsymbol{x}_2)\tilde{\boldsymbol{v}}_1 \qquad \tilde{\boldsymbol{v}}_2 = \frac{\boldsymbol{v}_2}{\|\boldsymbol{v}_2\|}$$

$$\boldsymbol{v}_3 = \boldsymbol{x}_3 - (\tilde{\boldsymbol{v}}_1 \bullet \boldsymbol{x}_3)\tilde{\boldsymbol{v}}_1 - (\tilde{\boldsymbol{v}}_2 \bullet \boldsymbol{x}_3)\tilde{\boldsymbol{v}}_2 \qquad \tilde{\boldsymbol{v}}_3 = \frac{\boldsymbol{v}_3}{\|\boldsymbol{v}_3\|}$$

$$\vdots$$

$$\boldsymbol{v}_k = \boldsymbol{x}_k - (\tilde{\boldsymbol{v}}_1 \bullet \boldsymbol{x}_k)\tilde{\boldsymbol{v}}_1 - \cdots - (\tilde{\boldsymbol{v}}_{k-1} \bullet \boldsymbol{x}_k)\tilde{\boldsymbol{v}}_{k-1} \qquad \tilde{\boldsymbol{v}}_k = \frac{\boldsymbol{v}_k}{\|\boldsymbol{v}_k\|}$$

and

$$W_1 = \operatorname{span}(\boldsymbol{x}_1), W_2 = \operatorname{span}(\boldsymbol{x}_1, \boldsymbol{x}_2), \ldots, W_k = \operatorname{span}(\boldsymbol{x}_1, \ldots, \boldsymbol{x}_k)$$

Then $\forall i = 1, \ldots, k$, $\{\tilde{\boldsymbol{v}}_1, \ldots, \tilde{\boldsymbol{v}}_i\}$ is an orthonormal basis of W_i

Links between the two processes

- To get an orthonormal basis, you can also use the orthogonalisation process then normalise the result

- There is no difference between the two methods, the only gain in the orthonormalisation process is that you don't need to divide by $\|\cdot\|^2$ when projecting, since you have already done so just before

The QR decomposition

- Orthogonality and projections
- The Gram-Schmidt orthogonalisation process
- **Orthogonal matrices**
- The QR factorisation & Least squares

Theorem 66

Let $Q \in \mathcal{M}_{np}$. The columns of Q form an orthonormal set if and only if

$$Q^T Q = \mathbb{I}_p$$

Definition 67 (Orthogonal matrix)

*$Q \in \mathcal{M}_n$ is an **orthogonal matrix** if its columns form an orthonormal set*

So $Q \in \mathcal{M}_n$ orthogonal if $Q^T Q = \mathbb{I}$, i.e., $Q^T = Q^{-1}$

Theorem 68 (NSC for orthogonality)

$Q \in \mathcal{M}_n$ orthogonal $\iff$ $Q^{-1} = Q^T$

Orthogonal matrices “encode" isometries

Theorem 69 (Orthogonal matrices “encode" isometries)

Let $Q \in \mathcal{M}_n$. TFAE

1. *Q orthogonal*
2. $\forall \boldsymbol{x} \in \mathbb{R}^n$, $\|Q\boldsymbol{x}\| = \|\boldsymbol{x}\|$
3. $\forall \boldsymbol{x}, \boldsymbol{y} \in \mathbb{R}^n$, $Q\boldsymbol{x} \bullet Q\boldsymbol{y} = \boldsymbol{x} \bullet \boldsymbol{y}$

Proof of Theorem 69

1 $\implies$ **3**: Assume Q is orthogonal. Then $Q^TQ = \mathbb{I}_n$, so

$$Q\boldsymbol{x} \bullet Q\boldsymbol{y} = (Q\boldsymbol{x})^T(Q\boldsymbol{y}) = \boldsymbol{x}^TQ^TQ\boldsymbol{y} = \boldsymbol{x}^T\mathbb{I}_n\boldsymbol{y} = \boldsymbol{x} \bullet \boldsymbol{y}$$

3 $\implies$ **1**: Assume $Q\boldsymbol{x} \bullet Q\boldsymbol{y} = \boldsymbol{x} \bullet \boldsymbol{y}\ \forall \boldsymbol{x}, \boldsymbol{y}$. For standard basis vectors $\boldsymbol{e}_i, \boldsymbol{e}_j$:

$$(Q\boldsymbol{e}_i) \bullet (Q\boldsymbol{e}_j) = \boldsymbol{e}_i \bullet \boldsymbol{e}_j = \begin{cases} 1 & \text{if } i = j \\ 0 & \text{if } i \neq j \end{cases}$$

Since $Q\boldsymbol{e}_i$ is the i-th column of Q, the columns of Q are orthonormal, so Q is orthogonal.

3 $\implies$ **2**: Assume $Q\boldsymbol{x} \bullet Q\boldsymbol{y} = \boldsymbol{x} \bullet \boldsymbol{y}$. Letting $\boldsymbol{y} = \boldsymbol{x}$ gives

$$\|Q\boldsymbol{x}\|^2 = Q\boldsymbol{x} \bullet Q\boldsymbol{x} = \boldsymbol{x} \bullet \boldsymbol{x} = \|\boldsymbol{x}\|^2 \implies \|Q\boldsymbol{x}\| = \|\boldsymbol{x}\|$$

2 $\implies$ **3**: Assume $\|Q\boldsymbol{z}\| = \|\boldsymbol{z}\|$ for all $\boldsymbol{z}$. Using $\boldsymbol{z} = \boldsymbol{x} + \boldsymbol{y}$, we expand $\|Q(\boldsymbol{x} + \boldsymbol{y})\|^2 = \|\boldsymbol{x} + \boldsymbol{y}\|^2$:

$$\|Q\boldsymbol{x}\|^2 + 2Q\boldsymbol{x} \bullet Q\boldsymbol{y} + \|Q\boldsymbol{y}\|^2 = \|\boldsymbol{x}\|^2 + 2\boldsymbol{x} \bullet \boldsymbol{y} + \|\boldsymbol{y}\|^2$$

Since $\|Q\boldsymbol{x}\| = \|\boldsymbol{x}\|$ and $\|Q\boldsymbol{y}\| = \|\boldsymbol{y}\|$, we get $Q\boldsymbol{x} \bullet Q\boldsymbol{y} = \boldsymbol{x} \bullet \boldsymbol{y}$. □

Theorem 70

Let $Q \in \mathcal{M}_n$ be orthogonal. Then

1. *The rows of Q form an orthonormal set*
2. *Q^{-1} orthogonal*
3. $\det Q = \pm 1$
4. $\forall \lambda \in \sigma(Q)$, $|\lambda| = 1$
5. *If $Q_2 \in \mathcal{M}_n$ also orthogonal, then QQ_2 orthogonal*

Proof of 1 and 2 in Theorem 70

1. The rows of Q form an orthonormal set

- By definition, Q is orthogonal $\iff Q^T Q = \mathbb{I}_n$
- Since Q is square, $Q^T Q = \mathbb{I}_n \implies Q^{-1} = Q^T \implies QQ^T = \mathbb{I}_n$
- This is equivalent to $(Q^T)^T Q^T = \mathbb{I}_n$, so the columns of Q^T (the rows of Q) are orthonormal □

2. Q^{-1} is orthogonal

- We must show that $(Q^{-1})^T Q^{-1} = \mathbb{I}_n$
- Using $Q^{-1} = Q^T$, we get $(Q^{-1})^T Q^{-1} = (Q^T)^T Q^T = QQ^T = \mathbb{I}_n$
- Thus Q^{-1} is an orthogonal matrix □

Proof of 3 and 5 in Theorem 70

3. $\det Q = \pm 1$

- We have $Q^T Q = \mathbb{I}_n$. Taking determinant on both sides

$$\det(Q^T Q) = \det(\mathbb{I}_n) = 1$$

- Using $\det(AB) = \det(A)\det(B)$, we get $\det(Q^T)\det(Q) = 1$
- From the general property $\det(A^T) = \det(A)$, it follows that $(\det(Q))^2 = 1$, so $\det Q = \pm 1$ □

5. If $Q_2 \in \mathcal{M}_n$ also orthogonal, then QQ_2 orthogonal

- We must show that $(QQ_2)^T(QQ_2) = \mathbb{I}_n$
- $(QQ_2)^T(QQ_2) = Q_2^T Q^T Q Q_2 = Q_2^T (Q^T Q) Q_2$
- Since Q is orthogonal, $Q^T Q = \mathbb{I}_n$, so the expression becomes $Q_2^T \mathbb{I}_n Q_2 = Q_2^T Q_2$
- Since Q_2 is orthogonal, $Q_2^T Q_2 = \mathbb{I}_n$ □

Proof of 4 in Theorem 70

4. $\forall \lambda \in \sigma(Q)$**,** $|\lambda| = 1$

Let λ be an eigenvalue of $Q \in \mathcal{M}_n$ orthogonal, i.e., $\exists \mathbb{R}^n \ni \boldsymbol{x} \neq \mathbf{0}$ s.t.

$$Q\boldsymbol{x} = \lambda \boldsymbol{x}$$

Take the norm on both sides

$$\|Q\boldsymbol{x}\| = \|\lambda \boldsymbol{x}\|$$

From 2 in Theorem 69, $\|Q\boldsymbol{x}\| = \|\boldsymbol{x}\|$ and from the properties of norms, $\|\lambda \boldsymbol{x}\| = |\lambda| \, \|\boldsymbol{x}\|$, so we have

$$\|Q\boldsymbol{x}\| = \|\lambda \boldsymbol{x}\| \iff \|\boldsymbol{x}\| = |\lambda| \, \|\boldsymbol{x}\| \iff 1 = |\lambda|$$

(we can divide by $\|\boldsymbol{x}\|$ since $\boldsymbol{x} \neq \mathbf{0}$ as an eigenvector) □

The QR decomposition

- Orthogonality and projections
- The Gram-Schmidt orthogonalisation process
- Orthogonal matrices
- **The QR factorisation & Least squares**

The QR factorisation

Theorem 71

Let $A \in \mathcal{M}_{np}$ with LI columns. Then A can be factored as

$$A = QR$$

where $Q \in \mathcal{M}_{np}$ has orthonormal columns and $R \in \mathcal{M}_p$ is nonsingular upper triangular

Back to least squares

So what was the point of all that..?

Theorem 72 (Least squares with QR factorisation)

$A \in \mathcal{M}_{np}$ with linearly independent columns, $\boldsymbol{b} \in \mathbb{R}^n$. If $A = QR$ is a QR factorisation of A, then the unique least squares solution $\tilde{\boldsymbol{x}}$ of $A\boldsymbol{x} = \boldsymbol{b}$ is

$$\tilde{\boldsymbol{x}} = R^{-1} Q^T \boldsymbol{b}$$

Proof of Theorem 72

A has LI columns so

- least squares $A\boldsymbol{x} = \boldsymbol{b}$ has unique solution $\tilde{\boldsymbol{x}} = (A^T A)^{-1} A^T \boldsymbol{b}$
- by Theorem 71, A can be written as $A = QR$ with $Q \in \mathcal{M}_{np}$ with orthonormal columns and $R \in \mathcal{M}_p$ nonsingular and upper triangular

So

$$
\begin{aligned}
A^T A\tilde{\boldsymbol{x}} = A^T \boldsymbol{b} &\implies (QR)^T QR\tilde{\boldsymbol{x}} = (QR)^T \boldsymbol{b} \\
&\implies R^T Q^T QR\tilde{\boldsymbol{x}} = R^T Q^T \boldsymbol{b} \\
&\implies R^T \mathbb{I}_p R\tilde{\boldsymbol{x}} = R^T Q^T \boldsymbol{b} \\
&\implies R^T R\tilde{\boldsymbol{x}} = R^T Q^T \boldsymbol{b} \\
&\implies (R^T)^{-1} R\tilde{\boldsymbol{x}} = (R^T)^{-1} R^T Q^T \boldsymbol{b} \\
&\implies R\tilde{\boldsymbol{x}} = Q^T \boldsymbol{b} \\
&\implies \tilde{\boldsymbol{x}} = R^{-1} Q^T \boldsymbol{b} \quad \square
\end{aligned}
$$

Other applications of the QR decomposition

Focus here on least squares, but QR decomposition fundamental tool in numerical linear algebra with other important applications

- **Solving linear systems**: to solve $A\boldsymbol{x} = \boldsymbol{b}$ for a non-singular square matrix A, write $QR\boldsymbol{x} = \boldsymbol{b} \implies R\boldsymbol{x} = Q^T\boldsymbol{b}$. Since R is upper triangular, this can be quickly solved by back-substitution. This method is often more numerically stable than Gaussian elimination (called the LU decomposition)

- **Computing eigenvalues**: the **QR algorithm** is a standard method for calculating the eigenvalues of a matrix. It uses the iterative process $A_k = Q_k R_k$ and $A_{k+1} = R_k Q_k$. Under certain conditions, A_k converges to an upper triangular matrix whose diagonal entries are the eigenvalues

- **Computing determinants**: if $A = QR$, then $\det(A) = \det(Q)\det(R)$. Since Q is orthogonal, $\det(Q) = \pm 1$, and since R is upper triangular, $\det(R)$ is simply the product of its diagonal entries

Example – Solving least squares with QR (1/5)

Consider the matrix A and a vector $\boldsymbol{b}$:

$$A = \begin{pmatrix} 1 & 1 \\ 1 & 0 \\ 0 & 1 \end{pmatrix} \quad \text{and} \quad \boldsymbol{b} = \begin{pmatrix} 2 \\ 3 \\ 4 \end{pmatrix}.$$

We want to find the unique least squares solution $\tilde{\boldsymbol{x}}$ to $A\boldsymbol{x} = \boldsymbol{b}$.

To do this using the QR decomposition, we must first compute Q and R.

Example – Solving least squares with QR (2/5)

We compute Q by applying the Gram-Schmidt process to the columns of A: $\boldsymbol{a}_1 = (1, 1, 0)$ and $\boldsymbol{a}_2 = (1, 0, 1)$.

First, we find the orthogonal vectors:

$$\boldsymbol{v}_1 = \boldsymbol{a}_1 = (1, 1, 0).$$

$$\boldsymbol{v}_2 = \boldsymbol{a}_2 - \frac{\boldsymbol{v}_1 \bullet \boldsymbol{a}_2}{\|\boldsymbol{v}_1\|^2}\boldsymbol{v}_1 = (1, 0, 1) - \frac{1}{2}(1, 1, 0) = \left(\frac{1}{2}, -\frac{1}{2}, 1\right).$$

Then, we normalise them to obtain the orthonormal columns of Q:

$$\boldsymbol{u}_1 = \frac{\boldsymbol{v}_1}{\|\boldsymbol{v}_1\|} = \left(\frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}}, 0\right).$$

$$\boldsymbol{u}_2 = \frac{\boldsymbol{v}_2}{\|\boldsymbol{v}_2\|} = \sqrt{\frac{2}{3}}\left(\frac{1}{2}, -\frac{1}{2}, 1\right) = \left(\frac{1}{\sqrt{6}}, -\frac{1}{\sqrt{6}}, \frac{2}{\sqrt{6}}\right).$$

Example – Solving least squares with QR (3/5)

Thus, the orthogonal matrix Q is:

$$Q = \begin{pmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{6}} \\ \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{6}} \\ 0 & \frac{2}{\sqrt{6}} \end{pmatrix}.$$

To find R, recall that $A = QR$. Since Q has orthonormal columns, $Q^T Q = \mathbb{I}$. Thus:

$$Q^T A = Q^T QR = \mathbb{I}R = R.$$

We can therefore compute R directly as $Q^T A$:

$$R = \begin{pmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} & 0 \\ \frac{1}{\sqrt{6}} & -\frac{1}{\sqrt{6}} & \frac{2}{\sqrt{6}} \end{pmatrix} \begin{pmatrix} 1 & 1 \\ 1 & 0 \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} \frac{2}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ 0 & \frac{3}{\sqrt{6}} \end{pmatrix}.$$

Example – Solving least squares with QR (4/5)

By Theorem 72, we know that $R\tilde{\boldsymbol{x}} = Q^T\boldsymbol{b}$. First, we compute $Q^T\boldsymbol{b}$:

$$Q^T\boldsymbol{b} = \begin{pmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} & 0 \\ \frac{1}{\sqrt{6}} & -\frac{1}{\sqrt{6}} & \frac{2}{\sqrt{6}} \end{pmatrix} \begin{pmatrix} 2 \\ 3 \\ 4 \end{pmatrix} = \begin{pmatrix} \frac{5}{\sqrt{2}} \\ \frac{7}{\sqrt{6}} \end{pmatrix}.$$

Now we solve the upper triangular system $R\tilde{\boldsymbol{x}} = Q^T\boldsymbol{b}$ via back-substitution:

$$\begin{pmatrix} \frac{2}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ 0 & \frac{3}{\sqrt{6}} \end{pmatrix} \begin{pmatrix} \tilde{x}_1 \\ \tilde{x}_2 \end{pmatrix} = \begin{pmatrix} \frac{5}{\sqrt{2}} \\ \frac{7}{\sqrt{6}} \end{pmatrix}.$$

The second equation gives $\frac{3}{\sqrt{6}}\tilde{x}_2 = \frac{7}{\sqrt{6}}$, which implies $\tilde{x}_2 = \frac{7}{3}$.

Example – Solving least squares with QR (5/5)

Substituting this into the first equation yields $\frac{2}{\sqrt{2}}\tilde{x}_1 + \frac{1}{\sqrt{2}}\left(\frac{7}{3}\right) = \frac{5}{\sqrt{2}}$, which simplifies to $2\tilde{x}_1 + \frac{7}{3} = 5$. Solving for $\tilde{x}_1$ yields $\tilde{x}_1 = \frac{4}{3}$.

Thus, the least squares solution is $\tilde{\boldsymbol{x}} = \begin{pmatrix} 4/3 \\ 7/3 \end{pmatrix}$.

We can easily confirm this result in R:

```
b <- c(2, 3, 4)
QT_b <- t(Q_A) %*% b
x_tilde <- solve(R_A, QT_b)
x_tilde
```

```
##          [,1]
## [1,] 1.333333
## [2,] 2.333333
```

```
qr.coef(qr_A, b) # with the built-in solver
```

```
## [1] 1.333333 2.333333
```

Fitting the Canada census data using QR

Let us return to the Canada census data we fitted earlier using the normal equations.

We saw that computing $A^T A$ and its inverse for a quadratic fit was problematic when using the raw years, because the matrix A was badly conditioned.

This led us to shift the x-axis to start at $t = 0$.

The QR decomposition provides a much more numerically stable way to solve the least squares problem $A\boldsymbol{x} = \boldsymbol{b}$.

To illustrate this, let us try to fit the quadratic model on the unshifted raw data using the QR approach.

Setup for unshifted quadratic fit

We first load the data and set up our matrix A for a quadratic fit without shifting the years.

```
data <- read.csv("../DATA/Canada_census.csv")
data <- data[which(data$year <= 1980),]
x <- data$year
y <- data$population
A_q <- matrix(c(rep(1, length(x)), x, x^2), nr = length(x), nc = 3)
```

Normal equations failure

Recall that using the normal equations on this matrix failed because $A^T A$ is computationally singular. We can see this in R:

```
ATA <- t(A_q) %*% A_q
try(solve(ATA) %*% t(A_q) %*% y)

## Error in solve.default(ATA) :
##   system is computationally singular: reciprocal condition number = 9.166
```

However, the QR decomposition avoids forming $A^T A$, preserving numerical stability.

Solving with QR decomposition

We can compute the QR decomposition using R's built-in `qr` function, extract the Q and R components, and then use `backsolve` to solve the upper triangular system $R\boldsymbol{x} = Q^T\boldsymbol{b}$.

```
# Compute the QR decomposition of A_q
QR <- qr(A_q)
Q <- qr.Q(QR)
R <- qr.R(QR)

# Solve Rx = Q^T y
sol_qr <- backsolve(R, t(Q) %*% y)
sol_qr
```

```
##                [,1]
## [1,]  5.133645e+09
## [2,] -5.513814e+06
## [3,]  1.481439e+03
```

Direct solution with `qr.solve`

The QR decomposition successfully finds the coefficients for the unshifted data!

Alternatively, R provides the function `qr.solve` which directly solves the least squares problem using the QR decomposition under the hood:

```
sol_qr_direct <- qr.solve(A_q, y)
sol_qr_direct
```

```
## [1]  5.133645e+09 -5.513814e+06  1.481439e+03
```

This demonstrates the practical advantage of using orthogonal matrix factorisations in data science applications over the normal equations.

The QR decomposition

The SVD (singular values decomposition)

The SVD (singular values decomposition)

Singular values
The SVD
An application of the SVD – Image compression
Computing the SVD
Applications of the SVD – Least squares
Summary – Least squares methods

Matrix factorisations (continued)

The singular value decomposition (known mostly by its acronym, SVD) is yet another type of factorisation/decomposition..

Singular values

Definition 73 (Singular value)

*Let $A \in \mathcal{M}_{np}(\mathbb{R})$. The **singular values** of A are the real numbers*

$$\sigma_1 \geq \sigma_2 \geq \cdots \sigma_p \geq 0$$

that are the square roots of the eigenvalues of $A^T A$

Singular values are real and nonnegative?

Recall that $\forall A \in \mathcal{M}_{np}$, $A^T A$ is symmetric

Theorem 74

Let $A \in \mathcal{M}_n(\mathbb{R})$ be symmetric. Then all eigenvalues of A are real

Proof of Theorem 74

$A \in \mathcal{M}_n(\mathbb{R})$ symmetric and $(\lambda, \boldsymbol{v})$ eigenpair of A, i.e, $A\boldsymbol{v} = \lambda \boldsymbol{v}$. Taking the complex conjugate, $\overline{A\boldsymbol{v}} = \overline{\lambda \boldsymbol{v}}$

Since $A \in \mathcal{M}_n(\mathbb{R})$, $\overline{A} = A$ $\qquad (z = \bar{z} \iff z \in \mathbb{R})$

So

$$A\bar{\boldsymbol{v}} = \overline{A}\bar{\boldsymbol{v}} = \overline{A\boldsymbol{v}} = \overline{\lambda \boldsymbol{v}} = \overline{\lambda}\bar{\boldsymbol{v}}$$

i.e., if $(\lambda, \boldsymbol{v})$ eigenpair, $(\bar{\lambda}, \bar{\boldsymbol{v}})$ also eigenpair

Proof of Theorem 74 (cont.)

Still assuming $A \in \mathcal{M}_n(\mathbb{R})$ symmetric and $(\lambda, \boldsymbol{v})$ eigenpair of A and using what we just proved (that $(\bar{\lambda}, \bar{\boldsymbol{v}})$ also eigenpair), take transposes

$$\begin{aligned} A\bar{\boldsymbol{v}} = \bar{\lambda}\bar{\boldsymbol{v}} &\iff (A\bar{\boldsymbol{v}})^T = (\bar{\lambda}\bar{\boldsymbol{v}})^T \\ &\iff \bar{\boldsymbol{v}}^T A^T = \bar{\lambda}\bar{\boldsymbol{v}}^T \\ &\iff \bar{\boldsymbol{v}}^T A = \bar{\lambda}\bar{\boldsymbol{v}}^T \qquad [A \text{ symmetric}] \end{aligned}$$

Let us now compute $\lambda(\bar{\boldsymbol{v}} \bullet \boldsymbol{v})$. We have

$$\begin{aligned} \lambda(\bar{\boldsymbol{v}} \bullet \boldsymbol{v}) &= \lambda\bar{\boldsymbol{v}}^T\boldsymbol{v} = \bar{\boldsymbol{v}}^T(\lambda\boldsymbol{v}) \\ &= \bar{\boldsymbol{v}}^T(A\boldsymbol{v}) = (\bar{\boldsymbol{v}}^T A)\boldsymbol{v} \\ &= (\bar{\lambda}\bar{\boldsymbol{v}}^T)\boldsymbol{v} = \bar{\lambda}(\bar{\boldsymbol{v}} \bullet \boldsymbol{v}) \\ &\iff (\lambda - \bar{\lambda})(\bar{\boldsymbol{v}} \bullet \boldsymbol{v}) = 0 \end{aligned}$$

Proof of Theorem 74 (end)

We have shown

$$(\lambda - \bar{\lambda})(\bar{\boldsymbol{v}} \bullet \boldsymbol{v}) = 0$$

Let

$$\boldsymbol{v} = \begin{pmatrix} a_1 + ib_1 \\ \vdots \\ a_n + ib_n \end{pmatrix}$$

Then

$$\bar{\boldsymbol{v}} = \begin{pmatrix} a_1 - ib_1 \\ \vdots \\ a_n - ib_n \end{pmatrix}$$

So

$$\bar{\boldsymbol{v}} \bullet \boldsymbol{v} = (a_1^2 + b_1^2) + \cdots + (a_n^2 + b_n^2)$$

But $\boldsymbol{v}$ eigenvector is $\neq \boldsymbol{0}$, so $\bar{\boldsymbol{v}} \bullet \boldsymbol{v} \neq 0$, so

$$(\lambda - \bar{\lambda})(\bar{\boldsymbol{v}} \bullet \boldsymbol{v}) = 0 \iff \lambda - \bar{\lambda} = 0 \iff \lambda = \bar{\lambda} \iff \lambda \in \mathbb{R} \quad \square$$

Eigenvalues of $A^T A$ are real and nonnegative

Theorem 75

For $A \in \mathcal{M}_{np}(\mathbb{R})$, the eigenvalues of $A^T A$ (and AA^T) are real and nonnegative

Proof of Theorem 75

We know that for $A \in \mathcal{M}_{np}$, A^TA symmetric and from Theorem 74, if $A \in \mathcal{M}_{np}(\mathbb{R})$, then A^TA is symmetric, real and with real eigenvalues

Let $(\lambda, \boldsymbol{v})$ be an eigenpair of A^TA, with $\boldsymbol{v}$ chosen so that $\|\boldsymbol{v}\| = 1$

Norms are functions $V \to \mathbb{R}_+$, so $\|A\boldsymbol{v}\|$ and $\|A\boldsymbol{v}\|^2$ are ≥ 0 and thus

$$\begin{aligned} 0 \leq \|A\boldsymbol{v}\|^2 &= (A\boldsymbol{v}) \bullet (A\boldsymbol{v}) = (A\boldsymbol{v})^T(A\boldsymbol{v}) \\ &= \boldsymbol{v}^TA^TA\boldsymbol{v} = \boldsymbol{v}^T(A^TA\boldsymbol{v}) = \boldsymbol{v}^T(\lambda\boldsymbol{v}) \\ &= \lambda(\boldsymbol{v}^T\boldsymbol{v}) = \lambda(\boldsymbol{v} \bullet \boldsymbol{v}) = \lambda\|\boldsymbol{v}\|^2 \\ &= \lambda \end{aligned}$$

Nonzero eigenvalues of A^TA and AA^T

Theorem 76

For $A \in \mathcal{M}_{np}(\mathbb{R})$, the nonzero eigenvalues of A^TA and AA^T are the same

Proof.

Let $(\lambda, \boldsymbol{v})$ be an eigenpair of A^TA with $\lambda \neq 0$. Then $\boldsymbol{v} \neq \mathbf{0}$ and

$$A^TA\boldsymbol{v} = \lambda\boldsymbol{v} \neq \mathbf{0}$$

Left multiply by A

$$AA^TA\boldsymbol{v} = \lambda A\boldsymbol{v}$$

Let $\boldsymbol{w} = A\boldsymbol{v}$, we thus have $AA^T\boldsymbol{w} = \lambda\boldsymbol{w}$; in other words, $A\boldsymbol{v}$ is an eigenvector of AA^T corresponding to the (nonzero) eigenvalue λ
The reverse works the same way..

The SVD (singular values decomposition)

Singular values

The SVD

An application of the SVD – Image compression

Computing the SVD

Applications of the SVD – Least squares

Summary – Least squares methods

The singular value decomposition (SVD)

Theorem 77 (SVD)

$A \in \mathcal{M}_{np}$ with singular values $\sigma_1 \geq \cdots \geq \sigma_r > 0$ and $\sigma_{r+1} = \cdots = \sigma_p = 0$

Then there exists $U \in \mathcal{M}_n$ orthogonal, $V \in \mathcal{M}_p$ orthogonal and a block matrix $\Sigma \in \mathcal{M}_{np}$ taking the form

$$\Sigma = \begin{pmatrix} D & \mathbf{0}_{r,p-r} \\ \mathbf{0}_{n-r,r} & \mathbf{0}_{n-r,p-r} \end{pmatrix}$$

where

$$D = \text{diag}(\sigma_1, \ldots, \sigma_r) \in \mathcal{M}_r$$

such that

$$A = U\Sigma V^T$$

Definition 78

*We call a factorisation as in Theorem 77 the **singular value decomposition** of A. The columns of U and V are, respectively, the **left** and **right singular vectors** of A*

U and V^T are *rotation* or *reflection* matrices, Σ is a *scaling* matrix

$U \in \mathcal{M}_n$ orthogonal matrix with columns the eigenvectors of AA^T

$V \in \mathcal{M}_p$ orthogonal matrix with columns the eigenvectors of A^TA

Outer product form of the SVD

Theorem 79 (Outer product form of the SVD)

$A \in \mathcal{M}_{np}$ with singular values $\sigma_1 \geq \cdots \geq \sigma_r > 0$ and $\sigma_{r+1} = \cdots = \sigma_p = 0$, $\boldsymbol{u}_1, \ldots, \boldsymbol{u}_r$ and $\boldsymbol{v}_1, \ldots, \boldsymbol{v}_r$, respectively, left and right singular vectors of A corresponding to these singular values

Then

$$A = \sigma_1 \boldsymbol{u}_1 \boldsymbol{v}_1^T + \cdots + \sigma_r \boldsymbol{u}_r \boldsymbol{v}_r^T \tag{2}$$

The SVD (singular values decomposition)

- Singular values
- The SVD
- **An application of the SVD – Image compression**
- Computing the SVD
- Applications of the SVD – Least squares
- Summary – Least squares methods

Applications of the SVD

Many applications of the SVD, both theoretical and practical..

1. Obtaining a unique solutions to least squares when $A^T A$ singular

2. Image compression

Compressing images

Consider an image (for simplicity, assume in shades of grey). This can be stored in a matrix $A \in \mathcal{M}_{np}$

Take the SVD of A. Then the small singular values carry information about the regions with little variation and can perhaps be omitted, whereas the large singular values carry information about more “dynamic” regions of the image

Suppose A has r nonzero singular values. For $k \leq r$, let

$$A_k = \sigma_1 \boldsymbol{u}_1 \boldsymbol{v}_1^T + \cdots + \sigma_k \boldsymbol{u}_k \boldsymbol{v}_k^T$$

For $k = r$ we get the usual outer product form (2) in Theorem 79

Load the image using `bmp::read.bmp`

```
my_image = bmp::read.bmp("../CODE/Julien_and_friend_1000x800.bmp")
my_image_g = pixmap::pixmapGrey(my_image)
my_image_g
```

```
## Pixmap image
##   Type          : pixmapGrey
##   Size          : 800x1000
##   Resolution    : 1x1
##   Bounding box  : 0 0 1000 800
```

Using built-in functions

We can also use the built-in function `svd` to compute the SVD of M

```
M.svd = svd(M)
```

The results are stored in a list with components `u`, `d` and `v`

Make function to recreate an image from the SVD

Given the SVD `im` of an image and a number of singular values to keep `n`, we can recreate the image using the function `compress_image`

We output the new image, but also, the amount of information required to encode this new image, as a percentage of the original image size

```
compress_image = function(im, n) {
  if (n > length(im$d)) {
    # Check that we gave a value of n within range, otherwise
    # just set to the max
    n = length(im$d)
  }
  d_tmp = im$d[1:n]
  u_tmp = im$u[,1:n]
  v_tmp = im$v[,1:n]
  # Store the results in a list (so we can return other information)
    out = list()
    # First, compute the resulting image
    out$img = mat.or.vec(nr = dim(im$u)[1], nc = dim(im$v)[1])
    for (i in 1:n) {
        out$img = out$img + d_tmp[i] * u_tmp[,i] %*% t(v_tmp[,i])
    }
```

```
    # Values of "colours" must be in [0,1], so shift and rescale
    if (min(min(out$img)) < 0 ) {
        out$img = out$img - min(min(out$img))
    }
    out$img = out$img / max(max(out$img))
    # Store some information: # points needed and % of the original
    out$nb_pixels_original = dim(im$u)[1] * dim(im$v)[2]
    out$nb_pixels_compressed = length(d_tmp) +
      dim(u_tmp)[1]*dim(u_tmp)[2] +
      dim(v_tmp)[1]*dim(v_tmp)[2]
    out$pct_of_original = out$nb_pixels_compressed /
      out$nb_pixels_original * 100
    # Return the result
    return(out)
}
```

Recreating the image

We can now recreate the image using the function `compress_image`

```
new_image = my_image_g
M.svd = svd(M)
M_tmp = compress_image(M.svd, 2)
new_image@grey = M_tmp$img
plot(new_image)
```

Using $n = 2$ singular values

Uses 0.56% of the original information

Using $n = 5$ singular values

Uses 1.41% of the original information

Using $n = 10$ singular values

Uses 2.81% of the original information

Using $n = 20$ singular values

Uses 5.63% of the original information

Using $n = 50$ singular values

Uses 14.07% of the original information

The SVD (singular values decomposition)

Singular values
The SVD
An application of the SVD – Image compression
Computing the SVD
Applications of the SVD – Least squares
Summary – Least squares methods

Computing the SVD (case of $\neq$ eigenvalues)

To compute the SVD, we use the following result

Theorem 80

Let $A \in \mathcal{M}_n$ symmetric, $(\lambda_1, \boldsymbol{u}_1)$ and $(\lambda_2, \boldsymbol{u}_2)$ be eigenpairs, $\lambda_1 \neq \lambda_2$. Then $\boldsymbol{u}_1 \bullet \boldsymbol{u}_2 = 0$

Proof of Theorem 80

$A \in \mathcal{M}_n$ symmetric, $(\lambda_1, \boldsymbol{u}_1)$ and $(\lambda_2, \boldsymbol{u}_2)$ eigenpairs with $\lambda_1 \neq \lambda_2$

$$\begin{aligned}
\lambda_1(\boldsymbol{v}_1 \bullet \boldsymbol{v}_2) &= (\lambda_1 \boldsymbol{v}_1) \bullet \boldsymbol{v}_2 \\
&= A\boldsymbol{v}_1 \bullet \boldsymbol{v}_2 \\
&= (A\boldsymbol{v}_1)^T \boldsymbol{v}_2 \\
&= \boldsymbol{v}_1^T A^T \boldsymbol{v}_2 \\
&= \boldsymbol{v}_1^T (A\boldsymbol{v}_2) \qquad [A \text{ symmetric so } A^T = A] \\
&= \boldsymbol{v}_1^T (\lambda_2 \boldsymbol{v}_2) \\
&= \lambda_2 (\boldsymbol{v}_1^T \boldsymbol{v}_2) \\
&= \lambda_2 (\boldsymbol{v}_1 \bullet \boldsymbol{v}_2)
\end{aligned}$$

So $(\lambda_1 - \lambda_2)(\boldsymbol{v}_1 \bullet \boldsymbol{v}_2) = 0$. But $\lambda_1 \neq \lambda_2$, so $\boldsymbol{v}_1 \bullet \boldsymbol{v}_2 = 0$ □

Computing the SVD (case of $\neq$ eigenvalues)

If all eigenvalues of A^TA (or AA^T) are distinct, we can use Theorem 80

1. Compute $A^TA \in \mathcal{M}_p$
2. Compute eigenvalues $\lambda_1, \ldots, \lambda_p$ of A^TA; order them as $\lambda_1 > \cdots > \lambda_p \geq 0$ ($>$ not $\geq$ since $\neq$)
3. Compute singular values $\sigma_1 = \sqrt{\lambda_1}, \ldots, \sigma_p = \sqrt{\lambda_p}$
4. Diagonal matrix D in Σ is either in $\mathcal{M}_p$ (if $\sigma_p > 0$) or in $\mathcal{M}_{p-1}$ (if $\sigma_p = 0$)

5. Since eigenvalues are distinct, Theorem 80 $\implies$ eigenvectors are orthogonal set. Compute these eigenvectors in the same order as the eigenvalues
6. Normalise them and use them to make the matrix V, i.e., $V = [\boldsymbol{v}_1 \cdots \boldsymbol{v}_p]$
7. To find the $\boldsymbol{u}_i$, compute, for $i = 1, \ldots, r$,

$$\boldsymbol{u}_i = \frac{1}{\sigma_i} A \boldsymbol{v}_i$$

and ensure that $\|\boldsymbol{u}_i\| = 1$

Why do we compute $\boldsymbol{u}_i$ this way?

Recall the definition of the SVD:

$$A = U\Sigma V^T$$

Right-multiply by V; since V is an orthogonal matrix ($V^T V = VV^T = \mathbb{I}$)

$$AV = U\Sigma V^T V = U\Sigma$$

Looking at the i-th column of both sides of this equation:

- The i-th column of AV is $A\boldsymbol{v}_i$
- The i-th column of $U\Sigma$ is $\sigma_i \boldsymbol{u}_i$

Therefore, we have $A\boldsymbol{v}_i = \sigma_i \boldsymbol{u}_i$, which yields for $\sigma_i > 0$:

$$\boldsymbol{u}_i = \frac{1}{\sigma_i} A\boldsymbol{v}_i$$

Example 1: Square matrix

Let $A = \begin{pmatrix} 2 & 2 \\ -1 & 1 \end{pmatrix}$.

1. Compute $A^T A = \begin{pmatrix} 2 & -1 \\ 2 & 1 \end{pmatrix} \begin{pmatrix} 2 & 2 \\ -1 & 1 \end{pmatrix} = \begin{pmatrix} 5 & 3 \\ 3 & 5 \end{pmatrix}$
2. Eigenvalues of $A^T A$: characteristic equation is $(5-\lambda)^2 - 9 = 0 \implies \lambda = 8, 2$. So $\lambda_1 = 8, \lambda_2 = 2$.
3. Singular values: $\sigma_1 = \sqrt{8} = 2\sqrt{2}, \sigma_2 = \sqrt{2}$. Matrix $\Sigma = \begin{pmatrix} 2\sqrt{2} & 0 \\ 0 & \sqrt{2} \end{pmatrix}$.

Example 1: Square matrix (continued)

4. Eigenvectors of A^TA:
   - For $\lambda_1 = 8$: null space of $\begin{pmatrix} -3 & 3 \\ 3 & -3 \end{pmatrix}$, giving $\boldsymbol{v}_1 = \frac{1}{\sqrt{2}}\begin{pmatrix} 1 \\ 1 \end{pmatrix}$
   - For $\lambda_2 = 2$: null space of $\begin{pmatrix} 3 & 3 \\ 3 & 3 \end{pmatrix}$, giving $\boldsymbol{v}_2 = \frac{1}{\sqrt{2}}\begin{pmatrix} -1 \\ 1 \end{pmatrix}$

   So $V = \frac{1}{\sqrt{2}}\begin{pmatrix} 1 & -1 \\ 1 & 1 \end{pmatrix}$.
5. Compute the $\boldsymbol{u}_i$:
   - $\boldsymbol{u}_1 = \frac{1}{\sigma_1}A\boldsymbol{v}_1 = \frac{1}{2\sqrt{2}}\begin{pmatrix} 2 & 2 \\ -1 & 1 \end{pmatrix}\frac{1}{\sqrt{2}}\begin{pmatrix} 1 \\ 1 \end{pmatrix} = \frac{1}{4}\begin{pmatrix} 4 \\ 0 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$
   - $\boldsymbol{u}_2 = \frac{1}{\sigma_2}A\boldsymbol{v}_2 = \frac{1}{\sqrt{2}}\begin{pmatrix} 2 & 2 \\ -1 & 1 \end{pmatrix}\frac{1}{\sqrt{2}}\begin{pmatrix} -1 \\ 1 \end{pmatrix} = \frac{1}{2}\begin{pmatrix} 0 \\ 2 \end{pmatrix} = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$

   So $U = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$.

Example 2: Nonsquare matrix

Let $A = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 0 \end{pmatrix}$.

1. Compute $A^T A = \begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix} \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} 2 & 0 \\ 0 & 1 \end{pmatrix}$
2. Eigenvalues of $A^T A$ are clearly $\lambda_1 = 2, \lambda_2 = 1$.
3. Singular values: $\sigma_1 = \sqrt{2}, \sigma_2 = 1$. The matrix Σ is 3×2, so $\Sigma = \begin{pmatrix} \sqrt{2} & 0 \\ 0 & 1 \\ 0 & 0 \end{pmatrix}$.
4. Eigenvectors of $A^T A$ are the standard basis vectors: $\boldsymbol{v}_1 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \boldsymbol{v}_2 = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$.

   So $V = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$.

Example 2: Nonsquare matrix (continued)

5. Compute the $\boldsymbol{u}_i$ for $i = 1, 2$:
   - $\boldsymbol{u}_1 = \frac{1}{\sigma_1} A\boldsymbol{v}_1 = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix}$
   - $\boldsymbol{u}_2 = \frac{1}{\sigma_2} A\boldsymbol{v}_2 = \frac{1}{1} \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} 0 \\ 1 \end{pmatrix} = \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix}$
6. Find $\boldsymbol{u}_3$ to complete the orthonormal basis U of $\mathcal{R}^3$. We need $\boldsymbol{u}_3$ orthogonal to $\boldsymbol{u}_1$ and $\boldsymbol{u}_2$, with $\|\boldsymbol{u}_3\| = 1$. By inspection, $\boldsymbol{u}_3 = \frac{1}{\sqrt{2}} \begin{pmatrix} -1 \\ 0 \\ 1 \end{pmatrix}$ works.
7. Finally, $U = \begin{pmatrix} \frac{1}{\sqrt{2}} & 0 & -\frac{1}{\sqrt{2}} \\ 0 & 1 & 0 \\ \frac{1}{\sqrt{2}} & 0 & \frac{1}{\sqrt{2}} \end{pmatrix}$.

Return to image compression – Computations “by hand”

```
M = my_image_g@grey
MTM = t(M) %*% M
# Ensure matrix is symmetric
MTM = (MTM+t(MTM))/2
ev = eigen(MTM)
```

Given the size and nature of the entries, the matrix M^TM is symmetric only to `1e-5` precision, so we use a little trick to make it symmetric no matter what: take the average of M^TM and its transpose MM^T

Which version of the algorithm to use?

Make zero the eigenvalues that are close to zero (200 out of 1000)

```
ev$values = ev$values*(ev$values>1e-10)
```

Can we use the algorithm for all eigenvalues being distinct or do we have repeated ones?

```
any(duplicated(ev$values[ev$values>1e-10]))

## [1] FALSE
```

So we can use the standard algorithm

Computing the SVD

```
idx_positive_ev = which(ev$values>1e-10)
sv = sqrt(ev$values[idx_positive_ev])
```

Computing the SVD

Then $D = \text{diag}(\sigma_1, \ldots, \sigma_r)$, V is the matrix of normalised eigenvectors in the same order as the σ_i and for $i = 1, \ldots, r$

$$\mathbf{u}_i = \frac{1}{\sigma_i} A \mathbf{v}_i$$

ensuring that $\|\mathbf{u}_i\| = 1$

```
D = diag(sv)
V = ev$vectors[, idx_positive_ev]
# Not needed with R, other languages might not normalise
c1 = sqrt(colSums(V^2))
for (i in 1:dim(V)[2]) {
    V[,i] = V[,i]/c1[i]
}
```

Computing the SVD

Finally, we compute the $\boldsymbol{u}_i$'s

```
U = M %*% V %*% diag(1/sv)
r = length(sv)
im = list(u=U, d=sv, v=V)
```

Computing the SVD (case where some eigenvalues are $=$)

1. Compute $A^T A \in \mathcal{M}_p$
2. Compute eigenvalues $\lambda_1, \ldots, \lambda_p$ of $A^T A$; order them as $\lambda_1 \geq \cdots \geq \lambda_p \geq 0$
3. Compute singular values $\sigma_1 = \sqrt{\lambda_1}, \ldots, \sigma_p = \sqrt{\lambda_p}$, with $r \leq p$ the index of the last positive singular value
4. For eigenvalues that are distinct, proceed as before
5. For eigenvalues with multiplicity > 1, we need to ensure that the resulting eigenvectors are LI *and* orthogonal

Dealing with eigenvalues with multiplicity > 1

When an eigenvalue has (algebraic) multiplicity > 1, e.g., characteristic polynomial contains a factor like $(\lambda - 2)^2$, things can become a little bit more complicated

The proper way to deal with this involves the so-called Jordan Normal Form (JNF, another matrix decomposition – for square matrices)

In short: not all square matrices are diagonalisable, but all square matrices admit a JNF

Sometimes, we can find several LI eigenvectors associated to the same eigenvalue. Check this. If not, need to use the following

Definition 81 (Generalised eigenvectors)

The vector $\boldsymbol{x} \neq \mathbf{0}$ is a **generalized eigenvector** of rank m of $A \in \mathcal{M}_n$ corresponding to eigenvalue λ if

$$(A - \lambda \mathbb{I})^m \boldsymbol{x} = \mathbf{0}$$

but

$$(A - \lambda \mathbb{I})^{m-1} \boldsymbol{x} \neq \mathbf{0}$$

Procedure for generalised eigenvectors

$A \in \mathcal{M}_n$ and assume λ eigenvalue with algebraic multiplicity k

Find $\boldsymbol{v}_1$, “classic" eigenvector, i.e., $\boldsymbol{v}_1 \neq \mathbf{0}$ s.t. $(A - \lambda \mathbb{I})\boldsymbol{v}_1 = \mathbf{0}$

Find generalised eigenvector $\boldsymbol{v}_2$ of rank 2 by solving for $\boldsymbol{v}_2 \neq \mathbf{0}$,

$$(A - \lambda \mathbb{I})\boldsymbol{v}_2 = \boldsymbol{v}_1$$

...

Find generalised eigenvector $\boldsymbol{v}_k$ of rank k by solving for $\boldsymbol{v}_k \neq \mathbf{0}$,

$$(A - \lambda \mathbb{I})\boldsymbol{v}_k = \boldsymbol{v}_{k-1}$$

Then $\{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_k\}$ LI

Back to the normal procedure

With the LI eigenvectors $\{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_k\}$ corresponding to λ

Apply Gram-Schmidt to get orthogonal set

For all eigenvalues with multiplicity > 1, check that you either have LI eigenvectors or do what we just did

When you are done, be back on your merry way to step 6 in the case where eigenvalues are all $\neq$

I am caricaturing a little here: there can be cases that do not work exactly like this, but this is general enough..

Example 3: Repeated eigenvalues

Let $A = \begin{pmatrix} 1 & -1 & 0 \\ 0 & 1 & -1 \\ -1 & 0 & 1 \end{pmatrix}$.

1. Compute $A^T A = \begin{pmatrix} 1 & 0 & -1 \\ -1 & 1 & 0 \\ 0 & -1 & 1 \end{pmatrix} \begin{pmatrix} 1 & -1 & 0 \\ 0 & 1 & -1 \\ -1 & 0 & 1 \end{pmatrix} = \begin{pmatrix} 2 & -1 & -1 \\ -1 & 2 & -1 \\ -1 & -1 & 2 \end{pmatrix}$
2. Eigenvalues of $A^T A$: the characteristic polynomial is $-\lambda^3 + 6\lambda^2 - 9\lambda = 0$, which gives $\lambda_1 = 3, \lambda_2 = 3, \lambda_3 = 0$.
3. Singular values: $\sigma_1 = \sqrt{3}, \sigma_2 = \sqrt{3}, \sigma_3 = 0$.

Example 3: Repeated eigenvalues (continued)

4. Eigenvectors of A^TA:

   - For $\lambda_3 = 0$: null space of A^TA, giving $\boldsymbol{v}_3 = \frac{1}{\sqrt{3}}\begin{pmatrix}1\\1\\1\end{pmatrix}$
   - For $\lambda_1 = \lambda_2 = 3$: we need to find the null space of $A^TA - 3\mathbb{I} = \begin{pmatrix}-1 & -1 & -1\\-1 & -1 & -1\\-1 & -1 & -1\end{pmatrix}$. The eigenspace is the plane $x + y + z = 0$.

   We must choose two **orthogonal** eigenvectors in this plane.

   Let $\boldsymbol{x}_1 = \begin{pmatrix}1\\-1\\0\end{pmatrix}$. To find a second vector $\boldsymbol{x}_2$ orthogonal to $\boldsymbol{x}_1$ using Gram-Schmidt, we start with another vector in the plane, e.g., $\mathbf{y}_2 = \begin{pmatrix}1\\0\\-1\end{pmatrix}$.

   We compute $\boldsymbol{x}_2 = \mathbf{y}_2 - \mathrm{proj}_{\boldsymbol{x}_1}(\mathbf{y}_2) = \begin{pmatrix}1/2\\1/2\\-1\end{pmatrix}$, which we can scale to $\begin{pmatrix}1\\1\\-2\end{pmatrix}$.

Example 3: Repeated eigenvalues (continued)

5. Compute the $\boldsymbol{u}_i$ for $i = 1, 2$:
   - $\boldsymbol{u}_1 = \frac{1}{\sigma_1} A\boldsymbol{v}_1 = \frac{1}{\sqrt{3}} \begin{pmatrix} 1 & -1 & 0 \\ 0 & 1 & -1 \\ -1 & 0 & 1 \end{pmatrix} \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ -1 \\ 0 \end{pmatrix} = \frac{1}{\sqrt{6}} \begin{pmatrix} 2 \\ -1 \\ -1 \end{pmatrix}$
   - $\boldsymbol{u}_2 = \frac{1}{\sigma_2} A\boldsymbol{v}_2 = \frac{1}{\sqrt{3}} \begin{pmatrix} 1 & -1 & 0 \\ 0 & 1 & -1 \\ -1 & 0 & 1 \end{pmatrix} \frac{1}{\sqrt{6}} \begin{pmatrix} 1 \\ 1 \\ -2 \end{pmatrix} = \frac{1}{\sqrt{18}} \begin{pmatrix} 0 \\ 3 \\ -3 \end{pmatrix} = \frac{1}{\sqrt{2}} \begin{pmatrix} 0 \\ 1 \\ -1 \end{pmatrix}$
6. To complete the orthonormal basis U of $\mathcal{R}^3$, we find $\boldsymbol{u}_3$ orthogonal to $\boldsymbol{u}_1$ and $\boldsymbol{u}_2$. By inspection (or cross product), $\boldsymbol{u}_3 = \frac{1}{\sqrt{3}} \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix}$.
7. Finally, $U = \begin{pmatrix} \frac{2}{\sqrt{6}} & 0 & \frac{1}{\sqrt{3}} \\ -\frac{1}{\sqrt{6}} & \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{3}} \\ -\frac{1}{\sqrt{6}} & -\frac{1}{\sqrt{2}} & \frac{1}{\sqrt{3}} \end{pmatrix}$.

The SVD (singular values decomposition)

Singular values
The SVD
An application of the SVD – Image compression
Computing the SVD
Applications of the SVD – Least squares
Summary – Least squares methods

Pseudoinverse of a matrix

Definition 82 (Pseudoinverse)

$A = U\Sigma V^T$ an SVD for $A \in \mathcal{M}_{np}$, where

$$\Sigma = \begin{pmatrix} D & 0 \\ 0 & 0 \end{pmatrix}, \text{ with } D = \text{diag}(\sigma_1, \ldots, \sigma_r)$$

(D contains the nonzero singular values of A ordered as usual)

The **pseudoinverse** (or **Moore-Penrose inverse**) of A is $A^+ \in \mathcal{M}_{pn}$ given by

$$A^+ = V\Sigma^+ U^T$$

with

$$\Sigma^+ = \begin{pmatrix} D^{-1} & 0 \\ 0 & 0 \end{pmatrix} \in \mathcal{M}_{pn}$$

Properties of the pseudoinverse

The pseudoinverse A^+ generalises the concept of matrix inverse and satisfies the following properties (known as the Moore-Penrose conditions):

1. $AA^+A = A$
2. $A^+AA^+ = A^+$
3. $(AA^+)^T = AA^+$
4. $(A^+A)^T = A^+A$

Other useful properties:

- If A is square and invertible, then $A^+ = A^{-1}$
- $(A^+)^+ = A$
- $(A^T)^+ = (A^+)^T$
- If A has full column rank, then $A^+ = (A^TA)^{-1}A^T$
- If A has full row rank, then $A^+ = A^T(AA^T)^{-1}$

Least squares revisited

Theorem 83

Let $A \in \mathcal{M}_{np}$, $\boldsymbol{x} \in \mathbb{R}^p$ and $\boldsymbol{b} \in \mathbb{R}^n$. The least squares problem $A\boldsymbol{x} = \boldsymbol{b}$ has a unique least squares solution $\tilde{\boldsymbol{x}}$ of minimal length (closest to the origin) given by

$$\tilde{\boldsymbol{x}} = A^+ \boldsymbol{b}$$

where A^+ is the pseudoinverse of A

What does “minimal length” mean?

When A has *full column rank* (i.e., its columns are linearly independent), the least squares solution is unique:

$$\tilde{\boldsymbol{x}} = (A^T A)^{-1} A^T \boldsymbol{b}$$

However, if the columns of A are *linearly dependent*, the matrix $A^T A$ is not invertible. In this case, there are **infinitely many** least squares solutions that minimise $\|A\boldsymbol{x} - \boldsymbol{b}\|$

Among all these possible solutions, the pseudoinverse A^+ selects the unique solution $\tilde{\boldsymbol{x}} = A^+ \boldsymbol{b}$ that has the **smallest Euclidean norm** $\|\tilde{\boldsymbol{x}}\|$

Geometrically, it is the solution closest to the origin in $\mathbb{R}^p$

Example: Computing the pseudoinverse

Let $A = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$. The SVD of A is $U\Sigma V^T$ with:

$$U = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}, \quad \Sigma = \begin{pmatrix} 2 & 0 \\ 0 & 0 \end{pmatrix}, \quad V = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$$

To find A^+, we form Σ^+ by taking the reciprocal of the non-zero singular values (leaving zeros as zero):

$$\Sigma^+ = \begin{pmatrix} 1/2 & 0 \\ 0 & 0 \end{pmatrix}$$

Then the pseudoinverse is $A^+ = V\Sigma^+ U^T$:

$$A^+ = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \begin{pmatrix} 1/2 & 0 \\ 0 & 0 \end{pmatrix} \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}^T = \frac{1}{4} \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$$

Example: Least squares using the pseudoinverse

Consider $A\boldsymbol{x} = \boldsymbol{b}$ where $A = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$ and $\boldsymbol{b} = \begin{pmatrix} 3 \\ 1 \end{pmatrix}$

The system $\begin{pmatrix} x_1 + x_2 \\ x_1 + x_2 \end{pmatrix} = \begin{pmatrix} 3 \\ 1 \end{pmatrix}$ has no exact solution

The least squares approach minimises the error
$\|A\boldsymbol{x} - \boldsymbol{b}\|^2 = (x_1 + x_2 - 3)^2 + (x_1 + x_2 - 1)^2$. The minimum error is achieved when $x_1 + x_2 = 2$

Any vector $\boldsymbol{x} = \begin{pmatrix} x_1 \\ 2 - x_1 \end{pmatrix}$ is a least squares solution (e.g., $\begin{pmatrix} 2 \\ 0 \end{pmatrix}$, $\begin{pmatrix} 0 \\ 2 \end{pmatrix}$)

The **minimal length** solution is given by the pseudoinverse:

$$\tilde{\boldsymbol{x}} = A^+ \boldsymbol{b} = \frac{1}{4} \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} 3 \\ 1 \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$$

which is indeed the point on the line $x_1 + x_2 = 2$ closest to the origin

The SVD (singular values decomposition)

- Singular values
- The SVD
- An application of the SVD – Image compression
- Computing the SVD
- Applications of the SVD – Least squares
- **Summary – Least squares methods**

The least squares problem

Problem Statement:

Given a system $A\boldsymbol{x} = \boldsymbol{b}$ where $A \in \mathcal{M}_{np}$, $\boldsymbol{x} \in \mathbb{R}^p$, $\boldsymbol{b} \in \mathbb{R}^n$ (typically $n > p$), find $\tilde{\boldsymbol{x}}$ that minimizes

$$\|\boldsymbol{b} - A\boldsymbol{x}\|^2 = \sum_{i=1}^{n}(b_i - \sum_{j=1}^{p} A_{ij}x_j)^2$$

Geometric interpretation: Find the vector $A\tilde{\boldsymbol{x}}$ in the column space of A that is closest to $\boldsymbol{b}$

Solution: $A\tilde{\boldsymbol{x}} = \text{proj}_{\text{col}(A)}(\boldsymbol{b})$

Method 1: Normal equations

The normal equations:

$$A^T A\tilde{\boldsymbol{x}} = A^T \boldsymbol{b}$$

When this works:

- Always has at least one solution
- Any solution $\tilde{\boldsymbol{x}}$ to the normal equations is a least squares solution

Computational issues:

- Forming $A^T A$ can be numerically unstable
- Condition number of $A^T A$ is the square of the condition number of A
- Still useful for theoretical analysis

Method 2: when A Has linearly independent columns

Condition: $A \in \mathcal{M}_{np}$ has linearly independent columns

Then: $A^T A$ is invertible and the least squares solution is **unique**

$$\tilde{\boldsymbol{x}} = (A^T A)^{-1} A^T \boldsymbol{b}$$

Properties:

- $A^T A \in \mathcal{M}_p$ is square, symmetric, and positive definite
- $(A^T A)^{-1} A^T$ is called the *left pseudoinverse* of A
- This gives the unique least squares solution

Drawback: Computing $(A^T A)^{-1}$ directly can be numerically unstable

Method 3: QR factorization

QR Factorization: If $A \in \mathcal{M}_{np}$ has linearly independent columns, then

$$A = QR$$

where $Q \in \mathcal{M}_{np}$ has orthonormal columns and $R \in \mathcal{M}_p$ is upper triangular and nonsingular

Least squares solution:

$$\tilde{\boldsymbol{x}} = R^{-1} Q^T \boldsymbol{b}$$

Advantages:

- More numerically stable than forming $A^T A$
- R is upper triangular $\Rightarrow$ solving $R\tilde{\boldsymbol{x}} = Q^T \boldsymbol{b}$ by back substitution
- Condition number of R equals condition number of A
- Gram-Schmidt or Householder reflections can compute QR factorization

Method 4: Singular Value Decomposition (SVD)

SVD: For any $A \in \mathcal{M}_{np}$,

$$A = U\Sigma V^T$$

where $U \in \mathcal{M}_n$ orthogonal, $V \in \mathcal{M}_p$ orthogonal, $\Sigma \in \mathcal{M}_{np}$ with $\Sigma_{ii} = \sigma_i \geq 0$ (singular values)

Pseudoinverse: $A^+ = V\Sigma^+ U^T$ where Σ^+ has $(\Sigma^+)_{ii} = 1/\sigma_i$ if $\sigma_i > 0$, else 0

Least squares solution:

$$\tilde{\boldsymbol{x}} = A^+ \boldsymbol{b}$$

Key advantages:

- Works for *any* matrix A (even when columns are linearly dependent)
- Gives the solution of *minimal length* when multiple solutions exist
- Most numerically stable method
- Reveals the rank of A through the number of non-zero singular values

When to use which method

Method	**When to use**	**Advantages/Drawbacks**
Normal equations	Theory, small problems	Simple, but unstable
$(A^T A)^{-1} A^T$	A has LI columns	Explicit formula, unstable
QR Factorization	A has LI columns	Stable, efficient
SVD	Any A, rank-deficient	Most stable, handles all cases

Use QR for well-conditioned problems with LI columns, SVD for rank-deficient or ill-conditioned problems

Principal component analysis

MATH 2740 – Mathematics of Data Science – Unit U05

Julien Arino
julien.arino@umanitoba.ca

Department of Mathematics @ University of Manitoba

Fall 2026

The University of Manitoba campuses are located on original lands of Anishinaabeg, Ininew, Anisininew, Dakota and Dene peoples, and on the National Homeland of the Red River Métis. We respect the Treaties that were made on these territories, we acknowledge the harms and mistakes of the past, and we dedicate ourselves to move forward in partnership with Indigenous communities in a spirit of Reconciliation and collaboration.

Outline

A running example: hockey players

Change of basis

A crash course on probability

Back to PCA

Hockey players

Dimensionality reduction

In the previous unit, we saw that SVD could be used to reduce the dimension of data (matrices)

Now we look at another dimensionality reduction technique: principal component analysis (PCA)

PCA is often used as a blackbox technique, here we take a look at the math behind it

What is PCA?

Linear algebraic technique

Helps reduce a complex dataset to a lower dimensional one

Non-parametric method: does not assume anything about data distribution (distribution from the statistical point of view)

A running example: hockey players

Change of basis

A crash course on probability

Back to PCA

Hockey players

A 2D example

Dataset (link) of height and weight of some hockey players

```
# From https://figshare.com/ndownloader/files/5303173
data = read.csv("https://github.com/julien-arino/math-of-data-science/raw/re
dim(data)

## [1] 6292   13
```

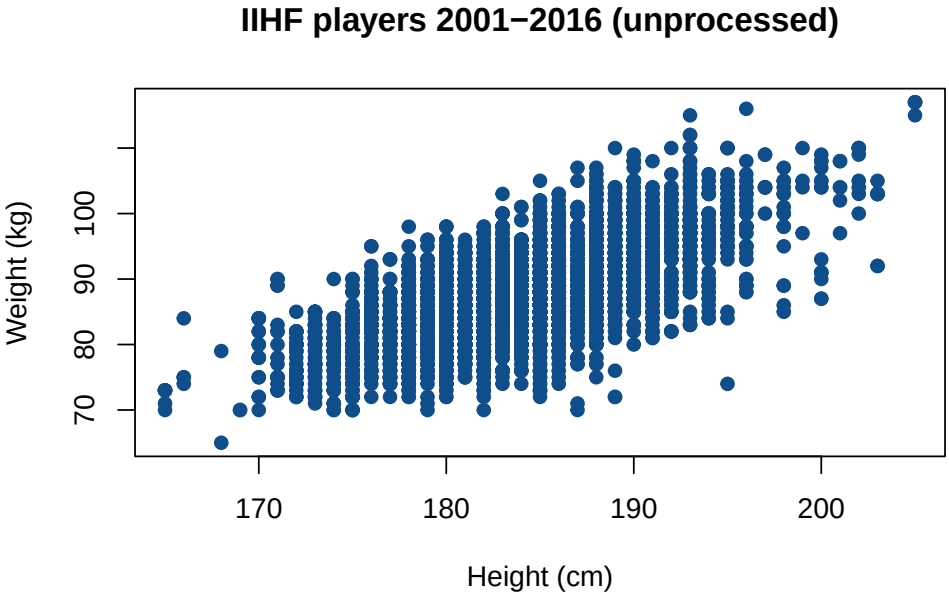
In case you are wondering, this is a database of ice hockey players at IIHF world championships, 2001-2016, assembled by the dataset's author

See some comments here

```r
head(data, n=3)
```

```
##   year country no                name position side height weight      bi
## 1 2001     RUS 10    tverdovsky oleg        D    L    185     84 1976-05
## 2 2001     RUS  2  vichnevsky vitali        D    L    188     86 1980-03
## 3 2001     RUS 26 petrochinin evgeni        D    L    182     95 1976-02
##                      club      age cohort      bmi
## 1   anaheim mighty ducks  24.95277   1976 24.54346
## 2   anaheim mighty ducks  21.11978   1980 24.33228
## 3 severstal cherepovetal  25.22930   1976 28.68011
```

As usual, it is a good idea to plot this to get a sense of the lay of the land


IIHF players 2001−2016 (unprocessed)
Weight (kg)
70
80
90
100
170
180
190
200
Height (cm)

The author of the study is interested in the evolution of weights, so it is likely that the same person will be in the dataset several times

Let us check this: first check below will be `FALSE` if the number of unique names does not match the number of rows in the dataset

```
length(unique(data$name)) == dim(data)[1]

## [1] FALSE

length(unique(data$name))

## [1] 3278

dim(data)[1]

## [1] 6292
```

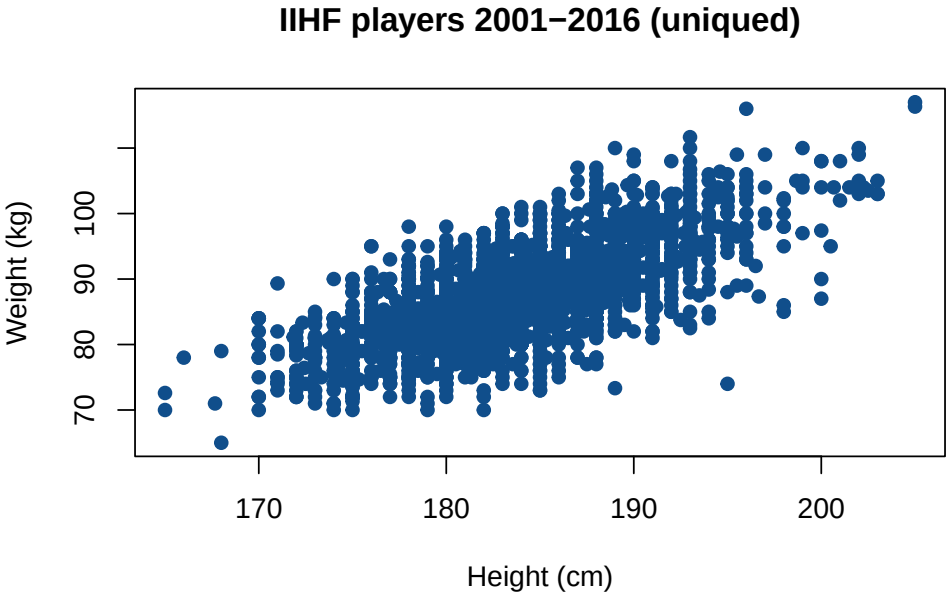
We are not interested in the evolution of weights, so we simplify: if more than one record for someone, take average of recorded weights and heights

To be extra careful, could check as well that there are no major variations on player height (homonymies?)

```
data_simplified = data.frame(name = unique(data$name))
w = c()
h = c()
for (n in data_simplified$name) {
    tmp = data[which(data$name == n),]
    h = c(h, mean(tmp$height))
    w = c(w, mean(tmp$weight))
}
data_simplified$weight = w
data_simplified$height = h
```

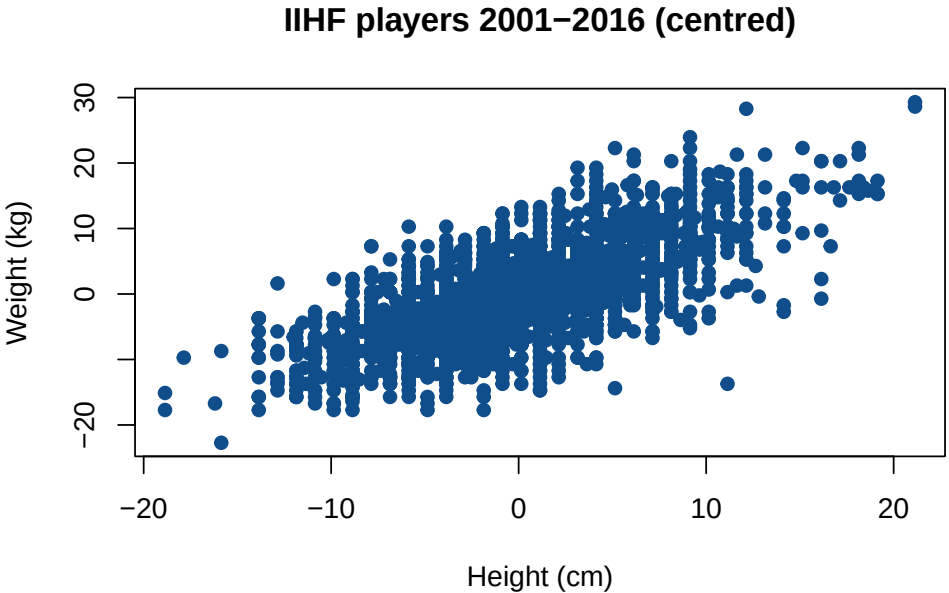
```r
data = data_simplified
head(data_simplified, n = 6)
```

```
##                   name weight height
## 1    tverdovsky oleg    84.0  185.0
## 2  vichnevsky vitali    86.0  188.0
## 3 petrochinin evgeni    95.0  182.0
## 4    zhdan alexander    85.5  178.5
## 5    orekhovsky oleg    88.0  175.0
## 6      zhukov sergei    92.5  193.0
```


IIHF players 2001−2016 (uniqued)
Weight (kg)
70
80
90
100
170
180
190
200
Height (cm)

Centre the data

```
mean(data$weight)

## [1] 87.71555

mean(data$height)

## [1] 183.8596

data$weight.c = data$weight-mean(data$weight)
data$height.c = data$height-mean(data$height)
```


IIHF players 2001−2016 (centred)
Weight (kg)
30
20
10
0
−20
−20
−10
0
10
20
Height (cm)

Setting things up

Each player is a row in the data matrix (an *observation*), each feature or variable (*height* and *weight*) is a column

After deduplication, we have an 3278×2 matrix (actually, 3278×4 if we consider the uncentred and centred features, but we will use one or the other, not both uncentred and centred)

We want to find what carries the most information

For this, we are going to project the information in a new basis in which the first "dimension" will carry most information (in a sense we'll define later), the second dimension will carry a little less, etc.

In order to do so, we need to learn how to change bases

A running example: hockey players

Change of basis

A crash course on probability

Back to PCA

Hockey players

Change of basis

Theory

A simple example

Basis for a trajectory in $\mathbb{R}^2$

In the following,

$$[\boldsymbol{x}]_{\mathcal{B}}$$

denotes the coordinates of $\boldsymbol{x}$ in the basis $\mathcal{B} = \{\boldsymbol{u}_1, \ldots, \boldsymbol{u}_n\}$, i.e., $[\boldsymbol{x}]_{\mathcal{B}} = (b_1, \ldots, b_n)^T$ such that $\boldsymbol{x} = b_1\boldsymbol{u}_1 + \cdots + b_n\boldsymbol{u}_n$

The aim of a change of basis is to express vectors in another coordinate system (another basis) $\mathcal{C} = \{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_n\}$

We do so by finding a matrix allowing to move from one basis to another

Change of basis

Definition 80 (Change of basis matrix)

Let $\mathcal{B} = \{\boldsymbol{u}_1, \ldots, \boldsymbol{u}_n\}$ and $\mathcal{C} = \{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_n\}$ be bases of vector space V

The **change of basis matrix** $P_{\mathcal{C} \leftarrow \mathcal{B}} \in \mathcal{M}_n$ from basis $\mathcal{B}$ to $\mathcal{C}$ is

$$P_{\mathcal{C} \leftarrow \mathcal{B}} = [[\boldsymbol{u}_1]_{\mathcal{C}} \cdots [\boldsymbol{u}_n]_{\mathcal{C}}]$$

i.e., its columns are the coordinate vectors $[\boldsymbol{u}_1]_{\mathcal{C}}, \ldots, [\boldsymbol{u}_n]_{\mathcal{C}}$ of vectors in $\mathcal{B}$ with respect to $\mathcal{C}$

Example – $\mathcal{B}$ is the canonical basis of $\mathbb{R}^3$

Let $\mathcal{B} = \{\boldsymbol{i}, \boldsymbol{j}, \boldsymbol{k}\}$ be the canonical basis of $\mathbb{R}^3$ and $\mathcal{C} = \{\boldsymbol{v}_1, \boldsymbol{v}_2, \boldsymbol{v}_3\}$, with

$$\boldsymbol{v}_1 = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}, \quad \boldsymbol{v}_2 = \begin{pmatrix} 1 \\ 1 \\ 0 \end{pmatrix}, \quad \boldsymbol{v}_3 = \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix}$$

To find $P_{\mathcal{C}\leftarrow\mathcal{B}} = [[\boldsymbol{i}]_{\mathcal{C}} \;\; [\boldsymbol{j}]_{\mathcal{C}} \;\; [\boldsymbol{k}]_{\mathcal{C}}]$, express each canonical vector in terms of $\mathcal{C}$:

$$\boldsymbol{i} = \boldsymbol{v}_1, \quad \boldsymbol{j} = \boldsymbol{v}_2 - \boldsymbol{v}_1, \quad \boldsymbol{k} = \boldsymbol{v}_3 - \boldsymbol{v}_2$$

$$\Longrightarrow \; [\boldsymbol{i}]_{\mathcal{C}} = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}, \quad [\boldsymbol{j}]_{\mathcal{C}} = \begin{pmatrix} -1 \\ 1 \\ 0 \end{pmatrix}, \quad [\boldsymbol{k}]_{\mathcal{C}} = \begin{pmatrix} 0 \\ -1 \\ 1 \end{pmatrix}$$

The change of basis matrix from $\mathcal{B}$ to $\mathcal{C}$ is

$$P_{\mathcal{C}\leftarrow\mathcal{B}} = [[\boldsymbol{i}]_{\mathcal{C}} \;\; [\boldsymbol{j}]_{\mathcal{C}} \;\; [\boldsymbol{k}]_{\mathcal{C}}] = \begin{pmatrix} 1 & -1 & 0 \\ 0 & 1 & -1 \\ 0 & 0 & 1 \end{pmatrix}$$

Example – $\mathcal{C}$ is the canonical basis of $\mathbb{R}^3$

Now consider the reverse: let $\mathcal{B} = \{\boldsymbol{u}_1, \boldsymbol{u}_2, \boldsymbol{u}_3\}$ with

$$\boldsymbol{u}_1 = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}, \quad \boldsymbol{u}_2 = \begin{pmatrix} 1 \\ 1 \\ 0 \end{pmatrix}, \quad \boldsymbol{u}_3 = \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix}$$

and let $\mathcal{C} = \{\boldsymbol{i}, \boldsymbol{j}, \boldsymbol{k}\}$ be the canonical basis of $\mathbb{R}^3$

Since $\mathcal{C}$ is the canonical basis, the coordinates in $\mathcal{C}$ are simply the vector itself:

$$[\boldsymbol{u}_1]_{\mathcal{C}} = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}, \quad [\boldsymbol{u}_2]_{\mathcal{C}} = \begin{pmatrix} 1 \\ 1 \\ 0 \end{pmatrix}, \quad [\boldsymbol{u}_3]_{\mathcal{C}} = \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix}$$

Thus, the change of basis matrix from $\mathcal{B}$ to the canonical basis $\mathcal{C}$ is

$$P_{\mathcal{C} \leftarrow \mathcal{B}} = [[\boldsymbol{u}_1]_{\mathcal{C}} \ \ [\boldsymbol{u}_2]_{\mathcal{C}} \ \ [\boldsymbol{u}_3]_{\mathcal{C}}] = \begin{pmatrix} 1 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{pmatrix}$$

Theorem 81

$\mathcal{B} = \{\boldsymbol{u}_1, \ldots, \boldsymbol{u}_n\}$ and $\mathcal{C} = \{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_n\}$ bases of vector space V and $P_{\mathcal{C}\leftarrow\mathcal{B}}$ a change of basis matrix from $\mathcal{B}$ to $\mathcal{C}$

1. $\forall \boldsymbol{x} \in V$, $P_{\mathcal{C}\leftarrow\mathcal{B}}[\boldsymbol{x}]_{\mathcal{B}} = [\boldsymbol{x}]_{\mathcal{C}}$
2. $P_{\mathcal{C}\leftarrow\mathcal{B}}$ *s.t.* $\forall \boldsymbol{x} \in V$, $P_{\mathcal{C}\leftarrow\mathcal{B}}[\boldsymbol{x}]_{\mathcal{B}} = [\boldsymbol{x}]_{\mathcal{C}}$ *is* ***unique***
3. $P_{\mathcal{C}\leftarrow\mathcal{B}}$ *invertible and* $P^{-1}_{\mathcal{C}\leftarrow\mathcal{B}} = P_{\mathcal{B}\leftarrow\mathcal{C}}$

Example – Linking the two directions

Let $\mathcal{B} = \{\boldsymbol{i}, \boldsymbol{j}, \boldsymbol{k}\}$ be the canonical basis and $\mathcal{C} = \{\boldsymbol{v}_1, \boldsymbol{v}_2, \boldsymbol{v}_3\}$ with

$$\boldsymbol{v}_1 = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}, \quad \boldsymbol{v}_2 = \begin{pmatrix} 1 \\ 1 \\ 0 \end{pmatrix}, \quad \boldsymbol{v}_3 = \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix}$$

From our two previous examples:

$$P_{\mathcal{C}\leftarrow\mathcal{B}} = \begin{pmatrix} 1 & -1 & 0 \\ 0 & 1 & -1 \\ 0 & 0 & 1 \end{pmatrix} \quad \text{and} \quad P_{\mathcal{B}\leftarrow\mathcal{C}} = \begin{pmatrix} 1 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{pmatrix}$$

By item 3 in Theorem 81, $P^{-1}_{\mathcal{C}\leftarrow\mathcal{B}} = P_{\mathcal{B}\leftarrow\mathcal{C}}$. Indeed:

$$P_{\mathcal{C}\leftarrow\mathcal{B}} P_{\mathcal{B}\leftarrow\mathcal{C}} = \begin{pmatrix} 1 & -1 & 0 \\ 0 & 1 & -1 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} = \mathbb{I}$$

Since $P_{\mathcal{B}\leftarrow\mathcal{C}} = [\boldsymbol{v}_1 \;\; \boldsymbol{v}_2 \;\; \boldsymbol{v}_3]$ is immediate, we have $P_{\mathcal{C}\leftarrow\mathcal{B}} = [\boldsymbol{v}_1 \;\; \boldsymbol{v}_2 \;\; \boldsymbol{v}_3]^{-1}$

Row-reduction method for changing bases

Theorem 82

$\mathcal{B} = \{\boldsymbol{u}_1, \ldots, \boldsymbol{u}_n\}$ and $\mathcal{C} = \{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_n\}$ bases of vector space V. Let $\mathcal{E}$ be any basis for V,

$$B = [[\boldsymbol{u}_1]_{\mathcal{E}}, \ldots, [\boldsymbol{u}_n]_{\mathcal{E}}] \ \textit{and}\ C = [[\boldsymbol{v}_1]_{\mathcal{E}}, \ldots, [\boldsymbol{v}_n]_{\mathcal{E}}]$$

and let $[C|B]$ be the augmented matrix constructed using C and B. Then

$$RREF\,([C|B]) = [\mathbb{I}|P_{\mathcal{C}\leftarrow\mathcal{B}}]$$

If working in $\mathbb{R}^n$, this is quite useful with $\mathcal{E}$ the standard basis of $\mathbb{R}^n$ (it does not matter if $\mathcal{B} = \mathcal{E}$)

So the question now becomes

How do we find what new basis to look at our data in?

(Changing the basis does not change the data, just the view you have of it)

(Think of what happens when you do a headstand.. your up becomes down, your right and left switch, but the world does not change, just your view of it)

(Changes of bases are *fundamental* operations in Science)

Change of basis

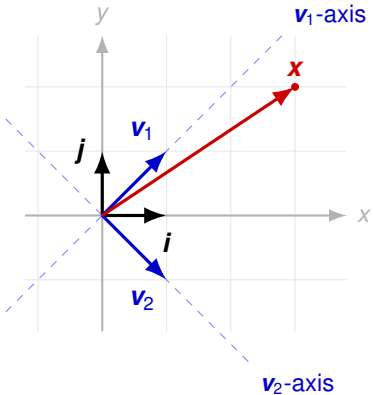
- Theory
- **A simple example**
- Basis for a trajectory in $\mathbb{R}^2$

Example: change of basis

Problem: Find the change of basis matrix from basis $\mathcal{B}$ to basis $\mathcal{C}$ in $\mathbb{R}^2$, where

$$\mathcal{B} = \left\{ \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 0 \\ 1 \end{pmatrix} \right\} \quad \text{and} \quad \mathcal{C} = \left\{ \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \begin{pmatrix} 1 \\ -1 \end{pmatrix} \right\}$$

Then use this matrix to find the coordinates of $\boldsymbol{x} = \begin{pmatrix} 3 \\ 2 \end{pmatrix}$ in basis $\mathcal{C}$



Step 1 – Set up the matrices

- $\mathcal{B} = \{\boldsymbol{i}, \boldsymbol{j}\}$ is the standard basis of $\mathbb{R}^2$, so using $\mathcal{E} = \mathcal{B}$, we have

$$[\boldsymbol{i}]_{\mathcal{E}} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad [\boldsymbol{j}]_{\mathcal{E}} = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

- For $\mathcal{C}$:

$$[\boldsymbol{v}_1]_{\mathcal{E}} = \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \quad [\boldsymbol{v}_2]_{\mathcal{E}} = \begin{pmatrix} 1 \\ -1 \end{pmatrix}$$

Step 2 – Row reduce

Using Theorem 82, we form the augmented matrix $[C|B]$:

$$[C|B] = \left[\begin{array}{cc|cc} 1 & 1 & 1 & 0 \\ 1 & -1 & 0 & 1 \end{array}\right]$$

Row reduce to RREF:

$$\left[\begin{array}{cc|cc} 1 & 1 & 1 & 0 \\ 1 & -1 & 0 & 1 \end{array}\right] \xrightarrow{R_2 \leftarrow R_2 - R_1} \left[\begin{array}{cc|cc} 1 & 1 & 1 & 0 \\ 0 & -2 & -1 & 1 \end{array}\right]$$

$$\xrightarrow{R_2 \leftarrow -\frac{1}{2}R_2} \left[\begin{array}{cc|cc} 1 & 1 & 1 & 0 \\ 0 & 1 & \frac{1}{2} & -\frac{1}{2} \end{array}\right]$$

$$\xrightarrow{R_1 \leftarrow R_1 - R_2} \left[\begin{array}{cc|cc} 1 & 0 & \frac{1}{2} & \frac{1}{2} \\ 0 & 1 & \frac{1}{2} & -\frac{1}{2} \end{array}\right]$$

Step 3 – Extract the change of basis matrix

From the RREF form $[\mathbb{I}|P_{\mathcal{C}\leftarrow\mathcal{B}}]$, we get:

$$P_{\mathcal{C}\leftarrow\mathcal{B}} = \begin{pmatrix} \frac{1}{2} & \frac{1}{2} \\ \frac{1}{2} & -\frac{1}{2} \end{pmatrix}$$

Verification: Let's check that this matrix works correctly.

- $[\boldsymbol{i}]_{\mathcal{C}} = P_{\mathcal{C}\leftarrow\mathcal{B}}[\boldsymbol{i}]_{\mathcal{B}} = \begin{pmatrix} \frac{1}{2} & \frac{1}{2} \\ \frac{1}{2} & -\frac{1}{2} \end{pmatrix} \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} \frac{1}{2} \\ \frac{1}{2} \end{pmatrix}$
- Check: $\frac{1}{2}\begin{pmatrix} 1 \\ 1 \end{pmatrix} + \frac{1}{2}\begin{pmatrix} 1 \\ -1 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$

Step 4 – Coordinates of $\boldsymbol{x}$ in basis $\mathcal{C}$

Now we find $[\boldsymbol{x}]_{\mathcal{C}}$ for $\boldsymbol{x} = \begin{pmatrix} 3 \\ 2 \end{pmatrix}$:

$$[\boldsymbol{x}]_{\mathcal{C}} = P_{\mathcal{C} \leftarrow \mathcal{B}} [\boldsymbol{x}]_{\mathcal{B}} = \begin{pmatrix} \frac{1}{2} & \frac{1}{2} \\ \frac{1}{2} & -\frac{1}{2} \end{pmatrix} \begin{pmatrix} 3 \\ 2 \end{pmatrix} = \begin{pmatrix} \frac{5}{2} \\ \frac{1}{2} \end{pmatrix}$$

Verification: Check that this gives us back the original vector:

$$\frac{5}{2} \begin{pmatrix} 1 \\ 1 \end{pmatrix} + \frac{1}{2} \begin{pmatrix} 1 \\ -1 \end{pmatrix} = \begin{pmatrix} \frac{5}{2} + \frac{1}{2} \\ \frac{5}{2} - \frac{1}{2} \end{pmatrix} = \begin{pmatrix} 3 \\ 2 \end{pmatrix}$$

Answer: The coordinates of $\boldsymbol{x} = \begin{pmatrix} 3 \\ 2 \end{pmatrix}$ in basis $\mathcal{C}$ are $[\boldsymbol{x}]_{\mathcal{C}} = \begin{pmatrix} \frac{5}{2} \\ \frac{1}{2} \end{pmatrix}$

Alternative method – Direct calculation

We could also solve this directly by setting up the system:

$$\boldsymbol{x} = c_1 \boldsymbol{v}_1 + c_2 \boldsymbol{v}_2$$

$$\begin{pmatrix} 3 \\ 2 \end{pmatrix} = c_1 \begin{pmatrix} 1 \\ 1 \end{pmatrix} + c_2 \begin{pmatrix} 1 \\ -1 \end{pmatrix}$$

This gives us the system:

$$\begin{aligned} c_1 + c_2 &= 3 \\ c_1 - c_2 &= 2 \end{aligned}$$

Solving: $c_1 = \frac{5}{2}$, $c_2 = \frac{1}{2}$

This confirms our result: $[\boldsymbol{x}]_{\mathcal{C}} = \begin{pmatrix} \frac{5}{2} \\ \frac{1}{2} \end{pmatrix}$

Computation in R – Finding $P_{\mathcal{C}\leftarrow\mathcal{B}}$

Set up $C = [\boldsymbol{v}_1, \boldsymbol{v}_2]$, $B = [\boldsymbol{i}, \boldsymbol{j}]$, form the augmented matrix $[C|B]$ and row reduce:

```
C = matrix(c(1, 1, 1, -1), nrow = 2)
B = diag(2)
CB = cbind(C, B)
(R = pracma::rref(CB))
```

```
##      [,1] [,2] [,3] [,4]
## [1,]    1    0  0.5  0.5
## [2,]    0    1  0.5 -0.5
```

```
(P = R[, 3:4])
```

```
##      [,1] [,2]
## [1,]  0.5  0.5
## [2,]  0.5 -0.5
```

Computation in R – Coordinates of $\boldsymbol{x}$

Now compute $[\boldsymbol{x}]_{\mathcal{C}} = P_{\mathcal{C}\leftarrow\mathcal{B}}[\boldsymbol{x}]_{\mathcal{B}}$ and verify that $C[\boldsymbol{x}]_{\mathcal{C}} = \boldsymbol{x}$:

```
x = c(3, 2)
(x_C = P %*% x)

##      [,1]
## [1,]  2.5
## [2,]  0.5

# Verification: reconstruct x in the canonical basis
C %*% x_C

##      [,1]
## [1,]    3
## [2,]    2
```

Change of basis

- Theory
- A simple example
- **Basis for a trajectory in $\mathbb{R}^2$**

Example – A trajectory in $\mathbb{R}^2$

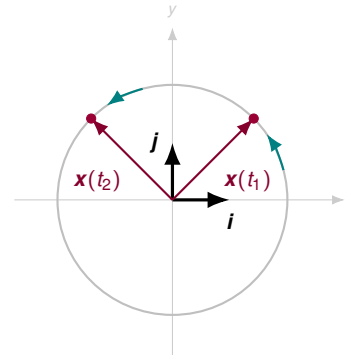
Let $\mathcal{B} = \{\boldsymbol{i}, \boldsymbol{j}\}$ be the canonical basis of $\mathbb{R}^2$.

Consider a trajectory along the unit circle:

$$\boldsymbol{x}(t) = \begin{pmatrix} \cos t \\ \sin t \end{pmatrix}, \quad t \in \mathbb{R}$$

This is a function $\boldsymbol{x} : \mathbb{R} \to \mathbb{R}^2$ that we call a **parametric equation**:

- t is the parameter (often interpreted as time)
- For each t, $\boldsymbol{x}(t)$ is a point in $\mathbb{R}^2$; as t varies, $\boldsymbol{x}(t)$ traces out the trajectory



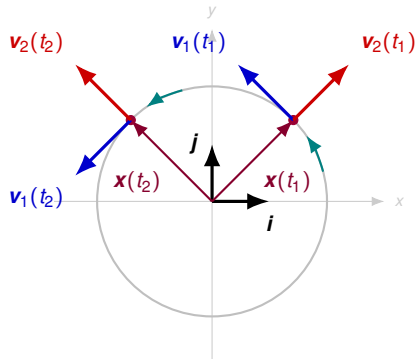
A moving basis along the trajectory

At each point $\boldsymbol{x}(t)$ on the trajectory, define a moving basis $\mathcal{C}(t) = \{\boldsymbol{v}_1(t), \boldsymbol{v}_2(t)\}$:

- $\boldsymbol{v}_1(t) = \boldsymbol{x}'(t) = \begin{pmatrix} -\sin t \\ \cos t \end{pmatrix}$ (unit tangent)
- $\boldsymbol{v}_2(t) = \begin{pmatrix} \cos t \\ \sin t \end{pmatrix}$ (outward unit normal)

Since $\boldsymbol{v}_1(t) \bullet \boldsymbol{v}_2(t) = 0$ and $\|\boldsymbol{v}_1\| = \|\boldsymbol{v}_2\| = 1$, $\mathcal{C}(t)$ is an orthonormal basis for all t

As t changes, the basis $\{\boldsymbol{v}_1(t), \boldsymbol{v}_2(t)\}$ moves and rotates along with $\boldsymbol{x}(t)$



Coordinates in the moving basis

The change of basis matrix from $\mathcal{C}(t)$ to the canonical basis $\mathcal{B}$ is

$$P_{\mathcal{B}\leftarrow\mathcal{C}(t)} = [\boldsymbol{v}_1(t) \;\; \boldsymbol{v}_2(t)] = \begin{pmatrix} -\sin t & \cos t \\ \cos t & \sin t \end{pmatrix}$$

Since $P_{\mathcal{B}\leftarrow\mathcal{C}(t)}$ is symmetric and orthogonal, the change of basis matrix to $\mathcal{C}(t)$ is:

$$P_{\mathcal{C}(t)\leftarrow\mathcal{B}} = P^{-1}_{\mathcal{B}\leftarrow\mathcal{C}(t)} = P^{T}_{\mathcal{B}\leftarrow\mathcal{C}(t)} = \begin{pmatrix} -\sin t & \cos t \\ \cos t & \sin t \end{pmatrix}$$

In the moving basis $\mathcal{C}(t)$, the coordinates of position and velocity are:

$$[\boldsymbol{x}(t)]_{\mathcal{C}(t)} = P_{\mathcal{C}(t)\leftarrow\mathcal{B}} \begin{pmatrix} \cos t \\ \sin t \end{pmatrix} = \begin{pmatrix} 0 \\ 1 \end{pmatrix}, \qquad [\boldsymbol{x}'(t)]_{\mathcal{C}(t)} = P_{\mathcal{C}(t)\leftarrow\mathcal{B}} \begin{pmatrix} -\sin t \\ \cos t \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$$

In the local frame, the particle is always at distance 1 along the normal axis with velocity 1 along the tangent

Choosing the inward normal $-\boldsymbol{v}_2(t)$ *simply gives* $[\boldsymbol{x}(t)]_{\mathcal{C}(t)} = \begin{pmatrix} 0 \\ -1 \end{pmatrix}$

A running example: hockey players

Change of basis

A crash course on probability

Back to PCA

Hockey players

Why probability?

We said earlier that we would look for a basis in which the first dimension carries most information

But how do we define *information*?

We use concepts from probability and statistics to do so

A good measure of information is *variance* (how much data varies around the mean)

Brief “review” of some probability concepts

Proper definition of *probability* requires to use *measure theory*.. will not get into details here

A **random variable** X is a *measurable* function $X : \Omega \to E$, where Ω is a set of outcomes (*sample space*) and E is a measurable space

$\mathbb{P}(X \in S \subseteq E) = \mathbb{P}(\omega \in \Omega | X(\omega) \in S)$

Distribution function of a r.v., $F(x) = \mathbb{P}(X \leq x)$, describes the distribution of a r.v.

R.v. can be discrete or continuous or .. other things.

Variance

Definition 83 (Variance)

Let X be a random variable. The **variance** of X is given by

$$\operatorname{Var} X = E\left[(X - E(X))^2\right]$$

where E is the expected value

Covariance

Definition 84 (Covariance)

Let X, Y be jointly distributed random variables. The **covariance** of X and Y is given by

$$\operatorname{cov}(X, Y) = E\left[(X - E(X))\,(Y - E(Y))\right]$$

Note that $\operatorname{cov}(X, X) = E\left[(X - E(X))^2\right] = \operatorname{Var} X$

In practice: “true law” versus “observation”

In statistics: we reason on the *true law* of distributions, but we usually have only access to a sample

We then use **estimators** to .. estimate the value of a parameter, e.g., the mean, variance and covariance

Definition 85 (Unbiased estimators of the mean and variance)

Let $x_1, \ldots, x_n$ be data points (the *sample*) and

$$\bar{x} = \frac{1}{n} \sum_{i=1}^{n} x_i$$

be the **mean** of the data. An unbiased estimator of the variance of the sample is

$$\sigma^2 = \frac{1}{n-1} \sum_{i=1}^{n} (x_i - \bar{x})^2$$

Unbiased: the expected value of the estimator equals the true parameter value ($E(\bar{x}) = E(X)$ and $E(\sigma^2) = \text{Var } X$): on average, it does not systematically overestimate or underestimate the parameter

Variance of hockey players' height and weight

We compute the sample variance in R using the function `var`:

```
var(data$height)

## [1] 29.66176

var(data$weight)

## [1] 47.81112
```

The variance is about 29.7 cm^2 for height and 47.8 kg^2 for weight

Normalising does not change the variance

Centring (normalising) shifts data by a constant, leaving spread unchanged:

$$\text{Var}\,(X - c) = \text{Var}\,X$$

Indeed, computing the variance using the centred values:

```
var(data$height.c)
```

```
## [1] 29.66176
```

```
var(data$weight.c)
```

```
## [1] 47.81112
```

The variance is identical to that of the uncentred data

Definition 86 (Unbiased estimator of the covariance - 2D case)

Let $(x_1, y_1), \ldots, (x_n, y_n)$ be data points,

$$\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i \text{ and } \bar{y} = \frac{1}{n}\sum_{i=1}^{n} y_i$$

be the means of the data. An unbiased estimator of the covariance of the sample is

$$\text{cov}(x, y) = \frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})$$

Computation of covariance in R – “By hand” and with a function

We compute the means and sum directly from the definition:

```
n = length(data$height)
x_bar = sum(data$height) / n
y_bar = sum(data$weight) / n
sum((data$height - x_bar) * (data$weight - y_bar)) / (n - 1)

## [1] 26.63506

cov(data$height, data$weight)

## [1] 26.63506
```

The covariance is about 26.6 cm · kg, matching R’s built-in `cov`

What does covariance do?

Variance explains how data disperses around the mean, in a 1-D case

Covariance measures the relationship between two dimensions. E.g., height and weight

More than the exact value, the sign is important:

- $\text{cov}(X, Y) > 0$: both dimensions change in the same "direction"; e.g., larger height usually means higher weight
- $\text{cov}(X, Y) < 0$: both dimensions change in reverse directions; e.g., time spent on social media and performance in this class
- $\text{cov}(X, Y) = 0$: the dimensions are independent from one another; e.g., sex/gender and "intelligence"

The covariance matrix

Definition 87 (Covariance matrix)

Suppose p random variables $X_1, \ldots, X_p$. Then the covariance matrix is the symmetric matrix

$$\begin{pmatrix} \text{Var } X_1 & \text{cov}(X_1, X_2) & \cdots & \text{cov}(X_1, X_p) \\ \text{cov}(X_1, X_2) & \text{Var } X_2 & \cdots & \text{cov}(X_2, X_p) \\ \vdots & \vdots & & \vdots \\ \text{cov}(X_1, X_p) & \text{cov}(X_2, X_p) & \cdots & \text{Var } X_p \end{pmatrix}$$

The actual form of the matrix is below but the form above stems from the properties of covariance

$$\begin{pmatrix} \text{cov}(X_1, X_1) & \text{cov}(X_1, X_2) & \cdots & \text{cov}(X_1, X_p) \\ \text{cov}(X_2, X_1) & \text{cov}(X_2, X_2) & \cdots & \text{cov}(X_2, X_p) \\ \vdots & \vdots & & \vdots \\ \text{cov}(X_p, X_1) & \text{cov}(X_p, X_2) & \cdots & \text{cov}(X_p, X_p) \end{pmatrix}$$

Sample covariance matrix

Let $X \in \mathcal{M}_{np}$ be the data matrix (n observations in rows, p features or variables in columns) and $\widetilde{X} \in \mathcal{M}_{np}$ be the **centred data matrix**. Then the (sample) covariance matrix $S_X \in \mathcal{M}_p$ takes the form

$$S_X = \frac{1}{n-1}\widetilde{X}^T\widetilde{X}$$

where the data is centred!

Sometimes you will see $S_X = 1/(n-1)\widetilde{X}\widetilde{X}^T$. This is for matrices with observations in columns and features (variables) in rows. Just remember that you want the covariance matrix to have size the number of features (p), not observations (n), this will give you the order in which to take the product

Why is $S_X = \frac{1}{n-1}\widetilde{X}^T\widetilde{X}$? – Data matrix setup

Let us prove why the sample covariance matrix takes this matrix form

Suppose we have n observations of p features (variables) $X_1, \ldots, X_p$

Organise the centred data into the **centred data matrix** $\widetilde{X} \in \mathcal{M}_{np}$ where each row is an observation and each column is a centred feature:

$$\widetilde{X} = \begin{pmatrix} \tilde{x}_{11} & \tilde{x}_{12} & \cdots & \tilde{x}_{1p} \\ \tilde{x}_{21} & \tilde{x}_{22} & \cdots & \tilde{x}_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ \tilde{x}_{n1} & \tilde{x}_{n2} & \cdots & \tilde{x}_{np} \end{pmatrix} = \begin{pmatrix} \tilde{\boldsymbol{c}}_1 & \tilde{\boldsymbol{c}}_2 & \cdots & \tilde{\boldsymbol{c}}_p \end{pmatrix}$$

where $\tilde{x}_{ij} = x_{ij} - \bar{x}_j$ and column $\tilde{\boldsymbol{c}}_j = (\tilde{x}_{1j}, \tilde{x}_{2j}, \ldots, \tilde{x}_{nj})^T \in \mathbb{R}^n$ contains all n centred measurements of feature X_j

Because the data is **centred**, $\frac{1}{n}\sum_{i=1}^{n} \tilde{x}_{ij} = 0$ for all $j = 1, \ldots, p$

Why is $S_X = \frac{1}{n-1}\widetilde{X}^T\widetilde{X}$? – Sample covariances

Recall the definition of the sample covariance between features X_j and X_k:

$$\operatorname{cov}(X_j, X_k) = \frac{1}{n-1}\sum_{i=1}^{n}(x_{ij} - \bar{x}_j)(x_{ik} - \bar{x}_k)$$

Writing $\tilde{x}_{ij} = x_{ij} - \bar{x}_j$ and $\tilde{x}_{ik} = x_{ik} - \bar{x}_k$, this simplifies to:

$$\operatorname{cov}(X_j, X_k) = \frac{1}{n-1}\sum_{i=1}^{n}\tilde{x}_{ij}\tilde{x}_{ik}$$

In particular, when $j = k$, we obtain the sample variance of feature X_j:

$$\operatorname{Var} X_j = \operatorname{cov}(X_j, X_j) = \frac{1}{n-1}\sum_{i=1}^{n}\tilde{x}_{ij}^2$$

Notice: $\sum_{i=1}^{n}\tilde{x}_{ij}\tilde{x}_{ik} = \tilde{\boldsymbol{c}}_j^T\tilde{\boldsymbol{c}}_k$, the dot product of column $\tilde{\boldsymbol{c}}_j$ and column $\tilde{\boldsymbol{c}}_k$!

Why is $S_X = \frac{1}{n-1}\widetilde{X}^T\widetilde{X}$? – Matrix product $\widetilde{X}^T\widetilde{X}$

Now consider the matrix product $\widetilde{X}^T\widetilde{X}$:

$\widetilde{X}^T \in \mathcal{M}_{pn}$ has the columns of $\widetilde{X}$ as its rows:

$$\widetilde{X}^T = \begin{pmatrix} \tilde{\boldsymbol{c}}_1^T \\ \tilde{\boldsymbol{c}}_2^T \\ \vdots \\ \tilde{\boldsymbol{c}}_p^T \end{pmatrix} = \begin{pmatrix} \tilde{x}_{11} & \tilde{x}_{21} & \cdots & \tilde{x}_{n1} \\ \tilde{x}_{12} & \tilde{x}_{22} & \cdots & \tilde{x}_{n2} \\ \vdots & \vdots & \ddots & \vdots \\ \tilde{x}_{1p} & \tilde{x}_{2p} & \cdots & \tilde{x}_{np} \end{pmatrix}$$

Since $\widetilde{X}^T \in \mathcal{M}_{pn}$ and $\widetilde{X} \in \mathcal{M}_{np}$, the product $\widetilde{X}^T\widetilde{X}$ is a $p \times p$ matrix

Its (j,k)-entry is the product of row j of $\widetilde{X}^T$ and column k of $\widetilde{X}$:

$$(\widetilde{X}^T\widetilde{X})_{jk} = \tilde{\boldsymbol{c}}_j^T\tilde{\boldsymbol{c}}_k = \sum_{i=1}^{n} \tilde{x}_{ij}\tilde{x}_{ik}$$

Why is $S_X = \frac{1}{n-1}\widetilde{X}^T\widetilde{X}$? – Conclusion

Dividing by $n-1$, each entry of $\frac{1}{n-1}\widetilde{X}^T\widetilde{X}$ is:

$$\left(\frac{1}{n-1}\widetilde{X}^T\widetilde{X}\right)_{jk} = \frac{1}{n-1}\sum_{i=1}^{n}\tilde{x}_{ij}\tilde{x}_{ik} = \begin{cases}\text{Var } X_j & \text{if } j = k\\ \text{cov}(X_j, X_k) & \text{if } j \neq k\end{cases}$$

Thus, writing out the matrix:

$$\frac{1}{n-1}\widetilde{X}^T\widetilde{X} = \begin{pmatrix}\text{Var } X_1 & \text{cov}(X_1, X_2) & \cdots & \text{cov}(X_1, X_p)\\ \text{cov}(X_2, X_1) & \text{Var } X_2 & \cdots & \text{cov}(X_2, X_p)\\ \vdots & \vdots & \ddots & \vdots\\ \text{cov}(X_p, X_1) & \text{cov}(X_p, X_2) & \cdots & \text{Var } X_p\end{pmatrix} = S_X$$

This proves that $S_X = \frac{1}{n-1}\widetilde{X}^T\widetilde{X}$ for the centred data matrix $\widetilde{X}$

What if observations are in columns?

Suppose the centred data is stored with observations in columns and features (variables) in rows:

$$\widetilde{Y} = \widetilde{X}^T \in \mathcal{M}_{pn}$$

where row j of $\widetilde{Y}$ is $\tilde{\boldsymbol{c}}_j^T = (\tilde{x}_{1j}, \ldots, \tilde{x}_{nj}) \in \mathbb{R}^{1\times n}$ (feature X_j)

Then $\widetilde{Y}\widetilde{Y}^T \in \mathcal{M}_p$, and its (j,k)-entry is:

$$(\widetilde{Y}\widetilde{Y}^T)_{jk} = \tilde{\boldsymbol{c}}_j^T \tilde{\boldsymbol{c}}_k = \sum_{i=1}^{n} \tilde{x}_{ij}\tilde{x}_{ik}$$

Dividing by $n-1$:

$$S_Y = \frac{1}{n-1}\widetilde{Y}\widetilde{Y}^T = S_X$$

The resulting sample covariance matrix is identical

What if the data is not centred beforehand?

Suppose $X \in \mathcal{M}_{np}$ is the **uncentred** data matrix

Let $\mathbf{1} = (1, 1, \ldots, 1)^T \in \mathbb{R}^n$. The vector of column means is $\bar{\boldsymbol{x}}^T = \frac{1}{n}\mathbf{1}^T X \in \mathbb{R}^{1\times p}$

Subtracting the means from each row of X gives the **centred data matrix** $\widetilde{X} \in \mathcal{M}_{np}$:

$$\widetilde{X} = X - \mathbf{1}\bar{\boldsymbol{x}}^T = X - \frac{1}{n}\mathbf{1}\mathbf{1}^T X = \left(\mathbb{I}_n - \frac{1}{n}\mathbf{1}\mathbf{1}^T\right) X = C_n X$$

where $C_n = \mathbb{I}_n - \frac{1}{n}\mathbf{1}\mathbf{1}^T \in \mathcal{M}_n$ is the **centring matrix**

C_n is symmetric ($C_n^T = C_n$) and idempotent ($C_n^2 = C_n$), so:

$$S_X = \frac{1}{n-1}\widetilde{X}^T\widetilde{X} = \frac{1}{n-1}X^T C_n^T C_n X = \frac{1}{n-1}X^T C_n X$$

Covariance matrix of hockey players in R

When passed a data frame or matrix, `cov` computes the covariance matrix:

```
cov(data[, c("height", "weight")])

##           height   weight
## height 29.66176 26.63506
## weight 26.63506 47.81112
```

The diagonal entries are the variances (Var (height) ≈ 29.7, Var (weight) ≈ 47.8) and the off-diagonal entries are cov(height, weight) ≈ 26.6

A running example: hockey players

Change of basis

A crash course on probability

Back to PCA

Hockey players

Setting things up

I will use notation (mostly) as in Joliffe's *Principal Component Analysis* (PDF of older version available for free from UofM Libraries)

$\boldsymbol{x} = (x_1, \ldots, x_p)$ vector of p random variables

We seek a linear function $\boldsymbol{\alpha}_1^T \boldsymbol{x}$ with maximum variance, where $\boldsymbol{\alpha}_1 = (\alpha_{11}, \ldots, \alpha_{1p})$, i.e.,

$$\boldsymbol{\alpha}_1^T \boldsymbol{x} = \sum_{j=1}^{p} \alpha_{1j} x_j$$

Then we seek a linear function $\boldsymbol{\alpha}_2^T \boldsymbol{x}$ with maximum variance, uncorrelated to $\boldsymbol{\alpha}_1^T \boldsymbol{x}$

And we continue...

At *k*th stage, we find a linear function $\boldsymbol{\alpha}_k^T \boldsymbol{x}$ with maximum variance, uncorrelated to $\boldsymbol{\alpha}_1^T \boldsymbol{x}, \ldots, \boldsymbol{\alpha}_{k-1}^T \boldsymbol{x}$

$\boldsymbol{\alpha}_i^T \boldsymbol{x}$ is the *i*th **principal component** (PC)

Case of known covariance matrix

Suppose we know Σ, covariance matrix of $\boldsymbol{x}$ (i.e., typically: we know $\boldsymbol{x}$)

Then the kth PC is

$$z_k = \boldsymbol{\alpha}_k^T \boldsymbol{x}$$

where $\boldsymbol{\alpha}_k$ is an eigenvector of Σ corresponding to the kth largest eigenvalue λ_k

If, additionally, $\|\boldsymbol{\alpha}_k\| = \boldsymbol{\alpha}_k^T \boldsymbol{\alpha} = 1$, then $\lambda_k = \text{Var } z_k$

Why is that?

Let us start with

$$\boldsymbol{\alpha}_1^T \boldsymbol{x}$$

We want maximum variance, where $\boldsymbol{\alpha}_1 = (\alpha_{11}, \ldots, \alpha_{1p})$, i.e.,

$$\boldsymbol{\alpha}_1^T \boldsymbol{x} = \sum_{j=1}^{p} \alpha_{1j} x_j$$

with the constraint that $\|\boldsymbol{\alpha}_1\| = 1$

We have

$$\text{Var } \boldsymbol{\alpha}_1^T \boldsymbol{x} = \boldsymbol{\alpha}_1^T \Sigma \boldsymbol{\alpha}_1$$

Computation of Var $(\alpha_1^T \boldsymbol{x})$

$$
\begin{aligned}
\text{Var}\,(\alpha_1^T \boldsymbol{x}) &= E\left[\left(\alpha_1^T \boldsymbol{x} - E\left(\alpha_1^T \boldsymbol{x}\right)\right)^2\right] \\
&= E\left[\left(\alpha_1^T \boldsymbol{x} - \alpha_1^T E(\boldsymbol{x})\right)^2\right] \\
&= E\left[\left(\alpha_1^T (\boldsymbol{x} - E(\boldsymbol{x}))\right)^2\right] \\
&= E\left[\left(\alpha_1^T (\boldsymbol{x} - E(\boldsymbol{x}))\right)\left(\alpha_1^T (\boldsymbol{x} - E(\boldsymbol{x}))\right)^T\right] \\
&= E\left[\alpha_1^T (\boldsymbol{x} - E(\boldsymbol{x}))(\boldsymbol{x} - E(\boldsymbol{x}))^T \alpha_1\right] \\
&= \alpha_1^T E\left[(\boldsymbol{x} - E(\boldsymbol{x}))(\boldsymbol{x} - E(\boldsymbol{x}))^T\right] \alpha_1 \\
&= \alpha_1^T \Sigma \alpha_1
\end{aligned}
$$

Objective

We want to maximise Var $\boldsymbol{\alpha}_1^T \boldsymbol{x}$, i.e.,

$$\boldsymbol{\alpha}_1^T \Sigma \boldsymbol{\alpha}_1$$

under the constraint that $\|\boldsymbol{\alpha}_1\| = 1$

$\Longrightarrow$ use **Lagrange multipliers**

Maximisation using Lagrange multipliers

(Recall from the mathematical preliminaries slides)

As presented in Unit 02 (*Mathematical preliminaries*), the method of **Lagrange multipliers** establishes that to find the extremum of a function $f(x_1, \ldots, x_n)$ under the constraint $g(x_1, \ldots, x_n) = k$:

1. Solve

$$\begin{aligned} \nabla f(x_1, \ldots, x_n) &= \lambda \nabla g(x_1, \ldots, x_n) \\ g(x_1, \ldots, x_n) &= k \end{aligned}$$

   where $\nabla = \left(\frac{\partial}{\partial x_1}, \ldots, \frac{\partial}{\partial x_n} \right)$ is the **gradient operator**

2. Plug all solutions into $f(x_1, \ldots, x_n)$ and find maximum values (provided values exist and $\nabla g \neq \mathbf{0}$ there)

$\lambda \in \mathbb{R}$ is the **Lagrange multiplier**

Geometrically, this corresponds to the tangency condition: gradient vectors are **parallel** ($\nabla f = \lambda \nabla g$)

Bear with me..

(You may experience a brief period of discomfort)

$\boldsymbol{\alpha}_1^T \Sigma \boldsymbol{\alpha}_1$ and $\|\boldsymbol{\alpha}_1\|^2 = \boldsymbol{\alpha}_1^T \boldsymbol{\alpha_1}$ are functions of $\boldsymbol{\alpha}_1 = (\alpha_{11}, \ldots, \alpha_{1p})$

In the notation of the previous slide, we want the max of

$$f(\alpha_{11}, \ldots, \alpha_{1p}) := \boldsymbol{\alpha}_1^T \Sigma \boldsymbol{\alpha}_1$$

under the constraint that

$$g(\alpha_{11}, \ldots, \alpha_{1p}) := \boldsymbol{\alpha}_1^T \boldsymbol{\alpha_1} = 1$$

and with gradient operator

$$\nabla = \left(\frac{\partial}{\partial \alpha_{11}}, \ldots, \frac{\partial}{\partial \alpha_{1p}} \right)$$

Effect of ∇ on g

g is easiest to see:

$$\begin{aligned}
\nabla g(\alpha_{11}, \ldots, \alpha_{1p}) &= \left(\frac{\partial}{\partial \alpha_{11}}, \ldots, \frac{\partial}{\partial \alpha_{1p}} \right) (\alpha_{11}, \ldots, \alpha_{1p}) \begin{pmatrix} \alpha_{11} \\ \vdots \\ \alpha_{1p} \end{pmatrix} \\
&= \left(\frac{\partial}{\partial \alpha_{11}}, \ldots, \frac{\partial}{\partial \alpha_{1p}} \right) \left(\alpha_{11}^2 + \cdots + \alpha_{1p}^2 \right) \\
&= (2\alpha_{11}, \ldots, 2\alpha_{1p}) \\
&= 2\boldsymbol{\alpha}_1
\end{aligned}$$

(And that's a general result: $\nabla \|\boldsymbol{x}\|_2^2 = 2\boldsymbol{x}$ with $\|\cdot\|_2$ the Euclidean norm)

Effect of ∇ on f

Expand (write $\Sigma = [s_{ij}]$ and do not exploit symmetry)

$$\begin{aligned}
\boldsymbol{\alpha}_1^T \Sigma \boldsymbol{\alpha}_1 &= (\alpha_{11}, \ldots, \alpha_{1p}) \begin{pmatrix} s_{11} & s_{12} & \cdots & s_{1p} \\ s_{21} & s_{22} & \cdots & s_{2p} \\ \vdots & \vdots & & \vdots \\ s_{p1} & s_{p2} & & s_{pp} \end{pmatrix} \begin{pmatrix} \alpha_{11} \\ \alpha_{12} \\ \vdots \\ \alpha_{1p} \end{pmatrix} \\
&= (\alpha_{11}, \ldots, \alpha_{1p}) \begin{pmatrix} s_{11}\alpha_{11} + s_{12}\alpha_{12} + \cdots + s_{1p}\alpha_{1p} \\ s_{21}\alpha_{11} + s_{22}\alpha_{12} + \cdots + s_{2p}\alpha_{1p} \\ \vdots \\ s_{p1}\alpha_{11} + s_{p2}\alpha_{12} + \cdots + s_{pp}\alpha_{1p} \end{pmatrix} \\
&= (s_{11}\alpha_{11} + s_{12}\alpha_{12} + \cdots + s_{1p}\alpha_{1p})\alpha_{11} \\
&\quad + (s_{21}\alpha_{11} + s_{22}\alpha_{12} + \cdots + s_{2p}\alpha_{1p})\alpha_{12} \\
&\quad \vdots \\
&\quad + (s_{p1}\alpha_{11} + s_{p2}\alpha_{12} + \cdots + s_{pp}\alpha_{1p})\alpha_{1p}
\end{aligned}$$

We have

$$\begin{aligned}\boldsymbol{\alpha}_1^T \Sigma \boldsymbol{\alpha}_1 = &(s_{11}\alpha_{11} + s_{12}\alpha_{12} + \cdots + s_{1p}\alpha_{1p})\alpha_{11} \\ &+ (s_{21}\alpha_{11} + s_{22}\alpha_{12} + \cdots + s_{2p}\alpha_{1p})\alpha_{12} \\ &\vdots \\ &+ (s_{p1}\alpha_{11} + s_{p2}\alpha_{12} + \cdots + s_{pp}\alpha_{1p})\alpha_{1p}\end{aligned}$$

$$\begin{aligned}\implies \frac{\partial}{\partial \alpha_{11}} \boldsymbol{\alpha}_1^T \Sigma \boldsymbol{\alpha}_1 &= (s_{11}\alpha_{11} + s_{12}\alpha_{12} + \cdots + s_{1p}\alpha_{1p}) + s_{11}\alpha_{11} \\ &\quad + s_{21}\alpha_{12} + \cdots + s_{p1}\alpha_{1p} \\ &= s_{11}\alpha_{11} + s_{12}\alpha_{12} + \cdots + s_{1p}\alpha_{1p} \\ &\quad + s_{11}\alpha_{11} + s_{21}\alpha_{12} + \cdots + s_{p1}\alpha_{1p} \\ &= 2(s_{11}\alpha_{11} + s_{12}\alpha_{12} + \cdots + s_{1p}\alpha_{1p})\end{aligned}$$

(last equality stems from symmetry of Σ)

In general, for $i = 1, \ldots, p$,

$$\begin{aligned}\frac{\partial}{\partial \alpha_{1i}} \boldsymbol{\alpha}_1^T \Sigma \boldsymbol{\alpha}_1 &= s_{i1}\alpha_{11} + s_{i2}\alpha_{12} + \cdots + s_{ip}\alpha_{1p} \\ &\quad + s_{i1}\alpha_{11} + s_{2i}\alpha_{12} + \cdots + s_{pi}\alpha_{1p} \\ &= 2(s_{i1}\alpha_{11} + s_{i2}\alpha_{12} + \cdots + s_{ip}\alpha_{1p})\end{aligned}$$

(because of symmetry of Σ)

As a consequence,

$$\nabla \boldsymbol{\alpha}_1^T \Sigma \boldsymbol{\alpha}_1 = 2\Sigma \boldsymbol{\alpha}_1$$

So solving

$$\nabla f(x_1, \ldots, x_n) = \lambda \nabla g(x_1, \ldots, x_n)$$

means solving

$$2\Sigma\boldsymbol{\alpha}_1 = \lambda 2\boldsymbol{\alpha}_1$$

i.e.,

$$\Sigma\boldsymbol{\alpha}_1 = \lambda\boldsymbol{\alpha}_1$$

$\implies$ $(\lambda, \boldsymbol{\alpha}_1)$ eigenpair of Σ, with $\boldsymbol{\alpha}_1$ having unit length

Picking the right eigenvalue

$(\lambda, \boldsymbol{\alpha}_1)$ eigenpair of Σ, with $\boldsymbol{\alpha}_1$ having unit length

But which λ to choose?

Recall that we want Var $\boldsymbol{\alpha}_1^T \boldsymbol{x} = \boldsymbol{\alpha}_1^T \Sigma \boldsymbol{\alpha}_1$ maximal

We have

$$\text{Var } \boldsymbol{\alpha}_1^T \boldsymbol{x} = \boldsymbol{\alpha}_1^T \Sigma \boldsymbol{\alpha}_1 = \boldsymbol{\alpha}_1^T (\Sigma \boldsymbol{\alpha}_1) = \boldsymbol{\alpha}_1^T (\lambda \boldsymbol{\alpha}_1) = \lambda (\boldsymbol{\alpha}_1^T \boldsymbol{\alpha}_1) = \lambda$$

$\Longrightarrow$ we pick $\lambda = \lambda_1$, the largest eigenvalue (covariance matrix symmetric so eigenvalues real)

What we have this far..

The first principal component is $\boldsymbol{\alpha}_1^T \boldsymbol{x}$ and has variance λ_1, where λ_1 the largest eigenvalue of Σ and $\boldsymbol{\alpha}_1$ an associated eigenvector with $\|\boldsymbol{\alpha}_1\| = 1$

We want the second principal component to be *uncorrelated* with $\boldsymbol{\alpha}_1^T \boldsymbol{x}$ and to have maximum variance Var $\boldsymbol{\alpha}_2^T \boldsymbol{x} = \boldsymbol{\alpha}_2^T \Sigma \boldsymbol{\alpha}_2$, under the constraint that $\|\boldsymbol{\alpha}_2\| = 1$

$\boldsymbol{\alpha}_2^T \boldsymbol{x}$ uncorrelated to $\boldsymbol{\alpha}_1^T \boldsymbol{x}$ if $\text{cov}(\boldsymbol{\alpha}_1^T \boldsymbol{x}, \boldsymbol{\alpha}_2^T \boldsymbol{x}) = 0$

We have

$$\begin{aligned}
\text{cov}(\boldsymbol{\alpha}_1^T \boldsymbol{x}, \boldsymbol{\alpha}_2^T \boldsymbol{x}) &= \boldsymbol{\alpha}_1^T \Sigma \boldsymbol{\alpha}_2 \\
&= \boldsymbol{\alpha}_2^T \Sigma^T \boldsymbol{\alpha}_1 \\
&= \boldsymbol{\alpha}_2^T \Sigma \boldsymbol{\alpha}_1 \quad [\Sigma \text{ symmetric}] \\
&= \boldsymbol{\alpha}_2^T (\lambda_1 \boldsymbol{\alpha}_1) \\
&= \lambda \boldsymbol{\alpha}_2^T \boldsymbol{\alpha}_1
\end{aligned}$$

So $\boldsymbol{\alpha}_2^T \boldsymbol{x}$ uncorrelated to $\boldsymbol{\alpha}_1^T \boldsymbol{x}$ if $\boldsymbol{\alpha}_1 \perp \boldsymbol{\alpha}_2$

This is beginning to sound a lot like Gram-Schmidt, no?

In short

Take whatever covariance matrix is available to you (known Σ or sample S_X) – assume sample from now on for simplicity

For $i = 1, \ldots, p$, the ith principal component is

$$z_i = \boldsymbol{v}_i^T \boldsymbol{x}$$

where $\boldsymbol{v}_i$ eigenvector of S_X associated to the ith largest eigenvalue λ_i

If $\boldsymbol{v}_i$ is normalised, then $\lambda_i = \text{Var } z_i$

A 3D example – Sample covariance matrix

Suppose we have a dataset with $p = 3$ variables, $\boldsymbol{x} = (x_1, x_2, x_3)^T$, centred so that sample means are zero

After computing all sample variances and covariances, we obtain the sample covariance matrix:

$$S_X = \begin{pmatrix} 15 & 10 & 4 \\ 10 & 14 & 6 \\ 4 & 6 & 7 \end{pmatrix}$$

- Sample variances (diagonal): Var $(x_1) = 15$, Var $(x_2) = 14$, Var $(x_3) = 7$
- Sample covariances (off-diagonal): all variables are positively correlated ($\text{cov}(x_1, x_2) = 10$, $\text{cov}(x_1, x_3) = 4$, $\text{cov}(x_2, x_3) = 6$)
- Total sample variance in the dataset is the trace:

$$\text{tr}(S_X) = 15 + 14 + 7 = 36$$

A 3D example – Eigenvalues

To compute the principal components, we find the eigenvalues of S_X by solving $\det(S_X - \lambda I) = 0$:

$$\det \begin{pmatrix} 15-\lambda & 10 & 4 \\ 10 & 14-\lambda & 6 \\ 4 & 6 & 7-\lambda \end{pmatrix} = 0$$

This yields three distinct positive eigenvalues:

$$\lambda_1 = 27, \qquad \lambda_2 = 6, \qquad \lambda_3 = 3$$

Note that:

$$\lambda_1 + \lambda_2 + \lambda_3 = 27 + 6 + 3 = 36 = \mathrm{tr}(S_X)$$

As expected from theory, the total variance is preserved!

A 3D example – Variance explained

The variance of each principal component $z_i = \boldsymbol{v}_i^T \boldsymbol{x}$ is given by Var $z_i = \lambda_i$ (for normalized $\boldsymbol{v}_i$)

Proportion of total variance explained by each principal component:

- **1st PC (z_1)**:
$$\frac{\lambda_1}{\text{tr}(S_X)} = \frac{27}{36} = \frac{3}{4} = 75\%$$
- **2nd PC (z_2)**:
$$\frac{\lambda_2}{\text{tr}(S_X)} = \frac{6}{36} = \frac{1}{6} \approx 16.67\%$$
- **3rd PC (z_3)**:
$$\frac{\lambda_3}{\text{tr}(S_X)} = \frac{3}{36} = \frac{1}{12} \approx 8.33\%$$

The first two principal components together account for $\frac{27+6}{36} = \frac{33}{36} \approx 91.67\%$ of the total variance

A 3D example – Eigenvectors

Next, we find the associated normalized eigenvectors by solving $(S_X - \lambda_i I)\boldsymbol{v}_i = \mathbf{0}$ subject to $\|\boldsymbol{v}_i\| = 1$:

- For $\lambda_1 = 27$:

$$\boldsymbol{v}_1 = \frac{1}{3}\begin{pmatrix}2\\2\\1\end{pmatrix} = \begin{pmatrix}2/3\\2/3\\1/3\end{pmatrix}, \qquad \|\boldsymbol{v}_1\|^2 = \frac{4+4+1}{9} = 1$$

- For $\lambda_2 = 6$:

$$\boldsymbol{v}_2 = \frac{1}{3}\begin{pmatrix}-2\\1\\2\end{pmatrix} = \begin{pmatrix}-2/3\\1/3\\2/3\end{pmatrix}, \qquad \|\boldsymbol{v}_2\|^2 = \frac{4+1+4}{9} = 1$$

- For $\lambda_3 = 3$:

$$\boldsymbol{v}_3 = \frac{1}{3}\begin{pmatrix}1\\-2\\2\end{pmatrix} = \begin{pmatrix}1/3\\-2/3\\2/3\end{pmatrix}, \qquad \|\boldsymbol{v}_3\|^2 = \frac{1+4+4}{9} = 1$$

A 3D example – Orthonormality check

Since S_X is symmetric with distinct eigenvalues, the eigenvectors are mutually orthogonal:

$$\boldsymbol{v}_1^T \boldsymbol{v}_2 = \frac{1}{9}\big(2(-2) + 2(1) + 1(2)\big) = \frac{-4+2+2}{9} = 0$$
$$\boldsymbol{v}_1^T \boldsymbol{v}_3 = \frac{1}{9}\big(2(1) + 2(-2) + 1(2)\big) = \frac{2-4+2}{9} = 0$$
$$\boldsymbol{v}_2^T \boldsymbol{v}_3 = \frac{1}{9}\big(-2(1) + 1(-2) + 2(2)\big) = \frac{-2-2+4}{9} = 0$$

Hence, $V = \begin{pmatrix} \boldsymbol{v}_1 & \boldsymbol{v}_2 & \boldsymbol{v}_3 \end{pmatrix}$ is an orthogonal matrix ($V^T V = I$), forming an orthonormal basis of $\mathbb{R}^3$

A 3D example – The principal components

The principal components are the linear combinations $z_i = \boldsymbol{v}_i^T \boldsymbol{x}$:

$$z_1 = \boldsymbol{v}_1^T \boldsymbol{x} = \frac{2}{3}x_1 + \frac{2}{3}x_2 + \frac{1}{3}x_3$$
$$z_2 = \boldsymbol{v}_2^T \boldsymbol{x} = -\frac{2}{3}x_1 + \frac{1}{3}x_2 + \frac{2}{3}x_3$$
$$z_3 = \boldsymbol{v}_3^T \boldsymbol{x} = \frac{1}{3}x_1 - \frac{2}{3}x_2 + \frac{2}{3}x_3$$

Properties of these new variables:

- Var $z_1 = \lambda_1 = 27$, Var $z_2 = \lambda_2 = 6$, Var $z_3 = \lambda_3 = 3$
- $\text{cov}(z_i, z_j) = 0$ for all $i \neq j$ (uncorrelated)
- In the new basis, the covariance matrix is diagonal:

$$S_Z = V^T S_X V = \text{diag}(27, 6, 3)$$

A 3D example – Dimensionality reduction

Suppose we want to reduce the dimension of the data from 3 to 2:

- Keep only the first two components, z_1 and z_2
- z_1 and z_2 together preserve 91.67% of the total variance
- Each 3D data point $\boldsymbol{x} = (x_1, x_2, x_3)^T$ is mapped to its 2D coordinates $(z_1, z_2)^T$
- Approximation of $\boldsymbol{x}$ using only 2 components:

$$\boldsymbol{x} \approx z_1 \boldsymbol{v}_1 + z_2 \boldsymbol{v}_2$$

- Discarding z_3 loses only 8.33% of the variance

A 3D example – Verifying in R

We can confirm our calculations directly in R:

```
S = matrix(c(15, 10, 4, 10, 14, 6, 4, 6, 7), nrow = 3, byrow = TRUE)
ev = eigen(S)
ev$values
```

```
## [1] 27  6  3
```

```
# Proportion and cumulative proportion of variance explained
ev$values / sum(ev$values)
```

```
## [1] 0.75000000 0.16666667 0.08333333
```

```
cumsum(ev$values) / sum(ev$values)
```

```
## [1] 0.7500000 0.9166667 1.0000000
```

A running example: hockey players

Change of basis

A crash course on probability

Back to PCA

Hockey players

Covariance of height and weight

The function `cov` returns the sample covariance of two variables

Recall that `cov` yields the same result whether the data is centred or uncentred:

```
cov(data$height, data$weight)

## [1] 26.63506

cov(data$height.c, data$weight.c)

## [1] 26.63506
```

As seen in the scatter plot, there is a positive linear relationship between height and weight

Sample covariance matrix

Let us form the $n \times 2$ centred data matrix $\widetilde{X}$ and compute the sample covariance matrix $S = \frac{1}{n-1}\widetilde{X}^T\widetilde{X}$

```
X = as.matrix(data[, c("height.c", "weight.c")])
S = 1 / (dim(X)[1] - 1) * t(X) %*% X
S

##           height.c weight.c
## height.c 29.66176 26.63506
## weight.c 26.63506 47.81112
```

The off-diagonal entries match $\text{cov}(X_1, X_2)$

Sample covariance matrix – Variances

Let us check that the diagonal entries indeed match the individual sample variances:

```
var(X[, 1])

## [1] 29.66176

var(X[, 2])

## [1] 47.81112
```

Both match perfectly

Computing the principal components

We compute the eigenvalues and eigenvectors of S using `eigen`

```
ev = eigen(S)
ev

## eigen() decomposition
## $values
## [1] 66.87496 10.59793
##
## $vectors
##            [,1]       [,2]
## [1,] 0.5820222 -0.8131729
## [2,] 0.8131729  0.5820222
```

`eigen` returns eigenvalues in decreasing order and orthonormal eigenvectors

Variance explained

The total variance is

$$\text{tr}(S) = \lambda_1 + \lambda_2 = 66.87 + 10.6 = 77.47$$

The first principal component explains

$$\frac{\lambda_1}{\lambda_1 + \lambda_2} = \frac{66.87}{77.47} \approx 86.3\%$$

of the total variance in the height and weight data

The second principal component accounts for the remaining 13.7%

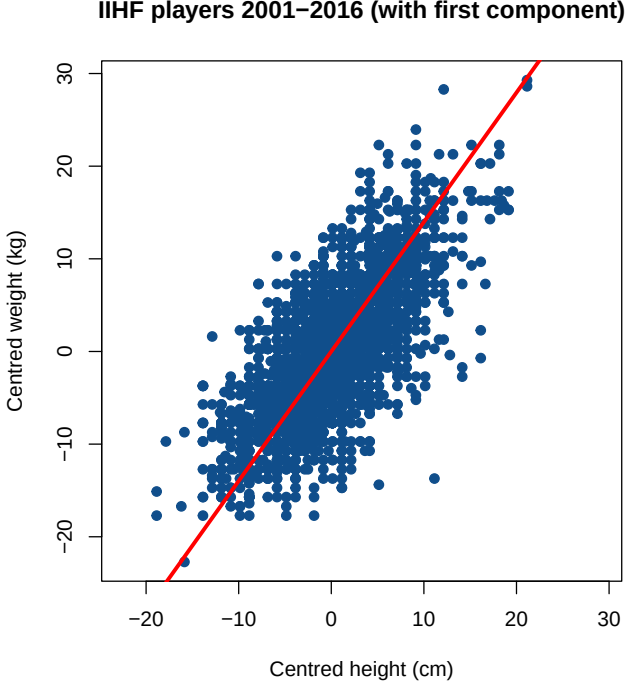
Plotting the first principal component

We superimpose the line spanned by the first eigenvector $\boldsymbol{v}_1$ on the centred data

The line passes through $(0, 0)$ and has slope $b = v_{21}/v_{11}$

To avoid visual distortion of angles, we use a square plot (`pty = "s"`, `asp = 1`) with equal ranges on both axes

```
par(pty = "s")
lims = range(c(data$height.c, data$weight.c))
plot(data$height.c, data$weight.c,
     xlim = lims, ylim = lims, asp = 1,
     pch = 19, col = "dodgerblue4",
     main = "IIHF players 2001-2016 (with first component)",
     xlab = "Centred height (cm)", ylab = "Centred weight (kg)")
abline(a = 0, b = ev$vectors[2, 1] / ev$vectors[1, 1],
       col = "red", lwd = 3)
```

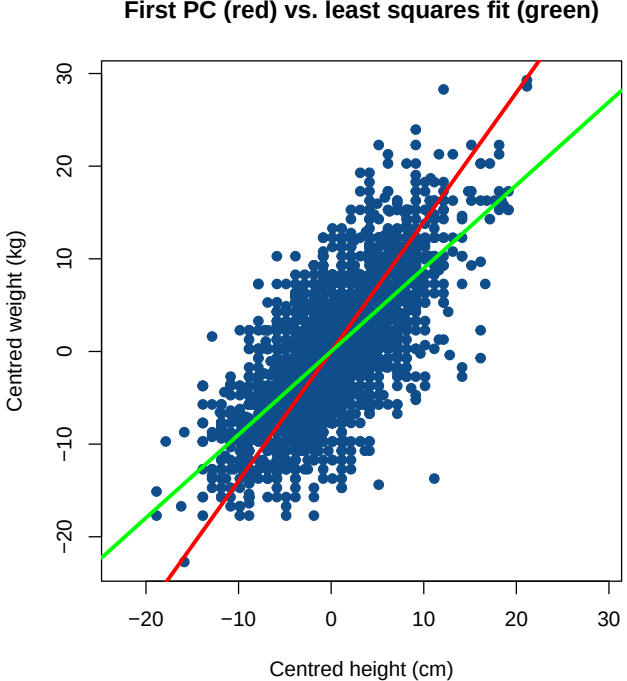

IIHF players 2001−2016 (with first component)
Centred weight (kg)
Centred height (cm)
30
20
10
0
−10
−20
−20
−10
0
10
20
30

First principal component vs. least squares

How does the first principal component line compare with ordinary least squares (OLS) regression?

Fit a linear model with zero intercept (since data is centred):

```
z = lm(weight.c ~ 0 + height.c, data = data)
coef(z)

##  height.c
## 0.8979593
```


First PC (red) vs. least squares fit (green)
Centred weight (kg)
Centred height (cm)
30
20
10
0
−10
−20
−20
−10
0
10
20
30

Why are the two lines different?

Ordinary least squares (OLS):

- Treats `height.c` as an exact predictor
- Minimises the sum of squared **vertical** residuals (errors parallel to the y-axis)

Principal component analysis (PCA):

- Treats both variables **symmetrically**
- Maximises the variance of the orthogonal projection
- Equivalently, minimises the sum of squared **orthogonal (perpendicular)** distances from points to the line

Rotating the data

Let us rotate the data so that the first principal component becomes the horizontal axis

A counter-clockwise rotation by an angle ϕ in $\mathbb{R}^2$ is represented by

$$R_\phi = \begin{pmatrix} \cos\phi & -\sin\phi \\ \sin\phi & \cos\phi \end{pmatrix}$$

The first eigenvector makes an angle $\theta > 0$ with the positive x-axis, $\tan\theta = v_{21}/v_{11}$

To align it with the horizontal axis, we rotate **clockwise** by θ, i.e., by $\phi = -\theta$

Computing the rotation matrix

```
theta = atan(ev$vectors[2, 1] / ev$vectors[1, 1])
theta
```

```
## [1] 0.949583
```

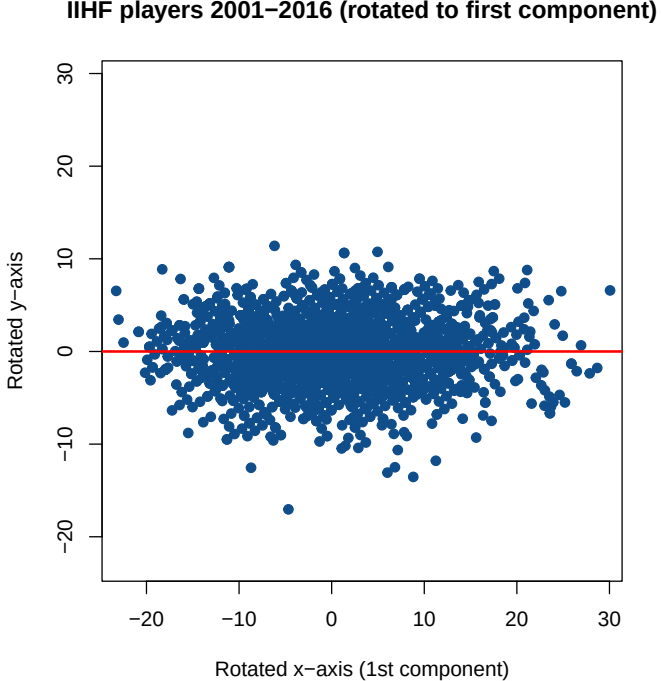
```
R_minus_theta = matrix(c(cos(-theta), -sin(-theta),
                         sin(-theta),  cos(-theta)),
                       nr = 2, byrow = TRUE)
R_minus_theta
```

```
##             [,1]      [,2]
## [1,]  0.5820222 0.8131729
## [2,] -0.8131729 0.5820222
```

Rotating the data

Apply the rotation matrix to each observation:

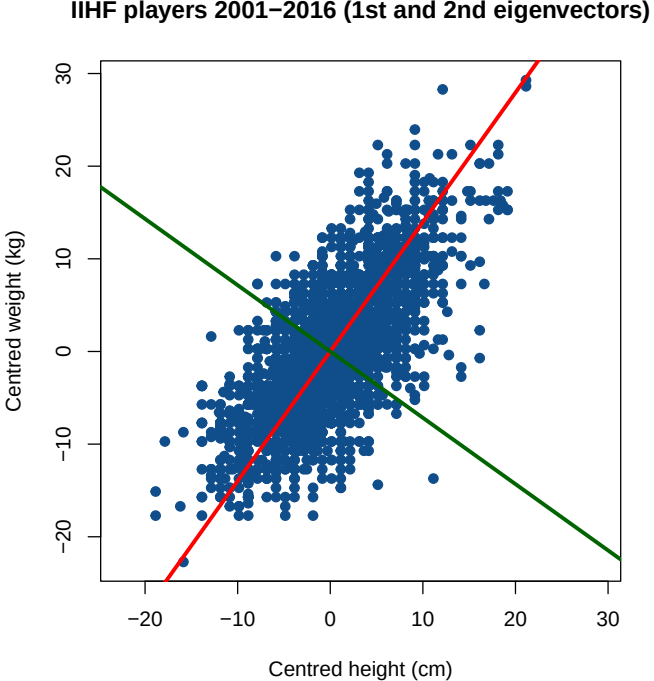
```
rot_pts = X %*% t(R_minus_theta)
data$height.c_r = rot_pts[, 1]
data$weight.c_r = rot_pts[, 2]
plot(data$height.c_r, data$weight.c_r,
     xlim = lims, ylim = lims, asp = 1,
     pch = 19, col = "dodgerblue4",
     xlab = "Rotated x-axis (1st component)",
     ylab = "Rotated y-axis",
     main = "IIHF players 2001-2016 (rotated to first component)")
abline(h = 0, col = "red", lwd = 2)
```


IIHF players 2001−2016 (rotated to first component)
Rotated y−axis
30
20
10
0
−10
−20
−20
−10
0
10
20
30
Rotated x−axis (1st component)

First and second principal components

Plot both eigenvectors on the centred data. Since S is symmetric, the eigenvectors are orthogonal (90° angle):

```
plot(data$height.c, data$weight.c,
     xlim = lims, ylim = lims, asp = 1,
     pch = 19, col = "dodgerblue4",
     main = "IIHF players 2001-2016 (1st and 2nd eigenvectors)",
     xlab = "Centred height (cm)", ylab = "Centred weight (kg)")
abline(a = 0, b = ev$vectors[2, 1] / ev$vectors[1, 1],
       col = "red", lwd = 3)
abline(a = 0, b = ev$vectors[2, 2] / ev$vectors[1, 2],
       col = "darkgreen", lwd = 3)
```


IIHF players 2001−2016 (1st and 2nd eigenvectors)
Centred weight (kg)
Centred height (cm)
30
20
10
0
−10
−20
−20
−10
0
10
20
30

Representing data in the new coordinate system

Rather than 2D trigonometry, express data in the orthonormal eigenbasis using our general change-of-basis machinery

Enforce orthonormality with `pracma::gramSchmidt` and compute the change-of-basis matrix using `pracma::rref`:

```
GS = pracma::gramSchmidt(A = ev$vectors, tol = 1e-10)
A = cbind(GS$Q, diag(2))
pracma::rref(A)
```

```
##      [,1] [,2]       [,3]      [,4]
## [1,]    1    0  0.5820222 0.8131729
## [2,]    0    1 -0.8131729 0.5820222
```

$P = Q^{-1} = Q^T$ is the change-of-basis matrix (right-hand block)

Projecting into the new basis

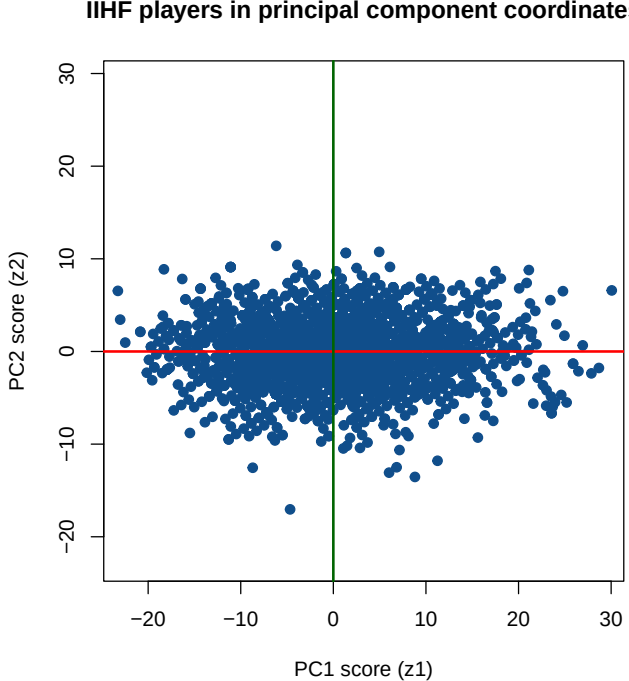
Multiplying $\widetilde{X}$ by P^T gives coordinates X_{new} in the new PC coordinate system:

```
P = pracma::rref(A)[, c(3, 4)]
P
```

```
##            [,1]      [,2]
## [1,]  0.5820222 0.8131729
## [2,] -0.8131729 0.5820222
```

```
X.new = X %*% t(P)
cov(X.new)
```

```
##               [,1]          [,2]
## [1,]  6.687496e+01 -2.328994e-13
## [2,] -2.328994e-13  1.059793e+01
```


IIHF players in principal component coordinates
PC2 score (z2)
PC1 score (z1)
30
20
10
0
−10
−20
−20
−10
0
10
20
30

Diagonal covariance in the new basis

As confirmed by `cov(X.new)`:

```
cov(X.new)

##                [,1]           [,2]
## [1,]  6.687496e+01 -2.328994e-13
## [2,] -2.328994e-13  1.059793e+01
```

- The sample covariance matrix in the new coordinate system is **diagonal**
- Diagonal entries equal $\lambda_1 = 66.87$ and $\lambda_2 = 10.6$
- Off-diagonal entry is zero: the new coordinates are completely **uncorrelated**

Hockey players

PCA with dedicated R packages

Multivariate hockey dataset

In practice, we use dedicated packages such as `prcomp`, `FactoMineR` and `ggbiplot`, especially with > 2 variables

We retain 4 numerical attributes (`height`, `weight`, `age`, `bmi`) and player `position` (`Defence`, `Forward`, `Goalkeeper`):

```
agg_num = aggregate(cbind(height, weight, age, bmi) ~ name,
                    data = raw_data, FUN = mean)
agg_pos = aggregate(position ~ name, data = raw_data,
                    FUN = function(x) x[1])
hockey_multi = merge(agg_pos, agg_num, by = "name")
hockey_multi$position = factor(
  hockey_multi$position,
  levels = c("D", "F", "G"),
  labels = c("Defence", "Forward", "Goalkeeper")
)
```

Multivariate hockey dataset

The aggregated multivariate dataset:

```
head(hockey_multi, n = 4)

##                  name position height weight      age      bmi
## 1     aagaard mikkel  Forward    180     81 20.53662 25.00000
## 2       aalto antti   Forward    187     96 27.15948 27.45289
## 3 aaltonen juhamatti  Forward    184     87 27.15606 25.69707
## 4  abdelkader justin  Forward    185     97 26.17933 28.34186
```

PCA using `prcomp`

Because variables are in different physical units (cm, kg, years, kg/m^2), we standardise to unit variance (`scale. = TRUE`):

```
hockey.pca = prcomp(
  hockey_multi[, c("height", "weight", "age", "bmi")],
  center = TRUE, scale. = TRUE
)
summary(hockey.pca)

## Importance of components:
##                           PC1    PC2    PC3     PC4
## Standard deviation     1.3933 1.1044 0.9154 0.03081
## Proportion of Variance 0.4853 0.3049 0.2095 0.00024
## Cumulative Proportion  0.4853 0.7903 0.9998 1.00000
```

Importance of components

- PC1 (48.5%) and PC2 (30.5%) explain 79.0% of the total variance across the four standardised variables
- The first three components capture 99.98% of the total variance
- PC4 accounts for almost zero variance (0.02%):

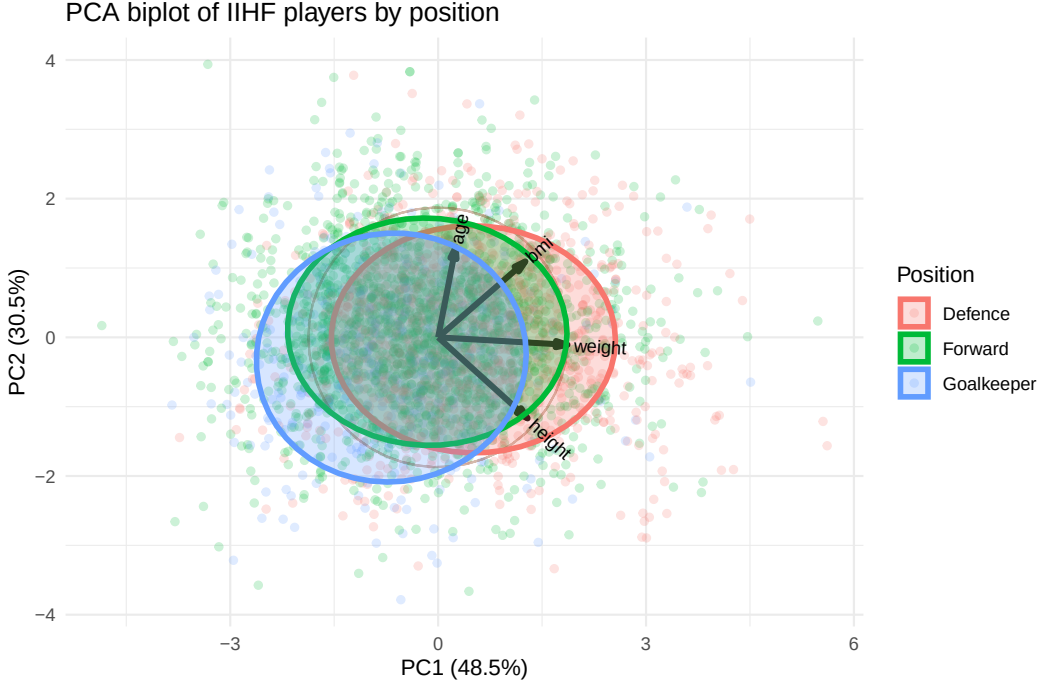
$$\text{BMI} = \frac{\text{weight}}{\text{height}^2}$$

Around the sample mean, BMI is nearly a linear combination of `height` and `weight`, so the 4D point cloud is practically three-dimensional!

PCA biplot with ggbiplot

The ggbiplot package overlays projected observations and variable loadings (arrows) with confidence ellipses:

```
library(ggbiplot)
p_biplot = ggbiplot(hockey.pca, obs.scale = 1, var.scale = 1,
         groups = hockey_multi$position, ellipse = TRUE,
         circle = TRUE, alpha = 0.2) +
  theme_minimal() +
  labs(title = "PCA biplot of IIHF players by position",
       color = "Position", fill = "Position")
print(p_biplot)
```


PCA biplot of IIHF players by position
PC2 (30.5%)
PC1 (48.5%)
4
2
0
−2
−4
−3
0
3
6
age
bmi
weight
height
Position
Defence
Forward
Goalkeeper

Interpreting the biplot

Variable loadings (arrows):

- `weight` points along positive PC1 (overall body size)
- PC2 contrasts `age` and `bmi` (pointing upward) against `height` (pointing downward)

Group ellipses (positions):

- Defencemen tend to have higher PC1 scores (taller and heavier on average)
- Forwards and goalkeepers cluster at lower PC1 scores

Multivariate analysis with FactoMineR

`FactoMineR::PCA` supports supplementary qualitative variables (`quali.sup = 1` for `position`):

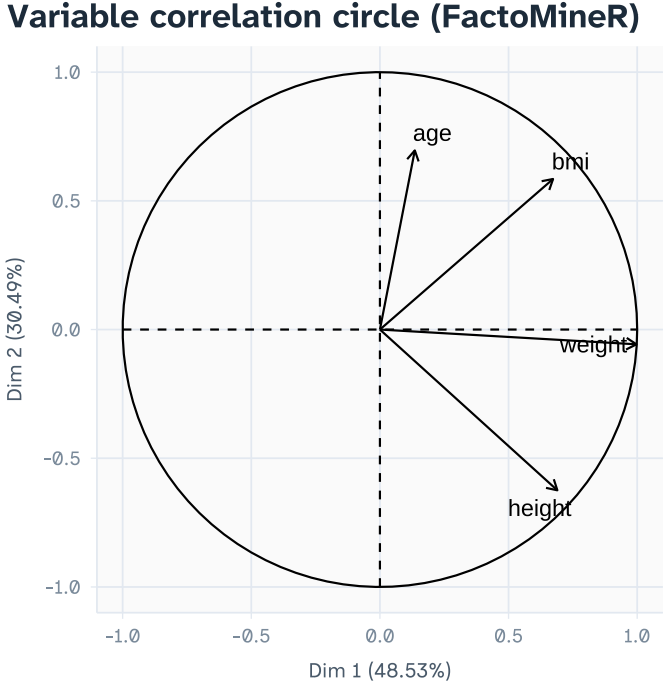
```
library(FactoMineR)
res.pca = PCA(hockey_multi[, c("position", "height", "weight",
                               "age", "bmi")],
              quali.sup = 1, scale.unit = TRUE, graph = FALSE)
```

Provides two key diagnostic visualisations:

1. **Variable correlation circle**: correlations with PC1 and PC2 inside the unit circle
2. **Factor map with group ellipses**: observations and barycentres by position

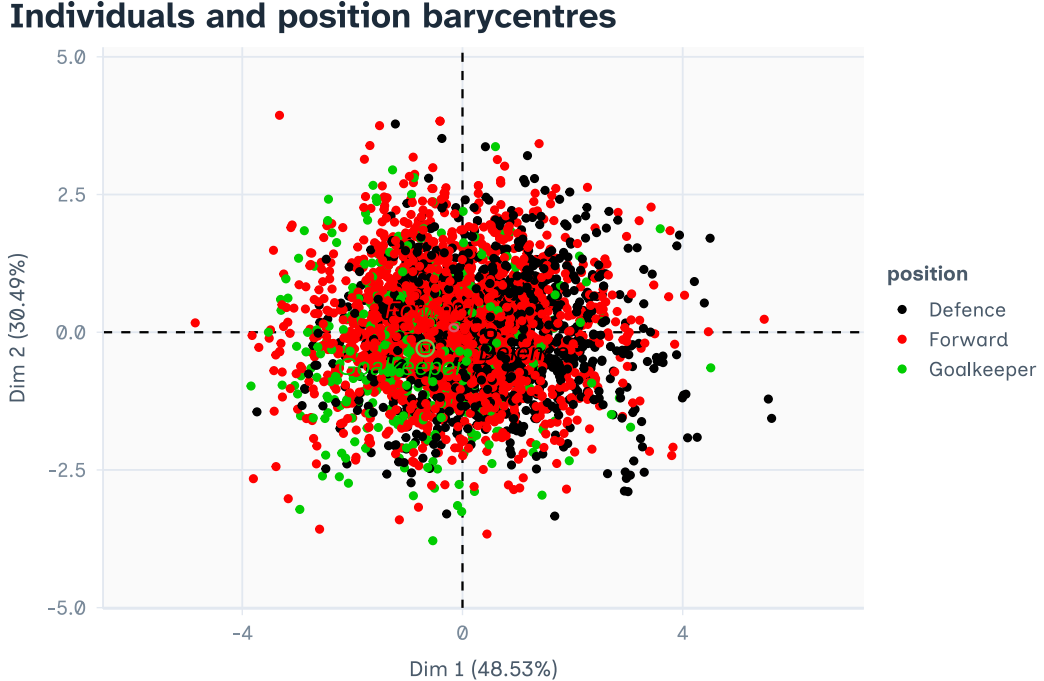
Variable correlation circle (FactoMineR)

```
plot.PCA(res.pca, axes = c(1, 2), choix = "var",
         title = "Variable correlation circle (FactoMineR)")
```


Variable correlation circle (FactoMineR)
1.0
0.5
0.0
-0.5
-1.0
Dim 2 (30.49%)
age
bmi
weight
height
-1.0
-0.5
0.0
0.5
1.0
Dim 1 (48.53%)

Position ellipses and barycentres (FactoMineR)

```
plotellipses(res.pca, keepvar = 1, label = "quali",
             title = "Individuals and position barycentres")
```


Individuals and position barycentres
5.0
2.5
0.0
-2.5
-5.0
Dim 2 (30.49%)
-4
0
4
Dim 1 (48.53%)
position
Defence
Forward
Goalkeeper

Hyperplanes, SVM and least-squares classification

MATH 2740 – Mathematics of Data Science – Unit U06

Julien Arino
julien.arino@umanitoba.ca

Department of Mathematics @ University of Manitoba

Fall 2026

The University of Manitoba campuses are located on original lands of Anishinaabeg, Ininew, Anisininew, Dakota and Dene peoples, and on the National Homeland of the Red River Métis. We respect the Treaties that were made on these territories, we acknowledge the harms and mistakes of the past, and we dedicate ourselves to move forward in partnership with Indigenous communities in a spirit of Reconciliation and collaboration.

Outline

Supervised vs unsupervised linear methods

Geometry of hyperplanes and support vector machines

Least-squares classification

Supervised vs unsupervised linear methods

Geometry of hyperplanes and support vector machines

Least-squares classification

Unsupervised vs supervised linear methods

In Unit 05 (PCA), we studied **unsupervised** dimensionality reduction:

- Given unlabelled feature vectors $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n \in \mathbb{R}^p$ (rows of the data matrix $X \in \mathcal{M}_{np}$)
- Find orthogonal directions that maximise total sample variance

Here, we study **supervised binary classification**:

- Given a labelled training dataset of n observations

$$(\boldsymbol{x}_1, y_1), \ldots, (\boldsymbol{x}_n, y_n)$$

  where $\boldsymbol{x}_i \in \mathbb{R}^p$ is a vector of p features and $y_i \in \{-1, +1\}$ (or $\{1, 2\}$) indicates the class to which observation $\boldsymbol{x}_i$ belongs
- Find a linear decision boundary or 1D projection that separates the two classes and predicts the label $\hat{y}(\boldsymbol{x})$ of any new point $\boldsymbol{x} \in \mathbb{R}^p$

Does maximum variance always separate classes?

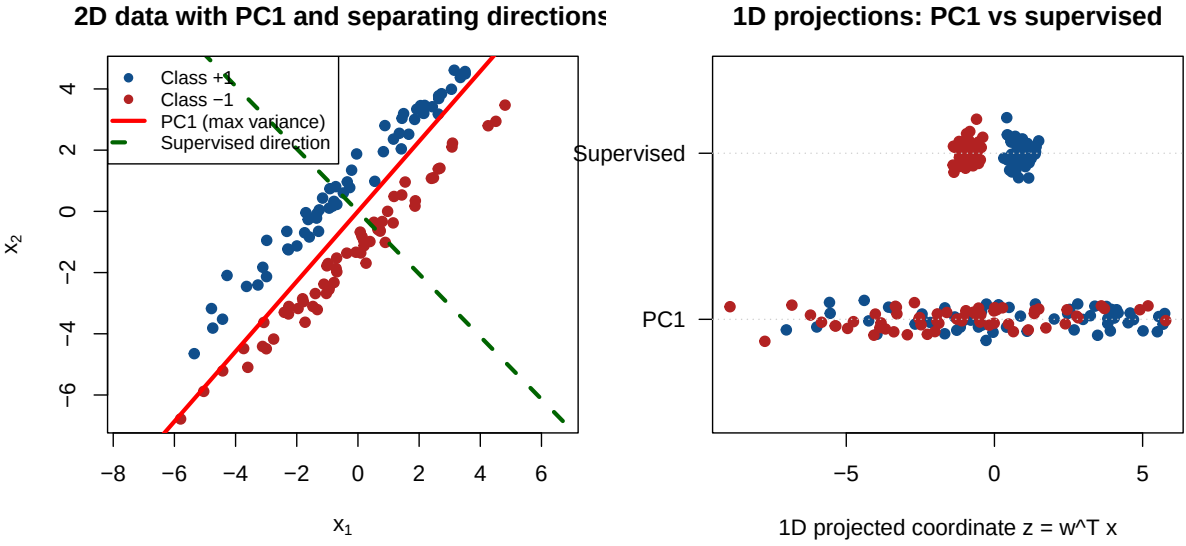
Suppose two classes form elongated, parallel point clouds that are closely spaced along their perpendicular direction

- Because PCA ignores the class labels y_i, it pools all n points together and finds the direction of largest overall spread
- When the spread *along* the clouds is larger than the distance *between* the clouds, the first principal component (PC1) runs parallel to the clouds
- Projecting onto PC1 mixes the two classes completely, whereas projecting onto the perpendicular direction separates them cleanly

When PCA fails to separate classes

Simulate two elongated parallel classes in $\mathbb{R}^2$ and compare PC1 (red) with the supervised separating direction (green)

```
set.seed(2740)
n_c = 60
t1 = rnorm(n_c, 0, 2.5); t2 = rnorm(n_c, 0, 2.5)
X1 = cbind(t1 + rnorm(n_c, 0, 0.25), t1 + 1.2 + rnorm(n_c, 0, 0.25))
X2 = cbind(t2 + rnorm(n_c, 0, 0.25), t2 - 1.2 + rnorm(n_c, 0, 0.25))
X_all = rbind(X1, X2)
y_all = c(rep(1, n_c), rep(-1, n_c))
pc1 = eigen(cov(X_all))$vectors[, 1]
SW = (n_c - 1) * (cov(X1) + cov(X2))
w_sep = solve(SW, colMeans(X1) - colMeans(X2))
w_sep = w_sep / sqrt(sum(w_sep^2))
```


2D data with PC1 and separating directions
Class +1
Class −1
PC1 (max variance)
Supervised direction
x2
x1
−8
−6
−4
−2
0
2
4
6
−6
−4
−2
0
2
4
1D projections: PC1 vs supervised
Supervised
PC1
−5
0
5
1D projected coordinate z = w^T x

Two linear classification frameworks in this unit

Both methods use a linear score $f(\boldsymbol{x}) = \boldsymbol{w}^T\boldsymbol{x} + b$ to classify points, but choose $\boldsymbol{w} \in \mathbb{R}^p$ and $b \in \mathbb{R}$ using different linear algebraic principles:

1. **Geometry of hyperplanes and support vector machines (SVM)**: characterise decision boundaries via normal vectors, orthogonal projections and maximum-margin separation

2. **Least-squares classification**: treat binary labels $y_i \in \{-1, +1\}$ as regression targets and solve the normal equations $(X^TX)\tilde{\boldsymbol{w}} = X^T\boldsymbol{y}$ from Unit 03

Supervised vs unsupervised linear methods

Geometry of hyperplanes and support vector machines

Least-squares classification

Geometry of hyperplanes and support vector machines

Hyperplanes and normal vectors

Orthogonal distance to a hyperplane

Linear separability and hard-margin SVM

Hyperplanes in $\mathbb{R}^p$

Definition 88 (Hyperplane in $\mathbb{R}^p$)

*Let $\boldsymbol{w} \in \mathbb{R}^p \setminus \{\mathbf{0}\}$ be a non-zero vector and $b \in \mathbb{R}$ be a scalar. The **hyperplane** determined by $\boldsymbol{w}$ and b is the set of points*

$$\mathcal{H} = \left\{ \boldsymbol{x} \in \mathbb{R}^p : \boldsymbol{w}^T \boldsymbol{x} + b = 0 \right\} = \{ \boldsymbol{x} \in \mathbb{R}^p : \boldsymbol{w} \bullet \boldsymbol{x} + b = 0 \}$$

*The vector $\boldsymbol{w}$ is called a **normal vector** to $\mathcal{H}$ and b is called the **offset** (or **bias**)*

- In $\mathbb{R}^2$, a hyperplane is a line $w_1 x_1 + w_2 x_2 + b = 0$
- In $\mathbb{R}^3$, a hyperplane is a plane $w_1 x_1 + w_2 x_2 + w_3 x_3 + b = 0$
- Multiplying $\boldsymbol{w}$ and b by any non-zero constant $\alpha \neq 0$ leaves the set $\mathcal{H}$ unchanged

Linear subspace vs affine subspace

When is a hyperplane $\mathcal{H} = \{\boldsymbol{x} \in \mathbb{R}^p : \boldsymbol{w}^T\boldsymbol{x} + b = 0\}$ a vector subspace of $\mathbb{R}^p$?

- A vector subspace must contain the zero vector $\mathbf{0}$, and $\mathbf{0} \in \mathcal{H} \iff \boldsymbol{w}^T\mathbf{0} + b = 0 \iff b = 0$
- When $b = 0$, the set $\mathcal{H}_0 = \{\boldsymbol{x} \in \mathbb{R}^p : \boldsymbol{w}^T\boldsymbol{x} = 0\} = \{\boldsymbol{w}\}^\perp$ is the orthogonal complement of span($\boldsymbol{w}$), which is a $(p-1)$-dimensional vector subspace of $\mathbb{R}^p$
- When $b \neq 0$, $\mathcal{H}$ does not pass through the origin; it is an **affine subspace** obtained by translating $\mathcal{H}_0$ by any fixed point $\boldsymbol{x}^* \in \mathcal{H}$:

$$\mathcal{H} = \boldsymbol{x}^* + \mathcal{H}_0 = \left\{\boldsymbol{x}^* + \boldsymbol{u} : \boldsymbol{w}^T\boldsymbol{u} = 0\right\}$$

Example in $\mathbb{R}^2$ – Hyperplanes with $b = 0$ and $b \neq 0$

In $\mathbb{R}^2$, hyperplanes are lines orthogonal to $\boldsymbol{w} \neq \mathbf{0}$

- **Line through the origin ($b = 0$):**

  $$\mathcal{H}_0 = \left\{ \boldsymbol{x} \in \mathbb{R}^2 : \boldsymbol{w}^T \boldsymbol{x} = 0 \right\} = \{\boldsymbol{w}\}^\perp$$

  Since $\mathbf{0} \in \mathcal{H}_0$, it is a 1D **vector subspace**

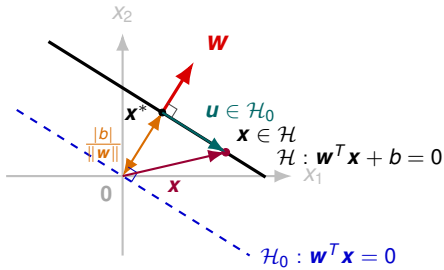
- **Line not through origin ($b \neq 0$):**

  $$\mathcal{H} = \left\{ \boldsymbol{x} \in \mathbb{R}^2 : \boldsymbol{w}^T \boldsymbol{x} + b = 0 \right\}$$

  $\mathbf{0} \notin \mathcal{H}$; **affine subspace** translated by $\boldsymbol{x}^* \in \mathcal{H}$:

  $$\mathcal{H} = \boldsymbol{x}^* + \mathcal{H}_0 = \{\boldsymbol{x}^* + \boldsymbol{u} : \boldsymbol{u} \in \mathcal{H}_0\}$$

- Every $\boldsymbol{x} \in \mathcal{H}$ satisfies $\boldsymbol{w}^T \boldsymbol{x} = -b$



Why $\boldsymbol{w}$ is orthogonal to the hyperplane

Proposition 89 (Orthogonality of the normal vector)

Let $\mathcal{H} = \{\boldsymbol{x} \in \mathbb{R}^p : \boldsymbol{w}^T\boldsymbol{x} + b = 0\}$. For any two points $\boldsymbol{x}_1, \boldsymbol{x}_2 \in \mathcal{H}$, the displacement vector $\boldsymbol{x}_1 - \boldsymbol{x}_2$ lying along $\mathcal{H}$ is orthogonal to $\boldsymbol{w}$:

$$\boldsymbol{w} \bullet (\boldsymbol{x}_1 - \boldsymbol{x}_2) = \boldsymbol{w}^T(\boldsymbol{x}_1 - \boldsymbol{x}_2) = 0$$

Proof:

- Since $\boldsymbol{x}_1 \in \mathcal{H}$ and $\boldsymbol{x}_2 \in \mathcal{H}$, both points satisfy the hyperplane equation:

$$\boldsymbol{w}^T\boldsymbol{x}_1 + b = 0 \qquad \text{and} \qquad \boldsymbol{w}^T\boldsymbol{x}_2 + b = 0$$

- Subtracting the second equation from the first eliminates b:

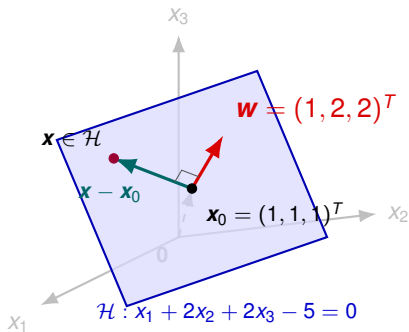
$$(\boldsymbol{w}^T\boldsymbol{x}_1 + b) - (\boldsymbol{w}^T\boldsymbol{x}_2 + b) = \boldsymbol{w}^T(\boldsymbol{x}_1 - \boldsymbol{x}_2) = 0$$

- Hence $\boldsymbol{w} \perp (\boldsymbol{x}_1 - \boldsymbol{x}_2)$ for every pair of points $\boldsymbol{x}_1, \boldsymbol{x}_2$ on $\mathcal{H}$ $\square$

Point-normal form of a (hyper)plane

A hyperplane is determined by a **point** $\boldsymbol{x}_0 \in \mathcal{H}$ and a **normal vector** $\boldsymbol{w} \neq \boldsymbol{0}$

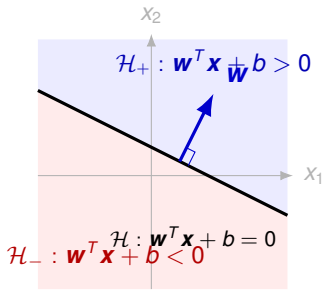
- For any $\boldsymbol{x} \in \mathcal{H}$, the displacement $\boldsymbol{x} - \boldsymbol{x}_0$ lies along $\mathcal{H}$, so $\boldsymbol{w} \perp (\boldsymbol{x} - \boldsymbol{x}_0)$:
  $\boldsymbol{w}^T(\boldsymbol{x} - \boldsymbol{x}_0) = 0$ (**point-normal form**)
- Expanding gives $\boldsymbol{w}^T\boldsymbol{x} - \boldsymbol{w}^T\boldsymbol{x}_0 = 0$, which is
  $\boldsymbol{w}^T\boldsymbol{x} + b = 0$ with $b = -\boldsymbol{w}^T\boldsymbol{x}_0$
- **Example in $\mathbb{R}^3$:** Point $\boldsymbol{x}_0 = (1, 1, 1)^T$ and normal $\boldsymbol{w} = (1, 2, 2)^T$:
  $$1(x_1 - 1) + 2(x_2 - 1) + 2(x_3 - 1) = 0$$
  $$\iff x_1 + 2x_2 + 2x_3 - 5 = 0$$
  Here $b = -\boldsymbol{w}^T\boldsymbol{x}_0 = -(1 + 2 + 2) = -5$



Half-spaces and linear classifiers

A hyperplane $\mathcal{H}$ divides $\mathbb{R}^p \setminus \mathcal{H}$ into two open **half-spaces**:

$$\mathcal{H}_+ = \left\{ \boldsymbol{x} \in \mathbb{R}^p : \boldsymbol{w}^T \boldsymbol{x} + b > 0 \right\}$$
$$\mathcal{H}_- = \left\{ \boldsymbol{x} \in \mathbb{R}^p : \boldsymbol{w}^T \boldsymbol{x} + b < 0 \right\}$$



- The normal vector $\boldsymbol{w}$ points into the positive half-space $\mathcal{H}_+$
- The corresponding **linear classifier** assigns class labels via the sign function:

$$\hat{y}(\boldsymbol{x}) = \text{sgn}(\boldsymbol{w}^T \boldsymbol{x} + b) = \begin{cases} +1 & \text{if } \boldsymbol{x} \in \mathcal{H}_+ \\ -1 & \text{if } \boldsymbol{x} \in \mathcal{H}_- \end{cases}$$

Example – Hyperplane geometry and classification in $\mathbb{R}^2$

Using the line $\mathcal{H}$ in $\mathbb{R}^2$ given by $2x_1 - x_2 - 3 = 0$, classify $\boldsymbol{p} = (3,1)^T$ and $\boldsymbol{q} = (0,0)^T$

- **Normal vector and offset:** $\boldsymbol{w} = \begin{pmatrix} 2 \\ -1 \end{pmatrix}$ and $b = -3$, with norm $\|\boldsymbol{w}\| = \sqrt{2^2 + (-1)^2} = \sqrt{5}$
- **Two points on $\mathcal{H}$:** setting $x_1 = 0$ gives $\boldsymbol{x}_1 = \begin{pmatrix} 0 \\ -3 \end{pmatrix}$; setting $x_2 = 1$ gives $\boldsymbol{x}_2 = \begin{pmatrix} 2 \\ 1 \end{pmatrix}$
- **Orthogonality check:** $\boldsymbol{x}_2 - \boldsymbol{x}_1 = \begin{pmatrix} 2 \\ 4 \end{pmatrix}$ and $\boldsymbol{w}^T(\boldsymbol{x}_2 - \boldsymbol{x}_1) = 2(2) + (-1)(4) = 0$
- **Classifying test points with $\hat{y}(\boldsymbol{x}) = \text{sgn}(\boldsymbol{w}^T\boldsymbol{x} + b)$:**
  - For $\boldsymbol{p} = (3,1)^T$: $\boldsymbol{w}^T\boldsymbol{p} + b = 2(3) - 1(1) - 3 = +2 > 0 \implies \hat{y}(\boldsymbol{p}) = +1$ ($\boldsymbol{p} \in \mathcal{H}_+$)
  - For $\boldsymbol{q} = (0,0)^T$: $\boldsymbol{w}^T\boldsymbol{q} + b = 2(0) - 1(0) - 3 = -3 < 0 \implies \hat{y}(\boldsymbol{q}) = -1$ ($\boldsymbol{q} \in \mathcal{H}_-$)

Geometry of hyperplanes and support vector machines

- Hyperplanes and normal vectors
- **Orthogonal distance to a hyperplane**
- Linear separability and hard-margin SVM

Orthogonal distance from a point to a hyperplane

Theorem 90 (Distance from a point to a hyperplane)

Let $\mathcal{H} = \{\boldsymbol{x} \in \mathbb{R}^p : \boldsymbol{w}^T\boldsymbol{x} + b = 0\}$ be a hyperplane with normal vector $\boldsymbol{w} \neq \boldsymbol{0}$, and let $\boldsymbol{x}_0 \in \mathbb{R}^p$ be any point. The Euclidean distance from $\boldsymbol{x}_0$ to $\mathcal{H}$ is

$$d(\boldsymbol{x}_0, \mathcal{H}) = \min_{\boldsymbol{x} \in \mathcal{H}} \|\boldsymbol{x}_0 - \boldsymbol{x}\| = \frac{|\boldsymbol{w}^T\boldsymbol{x}_0 + b|}{\|\boldsymbol{w}\|}$$

- Setting $\boldsymbol{x}_0 = \boldsymbol{0}$ gives the orthogonal distance from the origin to $\mathcal{H}$:

$$d(\boldsymbol{0}, \mathcal{H}) = \frac{|b|}{\|\boldsymbol{w}\|}$$

- The quantity $\frac{\boldsymbol{w}^T\boldsymbol{x}_0 + b}{\|\boldsymbol{w}\|}$ is the **signed distance** from $\boldsymbol{x}_0$ to $\mathcal{H}$ (positive on $\mathcal{H}_+$, negative on $\mathcal{H}_-$)

Proof – Distance from a point to a hyperplane

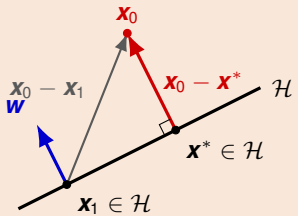
Let $\boldsymbol{x}^* \in \mathcal{H}$ be the orthogonal projection of $\boldsymbol{x}_0$ onto $\mathcal{H}$, and pick any $\boldsymbol{x}_1 \in \mathcal{H}$

- Since $\boldsymbol{x}_0 - \boldsymbol{x}^* \perp \mathcal{H}$, it equals the orthogonal projection of $\boldsymbol{x}_0 - \boldsymbol{x}_1$ onto $\boldsymbol{w}$:

$$\boldsymbol{x}_0 - \boldsymbol{x}^* = \text{proj}_{\boldsymbol{w}}(\boldsymbol{x}_0 - \boldsymbol{x}_1) = \frac{\boldsymbol{w}^T(\boldsymbol{x}_0 - \boldsymbol{x}_1)}{\|\boldsymbol{w}\|^2}\boldsymbol{w}$$

- Because $\boldsymbol{x}_1 \in \mathcal{H}$, we have $\boldsymbol{w}^T\boldsymbol{x}_1 = -b$, so $\boldsymbol{w}^T(\boldsymbol{x}_0 - \boldsymbol{x}_1) = \boldsymbol{w}^T\boldsymbol{x}_0 + b$
- Taking the Euclidean norm gives

$$d(\boldsymbol{x}_0, \mathcal{H}) = \|\boldsymbol{x}_0 - \boldsymbol{x}^*\| = \frac{|\boldsymbol{w}^T\boldsymbol{x}_0 + b|}{\|\boldsymbol{w}\|}$$



Example – Distance to a line in $\mathbb{R}^2$

Example 1 ($\mathbb{R}^2$): Find the distance from $\boldsymbol{x}_0 = \begin{pmatrix} 4 \\ 3 \end{pmatrix}$ to the line $\mathcal{H} : 3x_1 - 4x_2 + 2 = 0$ and the closest point $\boldsymbol{x}^* \in \mathcal{H}$

- Here $\boldsymbol{w} = (3, -4)^T$ and $b = 2$, so $\|\boldsymbol{w}\| = \sqrt{3^2 + (-4)^2} = \sqrt{25} = 5$
- Signed numerator: $\boldsymbol{w}^T \boldsymbol{x}_0 + b = 3(4) - 4(3) + 2 = 2 > 0$ ($\boldsymbol{x}_0$ lies in $\mathcal{H}_+$)
- Distance: $d(\boldsymbol{x}_0, \mathcal{H}) = \frac{|2|}{5} = \frac{2}{5} = 0.4$, and closest point:

$$\boldsymbol{x}^* = \boldsymbol{x}_0 - \frac{\boldsymbol{w}^T \boldsymbol{x}_0 + b}{\|\boldsymbol{w}\|^2} \boldsymbol{w} = \begin{pmatrix} 4 \\ 3 \end{pmatrix} - \frac{2}{25} \begin{pmatrix} 3 \\ -4 \end{pmatrix} = \begin{pmatrix} 94/25 \\ 83/25 \end{pmatrix}$$

Example – Distance to a plane in $\mathbb{R}^3$

Example 2 ($\mathbb{R}^3$): Find the distance from $\boldsymbol{x}_0 = (1, -2, 3)^T$ to the plane $\mathcal{H} : x_1 + 2x_2 + 2x_3 - 5 = 0$

- Here $\boldsymbol{w} = (1, 2, 2)^T$, $b = -5$ and $\|\boldsymbol{w}\| = \sqrt{1^2 + 2^2 + 2^2} = 3$, so

$$d(\boldsymbol{x}_0, \mathcal{H}) = \frac{|1(1) + 2(-2) + 2(3) - 5|}{3} = \frac{|-2|}{3} = \frac{2}{3}$$

Geometry of hyperplanes and support vector machines

- Hyperplanes and normal vectors
- Orthogonal distance to a hyperplane
- **Linear separability and hard-margin SVM**

Linearly separable point sets

Definition 91 (Linear separability)

Two sets of points $X_+ = \{\boldsymbol{x}_i : y_i = +1\}$ *and* $X_- = \{\boldsymbol{x}_i : y_i = -1\}$ *in* $\mathbb{R}^p$ *are* ***linearly separable*** *if there exist* $\boldsymbol{w} \in \mathbb{R}^p \setminus \{\mathbf{0}\}$ *and* $b \in \mathbb{R}$ *such that*

$$\boldsymbol{w}^T \boldsymbol{x}_i + b > 0 \quad \text{for all } \boldsymbol{x}_i \in X_+ \qquad \text{and} \qquad \boldsymbol{w}^T \boldsymbol{x}_i + b < 0 \quad \text{for all } \boldsymbol{x}_i \in X_-$$

- Since $y_i \in \{-1, +1\}$, both inequalities can be written as a single condition:

$$y_i(\boldsymbol{w}^T \boldsymbol{x}_i + b) > 0 \quad \text{for all } i = 1, \ldots, n$$

- Whenever a finite dataset is linearly separable, there are infinitely many separating hyperplanes – how should we pick the most robust one?

When are point sets not linearly separable? The XOR example

Not every two-class dataset in $\mathbb{R}^2$ is linearly separable

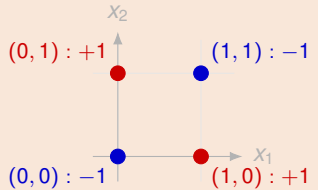
Example (XOR: false $= 0$, true $= 1$; class $y \in \{-1, +1\}$):

- Let $X_- = \{(0,0)^T, (1,1)^T\}$ $(y = -1)$ and $X_+ = \{(1,0)^T, (0,1)^T\}$ $(y = +1)$
- Suppose $\boldsymbol{w}^T\boldsymbol{x} + b < 0$ on X_- and $\boldsymbol{w}^T\boldsymbol{x} + b > 0$ on X_+:

$$\begin{aligned} \text{on } X_- : &\quad b < 0 \text{ and } w_1 + w_2 + b < 0 \\ \text{on } X_+ : &\quad w_1 + b > 0 \text{ and } w_2 + b > 0 \end{aligned}$$

- Adding the first pair gives $w_1 + w_2 + 2b < 0$, whereas adding the second pair gives $w_1 + w_2 + 2b > 0$ – contradiction!

x_1	x_2	x_1 or x_2	x_1 xor x_2
false (0)	false (0)	false	false (-1)
false (0)	true (1)	true	true $(+1)$
true (1)	false (0)	true	true $(+1)$
true (1)	true (1)	true	false (-1)



Scale invariance of separating hyperplanes

Let $\mathcal{H} = \{\boldsymbol{x} \in \mathbb{R}^p : \boldsymbol{w}^T\boldsymbol{x} + b = 0\}$ be a hyperplane. For any constant $k > 0$, consider the rescaled parameters $\widetilde{\boldsymbol{w}} = k\boldsymbol{w}$ and $\widetilde{b} = kb$:

- **Same geometric hyperplane:** for all $\boldsymbol{x} \in \mathbb{R}^p$,

$$\widetilde{\boldsymbol{w}}^T\boldsymbol{x} + \widetilde{b} = (k\boldsymbol{w})^T\boldsymbol{x} + kb = k(\boldsymbol{w}^T\boldsymbol{x} + b) = 0 \iff \boldsymbol{w}^T\boldsymbol{x} + b = 0$$

  Thus, $\{\boldsymbol{x} : \widetilde{\boldsymbol{w}}^T\boldsymbol{x} + \widetilde{b} = 0\}$ and $\{\boldsymbol{x} : \boldsymbol{w}^T\boldsymbol{x} + b = 0\}$ describe the exact same hyperplane $\mathcal{H}$

- **Same classification decisions:** since $k > 0$, signs are preserved:

$$\text{sgn}(\widetilde{\boldsymbol{w}}^T\boldsymbol{x} + \widetilde{b}) = \text{sgn}(k(\boldsymbol{w}^T\boldsymbol{x} + b)) = \text{sgn}(\boldsymbol{w}^T\boldsymbol{x} + b)$$

  Every point $\boldsymbol{x}$ is assigned to the exact same class ($+1$ or -1)

- **Same geometric distance:** by Theorem 90, $\frac{|\widetilde{\boldsymbol{w}}^T\boldsymbol{x}+\widetilde{b}|}{\|\widetilde{\boldsymbol{w}}\|} = \frac{k|\boldsymbol{w}^T\boldsymbol{x}+b|}{k\|\boldsymbol{w}\|} = \frac{|\boldsymbol{w}^T\boldsymbol{x}+b|}{\|\boldsymbol{w}\|}$

Takeaway: Multiplying or dividing both $\boldsymbol{w}$ and b by any positive scalar $k > 0$ does not change the decision boundary or the classifications

Suppose $(\boldsymbol{w}, b)$ separates a finite dataset, so

$$\min_{1 \leq i \leq n} y_i(\boldsymbol{w}^T \boldsymbol{x}_i + b) = m > 0$$

By scale invariance, dividing both $\boldsymbol{w}$ and b by m (setting $\widetilde{\boldsymbol{w}} = \frac{\boldsymbol{w}}{m}$ and $\widetilde{b} = \frac{b}{m}$) does not change the decision boundary $\mathcal{H}$, and scales the minimum to 1:

$$y_i(\widetilde{\boldsymbol{w}}^T \boldsymbol{x}_i + \widetilde{b}) = \frac{1}{m} y_i(\boldsymbol{w}^T \boldsymbol{x}_i + b) \geq \frac{m}{m} = 1$$

Support vectors and canonical margin hyperplanes

Dropping the tildes gives the **canonical form**:

$$y_i(\boldsymbol{w}^T \boldsymbol{x}_i + b) \geq 1 \quad \text{for all } i = 1, \ldots, n$$

The slab free of training points is bounded by two parallel hyperplanes:

- $\mathcal{H}_{+1} = \{\boldsymbol{x} \in \mathbb{R}^p : \boldsymbol{w}^T \boldsymbol{x} + b = +1\}$ (positive margin boundary)
- $\mathcal{H}_{-1} = \{\boldsymbol{x} \in \mathbb{R}^p : \boldsymbol{w}^T \boldsymbol{x} + b = -1\}$ (negative margin boundary)

Points on $\mathcal{H}_{+1}$ or $\mathcal{H}_{-1}$ (satisfying $y_i(\boldsymbol{w}^T \boldsymbol{x}_i + b) = 1$) are **support vectors**

Geometric derivation of the SVM margin width

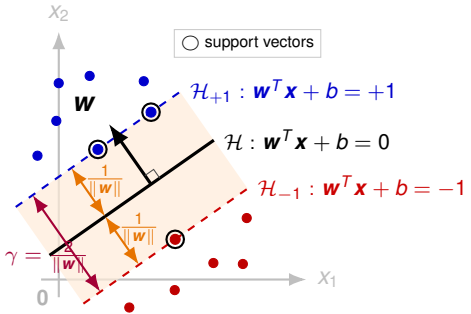
Using Theorem 90, how wide is the margin between $\mathcal{H}_{+1}$ and $\mathcal{H}_{-1}$?

- For any point $\boldsymbol{x}_+ \in \mathcal{H}_{+1}$, we have $\boldsymbol{w}^T\boldsymbol{x}_+ + b = +1$, so its distance to $\mathcal{H}$ is

$$d(\boldsymbol{x}_+, \mathcal{H}) = \frac{|\boldsymbol{w}^T\boldsymbol{x}_+ + b|}{\|\boldsymbol{w}\|} = \frac{1}{\|\boldsymbol{w}\|}$$

- For any point $\boldsymbol{x}_- \in \mathcal{H}_{-1}$, we have $\boldsymbol{w}^T\boldsymbol{x}_- + b = -1$, so its distance to $\mathcal{H}$ is also $\frac{1}{\|\boldsymbol{w}\|}$
- Therefore, the total **margin width** between $\mathcal{H}_{+1}$ and $\mathcal{H}_{-1}$ is

$$\gamma = \frac{2}{\|\boldsymbol{w}\|}$$



Hard-margin support vector machine (SVM)

To make the separating slab as wide as possible, we want to maximise $\gamma = \frac{2}{\|\boldsymbol{w}\|}$, which is equivalent to minimising $\|\boldsymbol{w}\|$ (or $\frac{1}{2}\|\boldsymbol{w}\|^2$):

$$\min_{\boldsymbol{w}\in\mathbb{R}^p,\, b\in\mathbb{R}} \frac{1}{2}\|\boldsymbol{w}\|^2 \quad \text{subject to} \quad y_i(\boldsymbol{w}^T\boldsymbol{x}_i + b) \geq 1 \quad \text{for } i = 1, \ldots, n$$

- This is the **hard-margin support vector machine (SVM)** optimisation problem
- The optimal hyperplane is completely determined by the support vectors lying on the margin boundaries $\boldsymbol{w}^T\boldsymbol{x}_i + b = \pm 1$
- For small problems in $\mathbb{R}^2$, we can find $(\boldsymbol{w}, b)$ directly by **geometry and linear equations**, without any Lagrange multiplier machinery

Example 1 – Hard-margin SVM for 2 points in $\mathbb{R}^2$

Problem: Find the canonical maximum-margin hyperplane $(\boldsymbol{w}, b)$, decision boundary and margin width γ for

$$\boldsymbol{x}_1 = \begin{pmatrix} 1 \\ 1 \end{pmatrix} \quad (y_1 = -1) \qquad \text{and} \qquad \boldsymbol{x}_2 = \begin{pmatrix} 3 \\ 5 \end{pmatrix} \quad (y_2 = +1)$$

Step 1 (Geometry):

- With one point in each class, both $\boldsymbol{x}_1$ and $\boldsymbol{x}_2$ are support vectors, and the decision boundary $\mathcal{H}$ is the perpendicular bisector of the segment $\boldsymbol{x}_1\boldsymbol{x}_2$
- The normal vector $\boldsymbol{w}$ must point from $\boldsymbol{x}_1$ ($y_1 = -1$) towards $\boldsymbol{x}_2$ ($y_2 = +1$), so $\boldsymbol{w}$ is a positive multiple of $\boldsymbol{x}_2 - \boldsymbol{x}_1$:

$$\boldsymbol{x}_2 - \boldsymbol{x}_1 = \begin{pmatrix} 3-1 \\ 5-1 \end{pmatrix} = \begin{pmatrix} 2 \\ 4 \end{pmatrix} \quad \Longrightarrow \quad \boldsymbol{w} = \alpha \begin{pmatrix} 1 \\ 2 \end{pmatrix} \quad (\alpha > 0)$$

Example 1 – Scaling $(\boldsymbol{w}, b)$ and computing the margin

Step 2 (Canonical equations): Since $\boldsymbol{x}_1 \in \mathcal{H}_{-1}$ and $\boldsymbol{x}_2 \in \mathcal{H}_{+1}$, we require

$$\boldsymbol{w}^T \boldsymbol{x}_1 + b = -1 \qquad \text{and} \qquad \boldsymbol{w}^T \boldsymbol{x}_2 + b = +1$$

- Subtracting the first equation from the second gives $\boldsymbol{w}^T(\boldsymbol{x}_2 - \boldsymbol{x}_1) = 2$:

$$\alpha \begin{pmatrix} 1 & 2 \end{pmatrix} \begin{pmatrix} 2 \\ 4 \end{pmatrix} = 2 \iff 10\alpha = 2 \iff \alpha = \frac{1}{5} \quad \Longrightarrow \quad \boldsymbol{w} = \begin{pmatrix} 1/5 \\ 2/5 \end{pmatrix}$$

- Substituting $\boldsymbol{w}$ into $\boldsymbol{w}^T \boldsymbol{x}_1 + b = -1$ gives the offset b:

$$\frac{1}{5}(1) + \frac{2}{5}(1) + b = -1 \quad \Longrightarrow \quad b = -1 - \frac{3}{5} = -\frac{8}{5}$$

Step 3 (Decision boundary and margin width):

- Decision boundary: $\frac{1}{5}x_1 + \frac{2}{5}x_2 - \frac{8}{5} = 0 \iff x_1 + 2x_2 - 8 = 0$ (passes through the midpoint $\boldsymbol{m} = (2, 3)^T$)
- Margin width: $\|\boldsymbol{w}\| = \sqrt{(1/5)^2 + (2/5)^2} = \frac{1}{\sqrt{5}}$, so $\gamma = \frac{2}{\|\boldsymbol{w}\|} = 2\sqrt{5} = \|\boldsymbol{x}_2 - \boldsymbol{x}_1\|$

Example 1 revisited – Solving via Lagrange multipliers

Recall Example 1: $\boldsymbol{x}_1 = (1,1)^T$ ($y_1 = -1$) and $\boldsymbol{x}_2 = (3,5)^T$ ($y_2 = +1$)

With multipliers $\alpha_1, \alpha_2 \geq 0$, form the Lagrangian for the hard-margin SVM:

$$L(\boldsymbol{w}, b, \alpha_1, \alpha_2) = \frac{1}{2}\|\boldsymbol{w}\|^2 - \sum_{i=1}^{2} \alpha_i \left[y_i(\boldsymbol{w}^T \boldsymbol{x}_i + b) - 1 \right]$$

Setting partial derivatives to zero gives the stationarity conditions:

- $\nabla_{\boldsymbol{w}} L = \boldsymbol{0} \implies \boldsymbol{w} = \sum_{i=1}^{2} \alpha_i y_i \boldsymbol{x}_i = -\alpha_1 (1,1)^T + \alpha_2 (3,5)^T$
- $\frac{\partial L}{\partial b} = 0 \implies \sum_{i=1}^{2} \alpha_i y_i = -\alpha_1 + \alpha_2 = 0 \implies \alpha_1 = \alpha_2 = \alpha$

Therefore $\boldsymbol{w} = \alpha(\boldsymbol{x}_2 - \boldsymbol{x}_1) = \alpha(2,4)^T = (2\alpha, 4\alpha)^T$

Example 1 revisited – Determining α, $\boldsymbol{w}$ and b

Since both points are support vectors ($\alpha > 0$), their margin constraints are active ($= 1$):

- At $\boldsymbol{x}_1$: $y_1(\boldsymbol{w}^T\boldsymbol{x}_1 + b) = -[2\alpha(1) + 4\alpha(1) + b] = 1 \implies 6\alpha + b = -1$
- At $\boldsymbol{x}_2$: $y_2(\boldsymbol{w}^T\boldsymbol{x}_2 + b) = 2\alpha(3) + 4\alpha(5) + b = 1 \implies 26\alpha + b = 1$

Subtracting the first equation from the second:

$$20\alpha = 2 \implies \alpha = \frac{1}{10}$$

Substituting $\alpha = 1/10$ gives the exact same parameters as the geometric approach:

$$\boldsymbol{w} = \begin{pmatrix} 2(1/10) \\ 4(1/10) \end{pmatrix} = \begin{pmatrix} 1/5 \\ 2/5 \end{pmatrix}, \qquad b = -1 - 6\left(\frac{1}{10}\right) = -\frac{8}{5}$$

Key insight: In general, $\boldsymbol{w} = \sum_{i=1}^{n} \alpha_i y_i \boldsymbol{x}_i$ is a linear combination of only the support vectors (points with $\alpha_i > 0$)

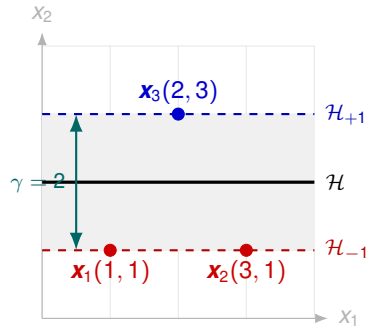
Example 2 – Hard-margin SVM for 3 points in $\mathbb{R}^2$

Problem: Find $(\boldsymbol{w}, b)$ and margin γ for:

- Class -1: $\boldsymbol{x}_1 = (1,1)^T$ and $\boldsymbol{x}_2 = (3,1)^T$
- Class $+1$: $\boldsymbol{x}_3 = (2,3)^T$

Step 1 (Direction of the boundary):

- Both $\boldsymbol{x}_1$ and $\boldsymbol{x}_2$ belong to class -1 and are equidistant from $\boldsymbol{x}_3$, so all three points are support vectors
- Since $\boldsymbol{x}_1, \boldsymbol{x}_2 \in \mathcal{H}_{-1}$, the vector $\boldsymbol{x}_2 - \boldsymbol{x}_1 = (2,0)^T$ lies along $\mathcal{H}_{-1}$
- By orthogonality, $\boldsymbol{w}^T(\boldsymbol{x}_2 - \boldsymbol{x}_1) = 2w_1 = 0 \implies w_1 = 0$, so $\boldsymbol{w} = (0, w_2)^T$



Example 2 – Solving the linear equations

Step 2 (Canonical equations): Substituting $\boldsymbol{w} = (0, w_2)^T$ into $\boldsymbol{w}^T \boldsymbol{x}_i + b = y_i$ for the support vectors gives

$$\begin{aligned} \text{at } \boldsymbol{x}_1 = (1,1)^T \text{ and } \boldsymbol{x}_2 = (3,1)^T: \quad & w_2(1) + b = -1 \\ \text{at } \boldsymbol{x}_3 = (2,3)^T: \quad & w_2(3) + b = +1 \end{aligned}$$

- Subtracting the first equation from the second gives $2w_2 = 2 \implies w_2 = 1$
- Substituting $w_2 = 1$ into $w_2 + b = -1$ gives $b = -2$, so

$$\boldsymbol{w} = \begin{pmatrix} 0 \\ 1 \end{pmatrix} \qquad \text{and} \qquad b = -2$$

- Decision boundary: $x_2 - 2 = 0$ (the horizontal line $x_2 = 2$)
- Margin width: $\|\boldsymbol{w}\| = 1 \implies \gamma = \frac{2}{\|\boldsymbol{w}\|} = 2$

Solving hard-margin SVMs on larger sets with R

For larger datasets ($n \gg 2$) or in higher dimensions ($p > 2$), quadratic programming solvers find the optimal multipliers $\alpha_i \geq 0$ numerically

In R, the package `e1071` (interface to LIBSVM) provides `svm()`:

```
fit = svm(y ~ ., data = dat, kernel = "linear", cost = 1000, scale =
                                    FALSE)
```

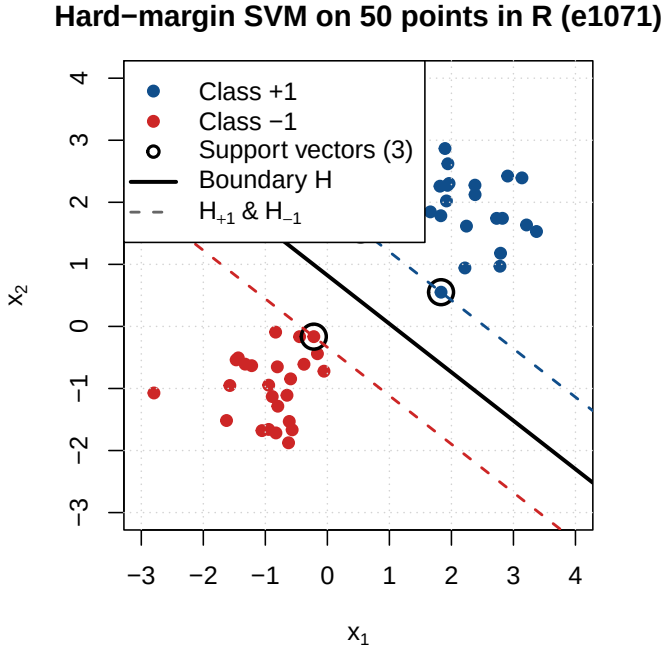
- `kernel = "linear"`: fits the linear decision boundary $\boldsymbol{w}^T \boldsymbol{x} + b = 0$
- `cost = 1000`: a large cost penalty enforces a hard margin (no margin violations)
- `scale = FALSE`: operates directly in raw feature coordinates

The fitted object contains:

- `fit$index`: indices of the support vectors ($\alpha_i > 0$)
- `fit$coefs`: the signed multipliers $\alpha_i y_i$ for each support vector
- $\boldsymbol{w} = \sum_{i \in \text{SV}} \alpha_i y_i \boldsymbol{x}_i$: computed via `drop(t(fit$coefs) %*% fit$SV)`
- Bias b: given by `-fit$rho`

Fitting a linear SVM in R – 50-point dataset

```
library(e1071)
set.seed(42); n = 25
X1 = matrix(rnorm(2 * n, mean = 2, sd = 0.6), n, 2)
X2 = matrix(rnorm(2 * n, mean = -1, sd = 0.6), n, 2)
dat = data.frame(x1 = c(X1[, 1], X2[, 1]), x2 = c(X1[, 2], X2[, 2]),
                 y = factor(c(rep(1, n), rep(-1, n))))
fit = svm(y ~ ., data = dat, kernel = "linear",
          cost = 1000, scale = FALSE)
w = drop(t(fit$coefs) %*% fit$SV)
b = -fit$rho
cat("Support vectors:", length(fit$index), "out of", nrow(dat), "\n")

## Support vectors: 3 out of 50

cat("w:", round(w, 4), " b:", round(b, 4),
    " gamma:", round(2 / sqrt(sum(w^2)), 4), "\n")

## w: 0.6734 0.8626  b: -0.7102  gamma: 1.8275
```


Hard−margin SVM on 50 points in R (e1071)
Class +1
Class −1
Support vectors (3)
Boundary H
H+1 & H−1
x1
x2
−3
−2
−1
0
1
2
3
4

Supervised vs unsupervised linear methods

Geometry of hyperplanes and support vector machines

Least-squares classification

Framing classification as linear least squares

Can we fit a linear classifier $\hat{y}(\boldsymbol{x}) = \operatorname{sgn}(\boldsymbol{w}^T\boldsymbol{x} + b)$ using the method of least squares from Unit 03?

We ask the affine score $f(\boldsymbol{x}_i) = \boldsymbol{w}^T\boldsymbol{x}_i + b$ to match the numeric class label $y_i \in \{-1, +1\}$ as closely as possible:

$$\boldsymbol{w}^T\boldsymbol{x}_i + b \approx y_i \quad \text{for } i = 1, \ldots, n$$

To write this as a single matrix product, absorb the intercept b by defining the **augmented feature vector** and **augmented weight vector**:

$$\tilde{\boldsymbol{x}}_i = \begin{pmatrix} \boldsymbol{x}_i \\ 1 \end{pmatrix} \in \mathbb{R}^{p+1}, \qquad \tilde{\boldsymbol{w}} = \begin{pmatrix} \boldsymbol{w} \\ b \end{pmatrix} \in \mathbb{R}^{p+1}$$

so that, by symmetry of the dot product ($\tilde{\boldsymbol{w}}^T\tilde{\boldsymbol{x}}_i = \tilde{\boldsymbol{x}}_i^T\tilde{\boldsymbol{w}}$):

$$\boldsymbol{w}^T\boldsymbol{x}_i + b = \tilde{\boldsymbol{w}}^T\tilde{\boldsymbol{x}}_i = \tilde{\boldsymbol{x}}_i^T\tilde{\boldsymbol{w}}$$

We write $\tilde{\boldsymbol{x}}_i^T\tilde{\boldsymbol{w}}$ so that the row vector $\tilde{\boldsymbol{x}}_i^T$ naturally matches the i-th row of the data matrix X (next slide).

Matrix formulation and normal equations

Stack the augmented rows $\tilde{\boldsymbol{x}}_i^T$ into the (augmented) **data matrix** $X \in \mathcal{M}_{n,p+1}$ (in regression and classification, often called the **design matrix** or **feature matrix**) and the labels into $\boldsymbol{y} \in \{-1, +1\}^n$:

$$X = \begin{pmatrix} \boldsymbol{x}_1^T & 1 \\ \vdots & \vdots \\ \boldsymbol{x}_n^T & 1 \end{pmatrix} \in \mathcal{M}_{n,p+1}, \qquad \boldsymbol{y} = \begin{pmatrix} y_1 \\ \vdots \\ y_n \end{pmatrix} \in \mathbb{R}^n$$

Minimising the sum of squared errors $\|X\tilde{\boldsymbol{w}} - \boldsymbol{y}\|^2 = \sum_{i=1}^n (\boldsymbol{w}^T \boldsymbol{x}_i + b - y_i)^2$ yields the **normal equations**:

$$X^T X \tilde{\boldsymbol{w}} = X^T \boldsymbol{y} \quad \Longrightarrow \quad \tilde{\boldsymbol{w}} = (X^T X)^{-1} X^T \boldsymbol{y}$$

- What we are explaining here **only works when we have more observations than features**: we require $n \geq p + 1$ and $\text{rank}(X) = p + 1$ for $X^T X \in \mathcal{M}_{p+1}$ to be invertible
- If $n < p + 1$ or if features are collinear, $X^T X$ is singular and $(X^T X)^{-1}$ does not exist

Example – 4-point least-squares classifier in $\mathbb{R}^2$

Problem: Find the least-squares linear classifier $(\boldsymbol{w}, b)$ for the $n = 4$ points:

- Class -1 ($y_1 = y_2 = -1$): $\boldsymbol{x}_1 = \begin{pmatrix} -1 \\ 0 \end{pmatrix}$ and $\boldsymbol{x}_2 = \begin{pmatrix} 0 \\ -1 \end{pmatrix}$
- Class $+1$ ($y_3 = y_4 = +1$): $\boldsymbol{x}_3 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$ and $\boldsymbol{x}_4 = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$

Step 1 (Set up X and $\boldsymbol{y}$):

$$X = \begin{pmatrix} -1 & 0 & 1 \\ 0 & -1 & 1 \\ 1 & 0 & 1 \\ 0 & 1 & 1 \end{pmatrix} \in \mathcal{M}_{43}, \qquad \boldsymbol{y} = \begin{pmatrix} -1 \\ -1 \\ 1 \\ 1 \end{pmatrix} \in \mathbb{R}^4$$

Example – Solving the normal equations by hand

Step 2 (Compute $X^T X$ and $X^T \boldsymbol{y}$):

$$X^T X = \begin{pmatrix} -1 & 0 & 1 & 0 \\ 0 & -1 & 0 & 1 \\ 1 & 1 & 1 & 1 \end{pmatrix} \begin{pmatrix} -1 & 0 & 1 \\ 0 & -1 & 1 \\ 1 & 0 & 1 \\ 0 & 1 & 1 \end{pmatrix} = \begin{pmatrix} 2 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 4 \end{pmatrix}, \qquad X^T \boldsymbol{y} = \begin{pmatrix} 2 \\ 2 \\ 0 \end{pmatrix}$$

Step 3 (Solve for $\tilde{\boldsymbol{w}}$): Since $X^T X = \mathrm{diag}(2, 2, 4)$ is diagonal, its inverse is immediate:

$$\tilde{\boldsymbol{w}} = (X^T X)^{-1} X^T \boldsymbol{y} = \begin{pmatrix} 1/2 & 0 & 0 \\ 0 & 1/2 & 0 \\ 0 & 0 & 1/4 \end{pmatrix} \begin{pmatrix} 2 \\ 2 \\ 0 \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \\ 0 \end{pmatrix} \implies \boldsymbol{w} = \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \quad b = 0$$

The decision boundary is $\boldsymbol{w}^T \boldsymbol{x} + b = x_1 + x_2 = 0$ (the line $x_2 = -x_1$), which separates the two classes cleanly

Example – Fitted values, residuals and classifying new points

With $\tilde{\boldsymbol{w}} = (1, 1, 0)^T$ ($\boldsymbol{w} = (1, 1)^T$ and $b = 0$), let us check the fitted scores $\hat{\boldsymbol{y}} = X\tilde{\boldsymbol{w}}$ and residual vector $\boldsymbol{e} = \boldsymbol{y} - \hat{\boldsymbol{y}}$:

$$\hat{\boldsymbol{y}} = X\tilde{\boldsymbol{w}} = \begin{pmatrix} -1 & 0 & 1 \\ 0 & -1 & 1 \\ 1 & 0 & 1 \\ 0 & 1 & 1 \end{pmatrix} \begin{pmatrix} 1 \\ 1 \\ 0 \end{pmatrix} = \begin{pmatrix} -1 \\ -1 \\ 1 \\ 1 \end{pmatrix} = \boldsymbol{y} \quad \Longrightarrow \quad \boldsymbol{e} = \boldsymbol{y} - X\tilde{\boldsymbol{w}} = \boldsymbol{0}$$

- Because these 4 points happen to lie on the lines $x_1 + x_2 = -1$ and $x_1 + x_2 = +1$, the least-squares residual is zero!
- For a new observation $\boldsymbol{x}_{\text{new}} = \begin{pmatrix} 2 \\ -0.5 \end{pmatrix}$, we compute $\boldsymbol{w}^T\boldsymbol{x}_{\text{new}} + b = 2 - 0.5 = 1.5 > 0$, so $\hat{y}(\boldsymbol{x}_{\text{new}}) = \text{sgn}(1.5) = +1$

Sensitivity of least-squares classification to outliers

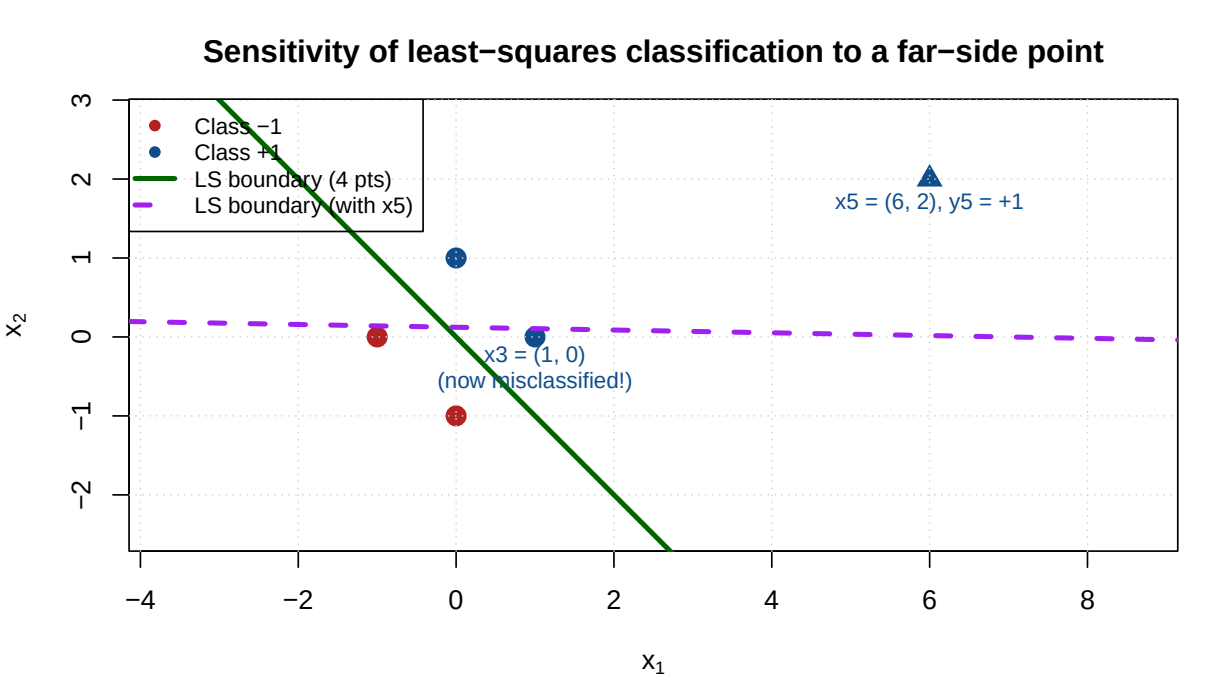
Why is least squares not always suitable for classification?

- The squared error $(\boldsymbol{w}^T \boldsymbol{x}_i + b - y_i)^2$ penalises predictions that are **too far on the correct side** of the decision boundary
- Suppose a point with $y_i = +1$ lies deep inside $\mathcal{H}_+$ so that $\boldsymbol{w}^T \boldsymbol{x}_i + b = 10 > 0$: it is *correctly classified*, yet least squares assigns a large squared penalty $(10 - 1)^2 = 81$
- To shrink that large residual, least squares tilts and shifts the hyperplane towards the extreme point, which can push the boundary across points near the margin and cause misclassifications

R: least-squares classifier and outlier sensitivity

Compare the 4-point least-squares boundary before and after adding a single "easy" Class $+1$ point at $\boldsymbol{x}_5 = (6, 2)^T$ (which lies well inside $x_1 + x_2 > 0$)

```
X4 = matrix(c(-1, 0, 1,  0, -1, 1,  1, 0, 1,  0, 1, 1), nrow = 4, byrow = TRUE)
y4 = c(-1, -1, 1, 1)
w_tilde_4 = solve(t(X4) %*% X4, t(X4) %*% y4)
# Add one outlier far on the correct (+1) side: x5 = (6, 2)^T
X5 = rbind(X4, c(6, 2, 1)); y5 = c(y4, 1)
w_tilde_5 = solve(t(X5) %*% X5, t(X5) %*% y5)
cbind(w_4pts = round(as.vector(w_tilde_4), 3), w_5pts = round(as.vector(w_tilde_5), 3))

##      w_4pts w_5pts
## [1,]      1  0.012
## [2,]      1  0.671
## [3,]      0 -0.082

# Fitted scores for all five points:
rbind(y = y5, score = round(as.vector(X5 %*% w_tilde_5), 3))

##         [,1]   [,2]   [,3]  [,4]  [,5]
## y     -1.000 -1.000  1.000 1.000 1.000
## score -0.094 -0.753 -0.071 0.588 1.329
```


Sensitivity of least−squares classification to a far−side point
Class −1
Class +1
LS boundary (4 pts)
LS boundary (with x5)
x5 = (6, 2), y5 = +1
x3 = (1, 0)
(now misclassified!)
x1
x2
−4
−2
0
2
4
6
8
3
2
1
0
−1
−2

Ridge regularisation when $X^T X$ is singular

What if we have more features than observations ($n < p + 1$) or collinear features?

- In that case, $\text{rank}(X) < p + 1$, so $X^T X$ is singular and cannot be inverted
- To fix this, we add a **ridge penalty** with regularisation parameter $\lambda > 0$:

$$\min_{\tilde{\boldsymbol{w}} \in \mathbb{R}^{p+1}} \left(\|X\tilde{\boldsymbol{w}} - \boldsymbol{y}\|^2 + \lambda \|\tilde{\boldsymbol{w}}\|^2 \right)$$

- Setting the gradient to zero yields the regularised normal equations:

$$(X^T X + \lambda \mathbb{I})\tilde{\boldsymbol{w}} = X^T \boldsymbol{y} \quad \Longrightarrow \quad \tilde{\boldsymbol{w}}_{\text{ridge}} = (X^T X + \lambda \mathbb{I})^{-1} X^T \boldsymbol{y}$$

- Adding $\lambda \mathbb{I}$ shifts all eigenvalues of $X^T X$ by $+\lambda > 0$: all its eigenvalues are strictly positive (we call such a matrix **positive definite**; we will discuss this in more detail later), which guarantees that it is always invertible

Summary – Comparing linear projection and classification methods

Method	Supervised?	Criterion optimised	Linear algebraic solution
PCA (Unit 05)	No	Max variance $\boldsymbol{w}^T \Sigma \boldsymbol{w}$	Top eigenvector of Σ
Hard-margin SVM	Yes	Max margin $\frac{2}{\|\boldsymbol{w}\|}$	Support vectors $\boldsymbol{w}^T \boldsymbol{x}_i + b = \pm 1$
Least squares	Yes	Min error $\|X\tilde{\boldsymbol{w}} - \boldsymbol{y}\|^2$	$\tilde{\boldsymbol{w}} = (X^T X)^{-1} X^T \boldsymbol{y}$

Distance matrices, Gram matrices and multidimensional scaling (MDS)

MATH 2740 – Mathematics of Data Science – Unit U07

Julien Arino
julien.arino@umanitoba.ca

Department of Mathematics @ University of Manitoba

Fall 2026

The University of Manitoba campuses are located on original lands of Anishinaabeg, Ininew, Anisininew, Dakota and Dene peoples, and on the National Homeland of the Red River Métis. We respect the Treaties that were made on these territories, we acknowledge the harms and mistakes of the past, and we dedicate ourselves to move forward in partnership with Indigenous communities in a spirit of Reconciliation and collaboration.

Outline

- Motivation: from distances back to coordinates
- Gram matrices and Euclidean distance matrices
- The centring matrix as an orthogonal projector
- Classical multidimensional scaling (MDS)
- Algebraic duality between PCA and classical MDS

Motivation: from distances back to coordinates

Gram matrices and Euclidean distance matrices

The centring matrix as an orthogonal projector

Classical multidimensional scaling (MDS)

Algebraic duality between PCA and classical MDS

From coordinates to distances and back

In principal component analysis (Unit U05), we start with a data matrix $X \in \mathcal{M}_{np}(\mathbb{R})$ containing the coordinates of n observations $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n \in \mathbb{R}^p$ and project them onto a lower-dimensional subspace $\mathbb{R}^k$ ($k \leq p$)

From the coordinates $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n \in \mathbb{R}^p$, it is straightforward to compute all pairwise Euclidean distances

$$d_{ij} = \|\boldsymbol{x}_i - \boldsymbol{x}_j\| = \sqrt{(\boldsymbol{x}_i - \boldsymbol{x}_j) \bullet (\boldsymbol{x}_i - \boldsymbol{x}_j)}, \qquad i, j = 1, \ldots, n$$

The inverse problem: what if we do *not* know the coordinates X, and only have a table of pairwise distances or dissimilarities $D = [d_{ij}] \in \mathcal{M}_n(\mathbb{R})$ between n objects?

Where do distance-only datasets arise?

In many scientific applications, objects do not come with explicit coordinates in $\mathbb{R}^p$, or only pairwise comparisons are measured:

- **Geography and transport**: road distances, flight distances or travel times between n cities
- **Genomics and evolutionary biology**: sequence edit distances or evolutionary divergence times between n species
- **Psychology and sensory science**: human judgements of dissimilarity between colours, sounds, faces or products
- **Cheminformatics and text analysis**: structural dissimilarity between molecules or cosine/Jaccard distances between documents

The goal of multidimensional scaling (MDS)

Given an $n \times n$ matrix $D = [d_{ij}]$ of pairwise distances between n objects:

1. **Exact Euclidean embedding**: do there exist points $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n \in \mathbb{R}^p$ (for some dimension p) such that

$$\|\boldsymbol{x}_i - \boldsymbol{x}_j\| = d_{ij} \qquad \forall i, j = 1, \ldots, n?$$

2. **Dimensionality reduction and visualisation**: for a chosen small dimension k (typically $k = 1, 2$ or 3), find coordinates $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n \in \mathbb{R}^k$ whose pairwise Euclidean distances $\|\boldsymbol{x}_i - \boldsymbol{x}_j\|$ best approximate d_{ij}

This family of methods is called **multidimensional scaling** (**MDS**). When D is treated as a Euclidean distance matrix and coordinates are recovered via spectral decomposition, it is called **classical MDS** (or **principal coordinate analysis**, **PCoA**)

Non-uniqueness: rigid motions preserve distances

Before trying to recover coordinates $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n \in \mathbb{R}^p$ from pairwise distances $d_{ij} = \|\boldsymbol{x}_i - \boldsymbol{x}_j\|$, observe that coordinates can *never* be unique!

Proposition 95 (Invariance of Euclidean distances under rigid motions)

Let $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n \in \mathbb{R}^p$, let $\boldsymbol{s} \in \mathbb{R}^p$ be a translation vector and let $Q \in \mathcal{M}_p(\mathbb{R})$ be an orthogonal matrix ($Q^T Q = \mathbb{I}_p$). Define transformed points

$$\boldsymbol{y}_i = Q\boldsymbol{x}_i + \boldsymbol{s}, \qquad i = 1, \ldots, n$$

Then all pairwise Euclidean distances are preserved:

$$\|\boldsymbol{y}_i - \boldsymbol{y}_j\| = \|\boldsymbol{x}_i - \boldsymbol{x}_j\| \qquad \forall i, j = 1, \ldots, n$$

Thus, distances only determine a point configuration up to **translation**, **rotation** and **reflection** (collectively called **rigid motions** or **Euclidean isometries**)

Proof of Proposition 95

Let $\boldsymbol{y}_i = Q\boldsymbol{x}_i + \boldsymbol{s}$ for $i = 1, \ldots, n$, where $\boldsymbol{s} \in \mathbb{R}^p$ and $Q^T Q = \mathbb{I}_p$

First, taking the difference between $\boldsymbol{y}_i$ and $\boldsymbol{y}_j$ eliminates the translation vector $\boldsymbol{s}$:

$$\boldsymbol{y}_i - \boldsymbol{y}_j = (Q\boldsymbol{x}_i + \boldsymbol{s}) - (Q\boldsymbol{x}_j + \boldsymbol{s}) = Q\boldsymbol{x}_i - Q\boldsymbol{x}_j = Q(\boldsymbol{x}_i - \boldsymbol{x}_j)$$

Next, compute the squared Euclidean norm using the orthogonality of Q:

$$\begin{aligned} \|\boldsymbol{y}_i - \boldsymbol{y}_j\|^2 &= \|Q(\boldsymbol{x}_i - \boldsymbol{x}_j)\|^2 \\ &= \big(Q(\boldsymbol{x}_i - \boldsymbol{x}_j)\big)^T \big(Q(\boldsymbol{x}_i - \boldsymbol{x}_j)\big) \\ &= (\boldsymbol{x}_i - \boldsymbol{x}_j)^T Q^T Q(\boldsymbol{x}_i - \boldsymbol{x}_j) \\ &= (\boldsymbol{x}_i - \boldsymbol{x}_j)^T \mathbb{I}_p (\boldsymbol{x}_i - \boldsymbol{x}_j) = \|\boldsymbol{x}_i - \boldsymbol{x}_j\|^2 \end{aligned}$$

Taking the non-negative square root yields $\|\boldsymbol{y}_i - \boldsymbol{y}_j\| = \|\boldsymbol{x}_i - \boldsymbol{x}_j\|$ □

Removing translation ambiguity by centring at the origin

Because shifting all points by a constant vector $\boldsymbol{s} \in \mathbb{R}^p$ leaves every pairwise distance unchanged, we must fix a reference location to obtain a well-defined reconstruction

Convention in classical MDS: we require the reconstructed points $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n \in \mathbb{R}^p$ to have their **centroid** (sample mean) at the origin $\mathbf{0} \in \mathbb{R}^p$:

$$\bar{\boldsymbol{x}} = \frac{1}{n} \sum_{i=1}^{n} \boldsymbol{x}_i = \mathbf{0} \iff \sum_{i=1}^{n} \boldsymbol{x}_i = \mathbf{0}$$

- Fixing $\sum_{i=1}^{n} \boldsymbol{x}_i = \mathbf{0}$ completely removes the translation ambiguity
- Any remaining ambiguity is purely an orthogonal transformation $Q \in \mathcal{M}_p(\mathbb{R})$ (rotation or reflection about the origin)
- Algebraically, why does fixing the origin help? Because once an origin is fixed, **distances determine inner products**, and inner products give us a **Gram matrix** that we can diagonalise!

Motivation: from distances back to coordinates

Gram matrices and Euclidean distance matrices

The centring matrix as an orthogonal projector

Classical multidimensional scaling (MDS)

Algebraic duality between PCA and classical MDS

Gram matrices and Euclidean distance matrices

The Gram matrix

The squared Euclidean distance matrix

Data matrix convention: observations in rows

Throughout this unit, we represent a configuration of n points in $\mathbb{R}^p$ by a data matrix $X \in \mathcal{M}_{np}(\mathbb{R})$ whose ith row is the transpose of the column vector $\boldsymbol{x}_i \in \mathbb{R}^p$:

$$X = \begin{pmatrix} \boldsymbol{x}_1^T \\ \boldsymbol{x}_2^T \\ \vdots \\ \boldsymbol{x}_n^T \end{pmatrix} = \begin{pmatrix} x_{11} & x_{12} & \cdots & x_{1p} \\ x_{21} & x_{22} & \cdots & x_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{np} \end{pmatrix} \in \mathcal{M}_{np}(\mathbb{R})$$

- Rows $i = 1, \ldots, n$ are **observations** (objects, samples, cities or individuals)
- Columns $j = 1, \ldots, p$ are **features** (also called variables or coordinate axes)

The Gram matrix of a set of vectors

Definition 96 (Gram matrix)

Let $X \in \mathcal{M}_{np}(\mathbb{R})$ have rows $\boldsymbol{x}_1^T, \ldots, \boldsymbol{x}_n^T \in \mathbb{R}^p$. The ***Gram matrix*** *(or* ***inner product matrix****) of $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n$ is the $n \times n$ matrix*

$$G = XX^T \in \mathcal{M}_n(\mathbb{R})$$

whose (i, j) entry is the standard inner product between $\boldsymbol{x}_i$ and $\boldsymbol{x}_j$:

$$g_{ij} = \boldsymbol{x}_i \bullet \boldsymbol{x}_j = \boldsymbol{x}_i^T \boldsymbol{x}_j = \sum_{m=1}^{p} x_{im} x_{jm}, \qquad i, j = 1, \ldots, n$$

Explicitly, in block row-times-column form:

$$G = XX^T = \begin{pmatrix} \boldsymbol{x}_1^T \\ \vdots \\ \boldsymbol{x}_n^T \end{pmatrix} \begin{pmatrix} \boldsymbol{x}_1 & \cdots & \boldsymbol{x}_n \end{pmatrix} = \begin{pmatrix} \boldsymbol{x}_1^T\boldsymbol{x}_1 & \boldsymbol{x}_1^T\boldsymbol{x}_2 & \cdots & \boldsymbol{x}_1^T\boldsymbol{x}_n \\ \boldsymbol{x}_2^T\boldsymbol{x}_1 & \boldsymbol{x}_2^T\boldsymbol{x}_2 & \cdots & \boldsymbol{x}_2^T\boldsymbol{x}_n \\ \vdots & \vdots & \ddots & \vdots \\ \boldsymbol{x}_n^T\boldsymbol{x}_1 & \boldsymbol{x}_n^T\boldsymbol{x}_2 & \cdots & \boldsymbol{x}_n^T\boldsymbol{x}_n \end{pmatrix}$$

Geometric meaning of the Gram matrix

Every entry of the Gram matrix $G = XX^T$ encodes geometric information relative to the origin **0**:

- **Diagonal entries** give squared lengths (squared distances from the origin):

$$g_{ii} = \boldsymbol{x}_i^T \boldsymbol{x}_i = \|\boldsymbol{x}_i\|^2 \geq 0, \qquad i = 1, \ldots, n$$

- **Off-diagonal entries** give lengths and angles θ_{ij} between vectors $\boldsymbol{x}_i$ and $\boldsymbol{x}_j$:

$$g_{ij} = \boldsymbol{x}_i^T \boldsymbol{x}_j = \|\boldsymbol{x}_i\| \, \|\boldsymbol{x}_j\| \cos\theta_{ij}, \qquad i \neq j$$

- **Orthogonal invariance**: if we rotate or reflect all points by an orthogonal matrix $Q \in \mathcal{M}_p(\mathbb{R})$ (so $\boldsymbol{x}_i \mapsto Q\boldsymbol{x}_i$ and $X \mapsto XQ^T$), the Gram matrix is **unchanged**:

$$(XQ^T)(XQ^T)^T = XQ^TQX^T = X\mathbb{I}_pX^T = XX^T = G$$

Positive semi-definite and positive definite matrices

Definition 97 (Positive semi-definite and positive definite matrices)

Let $A \in \mathcal{M}_n(\mathbb{R})$ be a symmetric matrix ($A^T = A$).

- *A is **positive semi-definite** (**PSD**) if*

$$\boldsymbol{v}^T A \boldsymbol{v} \geq 0 \qquad \forall \boldsymbol{v} \in \mathbb{R}^n$$

- *A is **positive definite** (**PD**) if*

$$\boldsymbol{v}^T A \boldsymbol{v} > 0 \qquad \forall \boldsymbol{v} \in \mathbb{R}^n \setminus \{\mathbf{0}\}$$

Recall from Theorem 74 (Unit U04) that all eigenvalues of a real symmetric matrix $A \in \mathcal{M}_n(\mathbb{R})$ are real. Equivalently

- A is positive semi-definite $\iff$ all eigenvalues of A satisfy $\lambda_i \geq 0$
- A is positive definite $\iff$ all eigenvalues of A satisfy $\lambda_i > 0$ (in which case A is invertible)

You have already encountered these matrices!

Even without the name, we have already proved and used these properties for matrices of the form A^TA and AA^T (with $A \in \mathcal{M}_{np}(\mathbb{R})$):

- **Unit U02 (Theorem 25)**: we proved that for any $A \in \mathcal{M}_{np}(\mathbb{R})$, both $AA^T \in \mathcal{M}_n(\mathbb{R})$ and $A^TA \in \mathcal{M}_p(\mathbb{R})$ are **symmetric**

- **Unit U03 (Least squares, invertibility of A^TA)**: for any $\boldsymbol{x} \in \mathbb{R}^p$, we wrote

$$\boldsymbol{x}^T(A^TA)\boldsymbol{x} = (A\boldsymbol{x})^T(A\boldsymbol{x}) = \|A\boldsymbol{x}\|^2 \geq 0$$

  which says precisely that A^TA is **positive semi-definite**! When A has linearly independent columns ($A\boldsymbol{x} = \boldsymbol{0} \iff \boldsymbol{x} = \boldsymbol{0}$), $\|A\boldsymbol{x}\|^2 > 0$ for all $\boldsymbol{x} \neq \boldsymbol{0}$, so A^TA is **positive definite** (and hence invertible)

- **Unit U04 (SVD, Theorem 75)**: we proved that all eigenvalues of $A^T A$ and AA^T are **real and non-negative** ($\lambda_i \geq 0$), which allowed us to define singular values $\sigma_i = \sqrt{\lambda_i}$

- **Unit U05 (PCA)**: the sample covariance matrix $S_X = \frac{1}{n-1} \widetilde{X}^T \widetilde{X}$ is symmetric positive semi-definite, and its eigenvalues $\lambda_i \geq 0$ are the variances along the principal axes

Fundamental properties of Gram matrices

Notice that for a data matrix $X \in \mathcal{M}_{np}(\mathbb{R})$, its Gram matrix $G = XX^T \in \mathcal{M}_n(\mathbb{R})$ is simply the matrix AA^T from the SVD in Unit U04 (with $A = X$), whose eigenvectors are the left singular vectors of X

Theorem 98 (Properties of the Gram matrix)

Let $X \in \mathcal{M}_{np}(\mathbb{R})$ and let $G = XX^T \in \mathcal{M}_n(\mathbb{R})$ be its Gram matrix. Then:

1. *G is symmetric and positive semi-definite, with real non-negative eigenvalues $\lambda_1 \geq \cdots \geq \lambda_n \geq 0$*
2. $\text{rank}(G) = \text{rank}(X) \leq \min(n, p)$
3. $\text{tr}(G) = \sum_{i=1}^{n} \|\boldsymbol{x}_i\|^2$

Characterisation of Gram matrices via spectral factorisation

In the SVD (Theorems 77 and 79, Unit U04), $X = U\Sigma V^T$ gives

$$XX^T = U\Sigma\Sigma^T U^T = \sum_{i=1}^{r} \sigma_i^2 \boldsymbol{u}_i \boldsymbol{u}_i^T$$

where $\lambda_i = \sigma_i^2 > 0$ are the positive eigenvalues of XX^T

Conversely, *any* symmetric PSD matrix B can be factored as a Gram matrix $B = XX^T$ by reversing this step, as shown by Theorem 99 later

Before stating that theorem, we need to define the square root of a matrix...

Square root of a matrix

A square matrix S is called a **square root** of A if $S^2 = SS = A$, and we write $S = A^{1/2}$

For a diagonal matrix with non-negative diagonal entries $\lambda_i \geq 0$, its square root is obtained by simply taking the square root of each diagonal entry:

$$\Lambda = \text{diag}(\lambda_1, \ldots, \lambda_n) \implies \Lambda^{1/2} = \text{diag}(\sqrt{\lambda_1}, \ldots, \sqrt{\lambda_n})$$

Because a symmetric PSD matrix has eigenvalues $\lambda_i \geq 0$, the square roots $\sqrt{\lambda_i}$ are always real numbers

Example (a simple 2×2 diagonal matrix)

$$\Lambda = \begin{pmatrix} 4 & 0 \\ 0 & 9 \end{pmatrix} \implies \Lambda^{1/2} = \begin{pmatrix} 2 & 0 \\ 0 & 3 \end{pmatrix}$$

since

$$\begin{pmatrix} 2 & 0 \\ 0 & 3 \end{pmatrix} \begin{pmatrix} 2 & 0 \\ 0 & 3 \end{pmatrix} = \begin{pmatrix} 2^2 & 0 \\ 0 & 3^2 \end{pmatrix} = \begin{pmatrix} 4 & 0 \\ 0 & 9 \end{pmatrix}$$

Theorem 99 (Spectral factorisation of symmetric PSD matrices)

Let $B \in \mathcal{M}_n(\mathbb{R})$ be symmetric and positive semi-definite with $\operatorname{rank}(B) = r \leq n$. *Let its ordered eigenvalues be*

$$\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_r > 0 \quad \textit{and} \quad \lambda_{r+1} = \cdots = \lambda_n = 0$$

with corresponding orthonormal eigenvectors $\boldsymbol{v}_1, \ldots, \boldsymbol{v}_n \in \mathbb{R}^n$. Define

$$V_r = \begin{pmatrix} \boldsymbol{v}_1 & \cdots & \boldsymbol{v}_r \end{pmatrix} \in \mathcal{M}_{nr}(\mathbb{R}), \qquad \Lambda_r = \operatorname{diag}(\lambda_1, \ldots, \lambda_r) \in \mathcal{M}_r(\mathbb{R})$$

Then the matrix $X = V_r \Lambda_r^{1/2} \in \mathcal{M}_{nr}(\mathbb{R})$ satisfies $XX^T = B$

Hey, lookee here, another matrix factorisation :) Conditions are much more restrictive than in past factorisations, though, since we need a PSD matrix as input

Gram matrices and Euclidean distance matrices

The Gram matrix

The squared Euclidean distance matrix

Euclidean and squared Euclidean distance matrices

Definition 100 (Euclidean distance matrix and squared distance matrix)

Let $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n \in \mathbb{R}^p$ be the rows of $X \in \mathcal{M}_{np}(\mathbb{R})$

- *The **Euclidean distance matrix** is $D = [d_{ij}] \in \mathcal{M}_n(\mathbb{R})$, where*

$$d_{ij} = \|\boldsymbol{x}_i - \boldsymbol{x}_j\| = \sqrt{\sum_{m=1}^{p} (x_{im} - x_{jm})^2}, \qquad i, j = 1, \ldots, n$$

- *The **squared Euclidean distance matrix** is $D^{(2)} = [d_{ij}^2] \in \mathcal{M}_n(\mathbb{R})$, obtained by squaring each entry of D elementwise (**Hadamard square** $D^{(2)} = D \circ D$):*

$$\left[D^{(2)}\right]_{ij} = d_{ij}^2 = \|\boldsymbol{x}_i - \boldsymbol{x}_j\|^2, \qquad i, j = 1, \ldots, n$$

Warning: $D^{(2)}$ denotes the elementwise square $[d_{ij}^2]$, *not* the matrix product DD!

Basic properties of distance matrices

For any configuration of points $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n \in \mathbb{R}^p$, the matrices $D = [d_{ij}]$ and $D^{(2)} = [d_{ij}^2]$ satisfy:

1. **Zero diagonal** (**hollow matrix**):

$$d_{ii} = \|\boldsymbol{x}_i - \boldsymbol{x}_i\| = 0 \implies \operatorname{diag}(D) = \operatorname{diag}(D^{(2)}) = \mathbf{0} \quad \text{and} \quad \operatorname{tr}(D) = \operatorname{tr}(D^{(2)}) = 0$$

2. **Symmetry**:

$$d_{ij} = \|\boldsymbol{x}_i - \boldsymbol{x}_j\| = \|\boldsymbol{x}_j - \boldsymbol{x}_i\| = d_{ji} \implies D^T = D \quad \text{and} \quad (D^{(2)})^T = D^{(2)}$$

3. **Non-negativity**: $d_{ij} \geq 0$ and $d_{ij}^2 \geq 0$ for all $i, j = 1, \ldots, n$, with $d_{ij} = 0 \iff \boldsymbol{x}_i = \boldsymbol{x}_j$
4. **Triangle inequality** (for D, though *not* necessarily for $D^{(2)}$):

$$d_{ij} \leq d_{ik} + d_{kj} \qquad \forall i, j, k = 1, \ldots, n$$

Relating squared distances to inner products

Why do we work with the *squared* distance matrix $D^{(2)}$ rather than D? Because squared Euclidean norms expand directly in terms of inner products!

For any two points $\boldsymbol{x}_i, \boldsymbol{x}_j \in \mathbb{R}^p$, using the bilinearity and symmetry of the inner product:

$$\begin{aligned} d_{ij}^2 = \|\boldsymbol{x}_i - \boldsymbol{x}_j\|^2 &= (\boldsymbol{x}_i - \boldsymbol{x}_j) \bullet (\boldsymbol{x}_i - \boldsymbol{x}_j) \\ &= (\boldsymbol{x}_i - \boldsymbol{x}_j)^T (\boldsymbol{x}_i - \boldsymbol{x}_j) \\ &= \boldsymbol{x}_i^T \boldsymbol{x}_i - \boldsymbol{x}_i^T \boldsymbol{x}_j - \boldsymbol{x}_j^T \boldsymbol{x}_i + \boldsymbol{x}_j^T \boldsymbol{x}_j \\ &= \|\boldsymbol{x}_i\|^2 - 2\boldsymbol{x}_i^T \boldsymbol{x}_j + \|\boldsymbol{x}_j\|^2 \end{aligned}$$

In terms of the entries $g_{ij} = \boldsymbol{x}_i^T \boldsymbol{x}_j$ of the Gram matrix $G = XX^T$:

$$d_{ij}^2 = g_{ii} - 2g_{ij} + g_{jj} \qquad \forall i, j = 1, \ldots, n$$

Notice that this is the law of cosines: $c^2 = a^2 + b^2 - 2ab\cos\theta$

Matrix formula for $D^{(2)}$ in terms of G

We can write the entrywise identity $d_{ij}^2 = g_{ii} - 2g_{ij} + g_{jj}$ as a single matrix equation

Theorem 101 (Squared distance matrix from the Gram matrix)

Let $X \in \mathcal{M}_{np}(\mathbb{R})$ have Gram matrix $G = XX^T \in \mathcal{M}_n(\mathbb{R})$. Let

$$\boldsymbol{g} = \operatorname{diag}(G) = \begin{pmatrix} g_{11} \\ g_{22} \\ \vdots \\ g_{nn} \end{pmatrix} = \begin{pmatrix} \|\boldsymbol{x}_1\|^2 \\ \|\boldsymbol{x}_2\|^2 \\ \vdots \\ \|\boldsymbol{x}_n\|^2 \end{pmatrix} \in \mathbb{R}^n \quad \textit{and} \quad \mathbb{1} = \begin{pmatrix} 1 \\ 1 \\ \vdots \\ 1 \end{pmatrix} \in \mathbb{R}^n$$

Then the squared Euclidean distance matrix $D^{(2)} = [d_{ij}^2] \in \mathcal{M}_n(\mathbb{R})$ is given by

$$D^{(2)} = \boldsymbol{g}\mathbb{1}^T - 2G + \mathbb{1}\boldsymbol{g}^T$$

Proof of Theorem 101

Examine the (i,j) entry of the rank-one outer product matrices $\boldsymbol{g}\mathbb{1}^T$ and $\mathbb{1}\boldsymbol{g}^T$:

$$\boldsymbol{g}\mathbb{1}^T = \begin{pmatrix} g_{11} \\ g_{22} \\ \vdots \\ g_{nn} \end{pmatrix} \begin{pmatrix} 1 & 1 & \cdots & 1 \end{pmatrix} = \begin{pmatrix} g_{11} & g_{11} & \cdots & g_{11} \\ g_{22} & g_{22} & \cdots & g_{22} \\ \vdots & \vdots & \ddots & \vdots \\ g_{nn} & g_{nn} & \cdots & g_{nn} \end{pmatrix} \implies \left[\boldsymbol{g}\mathbb{1}^T\right]_{ij} = g_{ii}$$

Similarly, taking the transpose:

$$\mathbb{1}\boldsymbol{g}^T = (\boldsymbol{g}\mathbb{1}^T)^T = \begin{pmatrix} g_{11} & g_{22} & \cdots & g_{nn} \\ g_{11} & g_{22} & \cdots & g_{nn} \\ \vdots & \vdots & \ddots & \vdots \\ g_{11} & g_{22} & \cdots & g_{nn} \end{pmatrix} \implies \left[\mathbb{1}\boldsymbol{g}^T\right]_{ij} = g_{jj}$$

Therefore, for every $(i,j) \in \{1,\ldots,n\}^2$,

$$\left[\boldsymbol{g}\mathbb{1}^T - 2G + \mathbb{1}\boldsymbol{g}^T\right]_{ij} = g_{ii} - 2g_{ij} + g_{jj} = \|\boldsymbol{x}_i\|^2 - 2\boldsymbol{x}_i^T\boldsymbol{x}_j + \|\boldsymbol{x}_j\|^2 = \|\boldsymbol{x}_i - \boldsymbol{x}_j\|^2 = d_{ij}^2 \quad \square$$

Example – Computing G and $D^{(2)}$ from X (1/2)

Consider $n = 3$ points in $\mathbb{R}^2$ given by the rows of

$$X = \begin{pmatrix} 1 & 2 \\ 3 & 1 \\ -1 & 0 \end{pmatrix} \in \mathcal{M}_{32}(\mathbb{R}), \qquad \text{i.e., } \boldsymbol{x}_1 = \begin{pmatrix} 1 \\ 2 \end{pmatrix}, \boldsymbol{x}_2 = \begin{pmatrix} 3 \\ 1 \end{pmatrix}, \boldsymbol{x}_3 = \begin{pmatrix} -1 \\ 0 \end{pmatrix}$$

First, compute the Gram matrix $G = XX^T$:

$$G = \begin{pmatrix} 1 & 2 \\ 3 & 1 \\ -1 & 0 \end{pmatrix} \begin{pmatrix} 1 & 3 & -1 \\ 2 & 1 & 0 \end{pmatrix} = \begin{pmatrix} 5 & 5 & -1 \\ 5 & 10 & -3 \\ -1 & -3 & 1 \end{pmatrix}$$

The vector of diagonal entries of G (squared norms of the points) is

$$\boldsymbol{g} = \operatorname{diag}(G) = \begin{pmatrix} 5 \\ 10 \\ 1 \end{pmatrix}$$

Example – Computing G and $D^{(2)}$ from X (2/2)

Using Theorem 101, we form $\boldsymbol{g}\mathbb{1}^T$, $\mathbb{1}\boldsymbol{g}^T$ and $D^{(2)} = \boldsymbol{g}\mathbb{1}^T - 2G + \mathbb{1}\boldsymbol{g}^T$:

$$\boldsymbol{g}\mathbb{1}^T = \begin{pmatrix} 5 & 5 & 5 \\ 10 & 10 & 10 \\ 1 & 1 & 1 \end{pmatrix}, \qquad \mathbb{1}\boldsymbol{g}^T = \begin{pmatrix} 5 & 10 & 1 \\ 5 & 10 & 1 \\ 5 & 10 & 1 \end{pmatrix}$$

Hence:

$$D^{(2)} = \begin{pmatrix} 5 & 5 & 5 \\ 10 & 10 & 10 \\ 1 & 1 & 1 \end{pmatrix} - 2\begin{pmatrix} 5 & 5 & -1 \\ 5 & 10 & -3 \\ -1 & -3 & 1 \end{pmatrix} + \begin{pmatrix} 5 & 10 & 1 \\ 5 & 10 & 1 \\ 5 & 10 & 1 \end{pmatrix} = \begin{pmatrix} 0 & 5 & 8 \\ 5 & 0 & 17 \\ 8 & 17 & 0 \end{pmatrix}$$

Check directly:

- $d_{12}^2 = \|(1,2)^T - (3,1)^T\|^2 = (-2)^2 + 1^2 = 5$
- $d_{13}^2 = \|(1,2)^T - (-1,0)^T\|^2 = 2^2 + 2^2 = 8$
- $d_{23}^2 = \|(3,1)^T - (-1,0)^T\|^2 = 4^2 + 1^2 = 17$

Motivation: from distances back to coordinates

Gram matrices and Euclidean distance matrices

The centring matrix as an orthogonal projector

Classical multidimensional scaling (MDS)

Algebraic duality between PCA and classical MDS

The all-ones vector $\mathbb{1}$ and the all-ones matrix $\mathbb{J}_n$

Two simple building blocks in $\mathbb{R}^n$ and $\mathcal{M}_n(\mathbb{R})$:

- The **all-ones vector** $\mathbb{1} \in \mathbb{R}^n$:

$$\mathbb{1} = \begin{pmatrix} 1 \\ 1 \\ \vdots \\ 1 \end{pmatrix} \in \mathbb{R}^n \implies \mathbb{1}^T\mathbb{1} = \sum_{i=1}^{n} 1^2 = n \quad \text{and} \quad \|\mathbb{1}\| = \sqrt{n}$$

- The **all-ones matrix** $\mathbb{J}_n = \mathbb{1}\mathbb{1}^T \in \mathcal{M}_n(\mathbb{R})$:

$$\mathbb{J}_n = \begin{pmatrix} 1 & 1 & \cdots & 1 \\ 1 & 1 & \cdots & 1 \\ \vdots & \vdots & \ddots & \vdots \\ 1 & 1 & \cdots & 1 \end{pmatrix} \in \mathcal{M}_n(\mathbb{R}) \implies \mathbb{J}_n^2 = (\mathbb{1}\mathbb{1}^T)(\mathbb{1}\mathbb{1}^T) = \mathbb{1}(\mathbb{1}^T\mathbb{1})\mathbb{1}^T = n\mathbb{J}_n$$

Definition of the centring matrix

Definition 102 (Centring matrix)

*For $n \geq 1$, the **centring matrix** of order n is the $n \times n$ matrix*

$$C_n = \mathbb{I}_n - \frac{1}{n}\mathbb{1}\mathbb{1}^T = \mathbb{I}_n - \frac{1}{n}\mathbb{J}_n \in \mathcal{M}_n(\mathbb{R})$$

where $\mathbb{I}_n$ is the $n \times n$ identity matrix, $\mathbb{1} = (1, \ldots, 1)^T \in \mathbb{R}^n$ and $\mathbb{J}_n = \mathbb{1}\mathbb{1}^T$ is the $n \times n$ matrix of all ones

Entrywise, the (i, j) entry of C_n is

$$[C_n]_{ij} = \delta_{ij} - \frac{1}{n} = \begin{cases} 1 - \dfrac{1}{n} = \dfrac{n-1}{n} & \text{if } i = j \\ -\dfrac{1}{n} & \text{if } i \neq j \end{cases}$$

Explicit examples of C_n for small n

Writing out $C_n = \mathbb{I}_n - \frac{1}{n}\mathbb{J}_n$ explicitly for $n = 2, 3, 4$:

$$C_2 = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} - \frac{1}{2}\begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} = \frac{1}{2}\begin{pmatrix} 1 & -1 \\ -1 & 1 \end{pmatrix}$$

$$C_3 = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} - \frac{1}{3}\begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix} = \frac{1}{3}\begin{pmatrix} 2 & -1 & -1 \\ -1 & 2 & -1 \\ -1 & -1 & 2 \end{pmatrix}$$

$$C_4 = \frac{1}{4}\begin{pmatrix} 3 & -1 & -1 & -1 \\ -1 & 3 & -1 & -1 \\ -1 & -1 & 3 & -1 \\ -1 & -1 & -1 & 3 \end{pmatrix}$$

Notice that in every C_n, each row and each column sums to zero!

Algebraic and geometric properties of the centring matrix

Theorem 103 (Properties of the centring matrix C_n)

Let $C_n = \mathbb{I}_n - \frac{1}{n}\mathbb{1}\mathbb{1}^T \in \mathcal{M}_n(\mathbb{R})$. *Then:*

1. ***Symmetry****:* $C_n^T = C_n$
2. ***Idempotence****:* $C_n^2 = C_n$ *(and more generally* $C_n^k = C_n$ *for all integers* $k \geq 1$*)*
3. ***Null space****:* $C_n\mathbb{1} = \mathbf{0}$ *and* $\mathbb{1}^T C_n = \mathbf{0}^T$*; in fact,* $\text{null}(C_n) = \text{span}(\mathbb{1})$
4. ***Orthogonal projector****:* C_n *is the orthogonal projection matrix onto the* $(n-1)$*-dimensional subspace of zero-mean vectors*

$$W = \text{span}(\mathbb{1})^\perp = \left\{ \boldsymbol{v} \in \mathbb{R}^n : \mathbb{1}^T \boldsymbol{v} = \sum_{i=1}^{n} v_i = 0 \right\}$$

5. ***Spectrum, trace and rank****: the eigenvalues of* C_n *are* $\lambda = 0$ *(with multiplicity* 1*, eigenspace* $\text{span}(\mathbb{1})$*) and* $\lambda = 1$ *(with multiplicity* $n-1$*, eigenspace* W*). Consequently,* $\text{rank}(C_n) = \text{tr}(C_n) = n - 1$

Proof of Theorem 103 – Parts 1 and 2

1. C_n **is symmetric (**$C_n^T = C_n$**)**: Using linearity of the transpose, $\mathbb{I}_n^T = \mathbb{I}_n$ and $(\mathbb{1}\mathbb{1}^T)^T = (\mathbb{1}^T)^T\mathbb{1}^T = \mathbb{1}\mathbb{1}^T$:

$$C_n^T = \left(\mathbb{I}_n - \frac{1}{n}\mathbb{1}\mathbb{1}^T\right)^T = \mathbb{I}_n^T - \frac{1}{n}(\mathbb{1}\mathbb{1}^T)^T = \mathbb{I}_n - \frac{1}{n}\mathbb{1}\mathbb{1}^T = C_n \quad \square$$

2. C_n **is idempotent (**$C_n^2 = C_n$**)**: Expanding the square and using the associativity of matrix multiplication together with $\mathbb{1}^T\mathbb{1} = n$:

$$\begin{aligned} C_n^2 &= \left(\mathbb{I}_n - \frac{1}{n}\mathbb{1}\mathbb{1}^T\right)\left(\mathbb{I}_n - \frac{1}{n}\mathbb{1}\mathbb{1}^T\right) \\ &= \mathbb{I}_n^2 - \frac{1}{n}\mathbb{1}\mathbb{1}^T - \frac{1}{n}\mathbb{1}\mathbb{1}^T + \frac{1}{n^2}(\mathbb{1}\mathbb{1}^T)(\mathbb{1}\mathbb{1}^T) \\ &= \mathbb{I}_n - \frac{2}{n}\mathbb{1}\mathbb{1}^T + \frac{1}{n^2}\mathbb{1}(\underbrace{\mathbb{1}^T\mathbb{1}}_{=n})\mathbb{1}^T \\ &= \mathbb{I}_n - \frac{2}{n}\mathbb{1}\mathbb{1}^T + \frac{1}{n}\mathbb{1}\mathbb{1}^T = \mathbb{I}_n - \frac{1}{n}\mathbb{1}\mathbb{1}^T = C_n \quad \square \end{aligned}$$

Proof of Theorem 103 – Part 3

3. $C_n\mathbb{1} = \mathbf{0}$, $\mathbb{1}^T C_n = \mathbf{0}^T$ **and** $\text{null}(C_n) = \text{span}(\mathbb{1})$:
First, multiply C_n on the right by $\mathbb{1}$:

$$C_n\mathbb{1} = \left(\mathbb{I}_n - \frac{1}{n}\mathbb{1}\mathbb{1}^T\right)\mathbb{1} = \mathbb{1} - \frac{1}{n}\mathbb{1}(\mathbb{1}^T\mathbb{1}) = \mathbb{1} - \frac{1}{n}\mathbb{1}(n) = \mathbb{1} - \mathbb{1} = \mathbf{0}$$

Taking transposes and using $C_n^T = C_n$ immediately gives

$$\mathbb{1}^T C_n = \mathbb{1}^T C_n^T = (C_n\mathbb{1})^T = \mathbf{0}^T$$

To show $\text{null}(C_n) = \text{span}(\mathbb{1})$, let $\boldsymbol{v} \in \mathbb{R}^n$. Then

$$\boldsymbol{v} \in \text{null}(C_n) \iff C_n\boldsymbol{v} = \mathbf{0} \iff \boldsymbol{v} - \left(\frac{1}{n}\mathbb{1}^T\boldsymbol{v}\right)\mathbb{1} = \mathbf{0} \iff \boldsymbol{v} = \bar{v}\mathbb{1} \iff \boldsymbol{v} \in \text{span}(\mathbb{1})$$

Thus $\text{null}(C_n) = \text{span}(\mathbb{1})$, which is a 1-dimensional subspace of $\mathbb{R}^n$
(nullity$(C_n) = 1$) □

Proof of Theorem 103 – Part 4

4. C_n is the orthogonal projection matrix onto $W = \text{span}(\mathbb{1})^\perp$:
Let $L = \text{span}(\mathbb{1})$. Since $\{\mathbb{1}\}$ is an orthogonal basis of the 1-dimensional subspace L with $\|\mathbb{1}\|^2 = \mathbb{1}^T\mathbb{1} = n$, the orthogonal projection of any vector $\boldsymbol{v} \in \mathbb{R}^n$ onto L is

$$\text{proj}_L(\boldsymbol{v}) = \frac{\mathbb{1} \bullet \boldsymbol{v}}{\|\mathbb{1}\|^2}\mathbb{1} = \frac{\mathbb{1}^T\boldsymbol{v}}{n}\mathbb{1} = \left(\frac{1}{n}\mathbb{1}\mathbb{1}^T\right)\boldsymbol{v}$$

By definition of the orthogonal complement $W = L^\perp = \{\boldsymbol{v} \in \mathbb{R}^n : \mathbb{1}^T\boldsymbol{v} = 0\}$, the orthogonal projection of $\boldsymbol{v}$ onto W is the component of $\boldsymbol{v}$ orthogonal to L:

$$\text{proj}_W(\boldsymbol{v}) = \text{perp}_L(\boldsymbol{v}) = \boldsymbol{v} - \text{proj}_L(\boldsymbol{v}) = \boldsymbol{v} - \left(\frac{1}{n}\mathbb{1}\mathbb{1}^T\right)\boldsymbol{v} = \left(\mathbb{I}_n - \frac{1}{n}\mathbb{1}\mathbb{1}^T\right)\boldsymbol{v} = C_n\boldsymbol{v}$$

Note also that for any $\boldsymbol{v} \in \mathbb{R}^n$, the centred vector $C_n\boldsymbol{v}$ and the mean vector $\bar{v}\mathbb{1} = (\mathbb{I}_n - C_n)\boldsymbol{v}$ are orthogonal:

$$(C_n\boldsymbol{v}) \bullet (\bar{v}\mathbb{1}) = \bar{v}\boldsymbol{v}^T C_n^T\mathbb{1} = \bar{v}\boldsymbol{v}^T(C_n\mathbb{1}) = 0 \quad \square$$

Proof of Theorem 103 – Part 5

5. Eigenvalues, trace and rank of C_n:

Let $(\lambda, \boldsymbol{v})$ be an eigenpair of C_n ($\boldsymbol{v} \neq \mathbf{0}$ and $C_n\boldsymbol{v} = \lambda\boldsymbol{v}$). Applying C_n to both sides and using idempotence $C_n^2 = C_n$:

$$C_n\boldsymbol{v} = C_n^2\boldsymbol{v} = C_n(C_n\boldsymbol{v}) = C_n(\lambda\boldsymbol{v}) = \lambda(C_n\boldsymbol{v}) = \lambda^2\boldsymbol{v}$$

Since $\boldsymbol{v} \neq \mathbf{0}$, $(\lambda^2 - \lambda)\boldsymbol{v} = \mathbf{0} \implies \lambda(\lambda - 1) = 0$, so every eigenvalue of C_n is either $\lambda = 0$ or $\lambda = 1$:

- For $\lambda = 0$: the eigenspace is $\text{null}(C_n) = \text{span}(\mathbb{1})$, which has dimension 1 (multiplicity 1)
- For $\lambda = 1$: $\boldsymbol{v} \in W = \text{span}(\mathbb{1})^\perp \iff \mathbb{1}^T\boldsymbol{v} = 0 \implies C_n\boldsymbol{v} = \boldsymbol{v} - \frac{1}{n}\mathbb{1}(\mathbb{1}^T\boldsymbol{v}) = \boldsymbol{v}$, so the eigenspace for $\lambda = 1$ is W, which has dimension $n - 1$ (multiplicity $n - 1$)

Finally, the trace equals the sum of the diagonal entries (and the sum of the eigenvalues):

$$\text{tr}(C_n) = \sum_{i=1}^{n}\left(1 - \frac{1}{n}\right) = n\left(\frac{n-1}{n}\right) = n - 1 = \text{rank}(C_n) \quad \square$$

Action of C_n on a vector: subtracting the mean

Why is C_n called the *centring matrix*? Let $\boldsymbol{v} = (v_1, \ldots, v_n)^T \in \mathbb{R}^n$ be a column vector of n observations of a feature

First note that the inner product $\mathbb{1}^T \boldsymbol{v}$ is the sum of the entries of $\boldsymbol{v}$:

$$\mathbb{1}^T \boldsymbol{v} = \sum_{i=1}^{n} v_i = n\bar{v}, \qquad \text{where} \quad \bar{v} = \frac{1}{n} \sum_{i=1}^{n} v_i$$

Multiplying $\boldsymbol{v}$ on the left by C_n gives:

$$C_n \boldsymbol{v} = \left(\mathbb{I}_n - \frac{1}{n} \mathbb{1}\mathbb{1}^T \right) \boldsymbol{v} = \boldsymbol{v} - \frac{1}{n} \mathbb{1}(\mathbb{1}^T \boldsymbol{v}) = \boldsymbol{v} - \bar{v}\mathbb{1} = \begin{pmatrix} v_1 - \bar{v} \\ v_2 - \bar{v} \\ \vdots \\ v_n - \bar{v} \end{pmatrix}$$

Thus, $C_n \boldsymbol{v}$ subtracts the arithmetic mean $\bar{v}$ from every component of $\boldsymbol{v}$, producing a **zero-mean** (centred) vector

Action of C_n on a data matrix

Now let $X \in \mathcal{M}_{np}(\mathbb{R})$ be a data matrix with columns $\boldsymbol{c}_1, \ldots, \boldsymbol{c}_p \in \mathbb{R}^n$ (the p features) and rows $\boldsymbol{x}_1^T, \ldots, \boldsymbol{x}_n^T \in \mathbb{R}^p$ (the n observations)

- **Column-wise view**: left-multiplying X by C_n centres every column of X simultaneously:

$$\widetilde{X} = C_n X = \begin{pmatrix} C_n \boldsymbol{c}_1 & C_n \boldsymbol{c}_2 & \cdots & C_n \boldsymbol{c}_p \end{pmatrix}$$

- **Row-wise view**: if $\bar{\boldsymbol{x}} = \frac{1}{n} \sum_{i=1}^n \boldsymbol{x}_i = \frac{1}{n} X^T \mathbb{1} \in \mathbb{R}^p$ is the centroid of the n points, then

$$\widetilde{X} = C_n X = X - \frac{1}{n} \mathbb{1}\mathbb{1}^T X = X - \mathbb{1}\bar{\boldsymbol{x}}^T = \begin{pmatrix} (\boldsymbol{x}_1 - \bar{\boldsymbol{x}})^T \\ \vdots \\ (\boldsymbol{x}_n - \bar{\boldsymbol{x}})^T \end{pmatrix}$$

shifts the centroid of the n points to the origin $\mathbf{0} \in \mathbb{R}^p$

Link to covariance (Unit 05)

The sample covariance matrix $S_X \in \mathcal{M}_p(\mathbb{R})$ of X can be written compactly as

$$
\begin{aligned}
S_X &= \frac{1}{n-1}\widetilde{X}^T\widetilde{X} \\
&= \frac{1}{n-1}(C_nX)^T(C_nX) \\
&= \frac{1}{n-1}X^TC_n^TC_nX \\
&= \frac{1}{n-1}X^TC_nC_nX && \text{as } C_n \text{ is symmetric} \\
&= \frac{1}{n-1}X^TC_nX && \text{as } C_n \text{ is idempotent}
\end{aligned}
$$

Motivation: from distances back to coordinates

Gram matrices and Euclidean distance matrices

The centring matrix as an orthogonal projector

Classical multidimensional scaling (MDS)

Algebraic duality between PCA and classical MDS

Classical multidimensional scaling (MDS)

- **Double centring: recovering the Gram matrix from distances**
- Coordinate recovery via spectral decomposition
- Examples

From $D^{(2)}$ back to G: why double centring works

Recall Theorem 101: given the Gram matrix $G = XX^T$, the squared distance matrix is

$$D^{(2)} = \boldsymbol{g}\mathbb{1}^T - 2G + \mathbb{1}\boldsymbol{g}^T$$

We now want to **invert** this relationship: given only $D^{(2)}$, how can we isolate G?

Look closely at the two nuisance terms $\boldsymbol{g}\mathbb{1}^T$ and $\mathbb{1}\boldsymbol{g}^T$:

- $\boldsymbol{g}\mathbb{1}^T$ has $\mathbb{1}^T$ on the **right**, and we know that $\mathbb{1}^T C_n = \mathbf{0}^T$!
- $\mathbb{1}\boldsymbol{g}^T$ has $\mathbb{1}$ on the **left**, and we know that $C_n\mathbb{1} = \mathbf{0}$!

Therefore, if we multiply $D^{(2)}$ on the **left** by C_n and on the **right** by C_n, both rank-one nuisance terms $\boldsymbol{g}\mathbb{1}^T$ and $\mathbb{1}\boldsymbol{g}^T$ are completely annihilated!

This operation $D^{(2)} \mapsto -\frac{1}{2} C_n D^{(2)} C_n$ is called **double centring**

The double centring theorem

Theorem 104 (Double centring formula)

Let $X \in \mathcal{M}_{np}(\mathbb{R})$ be a data matrix with squared Euclidean distance matrix $D^{(2)} = [d_{ij}^2] \in \mathcal{M}_n(\mathbb{R})$, and let $\widetilde{X} = C_n X$ be the column-centred data matrix with centred Gram matrix $B = \widetilde{X}\widetilde{X}^T = C_n X X^T C_n$. Then

$$B = -\frac{1}{2} C_n D^{(2)} C_n = \widetilde{X}\widetilde{X}^T$$

In particular, if the points in X are already centred at the origin ($C_n X = X$, so $\sum_{i=1}^n \boldsymbol{x}_i = \mathbf{0}$), then $B = -\frac{1}{2} C_n D^{(2)} C_n = G = XX^T$ recovers the Gram matrix of X

Proof of Theorem 104

By Theorem 101, $D^{(2)} = \boldsymbol{g}\mathbb{1}^T - 2G + \mathbb{1}\boldsymbol{g}^T$, where $G = XX^T$ and $\boldsymbol{g} = \text{diag}(G)$
Multiply $D^{(2)}$ on the left by C_n and on the right by C_n:

$$C_n D^{(2)} C_n = C_n(\boldsymbol{g}\mathbb{1}^T)C_n - 2C_nGC_n + C_n(\mathbb{1}\boldsymbol{g}^T)C_n$$

Using associativity and Part 3 of Theorem 103 ($C_n\mathbb{1} = \mathbf{0}$ and $\mathbb{1}^T C_n = \mathbf{0}^T$):

$$C_n(\boldsymbol{g}\mathbb{1}^T)C_n = (C_n\boldsymbol{g})(\underbrace{\mathbb{1}^T C_n}_{=\mathbf{0}^T}) = (C_n\boldsymbol{g})\mathbf{0}^T = \mathbf{0}_{n\times n}$$

$$C_n(\mathbb{1}\boldsymbol{g}^T)C_n = (\underbrace{C_n\mathbb{1}}_{=\mathbf{0}})(\boldsymbol{g}^T C_n) = \mathbf{0}(\boldsymbol{g}^T C_n) = \mathbf{0}_{n\times n}$$

Thus both outer-product terms vanish, leaving

$$C_n D^{(2)} C_n = -2C_nGC_n = -2C_n(XX^T)C_n^T = -2(C_nX)(C_nX)^T = -2\widetilde{X}\widetilde{X}^T$$

Dividing both sides by -2 gives $-\frac{1}{2}C_nD^{(2)}C_n = C_nGC_n = \widetilde{X}\widetilde{X}^T$. If $C_nX = X$, then $\widetilde{X} = X$ and $-\frac{1}{2}C_nD^{(2)}C_n = XX^T = G$ □

Scalar form of double centring

Why is $B = -\frac{1}{2} C_n D^{(2)} C_n$ called *double* centring?

- Left-multiplying $D^{(2)}$ by C_n subtracts the **column means** of $D^{(2)}$ from each column
- Right-multiplying by $C_n = C_n^T$ subtracts the **row means** from each row

Let $\bar{d}^2_{i\cdot} = \frac{1}{n}\sum_{j=1}^n d^2_{ij}$ (mean of row i), $\bar{d}^2_{\cdot j} = \frac{1}{n}\sum_{i=1}^n d^2_{ij}$ (mean of column j) and $\bar{d}^2_{\cdot\cdot} = \frac{1}{n^2}\sum_{i=1}^n\sum_{j=1}^n d^2_{ij}$ (grand mean of $D^{(2)}$). Expanding $B = -\frac{1}{2}\left(\mathbb{I}_n - \frac{1}{n}\mathbb{J}_n\right) D^{(2)} \left(\mathbb{I}_n - \frac{1}{n}\mathbb{J}_n\right)$ gives the entrywise formula:

$$b_{ij} = -\frac{1}{2}\left(d^2_{ij} - \bar{d}^2_{i\cdot} - \bar{d}^2_{\cdot j} + \bar{d}^2_{\cdot\cdot}\right), \qquad i, j = 1, \ldots, n$$

Consequently, every row sum and every column sum of B is zero: $B\mathbb{1} = \mathbf{0}$ and $\mathbb{1}^T B = \mathbf{0}^T$, so $\lambda = 0$ is always an eigenvalue of B with eigenvector $\mathbb{1}$!

Classical multidimensional scaling (MDS)

- Double centring: recovering the Gram matrix from distances
- **Coordinate recovery via spectral decomposition**
- Examples

The classical MDS algorithm

Definition 105 (Classical MDS algorithm)

Given an $n \times n$ distance or dissimilarity matrix $D = [d_{ij}] \in \mathcal{M}_n(\mathbb{R})$ and a target embedding dimension $k \in \{1, \dots, n-1\}$:

1. ***Square the distances**: form $D^{(2)} = [d_{ij}^2] \in \mathcal{M}_n(\mathbb{R})$*
2. ***Double-centre**: compute the inner product matrix $B = -\frac{1}{2} C_n D^{(2)} C_n$*
3. ***Spectral decomposition**: compute the eigenvalues $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_n$ and orthonormal eigenvectors $\boldsymbol{v}_1, \boldsymbol{v}_2, \dots, \boldsymbol{v}_n \in \mathbb{R}^n$ of B*
4. ***Construct k-dimensional coordinates**: assuming $\lambda_1 \geq \cdots \geq \lambda_k > 0$, let $V_k = [\boldsymbol{v}_1 \ \cdots \ \boldsymbol{v}_k] \in \mathcal{M}_{nk}(\mathbb{R})$ and $\Lambda_k = \operatorname{diag}(\lambda_1, \dots, \lambda_k) \in \mathcal{M}_k(\mathbb{R})$. The k-dimensional MDS coordinate matrix is*

$$X_k = V_k \Lambda_k^{1/2} = \begin{pmatrix} \sqrt{\lambda_1}\, \boldsymbol{v}_1 & \sqrt{\lambda_2}\, \boldsymbol{v}_2 & \cdots & \sqrt{\lambda_k}\, \boldsymbol{v}_k \end{pmatrix} \in \mathcal{M}_{nk}(\mathbb{R})$$

Classical multidimensional scaling (MDS)

- Double centring: recovering the Gram matrix from distances
- Coordinate recovery via spectral decomposition
- **Examples**

Example 1 – Recovering 1D coordinates from $D^{(2)}$ (1/4)

Suppose $n = 3$ objects have the squared Euclidean distance matrix

$$D^{(2)} = \begin{pmatrix} 0 & 4 & 16 \\ 4 & 0 & 4 \\ 16 & 4 & 0 \end{pmatrix}$$

(i.e. $d_{12} = 2$, $d_{23} = 2$ and $d_{13} = 4$)

Problem:

1. Compute the double-centred Gram matrix $B = -\frac{1}{2} C_3 D^{(2)} C_3$
2. Find the eigenvalues and orthonormal eigenvectors of B
3. Determine the minimal embedding dimension $r = \text{rank}(B)$ and recover the centred coordinates X_r

Example 1 – Computing $B = -\frac{1}{2} C_3 D^{(2)} C_3$ (2/4)

Recall $C_3 = \frac{1}{3}\begin{pmatrix} 2 & -1 & -1 \\ -1 & 2 & -1 \\ -1 & -1 & 2 \end{pmatrix}$. First compute $C_3 D^{(2)}$:

$$C_3 D^{(2)} = \frac{1}{3}\begin{pmatrix} 2 & -1 & -1 \\ -1 & 2 & -1 \\ -1 & -1 & 2 \end{pmatrix}\begin{pmatrix} 0 & 4 & 16 \\ 4 & 0 & 4 \\ 16 & 4 & 0 \end{pmatrix} = \frac{1}{3}\begin{pmatrix} -20 & 4 & 28 \\ -16 & -8 & -16 \\ 36 & 4 & -12 \end{pmatrix} = \frac{4}{3}\begin{pmatrix} -5 & 1 & 7 \\ -4 & -2 & -4 \\ 9 & 1 & -3 \end{pmatrix}$$

Now right-multiply by C_3 and multiply by $-\frac{1}{2}$:

$$C_3 D^{(2)} C_3 = \frac{4}{9}\begin{pmatrix} -5 & 1 & 7 \\ -4 & -2 & -4 \\ 9 & 1 & -3 \end{pmatrix}\begin{pmatrix} 2 & -1 & -1 \\ -1 & 2 & -1 \\ -1 & -1 & 2 \end{pmatrix} = \frac{4}{9}\begin{pmatrix} -18 & 0 & 18 \\ 0 & 0 & 0 \\ 18 & 0 & -18 \end{pmatrix} = \begin{pmatrix} -8 & 0 & 8 \\ 0 & 0 & 0 \\ 8 & 0 & -8 \end{pmatrix}$$

$$\implies B = -\frac{1}{2} C_3 D^{(2)} C_3 = \begin{pmatrix} 4 & 0 & -4 \\ 0 & 0 & 0 \\ -4 & 0 & 4 \end{pmatrix}$$

Example 1 – Eigenpairs and coordinate recovery (3/4)

We have

$$B = \begin{pmatrix} 4 & 0 & -4 \\ 0 & 0 & 0 \\ -4 & 0 & 4 \end{pmatrix}$$

- Notice that row 1 and row 3 are opposites and row 2 is zero, so $\text{rank}(B) = 1$
- Since $\text{tr}(B) = 4 + 0 + 4 = 8$ and $\text{rank}(B) = 1$, the eigenvalues of B are immediately

$$\lambda_1 = 8, \qquad \lambda_2 = 0, \qquad \lambda_3 = 0$$

- For $\lambda_1 = 8$, $(B - 8\mathbb{I}_3)\boldsymbol{v}_1 = \mathbf{0} \implies v_{11} + v_{13} = 0$ and $v_{12} = 0$. Normalising gives the unit eigenvector

$$\boldsymbol{v}_1 = \frac{1}{\sqrt{2}} \begin{pmatrix} -1 \\ 0 \\ 1 \end{pmatrix} \qquad (\text{or } -\boldsymbol{v}_1 = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 0 \\ -1 \end{pmatrix})$$

Example 1 – Verification of recovered 1D coordinates (4/4)

Since B is positive semi-definite with $r = \text{rank}(B) = 1$, the 3 points lie on a 1-dimensional line ($\mathbb{R}^1$)!

Recovering the 1-dimensional coordinate vector $X_1 = \sqrt{\lambda_1}\, \boldsymbol{v}_1 \in \mathcal{M}_{31}(\mathbb{R})$:

$$X_1 = \sqrt{8}\left(\frac{1}{\sqrt{2}}\begin{pmatrix}-1\\0\\1\end{pmatrix}\right) = \frac{2\sqrt{2}}{\sqrt{2}}\begin{pmatrix}-1\\0\\1\end{pmatrix} = \begin{pmatrix}-2\\0\\2\end{pmatrix}$$

Check:

- Centred at the origin: $x_1 + x_2 + x_3 = -2 + 0 + 2 = 0$
- Pairwise squared distances:

$$(x_1 - x_2)^2 = (-2-0)^2 = 4, \quad (x_2 - x_3)^2 = (0-2)^2 = 4, \quad (x_1 - x_3)^2 = (-2-2)^2 = 16$$

which reproduces $D^{(2)}$ exactly! (Choosing $-\boldsymbol{v}_1$ gives $X_1 = (2, 0, -2)^T$, the reflection across the origin)

Example 2 – Recovering a 2D triangle from distances (1/4)

Now consider $n = 3$ points with squared Euclidean distance matrix

$$D^{(2)} = \begin{pmatrix} 0 & 6 & 6 \\ 6 & 0 & 12 \\ 6 & 12 & 0 \end{pmatrix}$$

(an isosceles right triangle with leg length $\sqrt{6}$ and hypotenuse $\sqrt{12}$, since $6 + 6 = 12$)

Let us compute $B = -\frac{1}{2} C_3 D^{(2)} C_3$ using the row/column mean formula $b_{ij} = -\frac{1}{2}(d_{ij}^2 - \bar{d}_{i\cdot}^2 - \bar{d}_{\cdot j}^2 + \bar{d}_{\cdot\cdot}^2)$:

- Row (and column) means of $D^{(2)}$:

$$\bar{d}_{1\cdot}^2 = \frac{0 + 6 + 6}{3} = 4, \qquad \bar{d}_{2\cdot}^2 = \frac{6 + 0 + 12}{3} = 6, \qquad \bar{d}_{3\cdot}^2 = \frac{6 + 12 + 0}{3} = 6$$

- Grand mean of $D^{(2)}$:

$$\bar{d}_{\cdot\cdot}^2 = \frac{4 + 6 + 6}{3} = \frac{16}{3}$$

Example 2 – Forming B and finding its eigenpairs (2/4)

Computing each entry $b_{ij} = -\frac{1}{2}(d_{ij}^2 - \bar{d}_{i\cdot}^2 - \bar{d}_{\cdot j}^2 + \frac{16}{3})$ (or multiplying $-\frac{1}{2}C_3 D^{(2)} C_3$ directly):

$$B = -\frac{1}{2}C_3 D^{(2)} C_3 = \frac{1}{3}\begin{pmatrix} 4 & -2 & -2 \\ -2 & 10 & -8 \\ -2 & -8 & 10 \end{pmatrix} = \begin{pmatrix} 4/3 & -2/3 & -2/3 \\ -2/3 & 10/3 & -8/3 \\ -2/3 & -8/3 & 10/3 \end{pmatrix}$$

Let us find the eigenvalues and orthonormal eigenvectors of B:

- We always know $\boldsymbol{v}_3 = \frac{1}{\sqrt{3}}(1,1,1)^T$ is an eigenvector with eigenvalue $\lambda_3 = 0$
- By symmetry between points 2 and 3, try $\boldsymbol{v}_1 = \frac{1}{\sqrt{2}}(0,1,-1)^T$:

$$B\boldsymbol{v}_1 = \frac{1}{3\sqrt{2}}\begin{pmatrix} 4 & -2 & -2 \\ -2 & 10 & -8 \\ -2 & -8 & 10 \end{pmatrix}\begin{pmatrix} 0 \\ 1 \\ -1 \end{pmatrix} = \frac{1}{3\sqrt{2}}\begin{pmatrix} 0 \\ 18 \\ -18 \end{pmatrix} = 6\boldsymbol{v}_1 \implies \lambda_1 = 6$$

Example 2 – Second eigenpair and 2D coordinates (3/4)

- Since $\text{tr}(B) = \frac{4+10+10}{3} = 8 = \lambda_1 + \lambda_2 + \lambda_3$ and $\lambda_1 = 6, \lambda_3 = 0$, the remaining eigenvalue is
$$\lambda_2 = 8 - 6 - 0 = 2$$
- The unit eigenvector $\boldsymbol{v}_2$ must be orthogonal to both $\boldsymbol{v}_3 = \frac{1}{\sqrt{3}}(1,1,1)^T$ and $\boldsymbol{v}_1 = \frac{1}{\sqrt{2}}(0,1,-1)^T$, which gives
$$\boldsymbol{v}_2 = \frac{1}{\sqrt{6}} \begin{pmatrix} 2 \\ -1 \\ -1 \end{pmatrix} \qquad \left(\text{check: } B\boldsymbol{v}_2 = \frac{1}{3\sqrt{6}} \begin{pmatrix} 12 \\ -6 \\ -6 \end{pmatrix} = 2\boldsymbol{v}_2 \right)$$

Since $\lambda_1 = 6 > 0$, $\lambda_2 = 2 > 0$ and $\lambda_3 = 0$, $\text{rank}(B) = 2$, so the 3 points embed isometrically in $\mathbb{R}^2$!

Example 2 – Constructing X_2 and verifying distances (4/4)

Using $X_2 = V_2 \Lambda_2^{1/2} = \begin{pmatrix} \sqrt{\lambda_1}\,\boldsymbol{v}_1 & \sqrt{\lambda_2}\,\boldsymbol{v}_2 \end{pmatrix}$ with $\sqrt{\lambda_1} = \sqrt{6}$ and $\sqrt{\lambda_2} = \sqrt{2}$:

$$X_2 = \begin{pmatrix} \sqrt{6} \cdot 0 & \sqrt{2} \cdot \frac{2}{\sqrt{6}} \\ \sqrt{6} \cdot \frac{1}{\sqrt{2}} & \sqrt{2} \cdot \frac{-1}{\sqrt{6}} \\ \sqrt{6} \cdot \frac{-1}{\sqrt{2}} & \sqrt{2} \cdot \frac{-1}{\sqrt{6}} \end{pmatrix} = \begin{pmatrix} 0 & \frac{2}{\sqrt{3}} \\ \sqrt{3} & -\frac{1}{\sqrt{3}} \\ -\sqrt{3} & -\frac{1}{\sqrt{3}} \end{pmatrix} \in \mathcal{M}_{32}(\mathbb{R})$$

Verification:

- Column sums are zero: $0 + \sqrt{3} - \sqrt{3} = 0$ and $\frac{2}{\sqrt{3}} - \frac{1}{\sqrt{3}} - \frac{1}{\sqrt{3}} = 0$
- Pairwise squared distances between rows $\boldsymbol{x}_1^T, \boldsymbol{x}_2^T, \boldsymbol{x}_3^T$ of X_2:

$$\|\boldsymbol{x}_1 - \boldsymbol{x}_2\|^2 = (0 - \sqrt{3})^2 + \left(\frac{2}{\sqrt{3}} + \frac{1}{\sqrt{3}}\right)^2 = 3 + 3 = 6 = d_{12}^2$$

$$\|\boldsymbol{x}_1 - \boldsymbol{x}_3\|^2 = (0 + \sqrt{3})^2 + \left(\frac{2}{\sqrt{3}} + \frac{1}{\sqrt{3}}\right)^2 = 3 + 3 = 6 = d_{13}^2$$

$$\|\boldsymbol{x}_2 - \boldsymbol{x}_3\|^2 = (\sqrt{3} + \sqrt{3})^2 + 0^2 = (2\sqrt{3})^2 = 12 = d_{23}^2$$

Motivation: from distances back to coordinates

Gram matrices and Euclidean distance matrices

The centring matrix as an orthogonal projector

Classical multidimensional scaling (MDS)

Algebraic duality between PCA and classical MDS

Algebraic duality between PCA and classical MDS

Equivalence via the singular value decomposition

Application in R: reconstructing maps from distances

Two $n \times p$ products: $\widetilde{X}^T\widetilde{X}$ versus $\widetilde{X}\widetilde{X}^T$

Suppose we have a data matrix $X \in \mathcal{M}_{np}(\mathbb{R})$ and its column-centred data matrix $\widetilde{X} = C_n X \in \mathcal{M}_{np}(\mathbb{R})$. From $\widetilde{X}$, we can form two symmetric positive semi-definite matrices:

Feature scatter / covariance (PCA, Unit U05)

$$\widetilde{X}^T\widetilde{X} \in \mathcal{M}_p(\mathbb{R}) \quad \left(\text{or } S_X = \frac{1}{n-1}\widetilde{X}^T\widetilde{X}\right)$$

- Size: $p \times p$ (features $\times$ features)
- (j, ℓ) entry: inner product between centred column j and centred column ℓ
- Diagonalising $\widetilde{X}^T\widetilde{X}$ gives the principal directions (loadings) in $\mathbb{R}^p$

Centred Gram matrix (Classical MDS, Unit U07)

$$B = \widetilde{X}\widetilde{X}^T = -\frac{1}{2}C_n D^{(2)} C_n \in \mathcal{M}_n(\mathbb{R})$$

- Size: $n \times n$ (observations $\times$ observations)
- (i, j) entry: inner product $\tilde{\boldsymbol{x}}_i^T \tilde{\boldsymbol{x}}_j$ between centred observation i and centred observation j
- Diagonalising $\widetilde{X}\widetilde{X}^T$ gives the principal coordinates in $\mathbb{R}^n$

Connecting PCA and classical MDS via the SVD

Let $\widetilde{X} = C_n X \in \mathcal{M}_{np}(\mathbb{R})$ be the column-centred data matrix of rank $r \leq \min(n-1, p) < n$, and write its singular value decomposition (Unit U04):

$$\widetilde{X} = U_r \Sigma_r V_r^T$$

where:

- $U_r = [\boldsymbol{u}_1 \ \cdots \ \boldsymbol{u}_r] \in \mathcal{M}_{nr}(\mathbb{R})$ has orthonormal columns ($U_r^T U_r = \mathbb{I}_r$)
- $\Sigma_r = \text{diag}(\sigma_1, \ldots, \sigma_r) \in \mathcal{M}_r(\mathbb{R})$ contains the positive singular values $\sigma_1 \geq \cdots \geq \sigma_r > 0$
- $V_r = [\boldsymbol{v}_1 \ \cdots \ \boldsymbol{v}_r] \in \mathcal{M}_{pr}(\mathbb{R})$ has orthonormal columns ($V_r^T V_r = \mathbb{I}_r$)

Now express both $\widetilde{X}^T \widetilde{X}$ and $B = \widetilde{X}\widetilde{X}^T$ in terms of the SVD of $\widetilde{X}$:

$$\widetilde{X}^T \widetilde{X} = V_r \Sigma_r^2 V_r^T \in \mathcal{M}_p(\mathbb{R}) \qquad \text{and} \qquad B = \widetilde{X}\widetilde{X}^T = U_r \Sigma_r^2 U_r^T \in \mathcal{M}_n(\mathbb{R})$$

Duality theorem: PCA scores equal classical MDS coordinates

Theorem 106 (Equivalence of PCA and Euclidean classical MDS)

Let $X \in \mathcal{M}_{np}(\mathbb{R})$ be a data matrix with column-centred data matrix $\widetilde{X} = C_n X$ of rank $r \leq \min(n-1, p) < n$ having singular value decomposition $\widetilde{X} = U_r \Sigma_r V_r^T$, and let $D = [\|\boldsymbol{x}_i - \boldsymbol{x}_j\|] \in \mathcal{M}_n(\mathbb{R})$ be the Euclidean distance matrix between the rows of X. Then for any $k \leq r$:

1. *The non-zero eigenvalues of $\widetilde{X}^T\widetilde{X} \in \mathcal{M}_p(\mathbb{R})$ and $B = -\frac{1}{2} C_n D^{(2)} C_n = \widetilde{X}\widetilde{X}^T \in \mathcal{M}_n(\mathbb{R})$ are identical:*

$$\lambda_i = \sigma_i^2 > 0, \qquad i = 1, \ldots, r$$

2. *The k-dimensional principal component score matrix $Y_k = \widetilde{X} V_k \in \mathcal{M}_{nk}(\mathbb{R})$ from PCA and the k-dimensional coordinate matrix $X_k = U_k \Lambda_k^{1/2} \in \mathcal{M}_{nk}(\mathbb{R})$ from classical MDS on D are **identical** (up to sign choices of the eigenvectors):*

$$Y_k = \widetilde{X} V_k = U_k \Sigma_k = U_k \Lambda_k^{1/2} = X_k$$

Proof of Theorem 106

Let $\widetilde{X} = U_r \Sigma_r V_r^T$, where $U_r = [\boldsymbol{u}_1 \ \cdots \ \boldsymbol{u}_r]$, $V_r = [\boldsymbol{v}_1 \ \cdots \ \boldsymbol{v}_r]$ and $\Sigma_r = \text{diag}(\sigma_1, \ldots, \sigma_r)$

1. PCA projection onto top k right singular vectors: The columns of $V_k = [\boldsymbol{v}_1 \ \cdots \ \boldsymbol{v}_k] \in \mathcal{M}_{pk}(\mathbb{R})$ are the top k orthonormal eigenvectors of $\widetilde{X}^T \widetilde{X}$ (and of $S_X = \frac{1}{n-1} \widetilde{X}^T \widetilde{X}$). Using

$$V_r^T V_k = \begin{pmatrix} \mathbb{I}_k \\ \mathbf{0}_{(r-k)\times k} \end{pmatrix}$$

the PCA score matrix is

$$Y_k = \widetilde{X} V_k = (U_r \Sigma_r V_r^T) V_k = U_k \Sigma_k$$

2. Classical MDS on Euclidean distances of X: By Theorem 104, double centring $D^{(2)}$ yields $B = -\frac{1}{2} C_n D^{(2)} C_n = \widetilde{X}\widetilde{X}^T = U_r \Sigma_r^2 U_r^T$. Thus the top k eigenvalues of B are $\lambda_i = \sigma_i^2$ ($\Lambda_k^{1/2} = \Sigma_k$) with orthonormal eigenvectors $U_k = [\boldsymbol{u}_1 \ \cdots \ \boldsymbol{u}_k] \in \mathcal{M}_{nk}(\mathbb{R})$. The classical MDS coordinate matrix is therefore

$$X_k = U_k \Lambda_k^{1/2} = U_k \Sigma_k = Y_k \quad \square$$

Computational and practical payoff of PCA–MDS duality

Why is Theorem 106 so useful in practice?

1. **Tall-and-skinny data ($n \gg p$, many observations, few features)**:
   - $\widetilde{X}^T\widetilde{X}$ is a small $p \times p$ matrix, whereas $\widetilde{X}\widetilde{X}^T$ is a huge $n \times n$ matrix
   - Computing PCA via $\widetilde{X}^T\widetilde{X}$ costs $O(np^2 + p^3)$, which is much faster than $O(n^3)$ for $\widetilde{X}\widetilde{X}^T$
2. **Short-and-wide data ($p \gg n$, few observations, huge number of features)**:
   - E.g. gene expression ($n = 50$ patients, $p = 20{,}000$ genes) or high-resolution images ($n = 100$ faces, $p = 10^6$ pixels)
   - $\widetilde{X}^T\widetilde{X}$ is $20{,}000 \times 20{,}000$, whereas the Gram matrix $B = \widetilde{X}\widetilde{X}^T$ is only 50×50!
   - Diagonalising the $n \times n$ Gram matrix $B = \widetilde{X}\widetilde{X}^T$ (classical MDS) gives the exact PCA scores in $O(n^2p + n^3)$ time!
3. **Coordinate-free and kernel extensions**:
   - Classical MDS works when X is unknown and only distances D are given
   - Replacing $\boldsymbol{x}_i^T\boldsymbol{x}_j$ by a general similarity kernel $k(\boldsymbol{x}_i, \boldsymbol{x}_j)$ leads directly to **kernel PCA**!

Algebraic duality between PCA and classical MDS

Equivalence via the singular value decomposition

Application in R: reconstructing maps from distances

Verifying PCA and classical MDS equivalence in R

Let us first verify Theorem 106 numerically in R on a synthetic 5×3 centred data matrix $\widetilde{X}$:

```
set.seed(2740)
X_tilde = scale(matrix(rnorm(15), nrow = 5, ncol = 3), center = TRUE, scale
pca_scores = prcomp(X_tilde, center = FALSE)$x[, 1:2]
mds_coords = cmdscale(dist(X_tilde), k = 2)
# Compare absolute values (eigenvectors are defined up to a sign +/- 1)
max(abs(abs(pca_scores) - abs(mds_coords)))

## [1] 2.442491e-15
```

Up to machine precision ($\approx 10^{-16}$) and column signs, the PCA scores `prcomp(X_tilde)$x[,1:2]` and the classical MDS coordinates `cmdscale(dist(X_tilde), k=2)` are identical!

Geographical example: Canadian cities distance matrix

Now suppose we are only given approximate great-circle distances (in kilometres) between $n = 8$ Canadian cities, with **no latitude or longitude coordinates**:

```
cities = c("Victoria", "Vancouver", "Calgary", "Regina",
           "Winnipeg", "Toronto", "Montreal", "Halifax")
D_can = matrix(c(
     0,   90,  725, 1215, 1690, 3390, 3760, 4490,
    90,    0,  675, 1165, 1640, 3355, 3685, 4430,
   725,  675,    0,  505,  990, 2705, 3020, 3770,
  1215, 1165,  505,    0,  500, 2030, 2360, 3125,
  1690, 1640,  990,  500,    0, 1515, 1825, 2590,
  3390, 3355, 2705, 2030, 1515,    0,  505, 1265,
  3760, 3685, 3020, 2360, 1825,  505,    0,  790,
  4490, 4430, 3770, 3125, 2590, 1265,  790,    0
), nrow = 8, byrow = TRUE, dimnames = list(cities, cities))
```

Step-by-step manual classical MDS vs `cmdscale`

Let us implement the classical MDS algorithm directly from $B = -\frac{1}{2} C_n D^{(2)} C_n$ and compare with R's built-in `cmdscale` function:

```
n = nrow(D_can)
C_n = diag(n) - matrix(1, n, n) / n
B = -0.5 * C_n %*% (D_can^2) %*% C_n
ev = eigen(B, symmetric = TRUE)
round(ev$values, 1)
```

```
## [1] 20985217.2   224941.6    67722.5    17544.1     3459.2        0.0    -
## [8]  -482729.4
```

```
X_manual = ev$vectors[, 1:2] %*% diag(sqrt(ev$values[1:2]))
rownames(X_manual) = cities
mds_builtin = cmdscale(as.dist(D_can), k = 2, eig = TRUE)
max(abs(abs(X_manual) - abs(mds_builtin$points)))
```

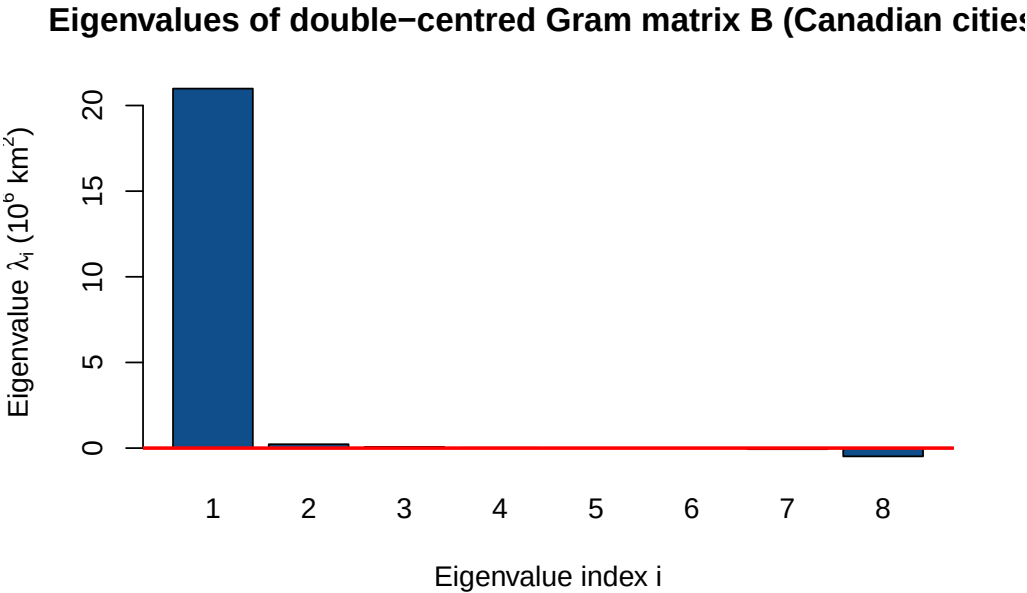
```
## [1] 0
```

Eigenvalue spectrum and goodness of fit

Notice how the first two eigenvalues dominate the spectrum of B:

```
gof_top2 = sum(ev$values[1:2]) / sum(ev$values[ev$values > 0])
gof_1d   = ev$values[1] / sum(ev$values[ev$values > 0])
c(GOF_1D = round(gof_1d, 4), GOF_2D = round(gof_top2, 4))

## GOF_1D GOF_2D
## 0.9853 0.9958
```

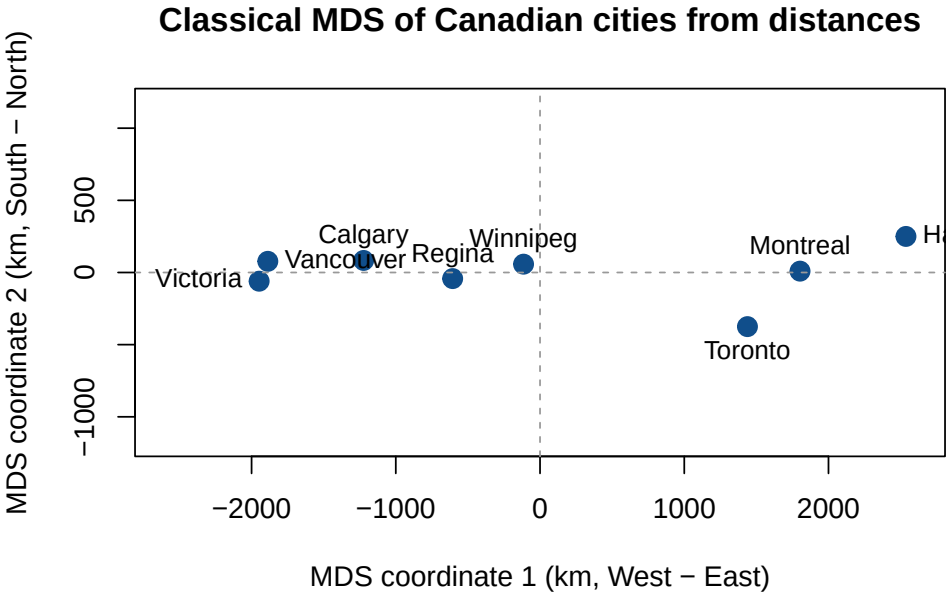
The first dimension alone (East–West axis across Canada) explains 98.5% of the positive trace, and 2 dimensions explain 99.58%!


Eigenvalues of double−centred Gram matrix B (Canadian cities
Eigenvalue λ_i (10^6 km^2)
0
5
10
15
20
1
2
3
4
5
6
7
8
Eigenvalue index i

Plotting the reconstructed 2D map of Canadian cities

Recall that classical MDS only determines coordinates up to orthogonal transformations (rotations and reflections). We plot the raw MDS coordinates and then flip signs if needed so West is left and North is up:

```
coords = X_manual
# Ensure West < 0 and Winnipeg is North of Toronto
if (coords["Vancouver", 1] > 0) coords[, 1] = -coords[, 1]
if (coords["Winnipeg", 2] < coords["Toronto", 2]) coords[, 2] = -coords[, 2]
plot(coords[, 1], coords[, 2], pch = 19, cex = 1.5, col = "dodgerblue4",
     asp = 1, xlim = c(-2600, 2600), ylim = c(-900, 900),
     xlab = "MDS coordinate 1 (km, West - East)",
     ylab = "MDS coordinate 2 (km, South - North)",
     main = "Classical MDS of Canadian cities from distances")
abline(h = 0, v = 0, lty = 2, col = "grey60")
```

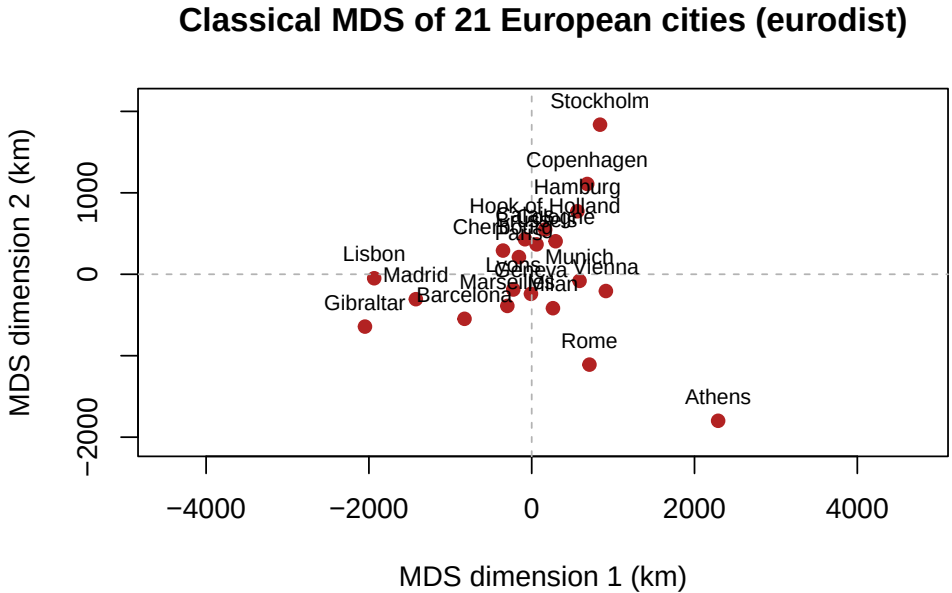

Classical MDS of Canadian cities from distances
MDS coordinate 2 (km, South − North)
500
0
−1000
Calgary
Winnipeg
Vancouver
Regina
Victoria
Montreal
Toronto
−2000
−1000
0
1000
2000
MDS coordinate 1 (km, West − East)

Classical example: European road distances (`eurodist`)

R's built-in `datasets` package includes `eurodist`, a table of road distances (in km) between 21 European cities:

```
D_euro = as.matrix(eurodist)
mds_euro = cmdscale(eurodist, k = 2, eig = TRUE)
x_eu = mds_euro$points[, 1]
y_eu = -mds_euro$points[, 2] # Flip North-South axis
plot(x_eu, y_eu, pch = 19, col = "firebrick", asp = 1,
     xlim = range(x_eu) * 1.15, ylim = range(y_eu) * 1.15,
     xlab = "MDS dimension 1 (km)", ylab = "MDS dimension 2 (km)",
     main = "Classical MDS of 21 European cities (eurodist)")
abline(h = 0, v = 0, lty = 2, col = "grey70")
```

Even though road distances include detours around mountains and seas (so B has small negative eigenvalues), the top 2 eigenvectors of $-\frac{1}{2}C_{21}D^{(2)}C_{21}$ faithfully reconstruct the map of Europe!


Classical MDS of 21 European cities (eurodist)
Stockholm
Copenhagen
Hamburg
Hook of Holland
Lisbon
Madrid
Gibraltar
Barcelona
Munich
Vienna
Rome
Athens
MDS dimension 2 (km)
1000
0
−2000
−4000
−2000
0
2000
4000
MDS dimension 1 (km)

Summary of Unit U07

- **Gram matrix**: for $X \in \mathcal{M}_{np}(\mathbb{R})$, $G = XX^T \in \mathcal{M}_n(\mathbb{R})$ has entries $g_{ij} = \boldsymbol{x}_i^T \boldsymbol{x}_j$, is symmetric positive semi-definite and satisfies $\operatorname{rank}(G) = \operatorname{rank}(X)$
- **From Gram to squared distances**: with $\boldsymbol{g} = \operatorname{diag}(G)$,

$$D^{(2)} = \boldsymbol{g}\mathbb{1}^T - 2G + \mathbb{1}\boldsymbol{g}^T$$

- **Centring matrix**: $C_n = \mathbb{I}_n - \frac{1}{n}\mathbb{1}\mathbb{1}^T$ is the symmetric, idempotent orthogonal projection matrix onto $\operatorname{span}(\mathbb{1})^\perp$ with $\operatorname{rank}(C_n) = \operatorname{tr}(C_n) = n - 1$ and $C_n\mathbb{1} = \mathbf{0}$
- **Double centring and classical MDS**: $B = -\frac{1}{2}C_n D^{(2)} C_n = \widetilde{X}\widetilde{X}^T$ recovers the centred Gram matrix from distances alone, and $X_k = V_k \Lambda_k^{1/2}$ recovers k-dimensional coordinates
- **PCA–MDS duality**: via the SVD $\widetilde{X} = U\Sigma V^T$, classical MDS on Euclidean distances of X produces the exact same low-dimensional coordinates $U_k\Sigma_k$ as PCA on X

Non-negative matrices, digraphs and regular Markov chains

MATH 2740 – Mathematics of Data Science – Unit U08

Julien Arino
julien.arino@umanitoba.ca

Department of Mathematics @ University of Manitoba

Fall 2026

The University of Manitoba campuses are located on original lands of Anishinaabeg, Ininew, Anisininew, Dakota and Dene peoples, and on the National Homeland of the Red River Métis. We respect the Treaties that were made on these territories, we acknowledge the harms and mistakes of the past, and we dedicate ourselves to move forward in partnership with Indigenous communities in a spirit of Reconciliation and collaboration.

Outline

Non-negative matrices and their digraphs

Matrix powers, walks and connectivity

The Perron-Frobenius theorem

Regular discrete-time Markov chains

Non-negative matrices and their digraphs

Matrix powers, walks and connectivity

The Perron-Frobenius theorem

Regular discrete-time Markov chains

Why study non-negative matrices and digraphs together?

Many matrices in data science have entries that represent quantities that cannot be negative:

- counts, weights, distances or similarity scores
- transition probabilities between states in a stochastic process
- hyperlinks between web pages or flows in a network

Such matrices have a rich geometric and spectral theory (Perron-Frobenius theory)

Their zero/non-zero sparsity pattern is naturally encoded by a **directed graph** (**digraph**), turning algebraic questions about matrix powers into visual questions about walks on a graph

Non-negative and positive vectors and matrices

Definition 107 (Non-negative and positive matrices)

Let $A = [a_{ij}] \in \mathcal{M}_{np}(\mathbb{R})$ be a real matrix

- *A is **non-negative**, written $A \geq 0$ (or $A \in \mathcal{M}_{np}(\mathbb{R}_{\geq 0})$), if*

$$a_{ij} \geq 0 \quad \text{for all } i = 1, \ldots, n \text{ and } j = 1, \ldots, p$$

- *A is **positive** (or **strictly positive**), written $A > 0$, if*

$$a_{ij} > 0 \quad \text{for all } i = 1, \ldots, n \text{ and } j = 1, \ldots, p$$

The same notation applies to vectors $\boldsymbol{x} = (x_1, \ldots, x_n)^T \in \mathbb{R}^n$:

- $\boldsymbol{x} \geq \mathbf{0}$ means $x_i \geq 0$ for all $i = 1, \ldots, n$
- $\boldsymbol{x} > \mathbf{0}$ means $x_i > 0$ for all $i = 1, \ldots, n$

Componentwise inequalities and basic properties

For two matrices $A, B \in \mathcal{M}_{np}(\mathbb{R})$, we write $A \geq B$ if $A - B \geq 0$, and $A > B$ if $A - B > 0$

Proposition 108 (Arithmetic of non-negative matrices)

Let $A, B \in \mathcal{M}_n(\mathbb{R}_{\geq 0})$ and $\boldsymbol{x} \in \mathbb{R}^n_{\geq 0}$

1. *$A + B \geq 0$ and $AB \geq 0$*
2. *For any integer $k \geq 1$, $A^k \geq 0$*
3. *If $A > 0$ and $\boldsymbol{x} \geq \mathbf{0}$ with $\boldsymbol{x} \neq \mathbf{0}$, then $A\boldsymbol{x} > \mathbf{0}$*
4. *If $A \geq 0$ and $\boldsymbol{x} > \mathbf{0}$, then $A\boldsymbol{x} > \mathbf{0}$ if and only if A has no zero row*

Why does property 3 hold? Because $\boldsymbol{x} \neq \mathbf{0}$ has at least one entry $x_{j_0} > 0$, so for every row i,

$$(A\boldsymbol{x})_i = \sum_{j=1}^{n} a_{ij} x_j \geq a_{i,j_0} x_{j_0} > 0$$

Directed graphs (digraphs)

Definition 109 (Directed graph)

*A **directed graph** (or **digraph**) is a pair $G = (V, A)$ where:*

- $V = \{1, 2, \ldots, n\}$ *is a finite set of elements called **vertices** (or **nodes**)*
- $A \subseteq V \times V$ *is a set of ordered pairs of vertices called **arcs** (or **directed edges**)*

Definition 110 (Arc, tail, head and loop)

An **arc** $e = (u, v) \in A$ represents a directed connection from vertex u to vertex v

- u is the **initial endpoint** (or **tail**) and v is the **terminal endpoint** (or **head**)
- u is a **predecessor** of v and v is a **successor** of u
- An arc of the form $(u, u) \in A$ from a vertex to itself is called a **loop** (or **self-loop**)

Indegree, outdegree, sources and sinks

Let $G = (V, A)$ be a digraph and $v \in V$

Definition 111 (Outdegree and indegree)

- The **outdegree** $d_G^+(v)$ is the number of arcs directed away from v:

$$d_G^+(v) = |\{w \in V : (v, w) \in A\}|$$

- The **indegree** $d_G^-(v)$ is the number of arcs directed towards v:

$$d_G^-(v) = |\{u \in V : (u, v) \in A\}|$$

- A vertex with $d_G^-(v) = 0$ is a **source**; a vertex with $d_G^+(v) = 0$ is a **sink**

Since every arc has one tail and one head,

$$\sum_{v \in V} d_G^+(v) = \sum_{v \in V} d_G^-(v) = |A|$$

The digraph associated with a non-negative matrix

Definition 112 (Digraph of a matrix and adjacency matrix)

- *Given a square non-negative matrix* $M = [m_{ij}] \in \mathcal{M}_n(\mathbb{R}_{\geq 0})$*, its associated* ***digraph*** *is* $\mathcal{G}(M) = (V, A)$ *with vertex set* $V = \{1, \ldots, n\}$ *and arc set*

$$A = \{(i, j) \in V \times V : m_{ij} > 0\}$$

  If we attach weight m_{ij} *to each arc* $(i, j) \in A$*, we obtain a* ***weighted digraph*** *(or* ***directed network****)*

- *Conversely, given a digraph* $G = (V, A)$ *on* $V = \{1, \ldots, n\}$*, its (binary)* ***adjacency matrix*** *is* $A(G) = [a_{ij}] \in \mathcal{M}_n(\mathbb{R}_{\geq 0})$ *defined by*

$$a_{ij} = \begin{cases} 1 & \text{if } (i, j) \in A \\ 0 & \text{otherwise} \end{cases}$$

Note that diagonal entries $m_{ii} > 0$ correspond to self-loops (i, i) at vertex i

Orientation convention: general matrices vs Markov chains

There are two natural conventions for drawing arcs from a matrix $M = [m_{ij}]$:

1. **Row-to-column (adjacency) convention**: arc (i, j) goes from i to j when $m_{ij} > 0$
   - Row i lists outgoing arcs from i; column j lists incoming arcs into j

2. **Column-to-row (transition) convention**: in column-stochastic Markov chains, $p_{ij} = \mathbb{P}(S_i(t+1) \mid S_j(t))$ is the probability of moving **from** j **to** i, so we draw an arc from j to i when $p_{ij} > 0$
   - Column j lists outgoing transitions from state j (summing to 1); row i lists incoming transitions into state i
   - This transition digraph is simply $\mathcal{G}(P^T)$, obtained by reversing all arrows of $\mathcal{G}(P)$

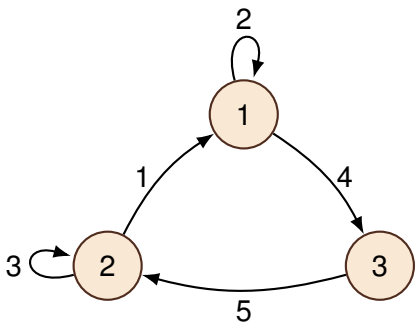
Crucial fact: Reversing all arrows preserves closed walks, cycle lengths and strong connectedness, so irreducibility and primitivity of M and M^T are identical

Example: 3×3 non-negative matrix and its digraph $\mathcal{G}(M)$

Consider the 3×3 non-negative matrix M and its binary sparsity pattern $A(\mathcal{G}(M))$:

$$M = \begin{pmatrix} 2 & 0 & 4 \\ 1 & 3 & 0 \\ 0 & 5 & 0 \end{pmatrix}, \qquad A(\mathcal{G}(M)) = \begin{pmatrix} 1 & 0 & 1 \\ 1 & 1 & 0 \\ 0 & 1 & 0 \end{pmatrix}$$

Weighted digraph $\mathcal{G}(M)$

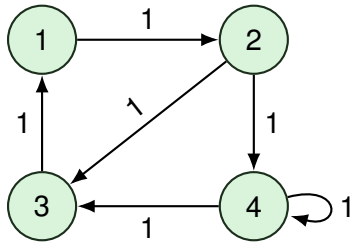


Reading $\mathcal{G}(M)$ directly from M:

- $m_{11} = 2 > 0 \implies$ self-loop $(1, 1)$
- $m_{13} = 4 > 0 \implies$ arc $(1, 3)$
- $m_{21} = 1 > 0 \implies$ arc $(2, 1)$
- $m_{22} = 3 > 0 \implies$ self-loop $(2, 2)$
- $m_{32} = 5 > 0 \implies$ arc $(3, 2)$
- All other entries are 0 (no other arcs)

Example: 4×4 sparse matrix and its digraph $\mathcal{G}(M)$

$$M = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 \end{pmatrix}$$



- Row sums of $A(\mathcal{G}(M))$ give the outdegrees:

$$d^+ = (1, 2, 1, 2)^T$$

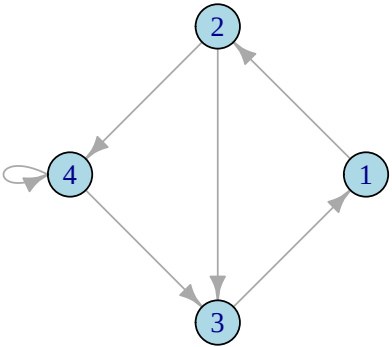
- Column sums of $A(\mathcal{G}(M))$ give the indegrees:

$$d^- = (1, 1, 2, 2)^T$$

Building a digraph from a matrix in igraph

```
M <- matrix(c(0, 1, 0, 0,
              0, 0, 1, 1,
              1, 0, 0, 0,
              0, 0, 1, 1), nrow = 4, byrow = TRUE)
G <- graph_from_adjacency_matrix(M, mode = "directed")
degree(G, mode = "out")

## [1] 1 2 1 2

degree(G, mode = "in")

## [1] 1 1 2 2

plot(G, edge.arrow.size = 0.7, vertex.color = "lightblue",
     vertex.size = 30, main = "Digraph G(M) from adjacency matrix M")
```

Digraph G(M) from adjacency matrix M



Non-negative matrices and their digraphs

Matrix powers, walks and connectivity

The Perron-Frobenius theorem

Regular discrete-time Markov chains

Matrix powers, walks and connectivity

Walks and powers of the adjacency matrix

Irreducibility and strong connectedness

Primitivity and aperiodicity

Walks, trails, paths and cycles in a digraph

Let $G = (V, A)$ be a digraph

Definition 113 (Directed walk, length and closed walk)

- *A **directed walk** from vertex v_0 to vertex v_k in G is a sequence of vertices $v_0, v_1, \ldots, v_k$ such that $(v_{r-1}, v_r) \in A$ is an arc in G for all $r = 1, \ldots, k$*
- *The **length** of the directed walk is k, the number of arcs traversed*
- *The walk is **closed** if its initial and terminal vertices coincide ($v_0 = v_k$)*

Definition 114 (Directed trail, path and cycle)

- A **directed trail** is a directed walk in which all arcs are distinct
- A **directed path** is a directed walk in which all vertices $v_0, v_1, \ldots, v_k$ are distinct
- A **directed cycle** (of length $k \geq 1$) is a closed walk $v_0, v_1, \ldots, v_k = v_0$ in which $v_0, v_1, \ldots, v_{k-1}$ are distinct (a cycle of length 1 is a self-loop)

Powers of a matrix and walks in its digraph

Theorem 115 (Matrix powers count walks)

Let $A = [a_{ij}] \in \mathcal{M}_n(\mathbb{R}_{\geq 0})$ and denote the (i,j) entry of A^k by $a_{ij}^{(k)}$ for $k \in \mathbb{N}_+$. Then

$$a_{ij}^{(k)} = \sum_{r_1=1}^{n} \sum_{r_2=1}^{n} \cdots \sum_{r_{k-1}=1}^{n} a_{i,r_1} a_{r_1,r_2} \cdots a_{r_{k-1},j}$$

In particular:

1. $a_{ij}^{(k)} > 0$ *if and only if there is at least one directed walk of length k from i to j in $\mathcal{G}(A)$*
2. *If $A \in \{0,1\}^{n \times n}$ is a binary adjacency matrix, then $a_{ij}^{(k)}$ equals the exact number of directed walks of length k from vertex i to vertex j in $\mathcal{G}(A)$*

Proof of Theorem 115

We prove the formula by mathematical induction on the walk length $k \geq 1$

Base case ($k = 1$): $A^1 = A$, and $a_{ij}^{(1)} = a_{ij}$ is the weight of the single arc (i, j) (a walk of length 1 from i to j), which equals 1 if (i, j) is an arc in the binary case and 0 otherwise

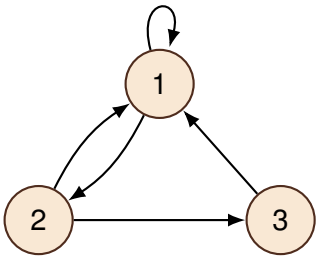
Inductive step: Suppose the result holds for $k \geq 1$. Since $A^{k+1} = A^k A$, the definition of matrix multiplication gives

$$a_{ij}^{(k+1)} = \sum_{r=1}^{n} a_{ir}^{(k)} a_{rj}$$

In the binary case $A \in \{0, 1\}^{n \times n}$, any walk of length $k + 1$ from i to j decomposes uniquely into a walk of length k from i to some penultimate vertex r, followed by the arc (r, j). For a fixed r, there are $a_{ir}^{(k)} a_{rj}$ such walks. Summing over all $r \in \{1, \ldots, n\}$ gives $a_{ij}^{(k+1)}$, completing the induction □

Example: counting walks of length 2 and 3

Consider the binary adjacency matrix A and its digraph $\mathcal{G}(A)$:



$$A = \begin{pmatrix} 1 & 1 & 0 \\ 1 & 0 & 1 \\ 1 & 0 & 0 \end{pmatrix}, \quad A^2 = \begin{pmatrix} 2 & 1 & 1 \\ 2 & 1 & 0 \\ 1 & 1 & 0 \end{pmatrix}$$

$$A^3 = A^2 A = \begin{pmatrix} 4 & 2 & 1 \\ 3 & 2 & 1 \\ 2 & 1 & 1 \end{pmatrix}$$

Let us verify individual entries directly from $\mathcal{G}(A)$:

- $(A^2)_{11} = 2$: the two walks of length 2 from 1 to 1 are $1 \to 1 \to 1$ and $1 \to 2 \to 1$
- $(A^2)_{23} = 0$: from 2, length-1 steps reach 1 or 3, neither of which has an arc to 3
- $(A^3)_{11} = 4$: the four walks of length 3 from 1 to 1 are

$$1 \to 1 \to 1 \to 1, \quad 1 \to 1 \to 2 \to 1, \quad 1 \to 2 \to 1 \to 1, \quad 1 \to 2 \to 3 \to 1$$

Matrix powers, walks and connectivity

Walks and powers of the adjacency matrix

Irreducibility and strong connectedness

Primitivity and aperiodicity

Strong connectedness and strongly connected components

Definition 116 (Strongly connected digraph)

*A digraph $G = (V, A)$ is **strongly connected** if for every ordered pair of vertices $(i, j) \in V \times V$ with $i \neq j$, there exists a directed walk (and hence a directed path) in G from i to j and from j to i*

Define the mutual reachability relation $i \equiv j$ on V by:

$$i \equiv j \iff i = j \text{ or there is a directed walk from } i \text{ to } j \text{ and from } j \text{ to } i$$

- $\equiv$ is reflexive, symmetric and transitive, so it is an **equivalence relation** on V
- Its equivalence classes partition V into maximal strongly connected subdigraphs called the **strongly connected components** (SCCs) of G
- G is strongly connected if and only if it has a single strongly connected component

Reducible and irreducible matrices

Definition 117 (Reducible and irreducible matrices)

A matrix $A \in \mathcal{M}_n(\mathbb{R}_{\geq 0})$ ($n \geq 2$) is ***reducible*** *if there exists a permutation matrix $P_\pi \in \mathcal{M}_n$ such that*

$$P_\pi^T A P_\pi = \begin{pmatrix} A_{11} & A_{12} \\ \mathbf{0} & A_{22} \end{pmatrix}$$

where $A_{11} \in \mathcal{M}_r$ and $A_{22} \in \mathcal{M}_{n-r}$ are square blocks with $1 \leq r < n$, and $\mathbf{0}$ is the $(n-r) \times r$ zero matrix

The matrix A is ***irreducible*** *if no such permutation matrix exists (by convention, any 1×1 matrix is irreducible)*

Note that $P_\pi^T A P_\pi$ simply relabels the rows and columns of A using the same permutation π of the vertices $\{1, \ldots, n\}$

Why block upper-triangular form prevents strong connectedness

Suppose A has been relabelled into the reducible block form

$$\tilde{A} = P_\pi^T A P_\pi = \begin{pmatrix} A_{11} & A_{12} \\ \mathbf{0} & A_{22} \end{pmatrix} \begin{matrix} \} \ V_1 = \{1, \ldots, r\} \\ \} \ V_2 = \{r+1, \ldots, n\} \end{matrix}$$

- The bottom-left block is $\mathbf{0}$, which means $\tilde{a}_{ij} = 0$ for all $i \in V_2$ and $j \in V_1$: there is **no arc** from any vertex in V_2 to any vertex in V_1
- Furthermore, for every power $k \geq 1$, block multiplication yields

$$\tilde{A}^k = \begin{pmatrix} A_{11}^k & * \\ \mathbf{0} & A_{22}^k \end{pmatrix}$$

  so $(\tilde{A}^k)_{ij} = 0$ for all $i \in V_2$ and $j \in V_1$
- Thus no walk of any length $k \geq 1$ can ever go from V_2 to V_1: the vertex set V_2 is a **closed trap** under the row-to-column walk dynamics

Characterisations of irreducibility

Theorem 118 (Characterisations of irreducible matrices)

For a non-negative matrix $A \in \mathcal{M}_n(\mathbb{R}_{\geq 0})$, the following statements are equivalent:

1. *A is irreducible*
2. *The associated digraph $\mathcal{G}(A)$ is strongly connected*
3. $\mathbb{I}_n + A + A^2 + \cdots + A^{n-1} > 0$ *(strictly positive entrywise)*
4. $(\mathbb{I}_n + A)^{n-1} > 0$ *(strictly positive entrywise)*

Items 3 and 4 provide a purely algebraic test for irreducibility without searching over all $n!$ permutation matrices

Proof of Theorem 118

(1) $\iff$ **(2)**: A is reducible $\iff$ vertices can be partitioned into non-empty sets V_1, V_2 with no arc from V_2 to V_1 $\iff$ no walk can go from V_2 to V_1 $\iff$ $\mathcal{G}(A)$ is not strongly connected

(2) $\iff$ **(3)**: By Theorem 115, $(A^k)_{ij} > 0$ $\iff$ there is a walk of length k from i to j. Any shortest walk between two distinct vertices $i \neq j$ is a simple path visiting at most n vertices, so its length is $k \in \{1, \ldots, n-1\}$. With $(\mathbb{I}_n)_{ii} = 1 > 0$ covering $i = j$, $\mathcal{G}(A)$ is strongly connected $\iff$ for every pair (i, j) there is some $k \in \{0, 1, \ldots, n-1\}$ with $(A^k)_{ij} > 0$ $\iff$ $\sum_{k=0}^{n-1} A^k > 0$

(3) $\iff$ **(4)**: By the binomial expansion with positive coefficients $\binom{n-1}{k} > 0$ and $A^k \geq 0$,

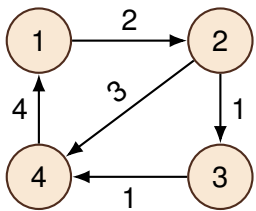
$$(\mathbb{I}_n + A)^{n-1} = \sum_{k=0}^{n-1} \binom{n-1}{k} A^k$$

A sum of non-negative terms with positive weights is strictly positive if and only if at least one term is strictly positive, so $(\mathbb{I}_n + A)^{n-1} > 0$ $\iff$ $\sum_{k=0}^{n-1} A^k > 0$ □

Exam-style example 1: testing irreducibility via $\mathcal{G}(A)$

Problem: Determine whether the following 4×4 matrix is irreducible:

$$A = \begin{pmatrix} 0 & 2 & 0 & 0 \\ 0 & 0 & 1 & 3 \\ 0 & 0 & 0 & 1 \\ 4 & 0 & 0 & 0 \end{pmatrix}$$



Solution:

- Draw the 4-node digraph $\mathcal{G}(A)$
- Observe the directed Hamiltonian cycle

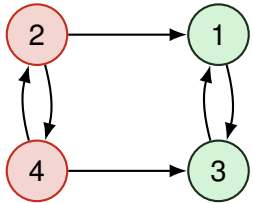
$$1 \to 2 \to 3 \to 4 \to 1$$

- Since this cycle visits all 4 vertices, every vertex can reach every other vertex along the cycle
- Therefore $\mathcal{G}(A)$ is **strongly connected**, so A is **irreducible**

Exam-style example 2: reducible matrix and block triangular form

Problem: Show that B is reducible, find its SCCs and write $P_\pi^T B P_\pi$ in block upper-triangular form:

$$B = \begin{pmatrix} 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \end{pmatrix}$$



- SCCs are $C_1 = \{2, 4\}$ and $C_2 = \{1, 3\}$
- Arcs go from C_1 to C_2, but **no arc goes from C_2 to C_1**, so $\mathcal{G}(B)$ is not strongly connected (B is **reducible**)
- Ordering vertices as $(2, 4, 1, 3)$ gives

$$P_\pi^T B P_\pi = \left(\begin{array}{cc|cc} 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ \hline 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{array}\right) = \begin{pmatrix} B_{11} & B_{12} \\ \mathbf{0} & B_{22} \end{pmatrix}$$

Checking strong connectedness and $(\mathbb{I}_n + A)^{n-1}$ in R

```
A <- matrix(c(0, 2, 0, 0,
              0, 0, 1, 3,
              0, 0, 0, 1,
              4, 0, 0, 0), nrow = 4, byrow = TRUE)
is_connected(graph_from_adjacency_matrix(A > 0), mode = "strong")

## [1] TRUE

# Compute (I_4 + A)^3
I4_plus_A <- diag(4) + A
I4_plus_A %*% I4_plus_A %*% I4_plus_A

##      [,1] [,2] [,3] [,4]
## [1,]   25    6    6   20
## [2,]   40   25    3   12
## [3,]   12    8    1    3
## [4,]   12   24    8   25
```

Matrix powers, walks and connectivity

- Walks and powers of the adjacency matrix
- Irreducibility and strong connectedness
- **Primitivity and aperiodicity**

Primitive matrices

Definition 119 (Primitive matrix)

A non-negative matrix $A \in \mathcal{M}_n(\mathbb{R}_{\geq 0})$ is ***primitive*** *if there exists a positive integer $m \in \mathbb{N}_+$ such that*

$$A^m > 0 \quad (\textit{every entry of } A^m \textit{ is strictly positive})$$

The smallest positive integer m such that $A^m > 0$ is called the ***index of primitivity*** *of A, denoted $\gamma(A)$*

An irreducible non-negative matrix that is not primitive is called ***imprimitive*** *(or* ***periodic****)*

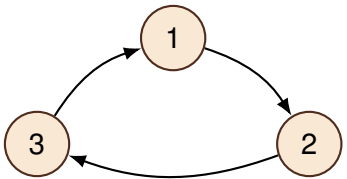
Proposition 120

1. *Every primitive matrix is irreducible*
2. *If $A^m > 0$ for some $m \in \mathbb{N}_+$, then $A^k > 0$ for all $k \geq m$*

Irreducible does not imply primitive!

Consider the 2×2 and 3×3 cyclic permutation matrices:

$$C_2 = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \qquad C_3 = \begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{pmatrix}$$



- Both $\mathcal{G}(C_2)$ and $\mathcal{G}(C_3)$ are single directed cycles, so C_2 and C_3 are **irreducible**
- Yet for C_2: $C_2^{2k} = \mathbb{I}_2$ and $C_2^{2k+1} = C_2$
- And for C_3: $C_3^2 = \begin{pmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix}$, $C_3^3 = \mathbb{I}_3$, $C_3^4 = C_3, \ldots$
- Every power C_2^m and C_3^m contains zeros, so neither matrix is primitive!

Why? Because walks from i to j only occur at lengths m belonging to a fixed congruence class modulo 2 (for C_2) or modulo 3 (for C_3)

Period of a digraph and characterisation of primitivity

Definition 121 (Period and aperiodicity)

*Let $G = (V, A)$ be a strongly connected digraph. The **period** (or **index of imprimitivity**) $d(G)$ of G is the greatest common divisor of the lengths of all directed cycles (or closed walks) in G:*

$$d(G) = \gcd\{\mathit{length}(C) : C \text{ is a directed cycle in } G\}$$

- *If $d(G) = 1$, we say that G (and its matrix A) is **aperiodic***
- *If $d(G) > 1$, we say that G (and its matrix A) is **periodic** with period $d(G)$*

Theorem 122 (Graphical characterisation of primitive matrices)

*A non-negative matrix $A \in \mathcal{M}_n(\mathbb{R}_{\geq 0})$ is **primitive** if and only if $\mathcal{G}(A)$ is **strongly connected** and **aperiodic** ($d(\mathcal{G}(A)) = 1$)*

Practical corollaries for exams

Checking gcd of cycle lengths is immediate in two common situations:

Corollary 123 (Self-loop test for primitivity)

*If $A \in \mathcal{M}_n(\mathbb{R}_{\geq 0})$ is irreducible ($\mathcal{G}(A)$ is strongly connected) and has **at least one positive diagonal entry** $a_{ii} > 0$ for some $i \in \{1, \ldots, n\}$, then A is **primitive***

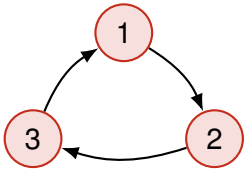
Reason: $a_{ii} > 0$ gives a self-loop $i \to i$ of length 1 in $\mathcal{G}(A)$, and $\gcd(1, \ldots) = 1$

Corollary 124 (Coprime cycle lengths)

*If $\mathcal{G}(A)$ is strongly connected and contains two directed cycles of coprime lengths ℓ_1 and ℓ_2 (i.e. $\gcd(\ell_1, \ell_2) = 1$, such as a 2-cycle and a 3-cycle), then A is **primitive***

Visualising periodic vs primitive digraphs

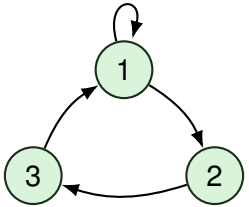
Case 1: Irreducible, periodic ($d = 3$)



$$A_1 = \begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{pmatrix}$$

Cycle length: $\{3\}$
$\gcd(3) = 3 \implies$ **not primitive**

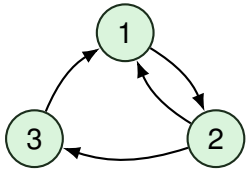
Case 2: Adding one self-loop ($d = 1$)



$$A_2 = \begin{pmatrix} 1 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{pmatrix}$$

Cycle lengths: $\{1, 3\}$
$\gcd(1, 3) = 1 \implies$ **primitive** ($A_2^4 > 0$)

Case 3: Zero trace, coprime cycles ($d = 1$)



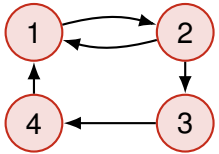
$$A_3 = \begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 1 & 0 & 0 \end{pmatrix}$$

Cycles: $1 \leftrightarrow 2$ (2),
$1 \to 2 \to 3 \to 1$ (3)
$\gcd(2, 3) = 1 \implies$ **primitive** ($A_3^5 > 0$)

Exam-style comparison: two 4×4 irreducible matrices

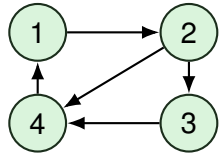
Both matrices below are irreducible with zero diagonal ($\mathrm{tr} = 0$). Which one is primitive?

$$M_1 = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \end{pmatrix}$$



- Directed cycles: $1 \to 2 \to 1$ (length 2) and $1 \to 2 \to 3 \to 4 \to 1$ (length 4)
- $d = \gcd(2, 4) = 2 > 1$:

$$M_2 = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \end{pmatrix}$$



- Directed cycles: $1 \to 2 \to 4 \to 1$ (length 3) and $1 \to 2 \to 3 \to 4 \to 1$ (length 4)
- $d = \gcd(3, 4) = 1$: **primitive**

Wielandt's bound on the index of primitivity

How large can the smallest exponent $\gamma(A)$ with $A^{\gamma(A)} > 0$ be for an $n \times n$ primitive matrix?

Theorem 125 (Wielandt's theorem)

Let $A \in \mathcal{M}_n(\mathbb{R}_{\geq 0})$ with $n \geq 2$. Then A is primitive if and only if

$$A^{n^2-2n+2} > 0$$

Moreover, if A is irreducible and has $d \geq 1$ positive diagonal entries, then

$$\gamma(A) \leq 2n - d - 1$$

- For $n = 3$, $n^2 - 2n + 2 = 5$: the matrix A_3 has $A_3^4 \not> 0$ and $A_3^5 > 0$
- For $n = 4$, $n^2 - 2n + 2 = 10$: the matrix M_2 on the previous slide has $M_2^9 \not> 0$ and $M_2^{10} > 0$, showing that Wielandt's bound $n^2 - 2n + 2$ is sharp!

Non-negative matrices and their digraphs

Matrix powers, walks and connectivity

The Perron-Frobenius theorem

Regular discrete-time Markov chains

Spectrum and spectral radius

Definition 126 (Spectrum and spectral radius)

Let $A \in \mathcal{M}_n(\mathbb{C})$ be a square matrix

- *The **spectrum** of A, denoted $\sigma(A)$, is the set of all eigenvalues of A:*

$$\sigma(A) = \{\lambda \in \mathbb{C} : \det(A - \lambda \mathbb{I}_n) = 0\}$$

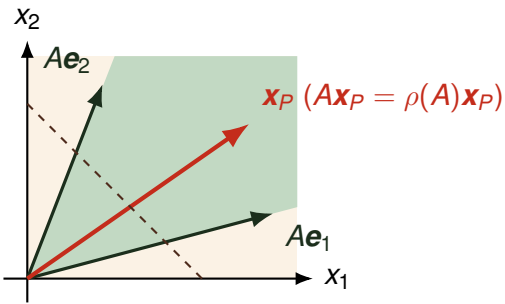
- *The **spectral radius** of A, denoted $\rho(A)$, is the maximum modulus of its eigenvalues:*

$$\rho(A) = \max_{\lambda \in \sigma(A)} |\lambda|$$

Geometrically, in the complex plane $\mathbb{C}$, all eigenvalues of A lie inside or on the closed disc $\{z \in \mathbb{C} : |z| \leq \rho(A)\}$ of radius $\rho(A)$ centred at the origin

Geometric intuition in $\mathbb{R}^2$: trapping an invariant ray

Why must a positive matrix $A > 0$ have a positive eigenvalue with a positive eigenvector?



- Consider the non-negative quadrant $\mathbb{R}^2_{\geq 0} = \{\boldsymbol{x} \geq \mathbf{0}\}$
- Since $A > 0$, both columns $A\boldsymbol{e}_1 > \mathbf{0}$ and $A\boldsymbol{e}_2 > \mathbf{0}$ lie **strictly inside** the open first quadrant
- Thus A maps the entire quadrant $\mathbb{R}^2_{\geq 0} \setminus \{\mathbf{0}\}$ into a narrower cone strictly inside the first quadrant
- By Brouwer's fixed-point theorem (or the intermediate value theorem on the angle of $\boldsymbol{x}$), A must map a unique ray in the first quadrant onto itself!

Perron's theorem for positive matrices

Theorem 127 (Perron's theorem, 1907)

Let $A \in \mathcal{M}_n(\mathbb{R})$ be a strictly positive matrix ($A > 0$). Then:

1. *$\rho(A) > 0$ is an eigenvalue of A (the **Perron root** or **dominant eigenvalue**)*
2. *$\rho(A)$ is algebraically and geometrically simple (multiplicity 1)*
3. *$\rho(A)$ strictly dominates every other eigenvalue: $|\lambda| < \rho(A)$ for all $\lambda \in \sigma(A) \setminus \{\rho(A)\}$*
4. *There exists a **unique** strictly positive right eigenvector $\boldsymbol{x} > \mathbf{0}$ such that*

$$A\boldsymbol{x} = \rho(A)\boldsymbol{x} \quad \text{and} \quad \mathbb{1}^T\boldsymbol{x} = \sum_{i=1}^{n} x_i = 1$$

   *(the **right Perron vector**, and similarly a unique left Perron vector $\boldsymbol{y} > \mathbf{0}$)*
5. *No other eigenvalue of A has a non-negative eigenvector*

The Perron-Frobenius theorem for primitive and irreducible matrices

Frobenius (1912) extended Perron's theorem to irreducible non-negative matrices:

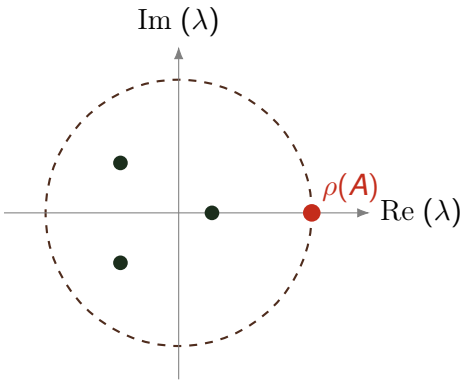
Theorem 128 (Perron-Frobenius theorem)

*Let $A \in \mathcal{M}_n(\mathbb{R}_{\geq 0})$ be an **irreducible** non-negative matrix*

1. *$\rho(A) > 0$ is a simple eigenvalue of A (algebraic and geometric multiplicity 1)*
2. *There exists a unique strictly positive right eigenvector $\boldsymbol{x} > \mathbf{0}$ with $A\boldsymbol{x} = \rho(A)\boldsymbol{x}$ and $\mathbb{1}^T\boldsymbol{x} = 1$ (and no other eigenvalue has a non-negative eigenvector)*
3. ***If** A **is primitive** ($d = 1$): $|\lambda| < \rho(A)$ for all $\lambda \in \sigma(A)$ with $\lambda \neq \rho(A)$*
4. ***If** A **is imprimitive with period** $d > 1$: A has exactly d eigenvalues of modulus $\rho(A)$, given by $\lambda_k = \rho(A)e^{2\pi ik/d}$ ($k = 0, 1, \ldots, d-1$), and $|\lambda| < \rho(A)$ for all others*

Spectral circle: primitive ($d = 1$) vs periodic ($d = 3$)

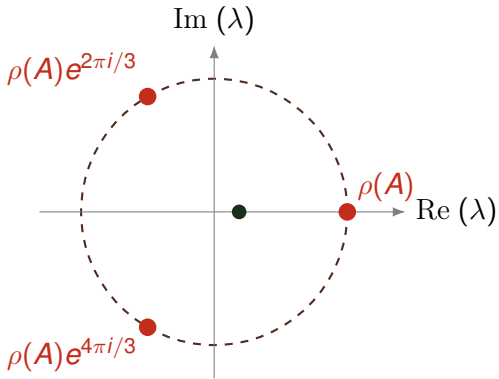
Primitive matrix ($d = 1$)



Strict spectral gap $|\lambda_j| < \rho(A)$ for $j \geq 2$

$\Longrightarrow \left(\frac{1}{\rho(A)}A\right)^k$ **converges** as $k \to \infty$

Irreducible periodic matrix ($d = 3$)

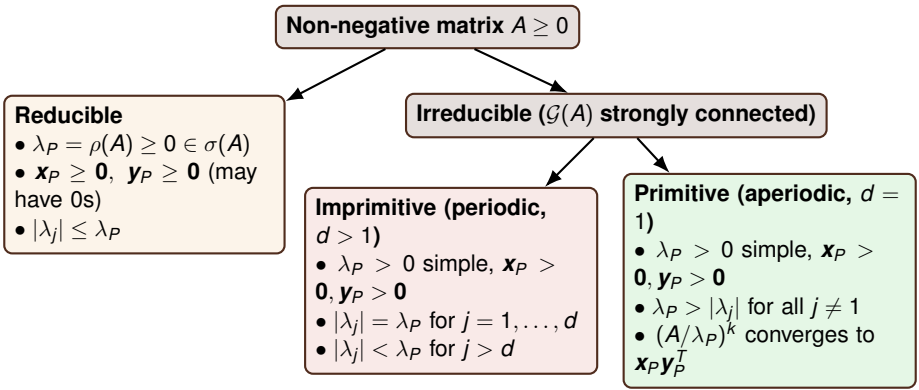


$d = 3$ eigenvalues on the circle $|\lambda| = \rho(A)$

$\Longrightarrow \left(\frac{1}{\rho(A)}A\right)^k$ **oscillates** with period 3

Summary of Perron-Frobenius across all non-negative matrices

Let $A \in \mathcal{M}_n(\mathbb{R}_{\geq 0})$ have Perron root $\lambda_P = \rho(A)$ and right/left Perron vectors $\boldsymbol{x}_P, \boldsymbol{y}_P$:



Asymptotic behaviour of powers of a primitive matrix

Theorem 129 (Asymptotics of primitive matrices)

Let $A \in \mathcal{M}_n(\mathbb{R}_{\geq 0})$ be a primitive matrix with Perron root $\rho(A) > 0$. Let $\boldsymbol{x} > \mathbf{0}$ and $\boldsymbol{y} > \mathbf{0}$ be the unique positive right and left eigenvectors satisfying $A\boldsymbol{x} = \rho(A)\boldsymbol{x}$, $\boldsymbol{y}^T A = \rho(A)\boldsymbol{y}^T$, $\mathbb{1}^T \boldsymbol{x} = 1$ and $\boldsymbol{y}^T \boldsymbol{x} = 1$. Then

$$\lim_{k\to\infty} \left(\frac{1}{\rho(A)} A\right)^k = \boldsymbol{x}\boldsymbol{y}^T > 0$$

Why this holds: Write $\boldsymbol{z} = (\boldsymbol{y}^T \boldsymbol{z})\boldsymbol{x} + \sum_{j=2}^n c_j \boldsymbol{v}_j$. Since $|\lambda_j/\rho(A)| < 1$ for all $j \geq 2$,

$$\left(\frac{1}{\rho(A)} A\right)^k \boldsymbol{z} = (\boldsymbol{y}^T \boldsymbol{z})\boldsymbol{x} + \sum_{j=2}^n c_j \left(\frac{\lambda_j}{\rho(A)}\right)^k \boldsymbol{v}_j \xrightarrow[k\to\infty]{} (\boldsymbol{y}^T \boldsymbol{z})\boldsymbol{x} = (\boldsymbol{x}\boldsymbol{y}^T)\boldsymbol{z}$$

Inspecting the Perron-Frobenius spectrum in R

Compare the eigenvalues of the periodic matrix A_1 ($d = 3$) and primitive matrix A_2 ($d = 1$):

```
A1 <- matrix(c(0, 1, 0,  0, 0, 1,  1, 0, 0), nrow = 3, byrow = TRUE)
A2 <- matrix(c(1, 1, 0,  0, 0, 1,  1, 0, 0), nrow = 3, byrow = TRUE)
# Moduli of eigenvalues of A1 (all 3 have modulus 1)
round(Mod(eigen(A1)$values), 4)

## [1] 1 1 1

# Eigenvalues and moduli for primitive A2 (unique dominant Perron root)
round(eigen(A2)$values, 4)

## [1]  1.4656+0.0000i -0.2328+0.7926i -0.2328-0.7926i

round(Mod(eigen(A2)$values), 4)

## [1] 1.4656 0.8260 0.8260
```

Non-negative matrices and their digraphs

Matrix powers, walks and connectivity

The Perron-Frobenius theorem

Regular discrete-time Markov chains

Regular discrete-time Markov chains

Markov chains and column-stochastic matrices

The fundamental limit theorem for regular Markov chains

Examples and running example: orange cats

Discrete-time Markov chains

A **Markov chain** is a discrete-time stochastic process whose evolution depends **only on the current state** of the system (the **memoryless** or **Markov property**):

- Consider a system that can be in one of n finite states $S = \{S_1, \ldots, S_n\}$ at discrete times $t = 0, 1, 2, \ldots$
- The probability of moving to state S_i at time $t + 1$ given that the system is in state S_j at time t is

$$p_{ij} = \mathbb{P}(S_i(t + 1) \mid S_j(t))$$

- Note the index order carefully: p_{ij} is the transition probability **from state S_j (column j) to state S_i (row i)**

Markov chains are a cornerstone of data science: they underpin PageRank, Hidden Markov Models (HMMs), Markov Chain Monte Carlo (MCMC) sampling and reinforcement learning

The transition matrix P

Definition 130 (Discrete-time Markov chain and transition matrix)

A sequence of random states from $S = \{S_1, \ldots, S_n\}$ is a (time-homogeneous) ***discrete-time Markov chain*** *(DTMC) if for all $t = 0, 1, 2, \ldots$ and all $1 \leq i, j \leq n$, the transition probability*

$$p_{ij} = \mathbb{P}(S_i(t+1) \mid S_j(t))$$

depends only on i and j. The matrix $P = [p_{ij}] \in \mathcal{M}_n(\mathbb{R}_{\geq 0})$ is called the ***transition matrix*** *of the Markov chain*

If the system is in state S_j at time t, it must transition to one of $S_1, \ldots, S_n$ at time $t+1$, so the entries in **column** j of P must sum to 1:

$$p_{1j} + p_{2j} + \cdots + p_{nj} = \sum_{i=1}^{n} p_{ij} = 1 \quad \text{for all } j = 1, \ldots, n$$

How do we get to state S_i at time $t+1$?

For $1 \leq i \leq n$, let $p_i(t) = \mathbb{P}(S_i(t))$ be the probability that the system is in state S_i at time t

List all n mutually exclusive ways to arrive in state S_i at time $t+1$:

1. We were in S_1 at time t (probability $p_1(t)$) and transitioned from S_1 to S_i (probability p_{i1}), contributing $p_{i1}p_1(t)$
2. We were in S_2 at time t (probability $p_2(t)$) and transitioned from S_2 to S_i (probability p_{i2}), contributing $p_{i2}p_2(t)$

⋮

n. We were in S_n at time t (probability $p_n(t)$) and transitioned from S_n to S_i (probability p_{in}), contributing $p_{in}p_n(t)$

Summing over all $j = 1, \ldots, n$ gives the law of total probability:

$$p_i(t+1) = \sum_{j=1}^{n} p_{ij}p_j(t) = p_{i1}p_1(t) + p_{i2}p_2(t) + \cdots + p_{in}p_n(t)$$

State distribution update in matrix form

Collecting the state probabilities into the column **probability vector**

$$\boldsymbol{p}(t) = \begin{pmatrix} p_1(t) \\ \vdots \\ p_n(t) \end{pmatrix} \geq \mathbf{0}, \qquad \mathbb{1}^T \boldsymbol{p}(t) = \sum_{i=1}^{n} p_i(t) = 1$$

the update equations for $i = 1, \ldots, n$ take the compact matrix form

$$\begin{pmatrix} p_1(t+1) \\ \vdots \\ p_n(t+1) \end{pmatrix} = \begin{pmatrix} p_{11} & p_{12} & \cdots & p_{1n} \\ \vdots & \vdots & \ddots & \vdots \\ p_{n1} & p_{n2} & \cdots & p_{nn} \end{pmatrix} \begin{pmatrix} p_1(t) \\ \vdots \\ p_n(t) \end{pmatrix} \qquad \Longleftrightarrow \qquad \boldsymbol{p}(t+1) = P\boldsymbol{p}(t)$$

Iterating from the initial distribution $\boldsymbol{p}(0)$ gives $\boldsymbol{p}(1) = P\boldsymbol{p}(0)$, $\boldsymbol{p}(2) = P\boldsymbol{p}(1) = P^2\boldsymbol{p}(0)$ and for any $t \in \mathbb{N}$:

$$\boldsymbol{p}(t) = P^t \boldsymbol{p}(0)$$

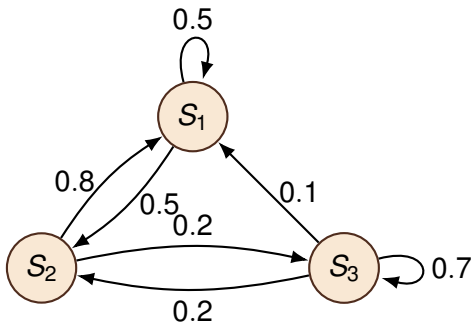
Transition matrix and transition digraph

For a column-stochastic transition matrix $P = [p_{ij}]$, recall that $p_{ij} > 0$ represents a transition **from state S_j to state S_i**:

Column-stochastic matrix P

$$P = \begin{pmatrix} 0.5 & 0.8 & 0.1 \\ 0.5 & 0 & 0.2 \\ 0 & 0.2 & 0.7 \end{pmatrix}$$

Transition digraph



- Column 1 (from S_1): $0.5 + 0.5 + 0 = 1$
- Column 2 (from S_2): $0.8 + 0 + 0.2 = 1$
- Column 3 (from S_3): $0.1 + 0.2 + 0.7 = 1$

Out-weights from each vertex S_j in the transition digraph correspond to **column** j of P and always sum to 1

Stochastic matrices

Definition 131 (Column-stochastic, row-stochastic and doubly stochastic matrices)

Let $M = [m_{ij}] \in \mathcal{M}_n(\mathbb{R}_{\geq 0})$ be a non-negative matrix and $\mathbb{1} = (1, \ldots, 1)^T \in \mathbb{R}^n$

- *M is **column-stochastic** if every column sums to 1:*

$$\sum_{i=1}^{n} m_{ij} = 1 \quad \text{for all } j = 1, \ldots, n \qquad \Longleftrightarrow \qquad \mathbb{1}^T M = \mathbb{1}^T$$

- *M is **row-stochastic** if every row sums to 1:*

$$\sum_{j=1}^{n} m_{ij} = 1 \quad \text{for all } i = 1, \ldots, n \qquad \Longleftrightarrow \qquad M\mathbb{1} = \mathbb{1}$$

- *M is **doubly stochastic** if it is both column-stochastic and row-stochastic ($\mathbb{1}^T M = \mathbb{1}^T$ and $M\mathbb{1} = \mathbb{1}$)*

Products and powers of column-stochastic matrices

Theorem 132

1. *If $M, N \in \mathcal{M}_n(\mathbb{R}_{\geq 0})$ are column-stochastic matrices, then their product MN is also column-stochastic*
2. *For any column-stochastic matrix $P \in \mathcal{M}_n(\mathbb{R}_{\geq 0})$ and any integer $t \geq 1$, P^t is column-stochastic, and if $\boldsymbol{p}(0)$ is a probability vector, then $\boldsymbol{p}(t) = P^t\boldsymbol{p}(0)$ is a probability vector*

Proof:

1. Since $M \geq 0$ and $N \geq 0$, their product satisfies $MN \geq 0$. Furthermore, using the associativity of matrix multiplication and $\mathbb{1}^T M = \mathbb{1}^T$, $\mathbb{1}^T N = \mathbb{1}^T$:
$$\mathbb{1}^T(MN) = (\mathbb{1}^T M)N = \mathbb{1}^T N = \mathbb{1}^T$$
Thus MN is non-negative and its columns sum to 1, so MN is column-stochastic
2. By induction on $t \geq 1$, P^t is column-stochastic, and $\boldsymbol{p}(t) = P^t\boldsymbol{p}(0) \geq \mathbf{0}$ with $\mathbb{1}^T\boldsymbol{p}(t) = (\mathbb{1}^T P^t)\boldsymbol{p}(0) = \mathbb{1}^T\boldsymbol{p}(0) = 1$

□

Spectral radius of a column-stochastic matrix

Theorem 133 (Spectrum of a stochastic matrix)

Let $P \in \mathcal{M}_n(\mathbb{R}_{\geq 0})$ be a column-stochastic matrix. Then:

1. $\lambda_1 = 1$ *is an eigenvalue of* P
2. *Every eigenvalue* $\lambda \in \sigma(P)$ *satisfies* $|\lambda| \leq 1$*, so the spectral radius is* $\rho(P) = 1$

Proof:

1. Since P is column-stochastic, $\mathbb{1}^T P = \mathbb{1}^T$. Taking the transpose gives $P^T \mathbb{1} = \mathbb{1}$ with $\mathbb{1} \neq \mathbf{0}$, so $\lambda = 1$ is an eigenvalue of P^T. Since $\det(P^T - \lambda \mathbb{I}_n) = \det(P - \lambda \mathbb{I}_n)$, P and P^T have the same eigenvalues ($\sigma(P) = \sigma(P^T)$), so $1 \in \sigma(P)$
2. Let $\lambda \in \sigma(P) = \sigma(P^T)$ and let $\boldsymbol{x} \neq \mathbf{0}$ be a right eigenvector of P^T: $P^T \boldsymbol{x} = \lambda \boldsymbol{x}$. Pick an index k maximising $|x_k| = \max_{1 \leq j \leq n} |x_j| > 0$. Then from row k of $P^T \boldsymbol{x} = \lambda \boldsymbol{x}$:

$$|\lambda|\,|x_k| = |\lambda x_k| = \left| \sum_{i=1}^{n} p_{ik} x_i \right| \leq \sum_{i=1}^{n} p_{ik} |x_i| \leq \left(\sum_{i=1}^{n} p_{ik} \right) |x_k| = 1 \cdot |x_k|$$

Regular discrete-time Markov chains

- Markov chains and column-stochastic matrices
- **The fundamental limit theorem for regular Markov chains**
- Examples and running example: orange cats

Regular Markov chains

Definition 134 (Regular Markov chain)

*A discrete-time Markov chain with column-stochastic transition matrix $P \in \mathcal{M}_n(\mathbb{R}_{\geq 0})$ is **regular** if P is **primitive**, i.e. there exists a positive integer $k \geq 1$ such that*

$$P^k > 0 \quad (\textit{every entry of } P^k \textit{ is strictly positive})$$

Interpretation:

- The entry $(P^k)_{ij}$ is the k-step transition probability of moving from state S_j to state S_i in exactly k steps
- Thus a Markov chain is regular if and only if there is some time horizon $k \geq 1$ such that from **any** starting state S_j, there is a strictly positive probability of being in **any** target state S_i after k steps
- By Theorem 122, a Markov chain is regular $\iff$ its transition digraph is **strongly connected** (irreducible) and **aperiodic** ($d = 1$, e.g. at least one state has a self-loop $p_{ii} > 0$)

The fundamental limit theorem for regular Markov chains

Applying Theorem 129 with $\rho(P) = 1$ and left Perron vector $\boldsymbol{y} = \mathbb{1}$ yields:

Theorem 135 (Fundamental limit theorem for regular Markov chains)

*Let $P \in \mathcal{M}_n(\mathbb{R}_{\geq 0})$ be the column-stochastic transition matrix of a **regular** Markov chain. Then:*

1. *There exists a **unique** strictly positive probability vector $\boldsymbol{v} = (v_1, \ldots, v_n)^T > \mathbf{0}$ satisfying $P\boldsymbol{v} = \boldsymbol{v}$ and $\mathbb{1}^T\boldsymbol{v} = \sum_{i=1}^n v_i = 1$, called the **stationary distribution** (or **equilibrium** / **steady-state distribution**) of the Markov chain*
2. *As $t \to \infty$, the matrix powers P^t converge to the rank-one column-stochastic matrix*
$$\lim_{t\to\infty} P^t = W = \begin{pmatrix} \boldsymbol{v} & \boldsymbol{v} & \cdots & \boldsymbol{v} \end{pmatrix} = \boldsymbol{v}\mathbb{1}^T > 0$$
3. *For **any** initial probability distribution $\boldsymbol{p}(0)$, the state vector $\boldsymbol{p}(t) = P^t\boldsymbol{p}(0)$ converges to $\boldsymbol{v}$:*
$$\lim_{t\to\infty} \boldsymbol{p}(t) = W\boldsymbol{p}(0) = \boldsymbol{v}(\mathbb{1}^T\boldsymbol{p}(0)) = \boldsymbol{v}$$

Finding the stationary distribution $\boldsymbol{v}$ in practice

How do we compute the unique stationary distribution $\boldsymbol{v}$ of a regular transition matrix $P \in \mathcal{M}_n(\mathbb{R}_{\geq 0})$ on an exam or by hand?

1. **Write the fixed-point equation $P\boldsymbol{v} = \boldsymbol{v}$ as a homogeneous linear system**:

$$(P - \mathbb{I}_n)\boldsymbol{v} = \mathbf{0} \qquad (\text{so } \boldsymbol{v} \in \operatorname{null}(P - \mathbb{I}_n))$$

   - Since $\lambda_1 = 1$ is a simple eigenvalue of P, $\dim \operatorname{null}(P - \mathbb{I}_n) = 1$
   - Because $\mathbb{1}^T(P - \mathbb{I}_n) = \mathbf{0}^T$, the rows of $P - \mathbb{I}_n$ always sum to zero (one row is redundant)

2. **Row-reduce** $P - \mathbb{I}_n$ (or solve the first $n - 1$ equations) to express $v_1, \ldots, v_n$ in terms of a single free parameter, giving a positive basis vector $\boldsymbol{w} \in \operatorname{null}(P - \mathbb{I}_n)$
3. **Normalise to a probability vector** using $\mathbb{1}^T\boldsymbol{v} = v_1 + \cdots + v_n = 1$:

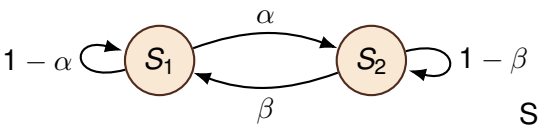
$$\boldsymbol{v} = \frac{\boldsymbol{w}}{\|\boldsymbol{w}\|_1} = \frac{\boldsymbol{w}}{w_1 + \cdots + w_n}$$

Regular discrete-time Markov chains

- Markov chains and column-stochastic matrices
- The fundamental limit theorem for regular Markov chains
- **Examples and running example: orange cats**

Exam-style example 1: general 2×2 regular Markov chain

Consider a two-state Markov chain with transition probabilities $\alpha = \mathbb{P}(S_2 \mid S_1)$ and $\beta = \mathbb{P}(S_1 \mid S_2)$, where $0 < \alpha, \beta < 1$:



$$P = \begin{pmatrix} 1-\alpha & \beta \\ \alpha & 1-\beta \end{pmatrix} > 0$$

Since $P > 0$, the chain is regular ($P^1 > 0$)

Finding the stationary distribution $\boldsymbol{v} = (v_1, v_2)^T$:

$$(P - \mathbb{I}_2)\boldsymbol{v} = \begin{pmatrix} -\alpha & \beta \\ \alpha & -\beta \end{pmatrix} \begin{pmatrix} v_1 \\ v_2 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix} \iff -\alpha v_1 + \beta v_2 = 0 \iff v_2 = \frac{\alpha}{\beta} v_1$$

Imposing $v_1 + v_2 = 1$ gives $v_1 \left(1 + \frac{\alpha}{\beta}\right) = 1$, hence:

$$\boldsymbol{v} = \frac{1}{\alpha + \beta} \begin{pmatrix} \beta \\ \alpha \end{pmatrix}, \qquad \lim_{t \to \infty} P^t = W = \frac{1}{\alpha + \beta} \begin{pmatrix} \beta & \beta \\ \alpha & \alpha \end{pmatrix}$$

Exam-style example 2: 3×3 regular chain via row reduction

Problem: Show that the Markov chain with transition matrix P is regular and find its stationary distribution $\boldsymbol{v}$ and $\lim_{t\to\infty} P^t$:

$$P = \begin{pmatrix} 0.5 & 0.4 & 0 \\ 0.5 & 0.2 & 0.5 \\ 0 & 0.4 & 0.5 \end{pmatrix}$$

Step 1 (Regularity):

- From P, we have transitions $S_1 \to S_2 \to S_3 \to S_2 \to S_1$, so all 3 states communicate ($\mathcal{G}(P^T)$ is strongly connected, P is irreducible)
- Diagonal entries $p_{11} = 0.5 > 0$, $p_{22} = 0.2 > 0$, $p_{33} = 0.5 > 0$ give self-loops, so by Corollary 123, P is **primitive** (in fact $P^2 > 0$) and the chain is **regular**

Exam-style example 2 (continued): solving $(P - \mathbb{I}_3)\boldsymbol{v} = \mathbf{0}$

Step 2 (Nullspace of $P - \mathbb{I}_3$): Form $P - \mathbb{I}_3$ (notice the columns sum to 0):

$$P - \mathbb{I}_3 = \begin{pmatrix} -0.5 & 0.4 & 0 \\ 0.5 & -0.8 & 0.5 \\ 0 & 0.4 & -0.5 \end{pmatrix} \xrightarrow{R_2 + R_1} \begin{pmatrix} -0.5 & 0.4 & 0 \\ 0 & -0.4 & 0.5 \\ 0 & 0.4 & -0.5 \end{pmatrix} \xrightarrow{R_3 + R_2} \begin{pmatrix} -0.5 & 0.4 & 0 \\ 0 & -0.4 & 0.5 \\ 0 & 0 & 0 \end{pmatrix}$$

From the two pivot rows with v_3 as free variable:

- Row 2: $-0.4v_2 + 0.5v_3 = 0 \implies v_2 = \frac{5}{4}v_3$
- Row 1: $-0.5v_1 + 0.4v_2 = 0 \implies v_1 = \frac{4}{5}v_2 = \frac{4}{5} \cdot \frac{5}{4}v_3 = v_3$

Step 3 (Normalisation): Impose $v_1 + v_2 + v_3 = 1$, i.e. $\left(1 + \frac{5}{4} + 1\right) v_3 = \frac{13}{4}v_3 = 1 \implies v_3 = \frac{4}{13}$, giving:

$$\boldsymbol{v} = \begin{pmatrix} 4/13 \\ 5/13 \\ 4/13 \end{pmatrix}, \qquad \lim_{t \to \infty} P^t = W = \begin{pmatrix} 4/13 & 4/13 & 4/13 \\ 5/13 & 5/13 & 5/13 \\ 4/13 & 4/13 & 4/13 \end{pmatrix}$$

Running example: the “orange” gene in cats

A cat’s coat colour is determined by many genes; the “orange” trait comes from a specific gene called the **orange locus**

It has two *alleles* (versions):

- **O** $\implies$ produces *phaeomelanin* (red/orange pigment)
- **o** $\implies$ produces *eumelanin* (black/brown pigment)

This gene is *sex-linked*: it is located on the **X chromosome**

- Females have two X chromosomes (**XX**), receiving one allele from each parent: genotypes $X^O X^O$, $X^o X^o$ or $X^O X^o$
- Males have one X and one Y chromosome (**XY**), receiving only one allele (always from the mother): genotypes $X^O Y$ or $X^o Y$

Genotype vs phenotype in cats

Males (simple):

- $X^O Y \implies$ **orange cat**
- $X^o Y \implies$ **non-orange cat** (e.g. black)

Females (the special case):

- $X^O X^O \implies$ **orange cat** (S_1)
- $X^o X^o \implies$ **non-orange cat** (S_2, e.g. black)
- $X^O X^o \implies$ **tortoiseshell cat** (S_3)

A “tortie” is not a simple blend: because of random X-inactivation in female cells, both alleles (O and o) are active in different patches of skin, creating the mottled orange and black pattern (male tortoiseshells are extremely rare genetic anomalies with **XXY** genotype $X^O X^o Y$)

Punnett square warm-up: crossing two parents

Cross 1: orange dad ($X^O Y$) + black mum ($X^o X^o$)

		Father	
		X^O	Y
Mother	X^o	$X^O X^o$	$X^o Y$
	X^o	$X^O X^o$	$X^o Y$

- All daughters ($X^O X^o$) are **tortoiseshell**
- All sons ($X^o Y$) are **black**

Cross 2: black dad ($X^o Y$) + tortie mum ($X^O X^o$)

		Father	
		X^o	Y
Mother	X^O	$X^O X^o$	$X^O Y$
	X^o	$X^o X^o$	$X^o Y$

- Daughters: 50% tortie ($X^O X^o$), 50% black ($X^o X^o$)
- Sons: 50% orange ($X^O Y$), 50% black ($X^o Y$)

Setting up the regular Markov chain for female genotypes

Track the 3 female genotypes across generations:

- S_1: $X^O X^O$ (orange female)
- S_2: $X^o X^o$ (black female)
- S_3: $X^O X^o$ (tortoiseshell female)

In each generation, we mate the current female with a male chosen at random from a **fixed population** consisting of:

- 50% orange males ($X^O Y$)
- 50% black males ($X^o Y$)

and select one of her daughters at random as the next generation's female

State 1: orange female ($X^O X^O$)

The mother is $X^O X^O$. We pick a father with 50/50 probability:

Case 1: father is $X^O Y$

Mother	X^O	Y
X^O	$X^O X^O$ (S_1)	Male
X^O	$X^O X^O$ (S_1)	Male

Daughters: 100% S_1

Case 2: father is $X^o Y$

Mother	X^o	Y
X^O	$X^O X^o$ (S_3)	Male
X^O	$X^O X^o$ (S_3)	Male

Daughters: 100% S_3

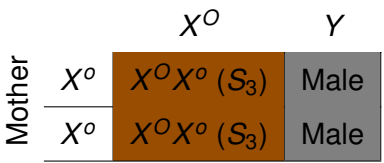
Transitions from S_1 (column 1 of P):

- $p_{11} = \mathbb{P}(S_1 \mid S_1) = 0.5 \times 1.0 = \mathbf{0.5}$
- $p_{21} = \mathbb{P}(S_2 \mid S_1) = \mathbf{0}$
- $p_{31} = \mathbb{P}(S_3 \mid S_1) = 0.5 \times 1.0 = \mathbf{0.5}$

State 2: black female (X^oX^o)

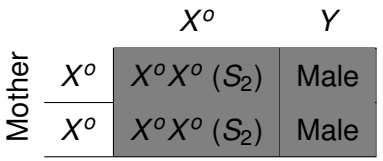
The mother is X^oX^o. We pick a father with 50/50 probability:

Case 1: father is X^OY



Daughters: 100% S_3

Case 2: father is X^oY



Daughters: 100% S_2

Transitions from S_2 (column 2 of P):

- $p_{12} = \mathbb{P}(S_1 \mid S_2) = \mathbf{0}$
- $p_{22} = \mathbb{P}(S_2 \mid S_2) = 0.5 \times 1.0 = \mathbf{0.5}$
- $p_{32} = \mathbb{P}(S_3 \mid S_2) = 0.5 \times 1.0 = \mathbf{0.5}$

State 3: tortoiseshell female ($X^O X^o$)

The mother is $X^O X^o$. We pick a father with 50/50 probability:

Case 1: father is $X^O Y$

Mother	X^O	Y
X^O	$X^O X^O$ (S_1)	Male
X^o	$X^O X^o$ (S_3)	Male

Daughters: 50% S_1, 50% S_3

Case 2: father is $X^o Y$

Mother	X^o	Y
X^O	$X^O X^o$ (S_3)	Male
X^o	$X^o X^o$ (S_2)	Male

Daughters: 50% S_2, 50% S_3

Transitions from S_3 (column 3 of P):

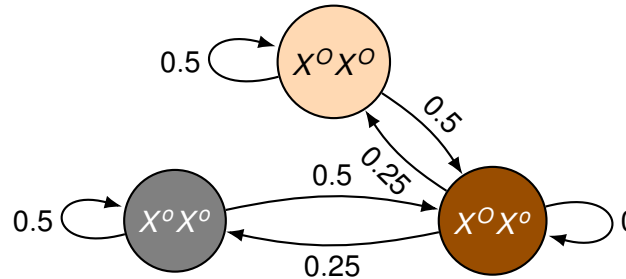
- $p_{13} = \mathbb{P}(S_1 \mid S_3) = 0.5 \times 0.5 = \mathbf{0.25}$
- $p_{23} = \mathbb{P}(S_2 \mid S_3) = 0.5 \times 0.5 = \mathbf{0.25}$
- $p_{33} = \mathbb{P}(S_3 \mid S_3) = 0.5 \times 0.5 + 0.5 \times 0.5 = \mathbf{0.5}$

Transition matrix and digraph of the orange cat chain

$$P = \begin{pmatrix} 0.5 & 0 & 0.25 \\ 0 & 0.5 & 0.25 \\ 0.5 & 0.5 & 0.5 \end{pmatrix}$$

Is this Markov chain regular?

- **Strongly connected**: $S_1 \to S_3 \to S_2$ and $S_2 \to S_3 \to S_1$, so all states communicate (P is irreducible)
- **Aperiodic**: $p_{11}, p_{22}, p_{33} > 0$ (self-loops), so P is **primitive** ($P^2 > 0$) and the chain is **regular**



Stationary distribution of the orange cat chain

We solve $(P - \mathbb{I}_3)\boldsymbol{v} = \mathbf{0}$ with $v_1 + v_2 + v_3 = 1$:

$$(P - \mathbb{I}_3)\boldsymbol{v} = \begin{pmatrix} -0.5 & 0 & 0.25 \\ 0 & -0.5 & 0.25 \\ 0.5 & 0.5 & -0.5 \end{pmatrix} \begin{pmatrix} v_1 \\ v_2 \\ v_3 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}$$

From the first two equations:

- $-0.5v_1 + 0.25v_3 = 0 \implies v_1 = 0.5v_3$
- $-0.5v_2 + 0.25v_3 = 0 \implies v_2 = 0.5v_3$

Substituting into the normalisation condition $v_1 + v_2 + v_3 = 1$:

$$0.5v_3 + 0.5v_3 + v_3 = 2v_3 = 1 \implies v_3 = 0.5, \quad v_1 = 0.25, \quad v_2 = 0.25$$

Thus the unique stationary distribution is $\boldsymbol{v} = (0.25, 0.25, 0.5)^T$: in the long run, regardless of the initial female's genotype, 25% of females are orange, 25% are black and 50% are tortoiseshell

Computing matrix powers and stationary distributions in R

Let us verify the convergence of P^t and the eigenpair $P\boldsymbol{v} = \boldsymbol{v}$ in R:

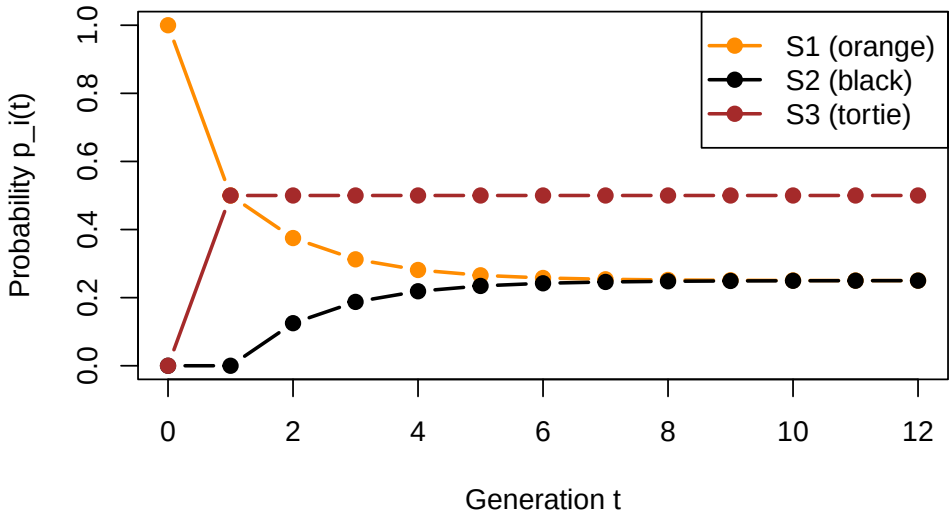
```
P_reg <- matrix(c(0.5, 0,   0.25,
                  0,   0.5, 0.25,
                  0.5, 0.5, 0.5),
                nrow = 3, byrow = TRUE)
# Dominant right eigenvector corresponding to eigenvalue 1
ev <- eigen(P_reg)
v_stat <- Re(ev$vectors[, 1])
v_stat <- v_stat / sum(v_stat)
v_stat
```

```
## [1] 0.25 0.25 0.50
```

```
# Compare with P^20 computed by repeated multiplication
P_20 <- Reduce(`%*%`, replicate(20, P_reg, simplify = FALSE))
round(P_20, 6)
```

Convergence of $\boldsymbol{p}(t) = P^t \boldsymbol{p}(0)$ from an initial state

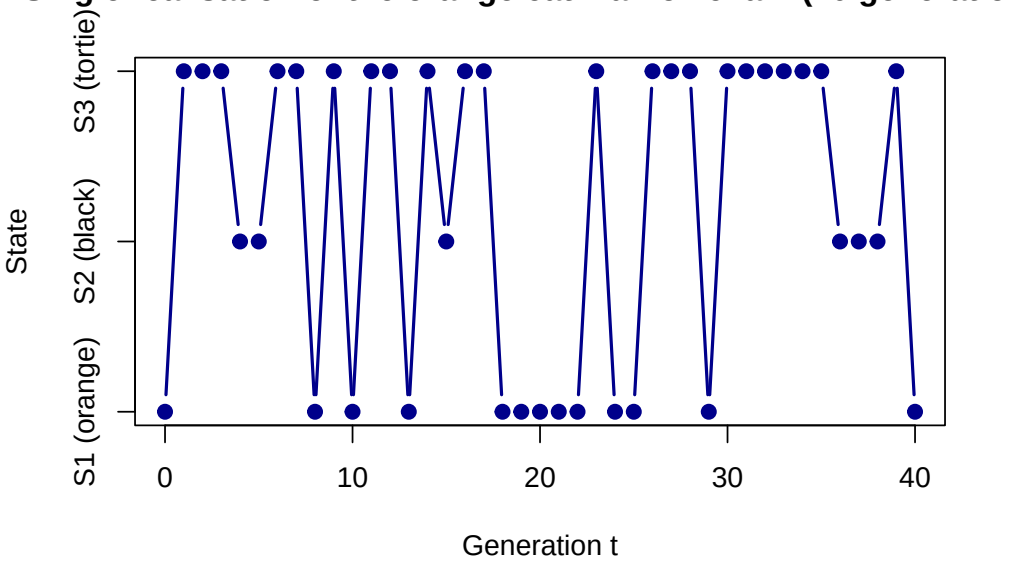
```r
t_max <- 12
p_hist <- matrix(0, nrow = 3, ncol = t_max + 1)
p_hist[, 1] <- c(1, 0, 0) # Start with 100% orange female (S_1)
for (t in 1:t_max) {
  p_hist[, t + 1] <- P_reg %*% p_hist[, t]
}
matplot(0:t_max, t(p_hist), type = "b", pch = 19, lty = 1, lwd = 2,
        col = c("darkorange", "black", "brown"),
        xlab = "Generation t", ylab = "Probability p_i(t)",
        ylim = c(0, 1), main = "Convergence to stationary distribution v = (
legend("topright", legend = c("S1 (orange)", "S2 (black)", "S3 (tortie)"),
       col = c("darkorange", "black", "brown"), lty = 1, pch = 19, lwd = 2)
```


Convergence to stationary distribution v = (0.25, 0.25, 0.5)
Probability p_i(t)
0.0
0.2
0.4
0.6
0.8
1.0
0
2
4
6
8
10
12
Generation t
S1 (orange)
S2 (black)
S3 (tortie)

Simulating a single realisation of the regular Markov chain

In a column-stochastic chain, when in state j at step t, the next state at step $t+1$ is drawn with probabilities given by **column** j of P:

```
set.seed(2740)
n_steps <- 40
states <- numeric(n_steps + 1)
states[1] <- 1 # Start in state S_1 (orange female)
for (t in 1:n_steps) {
  states[t + 1] <- sample(1:3, size = 1, prob = P_reg[, states[t]])
}
plot(0:n_steps, states, type = "b", pch = 19, lwd = 1.8, col = "darkblue",
     yaxt = "n", xlab = "Generation t", ylab = "State",
     main = "Single realisation of the orange cat Markov chain (40 generatio
axis(2, at = 1:3, labels = c("S1 (orange)", "S2 (black)", "S3 (tortie)"))
```


Single realisation of the orange cat Markov chain (40 generation
State
S3 (tortie)
S2 (black)
S1 (orange)
0
10
20
30
40
Generation t

Empirical state frequencies vs stationary distribution $\boldsymbol{v}$

By the ergodic theorem for regular Markov chains, the long-run fraction of time spent in each state along a single trajectory converges to $\boldsymbol{v} = (0.25, 0.25, 0.5)^T$:

```r
set.seed(2740)
n_long <- 10000
traj <- numeric(n_long)
traj[1] <- 1
for (t in 1:(n_long - 1)) {
  traj[t + 1] <- sample(1:3, size = 1, prob = P_reg[, traj[t]])
}
# Empirical frequencies over 10,000 generations vs exact v
rbind(empirical = prop.table(table(factor(traj, levels = 1:3))),
      stationary = v_stat)
```

```
##                 1      2      3
## empirical  0.2414 0.2505 0.5081
## stationary 0.2500 0.2500 0.5000
```

Absorbing Markov chains, spectral centrality and PageRank

MATH 2740 – Mathematics of Data Science – Unit U09

Julien Arino
julien.arino@umanitoba.ca

Department of Mathematics @ University of Manitoba

Fall 2026

The University of Manitoba campuses are located on original lands of Anishinaabeg, Ininew, Anisininew, Dakota and Dene peoples, and on the National Homeland of the Red River Métis. We respect the Treaties that were made on these territories, we acknowledge the harms and mistakes of the past, and we dedicate ourselves to move forward in partnership with Indigenous communities in a spirit of Reconciliation and collaboration.

Outline

Absorbing Markov chains

Spectral centrality in networks

PageRank and the Google matrix

From Unit 08 to Unit 09

In Unit 08, we studied irreducible and primitive non-negative matrices and saw that a regular Markov chain converges to a unique positive stationary distribution $\boldsymbol{w} = P\boldsymbol{w}$

Two natural questions arise when a directed network is *not* regular:

1. What happens when a Markov chain has **absorbing states** (states that can be entered but never left), making P reducible?
2. How do we rank the importance of vertices in large directed networks (citations, social networks or the World Wide Web) using eigenvectors and linear systems?

Both questions are answered by the same two linear algebra tools:
Perron-Frobenius theory and the **Neumann series** $(\mathbb{I} - M)^{-1} = \sum_{k=0}^{\infty} M^k$

Absorbing Markov chains

Spectral centrality in networks

PageRank and the Google matrix

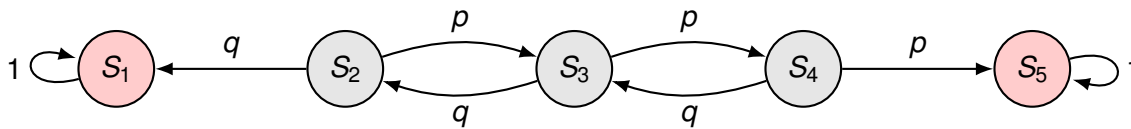
Absorbing Markov chains

- **Absorbing states and motivating examples**
- Canonical block form and reducibility
- The Neumann series and fundamental matrix
- Examples and R implementation

Motivating example 1 – 1D random walk with absorbing walls

Consider a particle (or gambler's fortune) moving on the states $\{S_1, S_2, S_3, S_4, S_5\}$:

- From an interior state S_j ($j \in \{2, 3, 4\}$), the particle moves left to S_{j-1} with probability $q = 1 - p$ and right to S_{j+1} with probability $p \in (0, 1)$
- States S_1 (ruin) and S_5 (target jackpot) are **absorbing boundaries**: once reached, the process stays there forever



Motivating example 2 – Modifying the breeding experiment

We modify the sex-linked orange cat breeding setup from Unit 08 by repeatedly mating a female offspring with an **orange male** ($X^O Y$) at every generation

The state of the chain is the genotype of the female offspring:

- S_1: $X^O X^O$ (orange female)
- S_2: $X^o X^o$ (black female)
- S_3: $X^O X^o$ (tortoiseshell female)

Let us compute the transition probabilities $p_{ij} = \mathbb{P}(S_i(t+1) \mid S_j(t))$ when mating each female genotype with an $X^O Y$ male

Transitions for the modified breeding experiment

Current state S_1 (X^OX^O)

Mom	X^O	Y
X^O	S_1	M
X^O	S_1	M

100% X^OX^O (S_1)

$\Longrightarrow$ $p_{11} = 1$

Current state S_2 (X^oX^o)

Mom	X^O	Y
X^o	S_3	M
X^o	S_3	M

100% X^OX^o (S_3)

$\Longrightarrow$ $p_{32} = 1$

Current state S_3 (X^OX^o)

Mom	X^O	Y
X^O	S_1	M
X^o	S_3	M

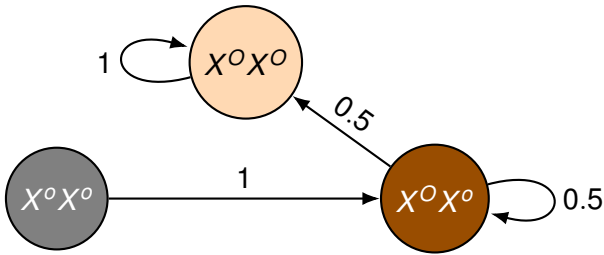
50% S_1, 50% S_3

$\Longrightarrow$ $p_{13} = 1/2$, $p_{33} = 1/2$

Summary of the modified breeding chain

In our column-stochastic convention ($p_{ij} = \mathbb{P}(S_i(t+1) \mid S_j(t))$), the transition matrix is

$$P = \begin{pmatrix} 1 & 0 & 1/2 \\ 0 & 0 & 0 \\ 0 & 1 & 1/2 \end{pmatrix}$$



Absorbing states and absorbing Markov chains

Definition 136 (Absorbing state)

A state S_i in a Markov chain with column-stochastic transition matrix $P = [p_{ij}]$ is ***absorbing*** *if once the chain enters S_i, it never leaves:*

$$p_{ii} = 1 \quad \text{and} \quad p_{ji} = 0 \text{ for all } j \neq i$$

Equivalently, column i of P is the standard basis vector $\mathbf{e}_i$, and in $\mathcal{G}(P)$ vertex S_i has a self-loop of weight 1 and no outgoing arcs to any other vertex

Definition 137 (Absorbing Markov chain)

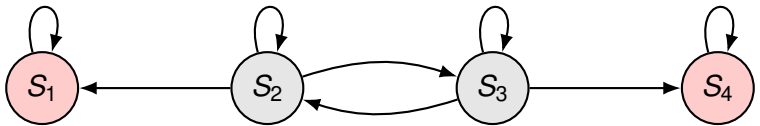
A Markov chain is ***absorbing*** *if:*

1. *It has at least one absorbing state*
2. *From every non-absorbing state, it is possible to reach at least one absorbing state in a finite number of steps (there is a directed walk in $\mathcal{G}(P)$ to an absorbing state)*

In an absorbing Markov chain, a state that is not absorbing is called ***transient***

Key questions about absorbing Markov chains

Suppose we have an absorbing chain such as a random walk with two absorbing boundaries S_1 and S_4:



1. Does the process eventually reach an absorbing state with probability 1?
2. Starting in transient state S_j, what is the expected number of visits to transient state S_i?
3. Starting in transient state S_j, what is the expected number of steps before absorption?
4. When there are multiple absorbing states, what is the probability of being absorbed in absorbing state S_i when starting in transient state S_j?

Absorbing Markov chains

- Absorbing states and motivating examples
- **Canonical block form and reducibility**
- The Neumann series and fundamental matrix
- Examples and R implementation

Absorbing chains are reducible

Suppose a Markov chain on n states has $r \geq 1$ absorbing states and $m = n - r \geq 1$ transient states

- Because an absorbing state S_i has no outgoing arcs to any other vertex ($p_{ji} = 0$ for all $j \neq i$), there is no directed path from S_i to any transient state
- Therefore, the transition digraph $\mathcal{G}(P)$ is **not strongly connected**
- By the characterisation from Unit 08, the transition matrix P is **reducible**

Relabelling the states of the chain corresponds to applying a permutation matrix $\Pi \in \mathcal{M}_n$:

$$\tilde{P} = \Pi^T P \Pi$$

We always relabel the states so that the r **absorbing states come first** $(S_1, \ldots, S_r)$ and the $m = n - r$ **transient states come last** $(S_{r+1}, \ldots, S_n)$

Example – Reordering the 5-state random walk into canonical form

In Motivating example 1 on $(S_1, S_2, S_3, S_4, S_5)$, states S_1 and S_5 are absorbing while S_2, S_3, S_4 are transient. In the original state ordering, the transition matrix is

$$P_{\text{orig}} = \begin{array}{c} \\ S_1 \\ S_2 \\ S_3 \\ S_4 \\ S_5 \end{array} \begin{array}{c} \begin{array}{ccccc} S_1 & S_2 & S_3 & S_4 & S_5 \end{array} \\ \begin{pmatrix} 1 & q & 0 & 0 & 0 \\ 0 & 0 & q & 0 & 0 \\ 0 & p & 0 & q & 0 \\ 0 & 0 & p & 0 & 0 \\ 0 & 0 & 0 & p & 1 \end{pmatrix} \end{array}$$

Reordering the states as $(S_1, S_5, S_2, S_3, S_4)$ via the permutation matrix $\Pi = [\boldsymbol{e}_1\ \boldsymbol{e}_5\ \boldsymbol{e}_2\ \boldsymbol{e}_3\ \boldsymbol{e}_4]$ gives the canonical block form $\tilde{P} = \Pi^T P_{\text{orig}} \Pi$:

$$\tilde{P} = \begin{array}{c} \\ S_1 \\ S_5 \\ \hline S_2 \\ S_3 \\ S_4 \end{array} \begin{array}{c} \begin{array}{cc|ccc} S_1 & S_5 & S_2 & S_3 & S_4 \end{array} \\ \left(\begin{array}{cc|ccc} 1 & 0 & q & 0 & 0 \\ 0 & 1 & 0 & 0 & p \\ \hline 0 & 0 & 0 & q & 0 \\ 0 & 0 & p & 0 & q \\ 0 & 0 & 0 & p & 0 \end{array}\right) \end{array} = \begin{pmatrix} \mathbb{I}_2 & R \\ \mathbf{0}_{3,2} & Q \end{pmatrix}$$

Canonical block form of an absorbing chain

Definition 138 (Canonical block form)

For an absorbing Markov chain with r absorbing states (listed first) and $m = n - r$ transient states (listed last), the column-stochastic transition matrix takes the ***canonical block form*** *(or* ***standard form****)*

$$P = \begin{pmatrix} \mathbb{I}_r & R \\ \mathbf{0}_{m,r} & Q \end{pmatrix}$$

where:

- $\mathbb{I}_r \in \mathcal{M}_r$ *is the $r \times r$ identity matrix (absorbing states stay put)*
- $\mathbf{0}_{m,r} \in \mathcal{M}_{mr}$ *is the $m \times r$ zero matrix (absorbing states never move to transient states)*
- $R \in \mathcal{M}_{rm}$ *contains the one-step transition probabilities from transient states to absorbing states*
- $Q \in \mathcal{M}_m$ *contains the one-step transition probabilities among transient states*

Visualising the canonical blocks

In our column-stochastic convention, columns correspond to the **from** state (at time t) and rows correspond to the **to** state (at time $t+1$):

	From absorbing (r cols)	**From transient** (m cols)
To absorbing (r rows)	$\mathbb{I}_r \in \mathcal{M}_r$	$R \in \mathcal{M}_{rm}$
To transient (m rows)	$\mathbf{0}_{m,r} \in \mathcal{M}_{mr}$	$Q \in \mathcal{M}_m$

Notice that:

- Each column of $\begin{pmatrix} R \\ Q \end{pmatrix}$ sums to 1: $\mathbb{1}_r^T R + \mathbb{1}_m^T Q = \mathbb{1}_m^T$
- Q is non-negative and **column-substochastic**: $\mathbb{1}_m^T Q \leq \mathbb{1}_m^T$, with strict inequality in every column j that has a direct transition to an absorbing state ($\sum_{i=1}^{r} r_{ij} > 0$)

Powers of P in canonical form

To understand the long-term behaviour $\boldsymbol{p}(k) = P^k \boldsymbol{p}(0)$, we compute the kth power of P in block form

Proposition 139 (Powers of the canonical transition matrix)

Let $P = \begin{pmatrix} \mathbb{I}_r & R \\ \mathbf{0}_{m,r} & Q \end{pmatrix}$ *be in canonical block form. For every integer* $k \geq 1$,

$$P^k = \begin{pmatrix} \mathbb{I}_r & R(\mathbb{I}_m + Q + Q^2 + \cdots + Q^{k-1}) \\ \mathbf{0}_{m,r} & Q^k \end{pmatrix} = \begin{pmatrix} \mathbb{I}_r & R\sum_{j=0}^{k-1} Q^j \\ \mathbf{0}_{m,r} & Q^k \end{pmatrix}$$

The (i, j) entry of Q^k is the probability that the chain is in the ith transient state after k steps, given that it started in the jth transient state

Proof of Proposition 139

We proceed by mathematical induction on $k \geq 1$

Base case ($k = 1$): Since $Q^0 = \mathbb{I}_m$, the formula gives $\begin{pmatrix} \mathbb{I}_r & R\mathbb{I}_m \\ \mathbf{0}_{m,r} & Q^1 \end{pmatrix} = P$, which holds

Inductive step: Suppose the formula holds for $k \geq 1$. Using block matrix multiplication for $P^{k+1} = P^k P$:

$$\begin{aligned} P^{k+1} &= \begin{pmatrix} \mathbb{I}_r & R(\mathbb{I}_m + Q + \cdots + Q^{k-1}) \\ \mathbf{0}_{m,r} & Q^k \end{pmatrix} \begin{pmatrix} \mathbb{I}_r & R \\ \mathbf{0}_{m,r} & Q \end{pmatrix} \\ &= \begin{pmatrix} \mathbb{I}_r\mathbb{I}_r + R(\cdots)\mathbf{0}_{m,r} & \mathbb{I}_r R + R(\mathbb{I}_m + Q + \cdots + Q^{k-1})Q \\ \mathbf{0}_{m,r}\mathbb{I}_r + Q^k\mathbf{0}_{m,r} & \mathbf{0}_{m,r}R + Q^k Q \end{pmatrix} \\ &= \begin{pmatrix} \mathbb{I}_r & R(\mathbb{I}_m + Q + Q^2 + \cdots + Q^k) \\ \mathbf{0}_{m,r} & Q^{k+1} \end{pmatrix} \end{aligned}$$

This completes the induction ■

Absorbing Markov chains

- Absorbing states and motivating examples
- Canonical block form and reducibility
- **The Neumann series and fundamental matrix**
- Examples and R implementation

Spectral radius of the transient block Q

Theorem 140 (Contraction of the transient block)

In any absorbing Markov chain with canonical transient block $Q \in \mathcal{M}_m$, the spectral radius satisfies

$$\rho(Q) < 1 \quad \textit{and consequently} \quad \lim_{k \to \infty} Q^k = \mathbf{0}_m$$

Why does this hold?

- By definition of an absorbing chain, from every transient state $j \in \{1, \ldots, m\}$ there is a directed walk to some absorbing state of length at most m
- Thus, starting from transient state j, the probability of being absorbed within m steps is strictly positive (> 0), so the probability of remaining in the transient states after m steps is strictly less than 1
- This means every column sum of Q^m is strictly less than 1: $\|Q^m\|_1 = \max_j \sum_{i=1}^{m} (Q^m)_{ij} < 1$
- Therefore $\rho(Q)^m = \rho(Q^m) \leq \|Q^m\|_1 < 1$, which implies $\rho(Q) < 1$ and $\lim_{k \to \infty} Q^k = \mathbf{0}$

Absorption occurs with probability 1

Because $(Q^k)_{ij}$ is the probability of being in transient state i at step k starting from transient state j, Theorem 140 immediately answers our first question

Theorem 141 (Eventual absorption)

In an absorbing Markov chain, the probability that the process eventually reaches an absorbing state is 1*, regardless of the initial state*

Indeed, starting from transient state j, the probability of still being in *some* transient state after k steps is the jth column sum $\sum_{i=1}^{m}(Q^k)_{ij} = (\mathbb{1}_m^T Q^k)_j \to 0$ as $k \to \infty$

The Neumann series for matrices

Recall the scalar geometric series: for $q \in \mathbb{R}$ with $|q| < 1$, $(1-q)^{-1} = \sum_{k=0}^{\infty} q^k$. The exact same identity holds for square matrices!

Theorem 142 (Neumann series)

Let $Q \in \mathcal{M}_m$ be a matrix with spectral radius $\rho(Q) < 1$ (equivalently, $\lim_{k\to\infty} Q^k = \mathbf{0}_m$). Then $\mathbb{I}_m - Q$ is invertible and its inverse is given by the convergent series

$$(\mathbb{I}_m - Q)^{-1} = \sum_{k=0}^{\infty} Q^k = \mathbb{I}_m + Q + Q^2 + Q^3 + \cdots$$

In particular, since $Q \geq 0$ for an absorbing Markov chain, every term $Q^k \geq 0$ is non-negative, so $(\mathbb{I}_m - Q)^{-1} \geq \mathbb{I}_m \geq 0$ has strictly non-negative entries (and diagonal entries ≥ 1)

Proof of Theorem 142 (Neumann series)

Consider the partial sum $S_k = \mathbb{I}_m + Q + Q^2 + \cdots + Q^{k-1}$ for $k \geq 1$. Multiplying by $\mathbb{I}_m - Q$ gives a telescoping sum:

$$\begin{aligned}(\mathbb{I}_m - Q)S_k &= (\mathbb{I}_m - Q)(\mathbb{I}_m + Q + Q^2 + \cdots + Q^{k-1}) \\ &= (\mathbb{I}_m + Q + Q^2 + \cdots + Q^{k-1}) - (Q + Q^2 + \cdots + Q^{k-1} + Q^k) \\ &= \mathbb{I}_m - Q^k\end{aligned}$$

First, suppose $(\mathbb{I}_m - Q)\boldsymbol{x} = \mathbf{0}$. Then $\boldsymbol{x} = Q\boldsymbol{x} = Q^2\boldsymbol{x} = \cdots = Q^k\boldsymbol{x} \to \mathbf{0}$ as $k \to \infty$ since $\rho(Q) < 1$. Hence $\text{null}(\mathbb{I}_m - Q) = \{\mathbf{0}\}$, so $\mathbb{I}_m - Q$ is invertible
Multiplying $(\mathbb{I}_m - Q)S_k = \mathbb{I}_m - Q^k$ on the left by $(\mathbb{I}_m - Q)^{-1}$ gives

$$S_k = (\mathbb{I}_m - Q)^{-1}(\mathbb{I}_m - Q^k) = (\mathbb{I}_m - Q)^{-1} - (\mathbb{I}_m - Q)^{-1}Q^k$$

Taking the limit as $k \to \infty$ and using $\lim_{k\to\infty} Q^k = \mathbf{0}_m$ yields

$$\sum_{j=0}^{\infty} Q^j = \lim_{k\to\infty} S_k = (\mathbb{I}_m - Q)^{-1} \qquad \blacksquare$$

The fundamental matrix and absorption quantities

Definition 143 (Fundamental matrix)

For an absorbing Markov chain with canonical transient block $Q \in \mathcal{M}_m$, the matrix

$$N = (\mathbb{I}_m - Q)^{-1} = \mathbb{I}_m + Q + Q^2 + \cdots \in \mathcal{M}_m$$

is called the ***fundamental matrix*** *of the absorbing Markov chain*

Theorem 144 (Quantitative analysis of absorbing Markov chains)

Let $P = \begin{pmatrix} \mathbb{I}_r & R \\ \mathbf{0}_{m,r} & Q \end{pmatrix}$ *be in canonical form with fundamental matrix* $N = [n_{ij}] = (\mathbb{I}_m - Q)^{-1}$. *Then:*

1. ***Expected visits to transient states:*** n_{ij} *is the expected total number of times the chain is in the ith transient state, given that it starts in the jth transient state*
2. ***Expected time to absorption:*** *Let* $\boldsymbol{t}^T = \mathbb{1}_m^T N = (t_1, \ldots, t_m)$. *Then* $t_j = \sum_{i=1}^{m} n_{ij}$ *(the jth column sum of N) is the expected number of steps before*

Proof of Theorem 144

1. Expected visits $N = (\mathbb{I}_m - Q)^{-1}$: Starting in transient state j, let $X_{ij}^{(k)} \in \{0, 1\}$ be the indicator random variable that equals 1 if the chain is in transient state i at step $k \geq 0$, and 0 otherwise. Then $\mathbb{E}[X_{ij}^{(k)}] = \mathbb{P}(\text{in transient state } i \text{ at step } k \mid \text{start in } j) = (Q^k)_{ij}$. By linearity of expectation, the expected total number of visits to transient state i is

$$\mathbb{E}\left[\sum_{k=0}^{\infty} X_{ij}^{(k)}\right] = \sum_{k=0}^{\infty} (Q^k)_{ij} = \left(\sum_{k=0}^{\infty} Q^k\right)_{ij} = n_{ij}$$

2. Time to absorption $t^T = \mathbb{1}_m^T N$: The total number of steps spent in *all* transient states before absorption starting from j is the sum over all $i = 1, \ldots, m$ of visits to transient state i:

$$t_j = \sum_{i=1}^{m} n_{ij} = (\mathbb{1}_m^T N)_j$$

3. Absorption probabilities $B = RN$: Taking $k \to \infty$ in Proposition 139 gives

$$\lim_{k \to \infty} P^k = \begin{pmatrix} \mathbb{I}_r & R\sum_{j=0}^{\infty} Q^j \\ \mathbf{0}_{m,r} & \lim_{k \to \infty} Q^k \end{pmatrix} = \begin{pmatrix} \mathbb{I}_r & RN \\ \mathbf{0}_{m,r} & \mathbf{0}_m \end{pmatrix} = \begin{pmatrix} \mathbb{I}_r & B \\ \mathbf{0}_{m,r} & \mathbf{0}_m \end{pmatrix} \quad \blacksquare$$

Absorbing Markov chains

- Absorbing states and motivating examples
- Canonical block form and reducibility
- The Neumann series and fundamental matrix
- **Examples and R implementation**

Example 1 – Back to the genetic example

In the modified orange cat breeding chain ($S_1 = X^O X^O$, $S_2 = X^o X^o$, $S_3 = X^O X^o$), state S_1 is absorbing ($r = 1$) and S_2, S_3 are transient ($m = 2$). The matrix is already in canonical form:

$$P = \begin{pmatrix} 1 & 0 & 1/2 \\ 0 & 0 & 0 \\ 0 & 1 & 1/2 \end{pmatrix} = \begin{pmatrix} \mathbb{I}_1 & R \\ \mathbf{0}_{2,1} & Q \end{pmatrix} \quad \text{with} \quad R = \begin{pmatrix} 0 & 1/2 \end{pmatrix}, \quad Q = \begin{pmatrix} 0 & 0 \\ 1 & 1/2 \end{pmatrix}$$

We compute $\mathbb{I}_2 - Q$ and invert it using $\begin{pmatrix} a & b \\ c & d \end{pmatrix}^{-1} = \frac{1}{ad-bc} \begin{pmatrix} d & -b \\ -c & a \end{pmatrix}$:

$$\mathbb{I}_2 - Q = \begin{pmatrix} 1 & 0 \\ -1 & 1/2 \end{pmatrix} \implies N = (\mathbb{I}_2 - Q)^{-1} = \frac{1}{1/2} \begin{pmatrix} 1/2 & 0 \\ 1 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 2 & 2 \end{pmatrix}$$

Interpreting Example 1

The fundamental matrix for transient states $(S_2, S_3) = (X^o X^o, X^O X^o)$ is

$$N = \begin{pmatrix} 1 & 0 \\ 2 & 2 \end{pmatrix}$$

- Starting with a black female (S_2, column 1 of N): expected visits are $n_{11} = 1$ generation in S_2 (at $t = 0$) and $n_{21} = 2$ generations in S_3 (tortoiseshell)
- Starting with a tortoiseshell female (S_3, column 2 of N): $n_{12} = 0$ generations in S_2 and $n_{22} = 2$ generations in S_3

Expected number of generations before absorption into S_1 ($X^O X^O$):

$$\boldsymbol{t}^T = \mathbb{1}_2^T N = \begin{pmatrix} 1 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 \\ 2 & 2 \end{pmatrix} = \begin{pmatrix} 3 & 2 \end{pmatrix}$$

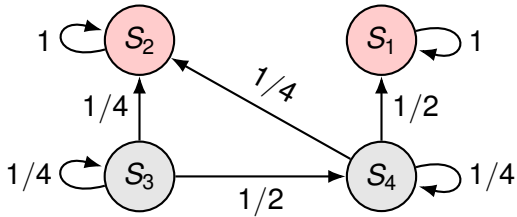
Absorption probability matrix (into the sole absorbing state S_1):

$$B = RN = \begin{pmatrix} 0 & 1/2 \end{pmatrix} \begin{pmatrix} 1 & 0 \\ 2 & 2 \end{pmatrix} = \begin{pmatrix} 1 & 1 \end{pmatrix}$$

Example 2 – 4-state chain with 2 absorbing states

Consider a 4-state absorbing chain where S_1 (pass/graduated) and S_2 (withdraw) are absorbing ($r = 2$), and S_3 (Year 1) and S_4 (Year 2) are transient ($m = 2$):

$$P = \begin{array}{c} \\ S_1 \\ S_2 \\ S_3 \\ S_4 \end{array} \begin{array}{c} \begin{array}{cccc} S_1 & S_2 & S_3 & S_4 \end{array} \\ \left(\begin{array}{cc|cc} 1 & 0 & 0 & 1/2 \\ 0 & 1 & 1/4 & 1/4 \\ \hline 0 & 0 & 1/4 & 0 \\ 0 & 0 & 1/2 & 1/4 \end{array}\right) \end{array}$$



Here the canonical blocks are

$$R = \begin{pmatrix} 0 & 1/2 \\ 1/4 & 1/4 \end{pmatrix} \in \mathcal{M}_2, \qquad Q = \begin{pmatrix} 1/4 & 0 \\ 1/2 & 1/4 \end{pmatrix} \in \mathcal{M}_2$$

Example 2 – Hand calculation of N, $\boldsymbol{t}^T$ and B

Step 1 (Fundamental matrix N): Compute $\mathbb{I}_2 - Q$ and its determinant:

$$\mathbb{I}_2 - Q = \begin{pmatrix} 3/4 & 0 \\ -1/2 & 3/4 \end{pmatrix}, \qquad \det(\mathbb{I}_2 - Q) = \frac{3}{4} \cdot \frac{3}{4} - 0 = \frac{9}{16}$$

Inverting via the 2×2 inverse formula gives

$$N = (\mathbb{I}_2 - Q)^{-1} = \frac{16}{9} \begin{pmatrix} 3/4 & 0 \\ 1/2 & 3/4 \end{pmatrix} = \begin{pmatrix} 4/3 & 0 \\ 8/9 & 4/3 \end{pmatrix}$$

Step 2 (Expected steps to absorption $\boldsymbol{t}^T = \mathbb{1}_2^T N$):

$$\boldsymbol{t}^T = \begin{pmatrix} 1 & 1 \end{pmatrix} \begin{pmatrix} 4/3 & 0 \\ 8/9 & 4/3 \end{pmatrix} = \begin{pmatrix} 20/9 & 4/3 \end{pmatrix} \quad \text{(starting from } S_3 \text{ and } S_4\text{)}$$

Step 3 (Absorption probabilities $B = RN$):

$$B = RN = \begin{pmatrix} 0 & 1/2 \\ 1/4 & 1/4 \end{pmatrix} \begin{pmatrix} 4/3 & 0 \\ 8/9 & 4/3 \end{pmatrix} = \begin{pmatrix} 4/9 & 2/3 \\ 5/9 & 1/3 \end{pmatrix}$$

Notice each column of B sums to 1: starting in S_3, a student graduates (S_1) with probability $4/9$ and withdraws (S_2) with probability $5/9$

Example 3 – Symmetric 5-state random walk

Return to Motivating example 1 on $\{S_1, S_2, S_3, S_4, S_5\}$ with $p = q = 1/2$ and absorbing walls S_1, S_5. Reordering states as $(S_1, S_5, S_2, S_3, S_4)$ puts P in canonical form with $r = 2$ and $m = 3$:

$$R = \begin{array}{c} \\ S_1 \\ S_5 \end{array} \begin{array}{c} \begin{array}{ccc} S_2 & S_3 & S_4 \end{array} \\ \begin{pmatrix} 1/2 & 0 & 0 \\ 0 & 0 & 1/2 \end{pmatrix} \end{array}, \qquad Q = \begin{array}{c} \\ S_2 \\ S_3 \\ S_4 \end{array} \begin{array}{c} \begin{array}{ccc} S_2 & S_3 & S_4 \end{array} \\ \begin{pmatrix} 0 & 1/2 & 0 \\ 1/2 & 0 & 1/2 \\ 0 & 1/2 & 0 \end{pmatrix} \end{array}$$

Inverting $\mathbb{I}_3 - Q = \begin{pmatrix} 1 & -1/2 & 0 \\ -1/2 & 1 & -1/2 \\ 0 & -1/2 & 1 \end{pmatrix}$ gives

$$N = (\mathbb{I}_3 - Q)^{-1} = \begin{pmatrix} 3/2 & 1 & 1/2 \\ 1 & 2 & 1 \\ 1/2 & 1 & 3/2 \end{pmatrix}, \quad \boldsymbol{t}^T = \mathbb{1}_3^T N = \begin{pmatrix} 3 & 4 & 3 \end{pmatrix}, \quad B = RN = \begin{pmatrix} 3/4 & 1/2 \\ 1/4 & 1/2 \end{pmatrix}$$

R implementation – Direct matrix calculation

We can verify Example 3 directly in base R using `solve`:

```
R_mat <- matrix(c(0.5, 0,   0,
                  0,   0,   0.5), nrow = 2, byrow = TRUE)
Q_mat <- matrix(c(0,   0.5, 0,
                  0.5, 0,   0.5,
                  0,   0.5, 0),   nrow = 3, byrow = TRUE)
colnames(R_mat) <- colnames(Q_mat) <- rownames(Q_mat) <- c("S2", "S3", "S4")
rownames(R_mat) <- c("S1", "S5")

N_mat <- solve(diag(3) - Q_mat)
t_vec <- colSums(N_mat)
B_mat <- R_mat %*% N_mat
N_mat

##     S2 S3  S4
## S2 1.5  1 0.5
## S3 1.0  2 1.0
```

Using the `markovchain` package

Note that `markovchain` uses row-stochastic matrices, so we pass `t(P_abs)`

```r
library(markovchain)
P_abs <- matrix(c(1, 0, 0.5,
                  0, 0, 0,
                  0, 1, 0.5), nrow = 3, byrow = TRUE)
mc_abs <- new("markovchain",
              states = sprintf("S_%d", 1:3),
              transitionMatrix = t(P_abs),
              name = "orange_abs")
summary(mc_abs)
```

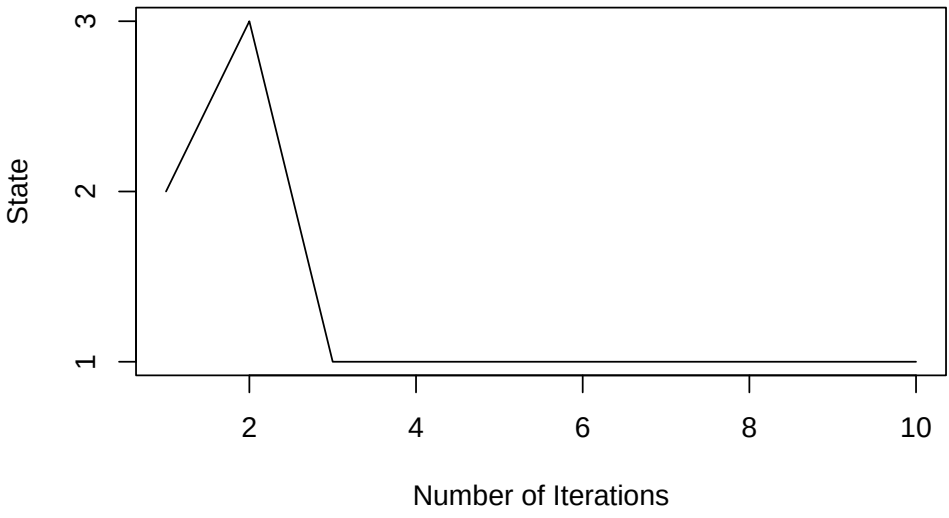
```
## orange_abs  Markov chain that is composed by:
## Closed classes:
## S_1
## Recurrent classes:
## {S_1}
## Transient classes:
```

Canonical form, absorption times and probabilities in R

```
canonicForm(mc_abs)

## orange_abs
##  A  3 - dimensional discrete Markov Chain defined by the following states
##  S_1, S_2, S_3
##  The transition matrix  (by rows)  is defined as follows:
##     S_1 S_2 S_3
## S_1 1.0   0 0.0
## S_2 0.0   0 1.0
## S_3 0.5   0 0.5

meanAbsorptionTime(mc_abs)

## S_2 S_3
##   3   2

absorptionProbabilities(mc_abs)
```

Simulating a realisation with DTMCPack

```
library(DTMCPack)
set.seed(2740)
IC <- c(0, 1, 0) # Start in S_2 (black female)
sol <- DTMC(t(P_abs), IC, 10, trace = TRUE)

## NULL
```


State
3
2
1
2
4
6
8
10
Number of Iterations

Absorbing Markov chains

Spectral centrality in networks

PageRank and the Google matrix

Spectral centrality in networks

Degree and eigenvector centrality

Issue with directed graphs and Katz centrality

Why spectral centrality?

Combinatorial centrality measures on graphs are often based on counting shortest paths (such as closeness and betweenness centrality)

Computational bottleneck on massive networks:

- All-pairs shortest-path algorithms require $O(nm)$ or $O(n^3)$ operations
- For a social network or web graph with $n \sim 10^8$ vertices, $n^2 \sim 10^{16}$ is completely intractable

The spectral alternative:

- Express vertex importance via matrix-vector products $A\boldsymbol{x}$, eigenvectors $A\boldsymbol{x} = \lambda\boldsymbol{x}$ or linear systems $(\mathbb{I} - \alpha A)\boldsymbol{x} = \boldsymbol{b}$
- When the adjacency matrix A is **sparse** ($m = |E| \ll n^2$ non-zero entries), one sparse matrix-vector product $A\boldsymbol{x}$ takes only $O(m)$ time!

Convention for directed adjacency matrices

To remain consistent with our column-stochastic Markov chain convention $\boldsymbol{p}(t+1) = P\boldsymbol{p}(t)$ where column j records transitions *from* j *to* i, we use the following convention for a directed graph $\mathcal{G} = (V, E)$ on n vertices:

$$a_{ij} = \begin{cases} 1 & \text{if there is a directed arc } j \to i \text{ (from } j \text{ to } i\text{)} \\ 0 & \text{otherwise} \end{cases}$$

With this convention:

- **Row** i of A records arcs *entering* vertex i, so the **in-degree** of vertex i is $d_i^{\text{in}} = \sum_{j=1}^{n} a_{ij} = (A\mathbb{1}_n)_i$
- **Column** j of A records arcs *leaving* vertex j, so the **out-degree** of vertex j is $d_j^{\text{out}} = \sum_{i=1}^{n} a_{ij} = (\mathbb{1}_n^T A)_j$
- For an undirected graph, $A = A^T$ is symmetric and $d_i^{\text{in}} = d_i^{\text{out}} = d_i$

First-order measure – In-degree centrality

The simplest measure of prestige in a directed network is **in-degree centrality**:

$$\boldsymbol{d}^{\text{in}} = A\mathbb{1}_n, \qquad d_i^{\text{in}} = \sum_{j=1}^{n} a_{ij}$$

Each arc $j \to i$ is treated as one "vote" for the importance of vertex i

Fundamental limitation:

- In-degree treats all incoming links as equal
- Receiving a link from a major hub or authoritative page counts the exact same $(+1)$ as receiving a link from an isolated spam page
- We want a **recursive** notion of prestige: *a vertex is important if it is pointed to by other important vertices*

Eigenvector centrality

Instead of summing 1 for each incoming neighbour $j \to i$, we sum the *importance scores* x_j of those neighbours and require x_i to be proportional to that sum

Definition 145 (Eigenvector centrality)

*Let $A = [a_{ij}] \in \mathcal{M}_n$ be the adjacency matrix of a graph or digraph (with $a_{ij} = 1$ if $j \to i$). The **eigenvector centrality** (or **prestige score**) of vertex i is the ith entry of the non-negative eigenvector $\boldsymbol{x} \geq \mathbf{0}$ satisfying*

$$\lambda x_i = \sum_{j=1}^{n} a_{ij} x_j, \quad i = 1, \ldots, n \iff A\boldsymbol{x} = \lambda \boldsymbol{x}$$

associated with the dominant eigenvalue $\lambda = \rho(A)$

Notice that $x_i = \frac{1}{\lambda} \sum_{j \in \mathcal{N}^{\text{in}}(i)} x_j$ depends on both the **quantity** of in-neighbours $|\mathcal{N}^{\text{in}}(i)|$ and the **quality** x_j of each in-neighbour

Why Perron-Frobenius theory is essential here

For $\boldsymbol{x}$ to be a valid centrality ranking, its entries must be **non-negative** ($x_i \geq 0$) and uniquely determined (up to normalisation $\mathbb{1}_n^T \boldsymbol{x} = 1$)

Theorem 146 (Existence and uniqueness of eigenvector centrality)

Let $\mathcal{G} = (V, E)$ be a strongly connected digraph (or connected undirected graph) with adjacency matrix $A \in \mathcal{M}_n$. Then A is irreducible and non-negative, so by the Perron-Frobenius theorem (Unit 08):

1. *The spectral radius $\lambda_1 = \rho(A) > 0$ is a simple eigenvalue of A*
2. *There exists a unique eigenvector $\boldsymbol{x} > \mathbf{0}$ (the **Perron eigenvector**) satisfying*

$$A\boldsymbol{x} = \rho(A)\boldsymbol{x} \quad \text{and} \quad \mathbb{1}_n^T \boldsymbol{x} = \sum_{i=1}^{n} x_i = 1$$

3. *No other eigenvalue has a non-negative eigenvector*

Constructing a stochastic matrix from an adjacency matrix

Let $A = [a_{ij}] \in \mathcal{M}_n$ be the adjacency matrix of a simple undirected graph $\mathcal{G} = (V, E)$ with no isolated vertices, and let $D = \text{diag}(d_{11}, \ldots, d_{nn})$ be its degree matrix, with diagonal entries

$$d_{ii} = \sum_{j=1}^{n} a_{ji} = \sum_{j=1}^{n} a_{ij} > 0$$

Proposition 147 (Random walk transition matrix on an undirected graph)

*The matrix $P = AD^{-1}$ is **column-stochastic**, and defines the transition matrix of a random walk on $\mathcal{G}$ where a walker at vertex j moves to a uniformly chosen neighbour $i \in \mathcal{N}(j)$ with probability $1/d_{jj}$*

Proof of Proposition 147

Write the entries of the diagonal matrix D^{-1} as d_{ij}^{-1}, where $d_{ii}^{-1} = 1/d_{ii}$ for $1 \leq i \leq n$ and $d_{ij}^{-1} = 0$ for $i \neq j$

By matrix multiplication, the (i, j) entry of AD^{-1} is

$$(AD^{-1})_{ij} = \sum_{k=1}^{n} a_{ik} d_{kj}^{-1} = a_{ij} d_{jj}^{-1} = \frac{a_{ij}}{d_{jj}} \geq 0, \qquad i, j = 1, \ldots, n$$

Therefore, the sum of column j of AD^{-1} is

$$\begin{aligned} \sum_{i=1}^{n} a_{ij} d_{jj}^{-1} &= d_{jj}^{-1} \sum_{i=1}^{n} a_{ij} \\ &= d_{jj}^{-1} d_{jj} = 1 \end{aligned}$$

Hence every column sums to 1 and AD^{-1} is column-stochastic ■

Computing eigenvector centrality – The power method

For large networks, computing the full characteristic polynomial is impossible. Instead, we compute the dominant eigenpair $(\lambda_1, \boldsymbol{x})$ using the **power method**: Start with an initial positive vector $\boldsymbol{x}^{(0)} > \mathbf{0}$ (such as $\boldsymbol{x}^{(0)} = \frac{1}{n}\mathbb{1}_n$). For $k = 1, 2, \ldots$:

1. Multiply by A: $\boldsymbol{y}^{(k)} = A\boldsymbol{x}^{(k-1)}$
2. Normalise (using the ℓ_1 norm $\|\boldsymbol{y}^{(k)}\|_1 = \mathbb{1}_n^T \boldsymbol{y}^{(k)}$ or the maximum-magnitude entry $m(\boldsymbol{y}^{(k)})$):

$$\boldsymbol{x}^{(k)} = \frac{\boldsymbol{y}^{(k)}}{\|\boldsymbol{y}^{(k)}\|_1}$$

until $\|\boldsymbol{x}^{(k)} - \boldsymbol{x}^{(k-1)}\|_1 < \varepsilon$

Theorem 148 (Convergence of the power method)

If A is primitive (so $\lambda_1 = \rho(A) > |\lambda_2| \geq \cdots \geq |\lambda_n|$), then $\boldsymbol{x}^{(k)} \to \boldsymbol{x}$ as $k \to \infty$, and the error decays at the geometric rate

$$\|\boldsymbol{x}^{(k)} - \boldsymbol{x}\| = O\left(\left|\frac{\lambda_2}{\lambda_1}\right|^k\right)$$

Spectral centrality in networks

Degree and eigenvector centrality

Issue with directed graphs and Katz centrality

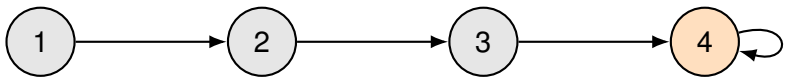
What goes wrong on general directed graphs?

Real-world directed networks (citation graphs, web graphs, DAGs) are rarely strongly connected! When $\mathcal{G}(A)$ is **reducible**, eigenvector centrality suffers from a severe pathology:

- Suppose vertex i is a **source node** (in-degree $d_i^{\text{in}} = 0$, so nobody points to i)
- Then row i of A is all zeros, so the eigenvector equation $A\boldsymbol{x} = \lambda \boldsymbol{x}$ with $\lambda > 0$ gives

$$\lambda x_i = \sum_{j=1}^{n} a_{ij} x_j = 0 \implies x_i = 0$$

- **Domino effect:** If vertex k only receives links from source nodes with $x_i = 0$, then $\lambda x_k = \sum_{j \to k} x_j = 0 \implies x_k = 0$ as well!



Here $x_1 = 0 \implies x_2 = 0 \implies x_3 = 0$, and only vertex 4 gets non-zero eigenvector centrality ($x_4 = 1$)!

Katz centrality (1953) – Baseline “free” importance

Leo Katz (1953) resolved the zero-propagation problem on directed graphs by giving every vertex a small baseline “free” importance $\beta > 0$, plus damped importance $\alpha > 0$ from its in-neighbours

Definition 149 (Katz centrality)

*Let $A \in \mathcal{M}_n$ be the adjacency matrix of a directed graph, $\alpha > 0$ be an **attenuation factor** (or damping parameter) and $\beta > 0$ be a baseline constant. The **Katz centrality** vector $\boldsymbol{x} \in \mathbb{R}^n$ is the solution to*

$$x_i = \alpha \sum_{j=1}^{n} a_{ij} x_j + \beta, \quad i = 1, \ldots, n \iff \boldsymbol{x} = \alpha A \boldsymbol{x} + \beta \mathbb{1}_n$$

Rearranging terms gives the linear system

$$(\mathbb{I}_n - \alpha A)\boldsymbol{x} = \beta \mathbb{1}_n$$

Since β only scales $\boldsymbol{x}$ by a positive constant, we typically set $\beta = 1$ (or normalise $\boldsymbol{x}$

Neumann series and walk-counting interpretation of Katz centrality

When is $\mathbb{I}_n - \alpha A$ guaranteed to be invertible with a strictly positive solution $\boldsymbol{x} > \mathbf{0}$?

Theorem 150 (Existence and walk-counting expansion of Katz centrality)

Let $A \in \mathcal{M}_n$ be a non-negative adjacency matrix and suppose $0 < \alpha < \frac{1}{\rho(A)}$ and $\beta > 0$. Then $\rho(\alpha A) = \alpha\rho(A) < 1$, so by Theorem 142 (the Neumann series), $\mathbb{I}_n - \alpha A$ is invertible and

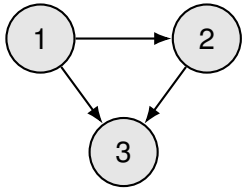
$$\boldsymbol{x} = \beta(\mathbb{I}_n - \alpha A)^{-1}\mathbb{1}_n = \beta\left(\sum_{k=0}^{\infty} \alpha^k A^k\right)\mathbb{1}_n = \beta\left(\mathbb{1}_n + \alpha A\mathbb{1}_n + \alpha^2 A^2\mathbb{1}_n + \cdots\right) > \mathbf{0}$$

Walk-counting interpretation:

- Recall from Unit 07 that $(A^k)_{ij}$ is the number of directed walks of length k from j to i
- Therefore $(A^k\mathbb{1}_n)_i = \sum_{j=1}^{n}(A^k)_{ij}$ is the total number of directed walks of length k from *any* vertex j terminating at vertex i
- Katz centrality sums walks of **all lengths** $k \geq 0$ ending at i, discounted by α^k:

Example – Katz centrality on a 3-vertex digraph

Consider the 3-vertex directed graph with arcs $1 \to 2$, $1 \to 3$ and $2 \to 3$:



With $a_{ij} = 1$ if $j \to i$, the adjacency matrix is strictly lower triangular:

$$A = \begin{pmatrix} 0 & 0 & 0 \\ 1 & 0 & 0 \\ 1 & 1 & 0 \end{pmatrix}$$

Here $\rho(A) = 0$, so any $\alpha > 0$ satisfies $\rho(\alpha A) = 0 < 1$

Let us compute Katz centrality with $\alpha = 1/2$ and $\beta = 1$ by solving $(\mathbb{I}_3 - \frac{1}{2}A)\boldsymbol{x} = \mathbb{1}_3$:

$$\mathbb{I}_3 - \frac{1}{2}A = \begin{pmatrix} 1 & 0 & 0 \\ -1/2 & 1 & 0 \\ -1/2 & -1/2 & 1 \end{pmatrix} \implies \begin{cases} x_1 = 1 \\ x_2 = \frac{1}{2}x_1 + 1 = \frac{3}{2} \\ x_3 = \frac{1}{2}x_1 + \frac{1}{2}x_2 + 1 = \frac{1}{2} + \frac{3}{4} + 1 = \frac{9}{4} \end{cases}$$

Normalising $\boldsymbol{x} = (1, 3/2, 9/4)^T = \frac{1}{4}(4, 6, 9)^T$ by $\mathbb{1}_3^T \boldsymbol{x} = 19/4$ gives $\tilde{\boldsymbol{x}} = (\frac{4}{19}, \frac{6}{19}, \frac{9}{19})^T$, ranking $3 \succ 2 \succ 1$

Absorbing Markov chains

Spectral centrality in networks

PageRank and the Google matrix

PageRank and the Google matrix

Motivation and vote splitting

Obstacles on real web graphs and the Google matrix

Linear system formulation and examples

What makes an important webpage?

In the early days of the World Wide Web (mid-1990s), search engines ranked pages primarily by keyword frequency on the page itself, which was easily manipulated

In 1998, Sergey Brin, Larry Page, Rajeev Motwani and Terry Winograd introduced **PageRank**, the algorithm at the heart of Google

Core idea: Use the directed hyperlink structure of the Web to measure authority

- Webpages are vertices $V = \{1, \ldots, n\}$; a hyperlink from page j to page i is a directed arc $j \to i$
- A link $j \to i$ is a recommendation or "vote" from page j for the importance of page i
- **Random surfer model:** Imagine a web surfer who clicks links at random from page to page. The PageRank of page i is the long-run proportion of time the random surfer spends on page i

From Katz centrality to vote splitting

Why not simply use Katz or eigenvector centrality on the Web adjacency matrix A?

The "generous hub" flaw of Katz/eigenvector centrality:

- In Katz centrality ($x_i = \alpha \sum_{j \to i} x_j + \beta$), if an important directory page j links to 10,000 pages, *every single one* of those 10,000 pages receives αx_j importance!
- A page with more outgoing links exerts more total influence on the network

Brin and Page's vote-splitting principle:

- Each page j has a single unit of voting power equal to its own importance x_j
- If page j has $d_j^{\text{out}} > 0$ outgoing links, it splits its importance **equally** among them, passing x_j / d_j^{out} along each outgoing link:

$$x_i = \sum_{j \to i} \frac{x_j}{d_j^{\text{out}}}$$

The hyperlink transition matrix

Let $A = [a_{ij}] \in \mathcal{M}_n$ be the directed adjacency matrix ($a_{ij} = 1$ if page j links to page i, with no self-loops). Suppose first that every page has at least one outgoing link ($d_j^{\text{out}} = \sum_{i=1}^{n} a_{ij} \geq 1$)

Define the diagonal out-degree matrix $D^{\text{out}} = \text{diag}(d_1^{\text{out}}, \ldots, d_n^{\text{out}})$ and the **hyperlink matrix** $P = [p_{ij}] \in \mathcal{M}_n$:

$$p_{ij} = \begin{cases} \dfrac{1}{d_j^{\text{out}}} & \text{if page } j \text{ links to page } i \\ 0 & \text{otherwise} \end{cases} \qquad \Longleftrightarrow \qquad P = A(D^{\text{out}})^{-1}$$

Proposition 151

*If $d_j^{out} \geq 1$ for all $j = 1, \ldots, n$, then $P = A(D^{out})^{-1}$ is **column-stochastic**, and the vote-splitting equations $x_i = \sum_{j \to i} x_j / d_j^{out}$ take the fixed-point form*

$$P\boldsymbol{x} = \boldsymbol{x}$$

PageRank and the Google matrix

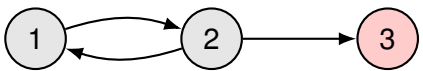
- Motivation and vote splitting
- **Obstacles on real web graphs and the Google matrix**
- Linear system formulation and examples

Two linear algebra obstacles on real web graphs

If we try to solve $P\boldsymbol{x} = \boldsymbol{x}$ directly on the real Web, two fatal problems occur:

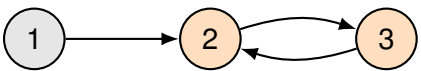
Obstacle 1: Dangling nodes ($d_j^{\text{out}} = 0$)

- Many pages (PDF files, images, dead-end pages) have **no outgoing links**: $d_j^{\text{out}} = 0$
- Column j of the raw hyperlink matrix is **0**, so probability mass "leaks" out of the network and P is not stochastic!



Obstacle 2: Reducibility and spider traps

- The Web is not strongly connected ($\mathcal{G}(P)$ is reducible) and contains **spider traps** (closed strongly connected components) and periodic cycles
- $P\boldsymbol{x} = \boldsymbol{x}$ may have multiple linearly independent solutions, assign $x_i = 0$ outside traps, or cause power iteration to oscillate!



Fixing Obstacle 1 – Handling dangling nodes

Before forming the Google matrix, we must ensure that P is genuinely column-stochastic ($\mathbb{1}_n^T P = \mathbb{1}_n^T$)

Whenever a vertex j is a **dangling node** ($d_j^{\text{out}} = 0$, giving a zero column in the raw hyperlink matrix), there are two standard conventions:

1. **Uniform teleportation from dead ends:** Replace the zero column j with the uniform probability vector $\frac{1}{n}\mathbb{1}_n = (1/n, \ldots, 1/n)^T$ (a random surfer reaching a dead-end page picks any page on the Web uniformly at random)
2. **Self-loop at dead ends:** Add a self-loop $j \to j$ so that $p_{jj} = 1$ (column j becomes $\boldsymbol{e}_j$)

In both conventions, every column of the resulting matrix P is a valid probability vector ($P \geq 0$ and $\mathbb{1}_n^T P = \mathbb{1}_n^T$). From now on, $P \in \mathcal{M}_n$ denotes this column-stochastic web transition matrix

Fixing Obstacle 2 – Teleportation and the Google matrix

To overcome reducibility, spider traps and periodicity, Brin and Page introduced a **damping factor** (or teleportation parameter) $\alpha \in (0, 1)$, typically chosen as $\alpha = 0.85$:

- With probability α, the random surfer follows an outgoing link on the current page (according to P)
- With probability $1 - \alpha$, the surfer gets bored and jumps (teleports) to a page chosen uniformly at random from all n pages (according to $\frac{1}{n}\mathbb{1}_n$)

Definition 152 (Google matrix)

*Let $P \in \mathcal{M}_n$ be a column-stochastic transition matrix and let $\alpha \in (0, 1)$ be a damping factor. The **Google matrix** is the $n \times n$ matrix*

$$G = \alpha P + (1 - \alpha)\frac{1}{n}\mathbb{1}_n\mathbb{1}_n^T = \alpha P + \frac{1 - \alpha}{n}\mathbb{J}_n$$

where $\mathbb{J}_n = \mathbb{1}_n\mathbb{1}_n^T \in \mathcal{M}_n$ is the $n \times n$ matrix of all ones

Existence, uniqueness and convergence of PageRank

Theorem 153 (PageRank theorem)

Let $P \in \mathcal{M}_n$ be a column-stochastic matrix and let $\alpha \in (0,1)$. Then the Google matrix

$$G = \alpha P + (1-\alpha)\frac{1}{n}\mathbb{1}_n\mathbb{1}_n^T$$

*is **column-stochastic** and **strictly positive** ($G > 0$), hence **primitive**. By Perron's theorem (Unit 08):*

1. *The spectral radius $\lambda_1 = \rho(G) = 1$ is a simple, strictly dominant eigenvalue of G (in fact, $|\lambda| \leq \alpha < 1$ for every other eigenvalue of G)*
2. *There exists a unique stationary probability vector $\boldsymbol{x} > \mathbf{0}$ (the **PageRank vector**) satisfying*
$$G\boldsymbol{x} = \boldsymbol{x} \quad \text{and} \quad \mathbb{1}_n^T\boldsymbol{x} = 1$$
3. *For any initial probability distribution $\boldsymbol{x}^{(0)}$, the power iteration $\boldsymbol{x}^{(k)} = G^k\boldsymbol{x}^{(0)}$ converges to $\boldsymbol{x}$ as $k \to \infty$ at the geometric rate $O(\alpha^k)$*

Proof of Theorem 153

1. Strict positivity ($G > 0$): Since $P \geq 0$ and $\alpha \in (0, 1)$, we have $\alpha P \geq 0$. Every entry of $\frac{1-\alpha}{n}\mathbb{J}_n$ equals $\frac{1-\alpha}{n} > 0$. Therefore, every entry of G satisfies

$$g_{ij} = \alpha p_{ij} + \frac{1-\alpha}{n} \geq \frac{1-\alpha}{n} > 0$$

so $G > 0$ is strictly positive (and therefore irreducible and primitive, as $G^1 > 0$)

2. Column-stochasticity ($\mathbb{1}_n^T G = \mathbb{1}_n^T$): Since $\mathbb{1}_n^T P = \mathbb{1}_n^T$ and $\mathbb{1}_n^T \mathbb{1}_n = n$, we compute

$$\mathbb{1}_n^T G = \alpha(\mathbb{1}_n^T P) + \frac{1-\alpha}{n}(\mathbb{1}_n^T \mathbb{1}_n)\mathbb{1}_n^T = \alpha \mathbb{1}_n^T + \frac{1-\alpha}{n} n \mathbb{1}_n^T = (\alpha + 1 - \alpha)\mathbb{1}_n^T = \mathbb{1}_n^T$$

Thus G is column-stochastic, which implies $\rho(G) = 1$

3. Application of Perron's theorem: Since $G > 0$ is a positive column-stochastic matrix, Perron's theorem guarantees that $\lambda_1 = \rho(G) = 1$ is simple and strictly dominant, with a unique positive eigenvector $\boldsymbol{x} > \mathbf{0}$ normalised by $\mathbb{1}_n^T \boldsymbol{x} = 1$, and $\lim_{k\to\infty} G^k \boldsymbol{x}^{(0)} = \boldsymbol{x}$ ■

PageRank and the Google matrix

- Motivation and vote splitting
- Obstacles on real web graphs and the Google matrix
- **Linear system formulation and examples**

Reformulating PageRank as a linear system

Although $G\boldsymbol{x} = \boldsymbol{x}$ is an eigenvalue problem for the eigenvalue $\lambda = 1$, we can use the normalisation condition $\mathbb{1}_n^T \boldsymbol{x} = 1$ to turn it into a **non-homogeneous linear system** $(\mathbb{I}_n - \alpha P)\boldsymbol{x} = \frac{1-\alpha}{n}\mathbb{1}_n$!

Theorem 154 (Linear system formulation of PageRank)

Let $P \in \mathcal{M}_n$ be column-stochastic and $\alpha \in (0, 1)$. A vector $\boldsymbol{x} \in \mathbb{R}^n$ with $\mathbb{1}_n^T \boldsymbol{x} = 1$ satisfies $G\boldsymbol{x} = \boldsymbol{x}$ if and only if it satisfies the linear system

$$(\mathbb{I}_n - \alpha P)\boldsymbol{x} = \frac{1-\alpha}{n}\mathbb{1}_n$$

Moreover, $\mathbb{I}_n - \alpha P$ is invertible and its inverse is given by the Neumann series $(\mathbb{I}_n - \alpha P)^{-1} = \sum_{k=0}^{\infty} \alpha^k P^k$, so the unique PageRank vector is

$$\boldsymbol{x} = \frac{1-\alpha}{n}(\mathbb{I}_n - \alpha P)^{-1}\mathbb{1}_n$$

Proof of Theorem 154

1. Equivalence with $G\boldsymbol{x} = \boldsymbol{x}$: For any vector $\boldsymbol{x}$ satisfying $\mathbb{1}_n^T \boldsymbol{x} = 1$, associativity of matrix multiplication gives

$$\mathbb{J}_n \boldsymbol{x} = (\mathbb{1}_n \mathbb{1}_n^T)\boldsymbol{x} = \mathbb{1}_n(\mathbb{1}_n^T \boldsymbol{x}) = \mathbb{1}_n(1) = \mathbb{1}_n$$

Substituting this into $G\boldsymbol{x} = \boldsymbol{x}$ gives

$$G\boldsymbol{x} = \boldsymbol{x} \iff \alpha P\boldsymbol{x} + \frac{1-\alpha}{n}\mathbb{1}_n\mathbb{1}_n^T\boldsymbol{x} = \boldsymbol{x} \iff \alpha P\boldsymbol{x} + \frac{1-\alpha}{n}\mathbb{1}_n = \boldsymbol{x} \iff (\mathbb{I}_n - \alpha P)\boldsymbol{x} = \frac{1-\alpha}{n}\mathbb{1}_n$$

2. Invertibility of $\mathbb{I}_n - \alpha P$: Since P is column-stochastic, $\rho(P) = 1$. Because $\alpha \in (0,1)$, we have $\rho(\alpha P) = \alpha\rho(P) = \alpha < 1$. By Theorem 142 (the Neumann series), $\mathbb{I}_n - \alpha P$ is invertible with $(\mathbb{I}_n - \alpha P)^{-1} = \sum_{k=0}^{\infty} \alpha^k P^k \geq \mathbf{0}$

3. Automatic normalisation ($\mathbb{1}_n^T \boldsymbol{x} = 1$): Left-multiplying $(\mathbb{I}_n - \alpha P)\boldsymbol{x} = \frac{1-\alpha}{n}\mathbb{1}_n$ by $\mathbb{1}_n^T$ and using $\mathbb{1}_n^T P = \mathbb{1}_n^T$ yields

$$\mathbb{1}_n^T(\mathbb{I}_n - \alpha P)\boldsymbol{x} = (1-\alpha)\mathbb{1}_n^T\boldsymbol{x} = \frac{1-\alpha}{n}\mathbb{1}_n^T\mathbb{1}_n = 1 - \alpha \implies \mathbb{1}_n^T\boldsymbol{x} = 1 \qquad \blacksquare$$

Unifying thread of Unit 09 – The resolvent $(\mathbb{I} - M)^{-1}$

Notice how all three topics in this unit are governed by the exact same matrix inverse (the **Neumann series**):

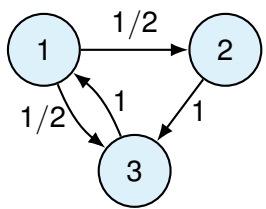
Problem	**Matrix** M	**Condition** $\rho(M) < 1$	**Key quant**
Absorbing Markov chains	Transient block Q	Absorption reachable	$N = (\mathbb{I}_m -$
Katz centrality	αA	$0 < \alpha < 1/\rho(A)$	$\boldsymbol{x} = \beta(\mathbb{I}_n - \alpha A$
PageRank	αP	$0 < \alpha < 1$ $(\rho(P) = 1)$	$\boldsymbol{x} = \frac{1-\alpha}{n}(\mathbb{I}_n -$

For hand calculations on small graphs ($n = 3$ or 4), solving the linear system $(\mathbb{I}_n - \alpha P)\boldsymbol{x} = \frac{1-\alpha}{n}\mathbb{1}_n$ is much faster than computing the dense matrix G and finding its nullspace!

Worked exam-style example 1 – 3-page web digraph

Consider a 3-page miniature web where:

- Page 1 links to Page 2 and Page 3 ($d_1^{\text{out}} = 2$)
- Page 2 links to Page 3 ($d_2^{\text{out}} = 1$)
- Page 3 links to Page 1 ($d_3^{\text{out}} = 1$)



The column-stochastic transition matrix $P = [p_{ij}]$ (where $p_{ij} = 1/d_j^{\text{out}}$ if $j \to i$) is

$$P = \begin{pmatrix} 0 & 0 & 1 \\ 1/2 & 0 & 0 \\ 1/2 & 1 & 0 \end{pmatrix}$$

With damping factor $\alpha = 1/2$, the Google matrix is

$$G = \frac{1}{2}P + \frac{1}{6}\mathbb{J}_3 = \begin{pmatrix} 1/6 & 1/6 & 2/3 \\ 5/12 & 1/6 & 1/6 \\ 5/12 & 2/3 & 1/6 \end{pmatrix}$$

Worked exam-style example 1 – Solving $(\mathbb{I}_3 - \alpha P)\boldsymbol{x} = \frac{1-\alpha}{3}\mathbb{1}_3$

Let us compute the PageRank vector $\boldsymbol{x}$ for $\alpha = 1/2$ by solving $(\mathbb{I}_3 - \frac{1}{2}P)\boldsymbol{x} = \frac{1}{6}\mathbb{1}_3$:

$$\mathbb{I}_3 - \frac{1}{2}P = \begin{pmatrix} 1 & 0 & -1/2 \\ -1/4 & 1 & 0 \\ -1/4 & -1/2 & 1 \end{pmatrix} \implies \begin{cases} x_1 - \frac{1}{2}x_3 = \frac{1}{6} & \text{(Row 1)} \\ -\frac{1}{4}x_1 + x_2 = \frac{1}{6} & \text{(Row 2)} \\ -\frac{1}{4}x_1 - \frac{1}{2}x_2 + x_3 = \frac{1}{6} & \text{(Row 3)} \end{cases}$$

Express x_2 and x_3 in terms of x_1 using Rows 1 and 2:

$$x_2 = \frac{1}{4}x_1 + \frac{1}{6}, \qquad x_3 = 2x_1 - \frac{1}{3}$$

Substitute into the normalisation condition $x_1 + x_2 + x_3 = 1$:

$$x_1 + \left(\frac{1}{4}x_1 + \frac{1}{6}\right) + \left(2x_1 - \frac{1}{3}\right) = 1 \implies \frac{13}{4}x_1 = \frac{7}{6} \implies x_1 = \frac{14}{39}$$

Then $x_2 = \frac{1}{4}\left(\frac{14}{39}\right) + \frac{1}{6} = \frac{10}{39}$ and $x_3 = 2\left(\frac{14}{39}\right) - \frac{13}{39} = \frac{15}{39}$, so

$$\boldsymbol{x} = \begin{pmatrix} 14/39 \\ 10/39 \\ 15/39 \end{pmatrix} \approx \begin{pmatrix} 0.359 \\ 0.256 \\ 0.385 \end{pmatrix} \implies \text{Ranking: Page 3} \succ \text{Page 1} \succ \text{Page 2}$$

Why Page 1 ranks almost as high as Page 3

Look closely at the ranking in our 3-page example:

- **In-degrees:** Page 3 has in-degree 2 (from 1 and 2), while Page 1 and Page 2 both have in-degree 1
- By raw in-degree, Page 1 and Page 2 would be tied!
- However, in PageRank, Page 1 receives the **undivided vote** of the most important page (Page 3, with $d_3^{\text{out}} = 1$), whereas Page 2 only receives **half a vote** from Page 1 ($d_1^{\text{out}} = 2$)
- Consequently, Page 1 ($x_1 = 14/39$) ranks far above Page 2 ($x_2 = 10/39$) and nearly matches Page 3 ($x_3 = 15/39$)

What if we use the standard Google damping factor $\alpha = 0.85$ on the same 3-page network?

$$(\mathbb{I}_3 - 0.85P)\boldsymbol{x} = \frac{0.15}{3}\mathbb{1}_3 \implies \boldsymbol{x} \approx \begin{pmatrix} 0.388 \\ 0.215 \\ 0.397 \end{pmatrix}$$

Worked exam-style example 2 – 4-page web with a dangling node

Suppose we add a dangling Page 4 ($d_4^{\text{out}} = 0$) to a 4-page web where $1 \to 2$, $1 \to 3$, $2 \to 3$, $3 \to 1$ and $2 \to 4$ (so $d_1^{\text{out}} = 2$, $d_2^{\text{out}} = 2$, $d_3^{\text{out}} = 1$ and $d_4^{\text{out}} = 0$):

$$P_{\text{raw}} = \begin{pmatrix} 0 & 0 & 1 & 0 \\ 1/2 & 0 & 0 & 0 \\ 1/2 & 1/2 & 0 & 0 \\ 0 & 1/2 & 0 & 0 \end{pmatrix} \xrightarrow{\text{replace zero col 4 by } \frac{1}{4}\mathbb{1}_4} P = \begin{pmatrix} 0 & 0 & 1 & 1/4 \\ 1/2 & 0 & 0 & 1/4 \\ 1/2 & 1/2 & 0 & 1/4 \\ 0 & 1/2 & 0 & 1/4 \end{pmatrix}$$

Notice why we can simplify $(\mathbb{I}_4 - \alpha P)\boldsymbol{x} = \frac{1-\alpha}{4}\mathbb{1}_4$ even faster when column 4 is $\frac{1}{4}\mathbb{1}_4$:

- Write $P = P_{\text{raw}} + \frac{1}{4}\mathbb{1}_4 \boldsymbol{e}_4^T$, so $P\boldsymbol{x} = P_{\text{raw}}\boldsymbol{x} + \frac{x_4}{4}\mathbb{1}_4$
- Then $G\boldsymbol{x} = \boldsymbol{x} \iff (\mathbb{I}_4 - \alpha P_{\text{raw}})\boldsymbol{x} = \left(\frac{1-\alpha}{4} + \frac{\alpha x_4}{4}\right)\mathbb{1}_4 = c\mathbb{1}_4$, where P_{raw} is sparse!
- Thus we can solve $(\mathbb{I}_4 - \alpha P_{\text{raw}})\boldsymbol{y} = \mathbb{1}_4$ by sparse back-substitution and normalise $\boldsymbol{x} = \boldsymbol{y}/(\mathbb{1}_4^T \boldsymbol{y})$!

Computing PageRank at web scale – Sparse power iteration

For the real Web ($n \sim 10^{10}$ pages), how is PageRank computed in practice?

- **Direct Gaussian elimination** on $(\mathbb{I}_n - \alpha P)\boldsymbol{x} = \frac{1-\alpha}{n}\mathbb{1}_n$ costs $O(n^3)$ time and $O(n^2)$ memory due to fill-in, which is completely impossible for $n = 10^{10}$
- **Forming the dense Google matrix** G explicitly also requires n^2 entries because $\mathbb{J}_n$ is completely dense!
- **Matrix-free power iteration:** Starting from $\boldsymbol{x}^{(0)} = \frac{1}{n}\mathbb{1}_n$, for any probability vector $\boldsymbol{x}^{(k)}$ ($\mathbb{1}_n^T \boldsymbol{x}^{(k)} = 1$) we compute

$$\boldsymbol{x}^{(k+1)} = G\boldsymbol{x}^{(k)} = \alpha P\boldsymbol{x}^{(k)} + \frac{1-\alpha}{n}\mathbb{1}_n$$

Why is this fast?

- We **never** form G: each step only requires one sparse matrix-vector multiplication $P\boldsymbol{x}^{(k)}$ ($O(m)$ operations where $m \approx 10n$ is the number of hyperlinks) plus adding the scalar $\frac{1-\alpha}{n}$ to each entry!
- Since $|\lambda_2(G)| \leq \alpha = 0.85$, after $k = 50$ iterations the error is bounded by $0.85^{50} \approx 3 \times 10^{-4}$

R implementation – Checking our 3-page example

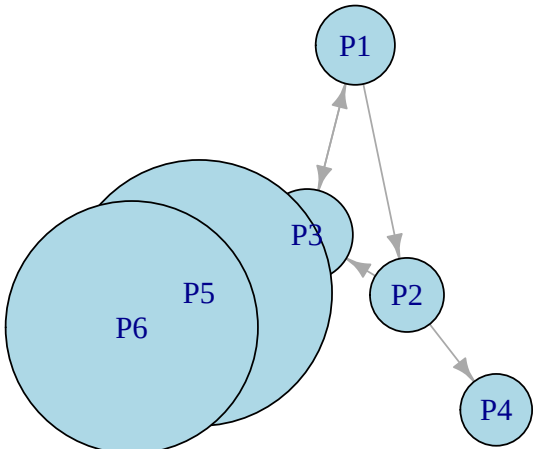
We verify our hand calculation for $\alpha = 0.5$ and $\alpha = 0.85$ in R:

```
P3 <- matrix(c(0,   0, 1,
               0.5, 0, 0,
               0.5, 1, 0), nrow = 3, byrow = TRUE)

# Solve (I - alpha * P) x = ((1 - alpha) / n) * 1 for alpha = 0.5
alpha <- 0.5
x_half <- solve(diag(3) - alpha * P3, rep((1 - alpha) / 3, 3))
MASS::fractions(x_half)

## [1] 14/39 10/39  5/13

# Compare with igraph::page_rank at alpha = 0.85
library(igraph)
g3 <- graph_from_edgelist(matrix(c(1,2, 1,3, 2,3, 3,1), ncol = 2, byrow = TR
page_rank(g3, damping = 0.85)$vector

## [1] 0.3877897 0.2148106 0.3973997
```

R example – 6-page web with a dangling node and spider trap

Consider a 6-page web graph where Page 4 is a dangling node ($d_4^{\text{out}} = 0$) and Pages $\{5, 6\}$ form a spider trap:

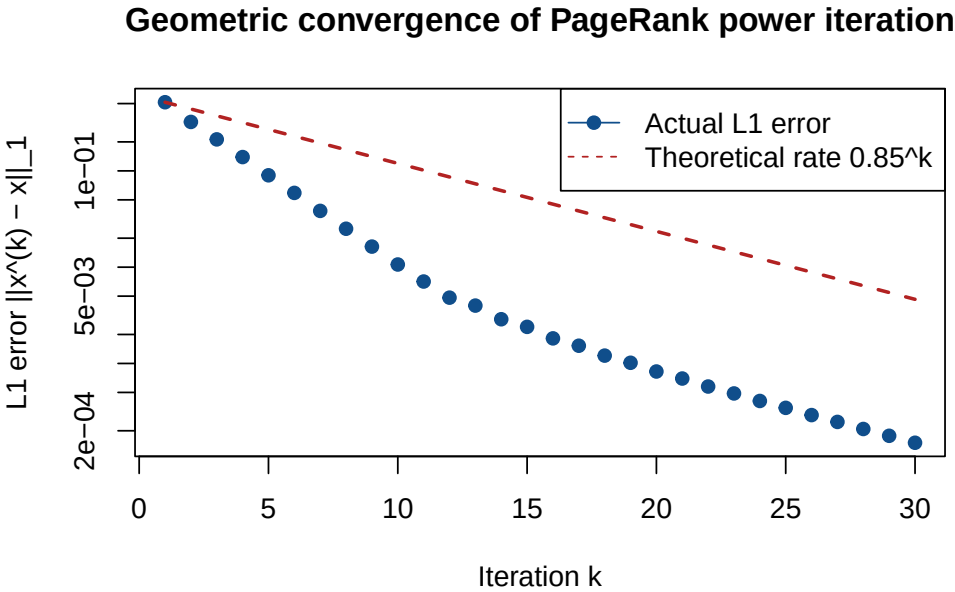
```
edges <- matrix(c(1,2, 1,3, 2,3, 3,1, 2,4, 3,5, 5,6, 6,5),
                ncol = 2, byrow = TRUE)
g6 <- graph_from_edgelist(edges, directed = TRUE)
V(g6)$name <- paste0("P", 1:6)
pr6 <- page_rank(g6, damping = 0.85)$vector
round(pr6, 4)

##     P1     P2     P3     P4     P5     P6
## 0.0728 0.0646 0.0920 0.0611 0.3653 0.3442

par(mar = c(0, 0, 0, 0))
plot(g6, vertex.size = 350 * pr6 + 18, vertex.color = "lightblue",
     vertex.label.cex = 1.1, edge.arrow.size = 0.7)
```


P1
P3
P5
P6
P2
P4

Power iteration vs linear system in R

We implement the dangling-node fix and sparse power iteration from scratch and plot convergence:

```
A6 <- as.matrix(as_adjacency_matrix(g6)) # row i -> col j in igraph, so tra
A6 <- t(A6)                              # now a_{ij} = 1 if j -> i
n <- nrow(A6)
out_deg <- colSums(A6)
P6 <- matrix(0, n, n)
for (j in 1:n) {
  if (out_deg[j] > 0) P6[, j] <- A6[, j] / out_deg[j] else P6[, j] <- rep(1
}
alpha <- 0.85
x_exact <- solve(diag(n) - alpha * P6, rep((1 - alpha) / n, n))

x_k <- rep(1 / n, n)
err <- numeric(30)
for (k in 1:30) {
```


Geometric convergence of PageRank power iteration
Actual L1 error
Theoretical rate 0.85^k
L1 error ||x^(k) − x||_1
2e−04
5e−03
1e−01
0
5
10
15
20
25
30
Iteration k

Summary of Unit 09

- **Absorbing Markov chains:** Reordering states so the r absorbing states precede the m transient states gives the canonical column-stochastic block form
$$P = \begin{pmatrix} \mathbb{I}_r & R \\ \mathbf{0}_{m,r} & Q \end{pmatrix}, \qquad \rho(Q) < 1$$
- **Fundamental matrix:** $N = (\mathbb{I}_m - Q)^{-1} = \sum_{k=0}^{\infty} Q^k$ counts expected visits to transient states, $\boldsymbol{t}^T = \mathbb{1}_m^T N$ gives expected absorption times and $B = RN$ gives absorption probabilities
- **Spectral centrality:** Eigenvector centrality ($A\boldsymbol{x} = \lambda \boldsymbol{x}$) requires strong connectivity; Katz centrality ($(\mathbb{I}_n - \alpha A)\boldsymbol{x} = \beta \mathbb{1}_n$ for $\alpha < 1/\rho(A)$) fixes directed acyclic and source nodes by adding baseline importance
- **PageRank:** Normalising votes by out-degree and adding teleportation yields the primitive column-stochastic Google matrix $G = \alpha P + \frac{1-\alpha}{n}\mathbb{1}_n\mathbb{1}_n^T$, whose unique stationary vector solves $(\mathbb{I}_n - \alpha P)\boldsymbol{x} = \frac{1-\alpha}{n}\mathbb{1}_n$